CISSP 800

Software Development Security



AL NAFI,
A company with a focus on education,
wellbeing and renewable energy.

© 2018 Al-Nafi. All Rights Reserved.

Forty Hadith on the importance of Knowledge, learning and Teaching.

© 2018 Al-Nafi. All Rights Reserved.

Hadith 1

The messenger of Allah peace be upon him said:

"Seeking 'Ilm is incumbent on every Muslim."

(Ibn Hajar al-Asqalani, Al-Matalib al-Aliyah)

Ibn Majah reported this hadith as follows: Anas Ibn Malik reported, the messenger of Allah (peace be upon Him) said:

"Seeking knowledge is incumbent on every Muslim; he who offers knowledge to those who do not appreciate it is like the one who decorated pigs with precious stones, pearls and gold."

How Nafi Members Study!

1. Please subscribe to our YouTube channel https://www.youtube.com/channel/UC2yAW4Oq27r1yuRE8ePKRvA
2. Follow us on Facebook https://www.facebook.com/info.alnafi/
3. Follow us on Twitter https://twitter.com/nafiPakistan
4. All Nafi members MUST study on the portal https://alnafi.com/login/ and connect using your Nafi member username and password. If you have problems connecting then please contact us via info@alnafi.com
5. To ask questions as it relates to studies please join our group https://www.facebook.com/groups/alnafi/
6. Once on the portal they can follow their classes by:
    - Watching videos
    - Asking questions
    - Attempting quizzes
    - Studying official Nafi notes
    - Keep track of their studies long with many more features

© 2018 Al-Nafi. All Rights Reserved.



Domain 8:
Software Development Security

8.1 Understand and integrate security in the Software Development Life Cycle (SDLC)

- Development methodologies
- Maturity models
- Operation and maintenance
- Change management
- Integrated product team

8.2 Identify and apply security controls in development environments

- Security of the software environments
- Configuration management as an aspect of secure coding
- Security of code repositories

8.3 Assess the effectiveness of software security

- Auditing and logging of changes
- Risk analysis and mitigation

© 2018 Al-Nafi. All Rights Reserved.



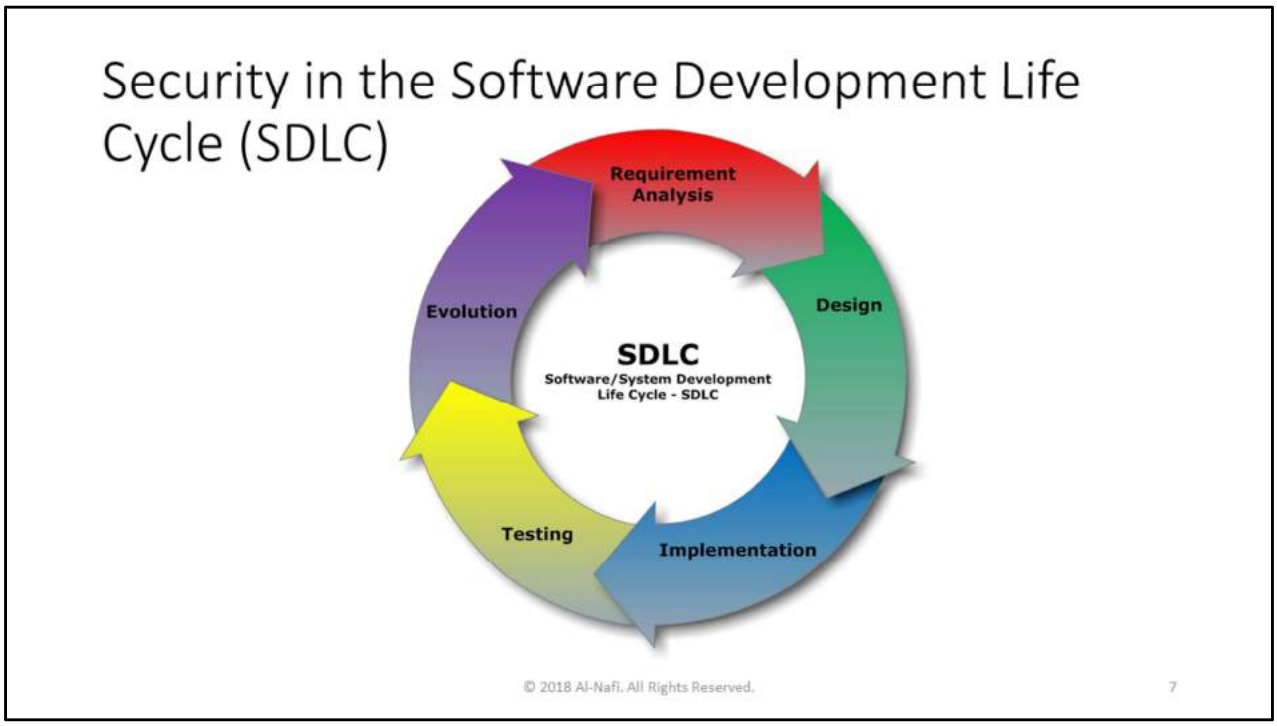
Domain 8:
Software Development Security

8.4 Assess security impact of acquired software

8.5 Define and apply secure coding guidelines and standards

- Security weaknesses and vulnerabilities at the source-code level
- Security of application programming interfaces
- Secure coding practices

© 2018 Al-Nafi. All Rights Reserved.



Reference https://www.dataversity.net/how-you-should-approach-the-secure-development-lifecycle/#

How You Should Approach the Secure Development Lifecycle

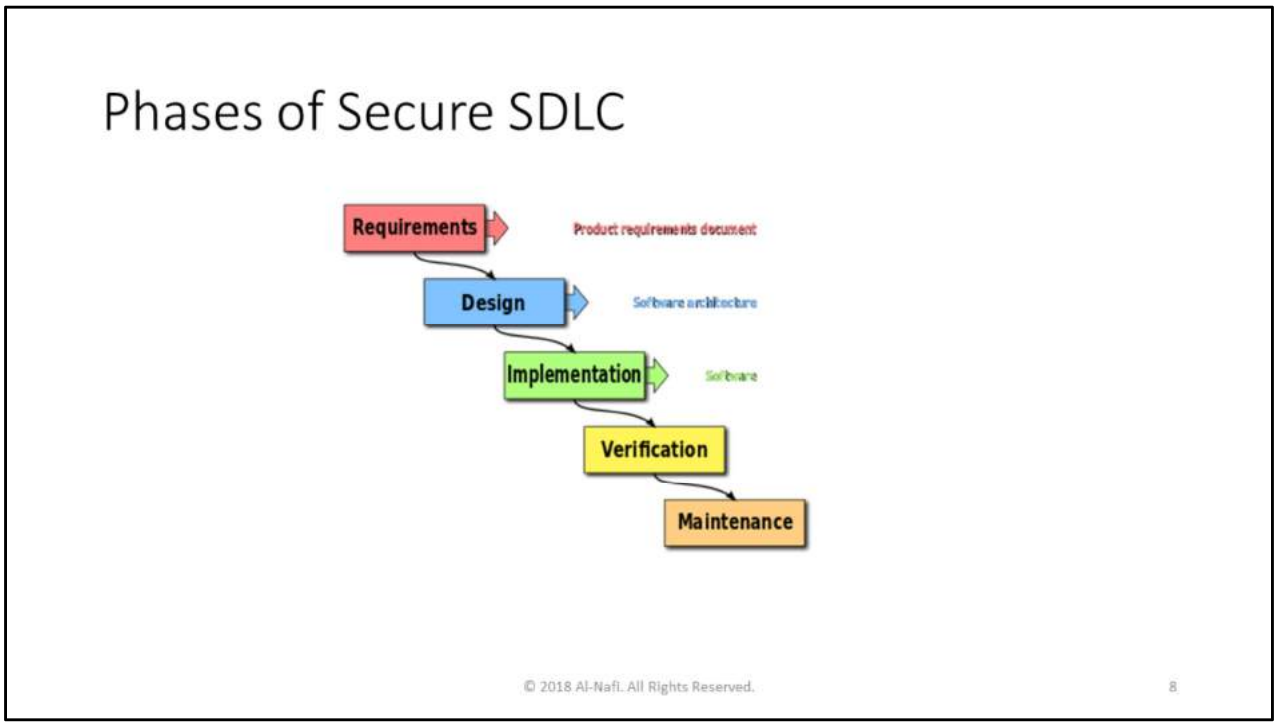
Is your development process producing secure software? Ensuring that their software is secure is one of the main challenges developers face daily. It is not enough to test the software only at the required stages, which can result in overlooking minor vulnerabilities. The attackers are always ready to exploit even the slightest flaw.

One of the key strategies you can use to secure your software is a Secure Software Development Lifecycle (Secure SDLC or SDL). Read on to learn about the SDL, why it is important, and how you can implement it.

What Is Secure SDLC and Why Is Important for You?

Secure Development Lifecycle (SDL) is the process of including security artifacts in the Software Development Lifecycle (SDLC). SDLC, in turn, consists of a detailed plan that defines the process organizations use to build an application from inception until decommission.

Development teams use different models such as Waterfall, Iterative or Agile. However, all models usually follow these phases:
Planning and requirements
Architecture and design
Test planning
Coding
Testing the code and results
Release and maintenance
Developers usually performed security-related tasks only at the testing stage, resulting in discovering issues too late or not at all. With time, teams started to integrate security activities to catch vulnerabilities early in the development cycle. With this in mind, the concept of secure SDLC started. Secure SDLC integrates activities such as penetration testing, code review, and architecture analysis into all steps of the development process.
The main benefits of adopting a secure SDLC include:
Makes security a continuous concern—including all stakeholders in the security considerations
Helps detect flaws early in the development process—reducing business risks for the organization
Reduces costs—by detecting and resolving issues early in the lifecycle.



Reference https://www.dataversity.net/how-you-should-approach-the-secure-development-lifecycle/#

How Does Secure SDLC work?

Most companies will implement a secure SDLC simply by adding security-related activities to their development process already in place. For example, they can perform an architecture risk analysis during the design phase.

There are seven phases in most SDLCs although they may vary according to the methodology used, such as Agile or Waterfall:

Concept
Planning
Design and Development
Testing
Release
Sustain
Disposal

For example, a development team implementing the waterfall methodology may

follow the following scheme:
Each security activity should correspond with a phase in the SDLC, as follows:

SDLC Artifacts

SDLC Phases	SDL Activities - SDL Artifacts			
		Who initiates this activity?		Who initiates this?
Concept	SDL discovery, preparation	Typically a sponsor	Security training	Everyone
Planning	Threat modeling	Senior engineers and project managers	Security requirements = Gap analysis Privacy Implementation Assessment (PIA)	Senior engineers and project managers
	Third party software tracking	Senior technical member/technical lead		
Design and Development	Threat modeling updates		Static analysis	Developers, QA or security expert
	Security design review	Development team	Vulnerability scanning	Developers, QA or security expert
	Code review	Development team		
Testing	Fuzzing	Developers, QA or security expert	Dynamic analysis Security review	Developers, QA or security expert
	Third-party penetration testing	Third-party certified pen tester		
Release	Final gap analysis		Final privacy review	
	Final security tests review		Open source licensing review	
Sustain	Third-party software tracking and review.	Senior technical member/technical lead	External vulnerability disclosure and response	

© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.dataversity.net/how-you-should-approach-the-secure-development-lifecycle/#

Some considerations to take into account when implementing a Secure SDLC are:

SDL Discovery

The goal should be to determine the security objectives required by the software, what are the possible threats and what regulations the organization needs to follow.

When working on the scope of the SDL, a development team should focus on deliverables such as security milestones, required certifications, risk assessments, key security resources, and required third-party resources.

Security Baselines

Here you should list what are the requirements your product need to comply with. For example, only using approved cryptography and libraries or using multifactor authentication. A Gap Analysis, contrasting the product's features against the baseline, is useful to identify the areas not complying with the security baseline. When finding a gap, this needs to be addressed as early as possible in the lifecycle. Since companies release a product based on the percentage of compliance with the

baseline, is important to work on gaps through the development process.

Security Training and Awareness

The company should provide security training sessions for developers, designers, architects, and QA. They can focus on secure design principles, security issues, web security or encryption.

Security awareness sessions are not geared specifically for the development team, involving everyone that is connected to the project within the organization. Sessions should be easy in terms of technical level and can include topics such as the various cybersecurity threats or risk impact and management.

Threat Modeling

Modeling the software components to identify and manage threats early in the development lifecycle. This helps the team to develop an incident response plan from the beginning, planing the appropriate mitigations early before the damage becomes more complicated to manage. Typically follows four steps, preparation, analysis, determine mitigations and validation. This activity can have different approaches such as protecting specific critical processes, exploit weaknesses or focus on the system design.

Third-Party Software Tracking

Since third-party software can be open source or commercial use, the team needs to list all third-party tools used in the project. This inventory should be done in the early stages of the development cycle. There are software tools that track and list the third-party components, sending you an alert if any component needs upgrading or has licensing issues.

Security Design and Peer Review

The development team should ensure the software is built with the most secure features. When reviewing the functional feature design, the developer should include a security design review, thinking like an attacker to discover the feature vulnerabilities. When reviewing the code, developers need to be aware of the most common coding security pitfalls. They can follow a checklist for secure coding, for example, that ensures that important security events are logged, check the permeability of the authentication process, or validate the user input.

Security Testing

While code review focus on functionality, security testing checks how vulnerable is the new product to attacks. Some of the testing activities include:

Static Analysis—identifies the exact location of weaknesses by analyzing the software without executing it.

Dynamic Analysis—identifies weaknesses by running the software, helping find infrastructure flaws and patch errors.

Vulnerability Scanning—injects malicious inputs against running software to check how the program reacts. Mostly used to scan applications with a web interface.

Fuzzing—involves giving invalid, random data to a program, to check for access

protocols and file formats. The test helps find bugs that humans often miss by generating random input and try all possible variations.

Third-party penetration testing—the tester simulates an attack to discover coding or system configuration flaws, and discover vulnerabilities a real attacker can exploit. It is required that the tester is an external party not connected to the team.

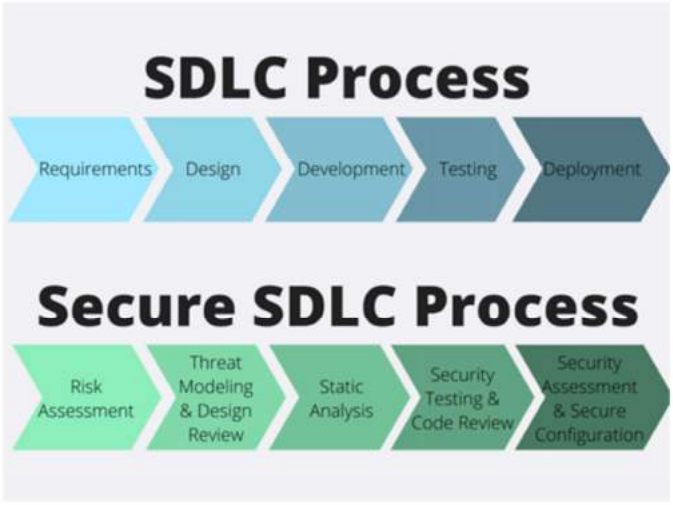
Data Disposal and Retention

Usually at the end of a product's life companies dispose of old products, or data that they don't need to use anymore. Many companies delete or overwrite encryption keys, in a process called "crypto-shredding". While getting rid of outdated data is a necessity, there are concerns when comes to keep the confidentiality of the information. Some regulations such as GDPR have specific requirements for data disposal and retention.

What's Next?

Implementing secure SDL helps you follow security best practices, integrating security activities and checkups across the development cycle. This will help to increase your product and company security posture.

Secure SDLC another Perspective



© 2018 Al-Nafi. All Rights Reserved.

To ensure software development success, organizations should choose appropriate development lifecycle methodologies to guide them in properly completing the phases involved in software development. As software development projects have become more complex, a number of methodologies have been created to manage this complexity, such as waterfall, agile, spiral, and a number of others. As a systems development effort goes through a lifecycle, the methodology can guide the engineers and developers in completing the phases properly. The theme of our presentation is that security needs to be involved in the phases, and therefore, the methodologies.

The software development lifecycle (SDLC) is a framework that can guide the phases of a software development project from inception to defining the functional requirements to implementation. As the word “development” implies, this lifecycle ends at the implementation phase. Regardless of the methodology used, the SDLC outlines the phases a software development project needs go to through. Organizations need to choose methodologies carefully, as the model chosen should be based on the requirements of the organization. As with any other project, understanding the requirements ahead of time is paramount for the success of the project itself. For example, some models work better with long-term, complex

projects, while others are more suited for short-term projects. However, the key point being made here is that a formalized SDLC needs to be utilized, but the entire process needs to involve security. The best security is always what is designed into the system, not what is added later.

The Role of ISO 27034 in Application Security

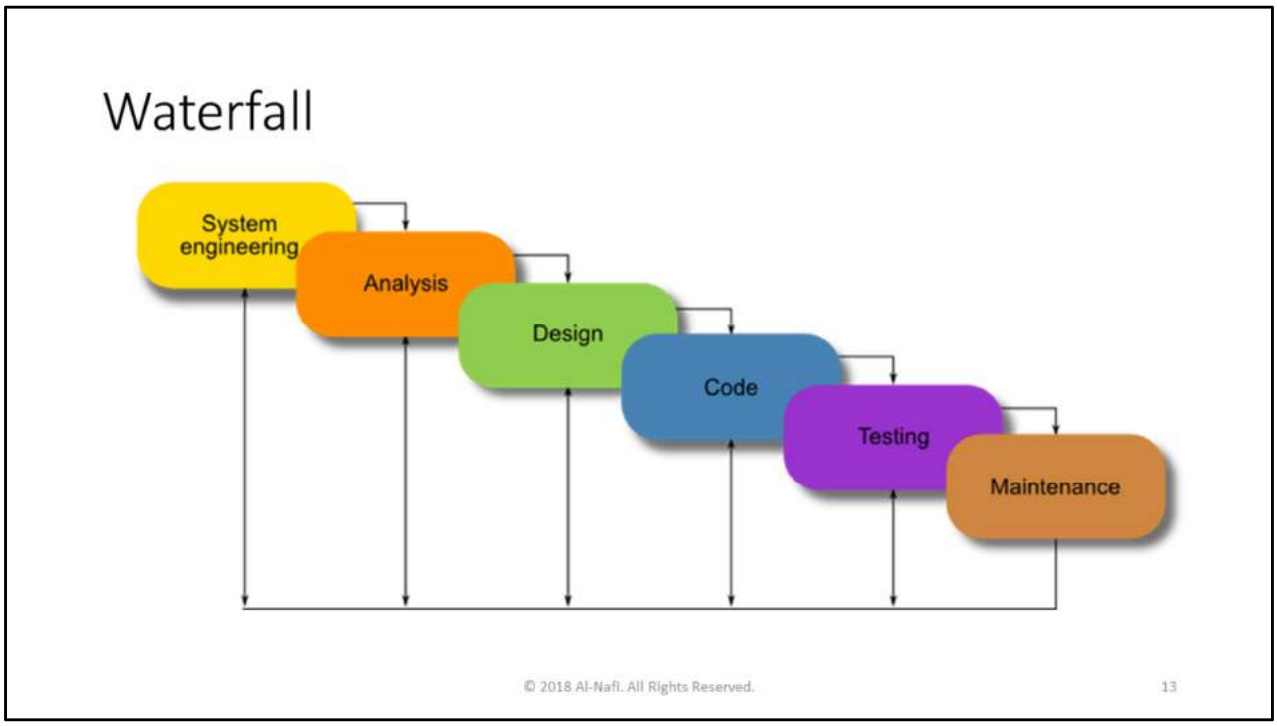
- At a later stage in CODS we will cover ISO 27034 standard and its various parts which goes into much detail. Exam preparation of CSSLP will also be introduced next year in 2020-2021 which goes into much greater detail about how to build security when developing an application or any software.

© 2018 Al-Nafi. All Rights Reserved.

Various SDLC Methods

- Waterfall
- Structured Programming Development
- Spiral Method
- Cleanroom
- Iterative Development
- Prototyping
- Modified Prototype Model (MPM)
- Rapid Application Development (RAD)
- Joint Analysis Development (JAD)
- Exploratory Model

© 2018 Al-Nafi. All Rights Reserved.



Reference : https://airbrake.io/blog/sdlc/waterfall-model

First introduced by Dr. Winston W. Royce in a paper published in 1970, the waterfall model is a software development process. The waterfall model emphasizes that a logical progression of steps be taken throughout the software development life cycle (SDLC), much like the cascading steps down an incremental waterfall. While the popularity of the waterfall model has waned over recent years in favor of more agile methodologies, the logical nature of the sequential process used in the waterfall method cannot be denied, and it remains a common design process in the industry.

Throughout this article we'll examine what specific stages make up the core of the waterfall model, when and where it is best implemented, and scenarios where it might be avoided in favor of other design philosophies.

Some more specific takes on SDLC include:

Rapid Application DevelopmentTest-Driven DevelopmentSoftware Development Life CycleIterative ModelExtreme ProgrammingScaled Agile FrameworkAgile ModelScrumRational Unified ProcessBig Bang ModelV-ModelConceptual ModelKaizen ModelKanban ModelSpiral ModelThe Six Stages of Falling Water

Actually implementing a waterfall model within a new software project is a rather straightforward process, thanks in large part due to the step-by-step nature of the method itself. There are minor differences in the numbers and descriptions of the steps involved in a waterfall method, depending on the developer you ask (and even the year during which you ask him or her). Regardless, the concepts are all the same and encompass the broad scope of what it takes to start with an idea and develop a full-scale, live application.

Requirements: During this initial phase, the potential requirements of the application are methodically analyzed and written down in a specification document that serves as the basis for all future development. The result is typically a requirements document that defines *what* the application should do, but not *how* it should do it.

Analysis: During this second stage, the system is analyzed in order to properly generate the models and business logic that will be used in the application.

Design: This stage largely covers technical design requirements, such as programming language, data layers, services, etc. A design specification will typically be created that outlines how exactly the business logic covered in analysis will be technically implemented.

Coding: The actual source code is finally written in this fourth stage, implementing all models, business logic, and service integrations that were specified in the prior stages.

Testing: During this stage, QA, beta testers, and all other testers systematically discover and report issues within the application that need to be resolved. It is not uncommon for this phase to cause a “necessary repeat” of the previous coding phase, in order for revealed bugs to be properly squashed.

Operations: Finally, the application is ready for deployment to a live environment. The operations stage entails not just the deployment of the application, but also subsequent support and maintenance that may be required to keep it functional and up-to-date.

The Advantages of the Waterfall Model

While the waterfall model has seen a slow phasing out in recent years in favor of more agile methods, it can still provide a number of benefits, particularly for larger projects and organizations that require the stringent stages and deadlines available within these cool, cascading waters.

Adapts to Shifting Teams: While not necessarily specific to the waterfall model only, using a waterfall method does allow the project as a whole to maintain a more detailed, robust scope and design structure due to all the upfront planning and documentation stages. This is particularly well suited to large teams that may see members come and go throughout the life cycle of the project, allowing the burden of design to be placed on the core documentation and less on any individual team member.

Forces Structured Organization: While some may argue this is a burden rather than a

benefit, the fact remains that the waterfall model *forces* the project, and even the organization building said project, to be extraordinarily disciplined in its design and structure. Most sizable projects will, by necessity, include detailed procedures to manage every aspect of the project, from design and development to testing and implementation.

Allows for Early Design Changes: While it can be difficult to make design changes later in the process, the waterfall approach lends itself well to alterations early in the life cycle. This is great when fleshing out the specification documents in the first couple stages with the development team and clients, as alterations can be made immediately and with minimal effort, since no coding or implementation has actually taken place up to that point.

Suited for Milestone-Focused Development: Due to the inherent linear structure of a waterfall project, such applications are always well-suited for organizations or teams that work well under a milestone- and date-focused paradigm. With clear, concrete, and well understood stages that everyone on the team can understand and prepare for, it is relatively simple to develop a time line for the entire process and assign particular markers and milestones for each stage and even completion. This isn't to suggest software development isn't often rife with delays (since it is), but waterfall is befitting the kind of project that needs deadlines.

The Disadvantages of the Waterfall Model

While some things in software development never really change, many others often fall by the wayside. While Dr. Royce's initial proposal of what is now known as the waterfall model was groundbreaking when first published back in 1970, over four decades later, a number of cracks are showing in the armor of this once heralded model.

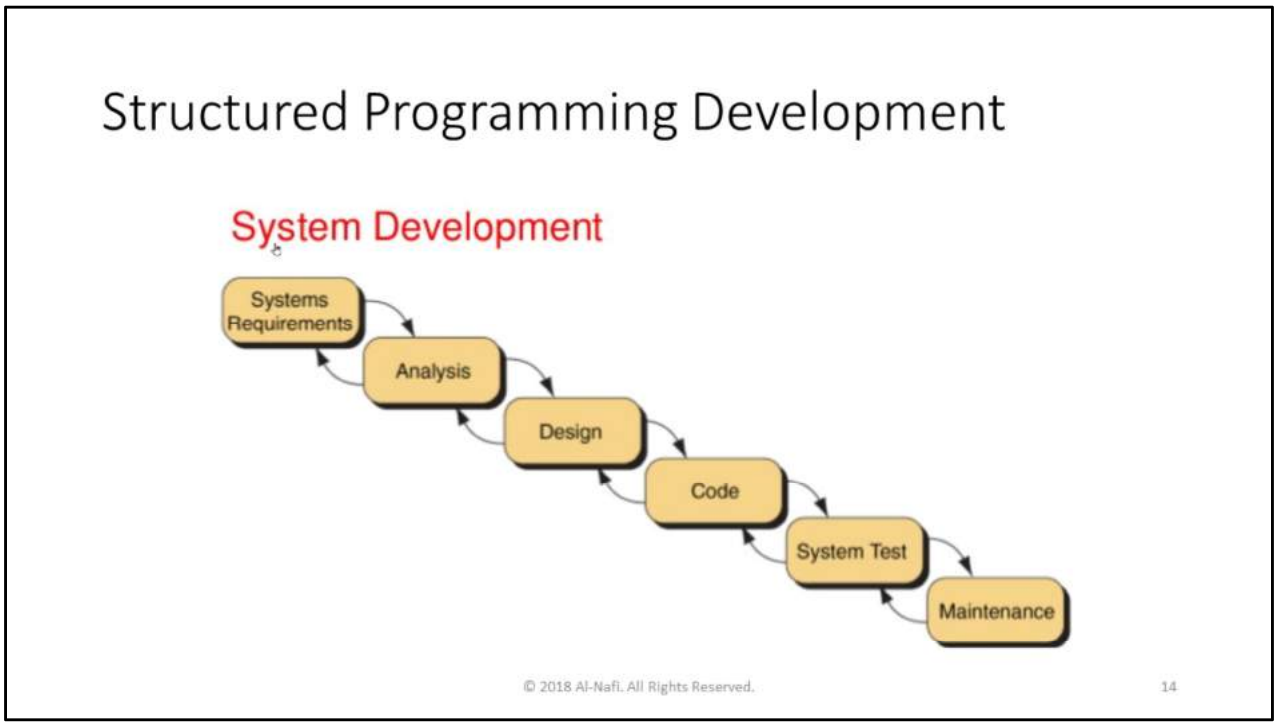
Nonadaptive Design Constraints: While arguably a whole book could be written on this topic alone, the most damning aspect of the waterfall model is its inherent lack of adaptability across all stages of the development life cycle. When a test in stage five reveals a fundamental flaw in the design of the system, it not only requires a dramatic leap backward in stages of the process, but in some cases, can be often lead to a devastating realization regarding the legitimacy of the entire system. While most experienced teams and developers would (rightfully) argue that such revelations shouldn't occur if the system was properly designed in the first place, not every possibility can be accounted for, especially when stages are so often delayed until the end of the process.

Ignores Mid-Process User/Client Feedback: Due to the strict step-by-step process that the waterfall model enforces, another particularly difficult issue to get around is that user or client feedback that is provided late into the development cycle can often be too little, too late. While project managers can obviously enforce a process to step back to a previous stage due to an unforeseen requirement or change coming from a client, it will be both costly and time-consuming, for both the development

team and the client.

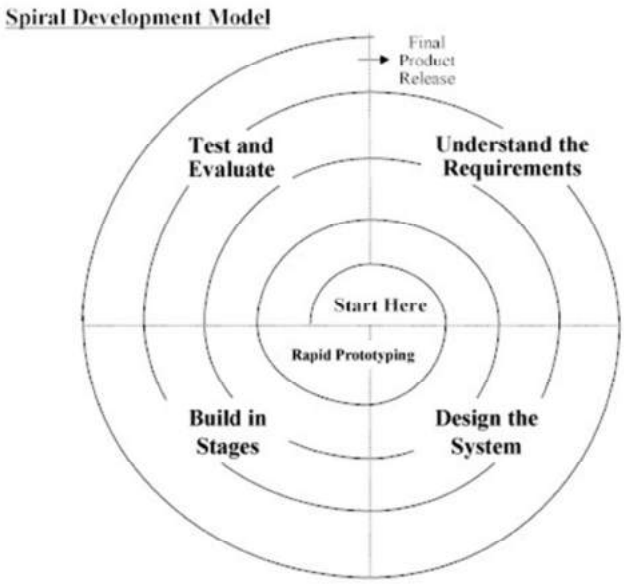
Delayed Testing Period: While most of the more modern SDLC models attempt to integrate testing as a fundamental and always-present process throughout development, the waterfall model largely shies away from testing until quite late into the life cycle. This not only means that most bugs or even design issues won't be discovered until very late into the process, but it also encourages lackadaisical coding practices since testing is only an afterthought.

In spite of going through an explicit testing phase during implementation of a waterfall model project, as discussed above, this testing is often too little, too late. *In addition to* the normal testing phase, you and your team should strongly consider introducing an effective error management tool into the development life cycle of your project. Airbrake's error monitoring software provides real-time error monitoring and automatic exception reporting for all your development projects. Airbrake's state of the art web dashboard ensures you receive round-the-clock status updates on your application's health and error rates. No matter what you're working on, Airbrake easily integrates with all the most popular languages and frameworks. Plus, Airbrake makes it easy to customize exception parameters, while giving you complete control of the active error filter system, so you only gather the errors that matter most.



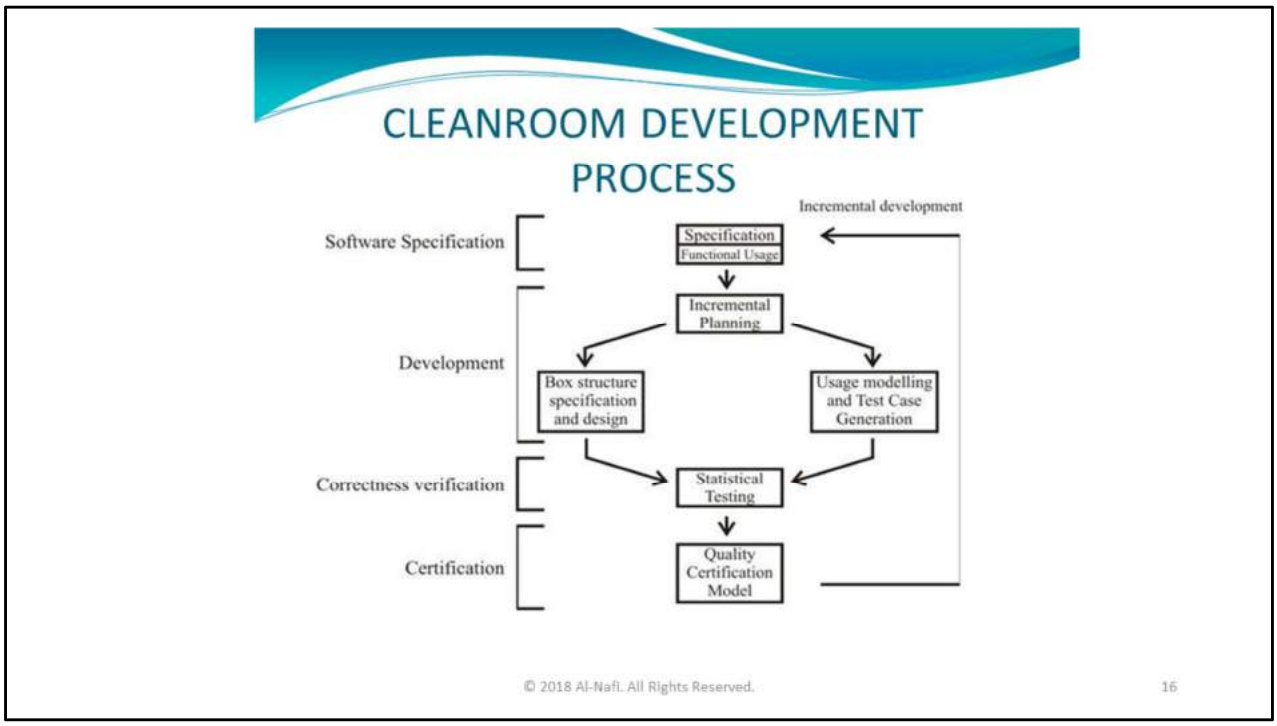
A method that programmers use to write programs allowing considerable influence on the quality of the finished products in terms of coherence, comprehensibility, freedom from faults, and security. This methodology uses extensive uses of subroutines and block structures that can be heavily reused. Structured programming also promotes discipline, allows introspection, and provides controlled flexibility. It requires defined processes and develops code into modules that are reused, and each phase is subject to reviews and approvals. It also provides a structured approach for security to be added as a formalized, involved approach.

Spiral Method

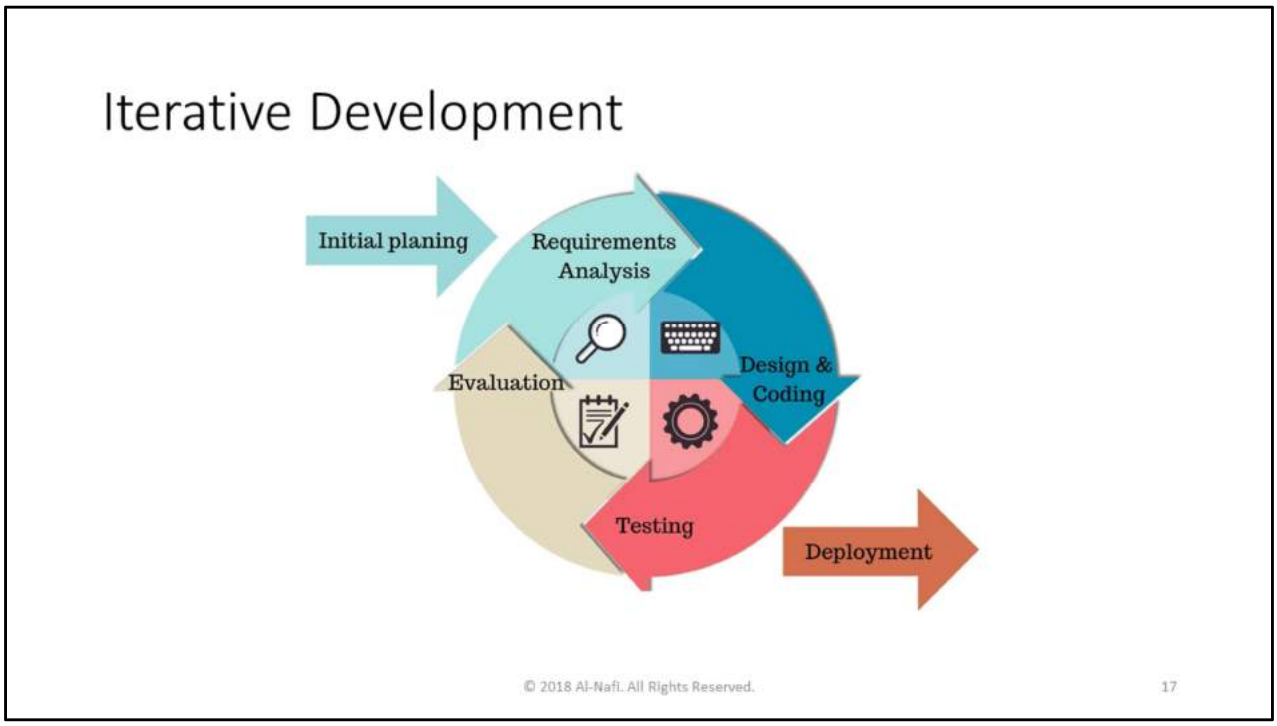


© 2018 Al-Nafi. All Rights Reserved.

A nested version of the original waterfall method, the development of each phase is carefully designed using the waterfall model, but the distinguishing feature of the spiral model is that in each phase we add four sub-stages, based on what is known as the Deming plan, do, check, act (PDCA) model. Specifically, a risk assessment review (Check) is done at each phase. The estimated costs to complete and the schedules are revised each time the risk assessment is performed. We can consider this model to be an improvement of the waterfall methodology based on being able to address, at each phase, the results of the risk sub-phase assessment. At this point, a decision is made to continue or cancel the project.



This methodology is focused on controlling and, at best, avoiding defects and bugs in the software. The emphasis is to write the code correctly the first time rather than trying to find the problems once they are already there and trying to address them later. Essentially, cleanroom software development focuses on defect prevention rather than defect removal. To allow this to happen, more time is spent in the early phases, focusing heavily on the assumption that the time spent in other phases, such as testing, is theoretically reduced. The basic premise, therefore, is that quality is achieved through proper design rather than testing and remediation later. In terms of security, the same pattern applies, if risk considerations are addressed up front, security becomes an integral part of the system as a design rather than adding it later. This is always preferred as far as security is concerned. Security should always be designed into the system based on requirements rather than being retrofitted later.

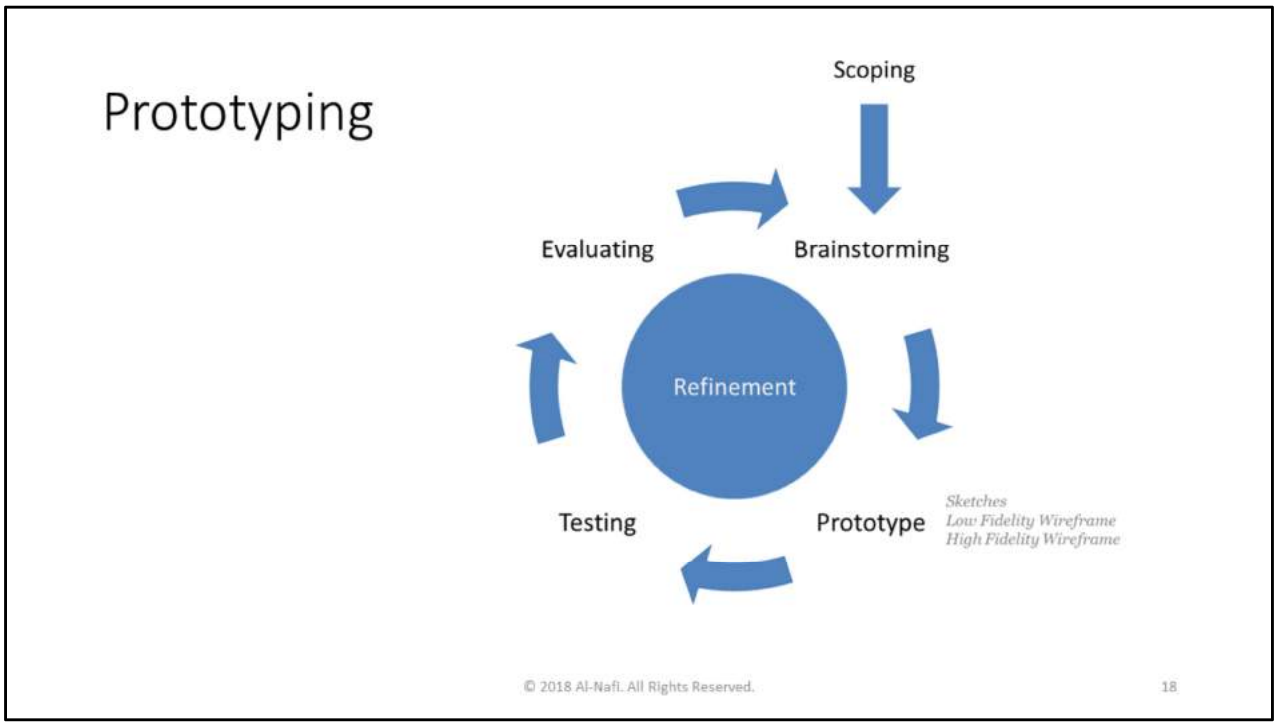


Reference https://searchsoftwarequality.techtarget.com/definition/iterative-development

Iterative development is a way of breaking down the software development of a large application into smaller chunks. In iterative development, feature code is designed, developed and tested in repeated cycles. With each iteration, additional features can be designed, developed and tested until there is a fully functional software application ready to be deployed to customers.
Typically iterative development is used in conjunction with incremental development in which a longer software development cycle is split into smaller segments that build upon each other.
Iterative and incremental development are key practices in Agile development methodologies. In Agile methodologies, the shorter development cycle, referred to as an iteration or sprint, is time-boxed (limited to a certain increment of time, such as two weeks). At the end of the iteration, working code is expected that can be demonstrated for a customer.
Iterative development contrasts with a traditional waterfall method in which each phase of the software development life cycle is "gated." Coding doesn't begin until design of the entire software application is complete and has gone through a phase

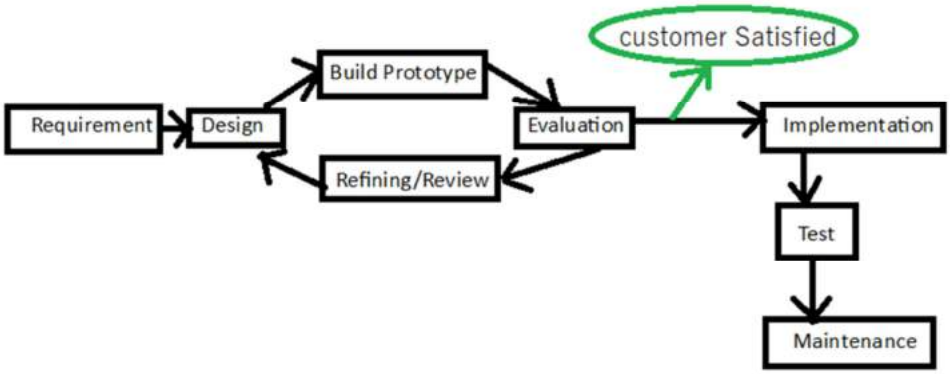
gate review. Likewise, testing doesn't begin until coding is complete and has passed necessary phase gate reviews.

The purpose of working iteratively is to allow more flexibility for changes. When requirements and design of a major application are done in the traditional method (sometimes referred to as *BDUF* or *Big Design Up Front*), there can be unforeseen problems that don't surface until development begins. By working iteratively, the project team goes through a cycle where they evaluate with each iteration, and determine what changes are needed to produce a satisfactory end product.



In prototyping, the objective is to build a simplified version of the entire application, release it for review, and use the feedback from the stakeholders to review to build a second, much better version. This is repeated until the owner and stakeholders are satisfied with the final product. Prototyping is broken down into a step-by-step process that includes initial concept, design and implementation of initial prototype, refining the prototype until acceptable to the owner, and complete and release final version.

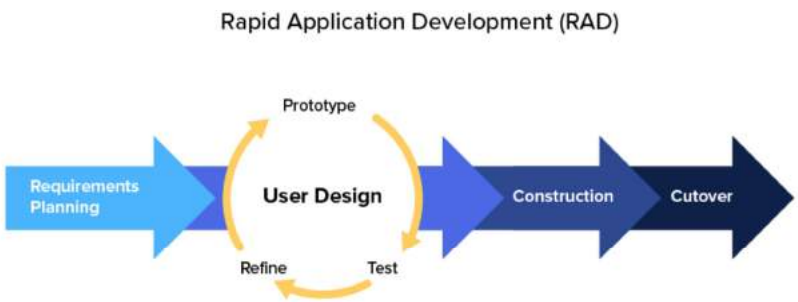
Modified Prototype Model (MPM)



© 2018 Al-Nafi. All Rights Reserved.

A refined form of the above prototyping methodology that is ideal for web application development, it allows for the basic functionality of a desired system or component to be formally deployed in a quick time frame. The maintenance phase is set to begin after the deployment. The goal is to have the process be flexible enough so the application is not based on the state of the organization at any given time. As the organization grows and the environment changes, the application evolves with it rather than being frozen in time.

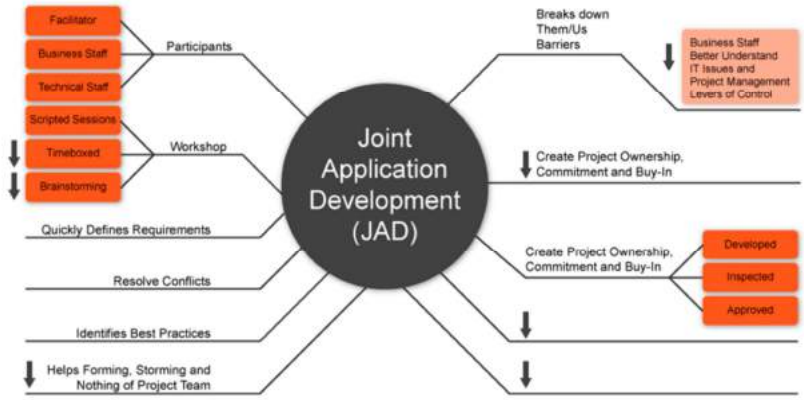
Rapid Application Development (RAD)



© 2018 Al-Nafi. All Rights Reserved.

Also, a refined form of prototyping, rapid application development (RAD) requires strict time limits on each phase and relies on efficient tools that enable quick development. The goal is to produce quality code quickly. While this sounds attractive, it must be handled properly because the quick development process may be a disadvantage if decisions are made so rapidly that it leads to poor design.

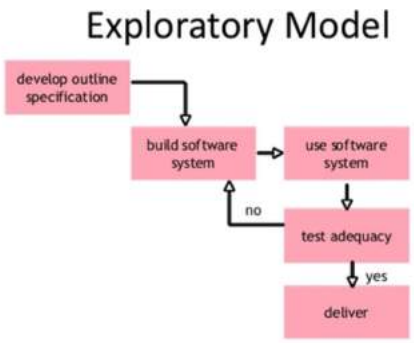
Joint Analysis Development (JAD)



© 2018 Al-Nafi. All Rights Reserved.

Originally invented to enhance the development of large mainframe systems, joint analysis development (JAD) has become very useful in today's environments. The premise is to have facilitation techniques that become an integral part of the management process that helps developers to work directly with owners and stakeholders to develop a working application. This is a novel idea that involves all stakeholders in the entire process. The success of this methodology is based on having key players communicating at all critical phases of the project. The focus, is in having the people who actually perform the job work together with those who have the best understanding of the technologies available to design the best solution. In other words, facilitation techniques bring together a team of stakeholders, including owner, expert systems developers, technical experts, and security professionals, throughout the development lifecycle. As we have mentioned, this needs to involve security as well. While input from the owner may result in a more functional program, the involvement of large numbers of stakeholder may help in addressing the security requirements, or at least that is the goal.

Exploratory Model



Incrementally build a system using test/user feedback to guide development. Find and refine the real requirements by building versions and getting real users to use them.

35

© 2018 Al-Nafi. All Rights Reserved.

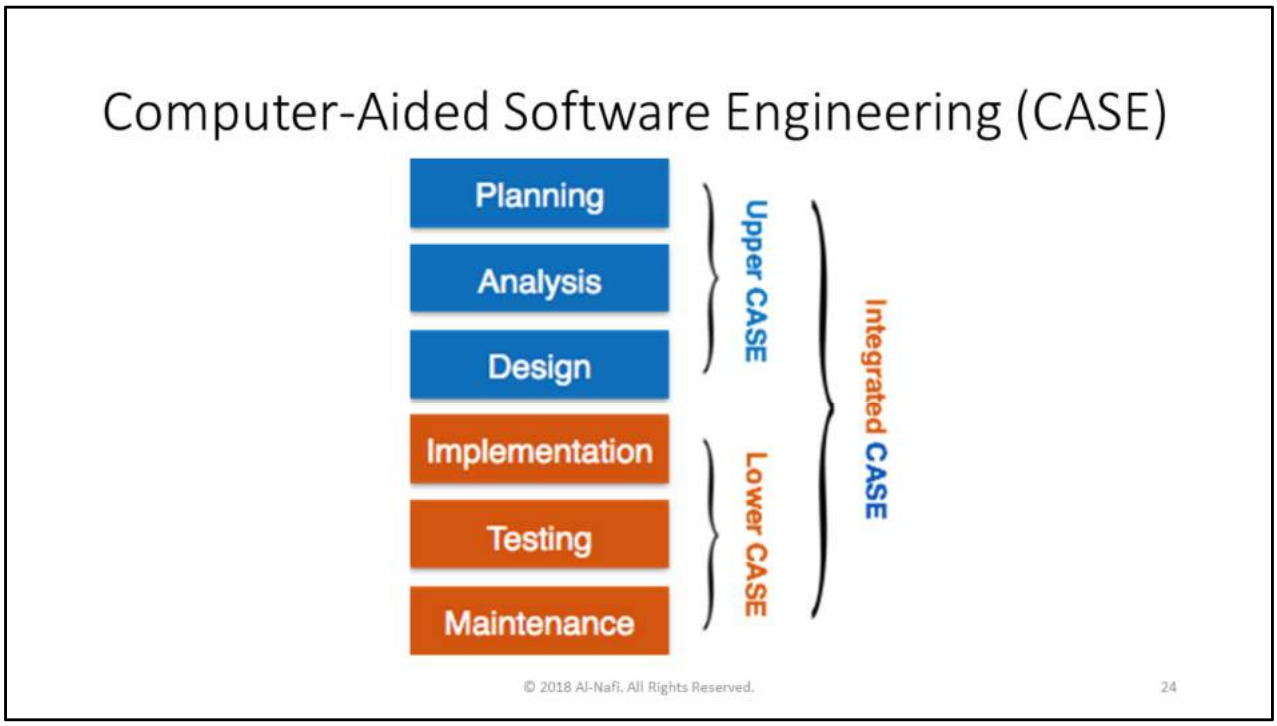
22

The exploratory model uses a set of requirements built with what is currently available. A big part of this model requires assumptions to be made as to how the system might work, and further insights and suggestions by interested parties, including security, are combined to create a usable system. Because of the lack of structure being the basis for this model, security requirements need to take priority to address the requirements properly. The security professionals need to ensure the requirements are addressed appropriately.

Other Methods and Models

- Computer-Aided Software Engineering (CASE)
- Computer-Based Development
- Reuse Model
- Extreme Modeling

© 2018 Al-Nafi. All Rights Reserved.



Reference https://searcherp.techtarget.com/definition/CASE-computer-aided-software-engineering

Computer-aided software engineering (CASE) describes a broad set of labor-saving tools used in software development. They create a framework for managing projects and are intended to help users stay organized and improve productivity. There was more interest in the concept of CASE tools years ago, but less so today, as the tools have morphed into different functions, often in reaction to software developer needs. The concept of CASE also received a heavy dose of criticism after its heyday.

Features of CASE tools

CASE tools, which are sometimes called integrated CASE or I-CASE tools, cover all aspects of the software development lifecycle, which includes writing the code, implementation and maintenance. The tools help in every aspect of development work: managing, modeling, error-checking, version control, designing, diagraming tools, prototyping and other aspects associated with software engineering.

Compilers and testing tools are also considered part of the CASE tool set.

Everything is centralized in a CASE repository, which provides an integrated system for project management information, code specifications, test cases and results, design specifications, diagrams and reports. This setup provides a place for teams and managers to keep track of what has been accomplished and what still needs to be done. This information is often displayed graphically so users can quickly find what they need, as well as get a quick overview of the project.

History of CASE tools and criticism

CASE software tools emerged as a significant product category in the 1980s. They were developed in a response to a need to bring order to large software development projects, and vendors claimed they would improve IT productivity and reduce errors.

A short explanation of CASE tools.

The U.S. government, a major builder of custom development projects, spent millions on CASE tools. But the government later became a critic of vendor claims about their capabilities. "Little evidence yet exists that CASE tools can improve software quality or productivity," wrote the Government Accountability Office in a 1993 report on the use of CASE tools by the U.S. Defense Department.

About a decade later, in 2002, a research paper also noted problems with CASE deployments. It found evidence of "a conceptual gap between the software engineers who develop CASE tools and the software engineers who use them," according to the paper, "Empirical Study of Software Developers' Experiences," by Ahmed Seffah and Rex B. Kline, computer science researchers at Concordia University.

Uses of CASE tools evolved

CASE tools have evolved to accommodate visual programming, object-oriented programming and Agile software development processes.

Tools that fit in the CASE category are widely available, but this umbrella term or approach doesn't have the relevance it once did in describing software engineering tools. Developers may be more likely to think in terms of specific tool categories, such as visual modeling and simulation software, system architecture tools and diagramming tools such as Microsoft Visio.

Computer-Based Development

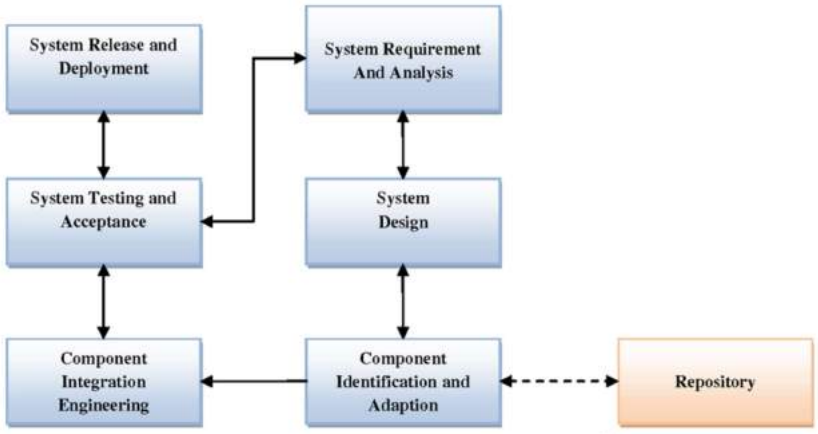
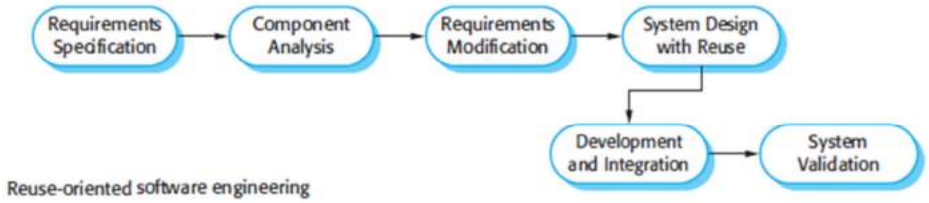


Fig. 1: an Improved Model for Component Based Software Development [12]

© 2018 Al-Nafi. All Rights Reserved.

This model is based on a process of using standardized and building blocks to assemble, rather than develop the application. The components are made up of sets of standardized data and standardized methods of processing that data. These sets, when used together, offer scheduling and cost-effective benefits to the development process and the team members involved. From a security perspective, the advantage might be that components have previously been tested for security functionality and assurance effectiveness. This is very similar to object-oriented programming (OOP) where objects and classes may be designed with security methods initially and then reused as required.

Reuse Model

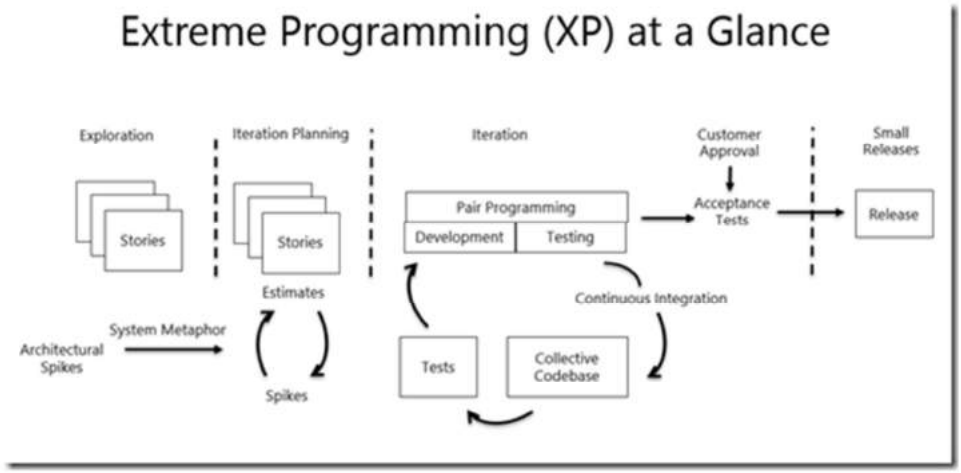


Reuse-oriented software engineering

© 2018 Al-Nafi. All Rights Reserved.

In this model, an application is built from already existing and tested components. The reuse model is best suited for projects using object oriented development because objects can be created, exported, reused, or modified as required. From a security perspective, the components would then be chosen based on the known effectiveness
of the security characteristics.

Extreme Modeling



© 2018 Al-Nafi. All Rights Reserved.

This discipline of software development is based on having several values and characteristics of software development. The values are simplicity, communication, and feedback all combined into the process. Despite the name, extreme programming is an attempt to use a structured approach to software development, relying on subprojects of limited and defined scope and developers always working in pairs. The team produces the software in a series of small, fully integrated releases that are supposed to fulfill the owner-defined needs. This implies that the owners need to be involved in defining the needs in the first place. It makes sense, as well, to involve security in defining those needs ahead of the developers programming the requirements. As we have mentioned earlier, this model relies on simplicity of the process, communication between all involved stakeholders, including security, and feedback to ensure requirements are addressed properly.

Software development Model Choice Considerations and Combinations



© 2018 Al-Nafi. All Rights Reserved.

The trends in software development combined with specific organizational needs have shown that companies tend to combine different software development methodologies to fit the specific design and development requirements. For example, an application may need a certain set of activities to take place to achieve success, or the organization may require certain standards or processes to meet industry or government requirements. In these cases, it would make sense to combine several models to allow that organization to develop the proper requirements in the most cost-effective and efficient way. However, as we have seen in all models, security needs to be included as part of the process from the start, to the end of not only the software development process, but also to the end of the SLC. Security, therefore, must be included, regardless of methodologies used. As security professionals know, the best type of security is what is designed into the system, not what is added later. Regardless of methodology used, security needs to be included right at the beginning as requirements for security functionality needs to be understood at that point. All stakeholders, including owners, must also be involved in determining those requirements. Historically, development has focused on functionality rather than security; therefore, it is critically important to educate those individuals responsible for the development,

the managers who oversee the projects, and the owners that are accountable for the protection of valuable assets. Development today is much more focused on security, and it is important for an organization to streamline the process of development of systems and applications and involve security in the early phases and throughout the SDLC.

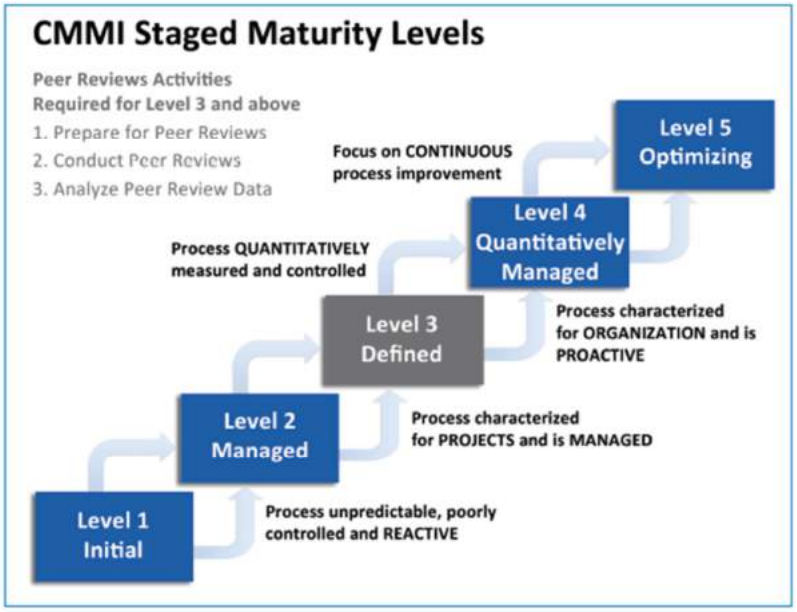
Maturity Models

- Capability Maturity Model (CMM) for Software or Software Capability Maturity Model (SW-CMM)
- ISO/IEC 21827:2008 Information technology — Security techniques — Systems Security Engineering — Capability Maturity Model® (SSE-CMM®)

© 2018 Al-Nafi. All Rights Reserved.

We will cover **ISO/IEC 21827:2008** later in CODS so that our students understand and can relate to CMM in further detail inshAllah.

CMMI Capability Maturity Model



© 2018 Al-Nafi. All Rights Reserved. 30

We will cover this later inshAllah within CODS later in 2020-2021.

The Software Engineering Institute (SEI) released the Capability Maturity Model for Software (CMM or SW-CMM) back in 1991. Even though software development has evolved in many ways since then, this model is very useful in allowing an organization to measure their current capability in software development and also to formulate a plan by which they can get better. The CMM focuses on quality management processes and contains five maturity levels that contain required measurement parameters within each maturity level. The five levels describe an evolutionary path from chaotic and unstructured processes to mature, disciplined, and optimized software processes. The whole purpose of using CMM is to allow organizations to mature to a higher level of quality in software development. So, to summarize, the CMM framework as shown in Figure 8.6 establishes a basis for evaluation of the reliability and improvement of the software development environment.

Initial: At the initial level, it typically means that good practices can be repeated, but they may be unorganized and chaotic. If an activity is not repeated, there is no

reason to improve it. Therefore, organizations would
be able to show that they have policies, procedures, and practices and commit to using them so that the organization can perform software development in a consistent manner.

Repeatable: In this level, best practices for software development are repeatable and can be rapidly transferred across various groups in the organization without problems. Practices need to be defined in such way so that the organizations allows for transfer of processes across project boundaries. This can provide for standardization and repeatable processes across the entire organization.

Defined: At the defined level, standard processes are formalized and all new developments happen with new, stricter, and standardized processes. The processes are well-understood and are very proactive. Managed: At this level, quantitative and measurable objectives are established for tasks. Quantitative measures are established, calculated, and maintained to form a baseline from which an assessment is possible. This can ensure that the best practices are followed and deviations from those measured objectives are reduced.

Optimizing: At the final level of the CMM, practices are continuously improved to enhance the organization's capability, and they are also optimized. This level also focuses on continuous improvement, and feedback from one phase will reach and positively impact development in other phases, all ensuring positive future results.



Reference https://www.cio.com/article/2439314/change-management-change-management-definition-and-solutions.html

We will do hands on implementation of ISO 20000 later inshAllah as it will during ISO 20000 Lead Implementer course.

What is change management?
In modern IT, change management has many different guises. Project managers view change management as the process used to obtain approval for changes to the scope, timeline, or budget of a project. Infrastructure professionals consider change management to be the process for approving, testing, and installing a new piece of equipment, a cloud instance, or a new release of an application. ITIL, ISO20000, PMP, Prince2, as well as other methodologies and standards, prescribe the process to gain approval and make changes to a project or operating environment.

The Association of Change Management Professionals (ACMP), PROSCI, the Innovation and Organizational Change Management Institute (IOCMI), and others view change management from an organizational perspective. While each

group has its own approaches, frameworks and language, these groups all address the human side of change in organizational contexts.
The following article focuses on change management from an organizational perspective, to distinguish it from the process-based changes of ITIL, Prince2, and so on. Here, “change” refers to any event or program the enterprise undertakes that causes major disruption to daily operations — for example, a new ERP installation or digital transformation. The clearest definition of this type of organizational change management (OCM) is provided by Sheila Cox of Performance Horizons who states: "Organizational change management ensures that the new processes resulting from a project are *actuall*y adopted by the people who are affected.”

What are the benefits of change management?
Change management reduces the risk that a new system or other change will be rejected by the enterprise. By itself OCM does not reduce costs or increase sales. Instead, it increases the teamwork required for the enterprise accept the change and operate more efficiently.

When is organizational change management needed?
OCM is needed whenever the enterprise undertakes a program or event that interrupts day-to-day operations. Such an undertaking will impact:
The work content of individual jobs. Many jobs require individuals or groups to perform tasks repeatedly. An accounting department has daily, weekly, monthly, and annual activities. Over time, most people become comfortable with the tools provided and the rhythm of the work calendar. Even simple changes may disrupt the workflow and be disconcerting for the staff.
The roles of individual employees. Many people view their value to the organization as being a good technical architect, programmer, or security specialist. When asked to take on a different role, they may become very uncomfortable. People with excellent technical skills often struggle when asked to become managers. Rather than performing all of the tasks, they have to learn to work through other people. Once they are no longer rewarded for the skills that made them successful, employees may question their purpose.
The organization itself. Executive teams debate major changes for months before making final decisions, enabling each member to gain a deeper understanding of the effects the change will have on the enterprise. Even if they don’t agree with the final decision, they have time to determine whether to accept the new direction or to depart gracefully. Individuals lower in the hierarchy rarely have time to process major changes. Executives do not want employees to worry about events that may never happen until it is clear the change will take place. In addition, tighter insider trading enforcement prohibits executives from sharing information about upcoming mergers,

acquisitions, or divestitures. As such, individuals who are not part of the executive team have much less time to prepare for the planned change and may decide to leave while the change is undertaken, making change management more difficult.

What are the requirements for change management success?

Organization change management programs require several things to be successful:

The right executive sponsor. Sponsorship is critical. The OCM sponsor is responsible for developing the case for change and obtaining the necessary OCM resources. For this, the sponsor needs the support of the CEO to make it clear that the effort is important.

The sponsor must understand the case for change clearly enough have a detailed discussion about the challenges that created the need for a different way of operating. She should be confident enough to confront skeptics and close enough to the details to justify the approach selected and the reasons the alternatives were rejected.

The sponsor needs to understand the impact on the staff. Good sponsors are concerned about the people who will be affected by the change. These sponsors communicate honestly while treating everyone fairly and respectfully. Rather than merely relating the facts, they take the time to listen to people and to empathize with the individuals who dislike the new way of operating. If people are to be terminated or reassigned, sponsors should know when it will happen and how everyone will be treated. They explain why the change was necessary, and do what they can to smooth the transition for individuals whose jobs are transformed. The best sponsors help everyone losing a job find the next opportunity

Cultural willingness to adapt and change. All organizations resist change to some degree, but ones that follow the dictum “if it ain’t broke don’t fix it” often need a major wake-up call to behave differently. The public revelation of sexual misconduct allegations against Harvey Weinstein provides a dramatic example of a call to address a long standing problem. A number of enterprises that had done little to stop sexual harassment suddenly took action.

Skilled change management teams embrace the organization’s emotional energy. They use company stories, language, and behavior to emphasize those parts of the current culture that are aligned with the planned change. These teams celebrate behaviors they wish to encourage by publicly recognizing individuals exhibiting these behaviors. Change management teams use every opportunity to reinforce the way the change helps the enterprise.

Individual willingness to change. Individuals must be willing to examine new information and adopt new behaviors and approaches. Since most people prefer the status quo, this can be difficult. Typically, most people only accept changes that make sense and improve their job content or their work environment.

Rewards and consequences. Major changes need to be reinforced by rewards and consequences. Individual performance plans with specific, measurable results need

to reinforce the desired future state. Individuals who meet their objectives need to be rewarded appropriately and those that do not need to face consequences.
A consulting firm, wanting broader market recognition, encouraged all partners to speak at industry conferences and to write for industry publications. Several partners became very successful at both. While their articles and talks generated new business, the client revenue each partner managed actually decreased. When the compensation plan did not reward them sufficiently for the additional firm revenue to offset their decreased client revenue, they were very unhappy. The firm's leadership team had to adjust the compensation plan quickly in order to prevent the partners from leaving.

Why is change management difficult?

It takes a great deal of time to change attitudes and behaviors. Application implementations, even large ones, are easier to plan and manage; project managers know when a module is tested or server installed. OCM managers have a much harder time measuring progress; gauging support can be tricky. Just when it appears that a key individual supports the change, the person raises another objection and returns to old behaviors.

Executives, often assume that everyone impacted will find the business case so compelling they will automatically accept the new way of operating. But most people resist change or are unpredictable. This creates several difficulties for the OCM team:

Change management is not deterministic. Unlike computer programs, people can be unpredictable and illogical. OCM activities that are effective with one group may be ineffective with another. Messages may resonate with some people but not with others.

Change management is a contact sport. The OCM team needs to interact one on one with individuals who will need to change. Emails, videos, and other mass communication can reinforce a message, but these don't make people feel the enterprise cares about their difficulties. Change is personal; sometimes people whose jobs have been transformed need someone else to listen to their frustrations before they will accept the new reality.

Midlevel and frontline staff must be engaged. Midlevel and frontline staff can make or break a major program. Since they understand the operational details of the current processes, they can anticipate potential problems and likely customer reactions. Individuals who are not sensitive to the disruption that major change can create often believe it is more efficient to involve fewer people early in the process. While involving more people in the change process creates additional work for the OCM team, it also builds commitment. Midlevel and frontline staff who see their suggestions accepted are more likely to support the final result.

Cultural differences can make OCM difficult. Cultural norms are different around the globe. The OCM effort needs to be aware of local customs even with a global system intended to standardize enterprise operations. Care needs to be taken to be sensitive

to these and other cultural norms:

Communications style. Denmark, Germany, Israel, Netherlands, and U.S. are very direct. India, Japan, Pakistan, and the Philippines tend to be indirect and believe it is very important for both parties to save face. In these cultures, individuals avoid saying no, and frequently mean, "I understand" rather than "I agree" when they say "yes."

Time orientation. Meetings in Germany, Switzerland, and the U.S. start and end when scheduled. Little time is dedicated to introductions, even when some attendees are meeting each other for the first time. Spain, Thailand, Brazil, and the Caribbean are less concerned about time. Things can wait until later in the day or even tomorrow. In such countries it is impolite to rush into a business discussion; only after the host and the visitor have shared refreshments and pleasantries can business begin.

Egalitarianism. Australia, Canada, Israel, New Zealand, U.S. have little hierarchy with almost everyone on a first-name basis. Conversely, hierarchy is very important in India, Iran, Japan, Saudi Arabia and other countries. Junior staff in these countries invariably defer to the senior person.

Violating cultural norms can cause great resentment. The best OCM teams are very sensitive to local cultural norms even when the people at headquarters demand a standard project rollout and standard OCM program globally.

Change management may be an afterthought. With major IT efforts, the project team is often consumed by business process changes, interfaces to other systems, data cleanup, etc. If the OCM effort is not started concurrently with the rest of the program, it may only be started when the program team experiences resistance from end users. Even enterprises that assert that OCM is critical sometime reduce or eliminate the OCM budget if the overall program gets too expensive.

Change management can be started too early. The OCM effort needs to be tightly coupled to the rest of the change program. This is particularly difficult with major IT programs when the OCM efforts begin before new system details have been finalized. In the absence of tangible information about the new system, the OCM team either sounds vague or describes what they hope the new system will do. When the new system fails to materialize quickly or has less functionality than anticipated, supporters often become disillusioned.

OCM and the change program may be disconnected. The rational and emotional cases for change need to be integrated tightly. Frequently, executives communicate a rational, logical case for change that lacks emotional appeal. People respond to calls to action that make them feel they are part of something that is more important than any single person and are energized by visions that capture their hearts as well as their minds.

Meg Whitman, former CEO of Hewlett-Packard, integrated appeals to the heart and mind. As she discussed in her post, "The Power of Transparent Communication," she

and her team attempted to build a strong connection to HP's history and traditions. They reinforced the "HP Way" cultural value that the quality of the work is as important as position in the hierarchy.

How should a change management team be structured?

The OCM team should be integrated with the team responsible for implementing the change. The OCM sponsor should be a senior executive, often the CEO. The sponsor is the cheerleader who describes why the change is important and how it will help the enterprise. This person acquires necessary resources, establishes OCM goals along with consequences for failure to support the change.

The OCM sponsor is supported by an OCM project manager who directs the day-to-day activities of the OCM team. The OCM project manager works closely with the overall program manager responsible for implementing the change. Together the OCM project manager and the overall program manager coordinate training, communications, and supporter recognition.

OCM staff, known as OCM Champions, are supporters of the change who "sell" the benefits to specifics departments, business units, and individuals. They start working with their target group shortly after the program team begins planning. As part of change training, these Champions explain how the change will help the individuals affected.

After implementation, Champions continue to make sure the change is supported and used by the individuals whose jobs have changed. They continue to espouse the benefits of the change and pay particularly attention to anyone having difficulty with the change. Sometimes they merely listen; in other cases they obtain additional training or other help for the struggling individual.

The best Champions are well respected even though they may not be very high in the organization chart. They wield informal power as opinion leaders, performing their duties competently and with grace. Many have been with the enterprise for a long time. Frequently, they serve as informal coaches to new employees who may be more senior in the hierarchy. They motivate others, inspiring them to do a good job. Other employees seek them out to determine if the people leading a major initiative will be persistent enough to make the change stick.

Change Targets are the groups and individuals who need to change their behaviors and their attitudes. They are the recipients of training necessary to implement the change. As they become supporters of the change, they are usually recognized for their support.

For more on leading change, see "8 secrets of effective change leaders."

What are the major steps in a change management program?

Organization change management programs typically have fewer tasks and greater complexity than the program they are supporting. The OCM program has to adapt and change on the fly to accommodate the vagaries of human nature as supporters

backslide and skeptics become supporters.

While there are different approaches to OCM, most can be summarized in the four major steps below:

Engage. The program begins when the sponsor creates a vision describing how the enterprise will operate after the change has been implemented. This vision should include the benefits that will accrue to the enterprise and should describe how the change will affect the staff. Ideally, improvements to the work environment will be obvious to the majority of the staff.

As part of engagement, the OCM team discusses the coming change with potential supporters to determine their willingness to support the change and to create a sense of urgency to implement the change. The OCM team also identifies likely skeptics and attempts to determine their concerns. In many cases, the team will commission a formal change readiness assessment to gain a more precise understanding of the enterprise's willingness to change.

Plan. The OCM team identifies all departments, business units and groups that will need to change along with key stakeholders in each. In parallel, the OCM team analyzes how the various parts of the change will impact the way that people perform their jobs. This analysis enables the OCM team to answer the most common question posed during a major change, "What's in it for me?"

As it becomes more obvious which stakeholders support the change, which are undecided, and which don't support the change, the OCM team creates a change plan with specific actions for each individual and group. Individual OCM members are assigned to work with individual stakeholders based in part on the strength of the relationship between the OCM team member and the specific stakeholder.

During this phase, the OCM team begins to assess the degree to which stakeholders accept the change. At this point, acceptance measures are informal and based on impressions from meeting behavior, one-on-one discussions and other interactions.

Rollout. During implementation, the OCM team communicates with individuals at all levels in the enterprise to gain their support for the change. Communications typically begin with a formal announcement from the CEO, supported by videos, emails, work station log-on announcements, town hall meetings, etc. The OCM team hopes to empower supporters and help individuals or groups become successful quickly. The OCM group identifies and celebrates successes publicly and rewards individuals responsible for each success.

As the rollout continues, attitudinal surveys are frequently employed to better gauge employee acceptance and commitment to the change. Special interventions are created and used for individuals and groups that appear reluctant to accept the change.

Reinforce. Because people rarely behave as others would like them to behave, the OCM team regularly revisits and updates change goals, rewards, communications and consequences. Experience is the best teacher. Repeated interactions with individual

stakeholders usually reveals their degree of acceptance, enabling the OCM team to adjust its approach as necessary

Tasks, projects and behaviors that support the change should be part of individual performance plans. Items in the performance plan need to be clear, measureable and achievable. In addition, these items need to be weighted appropriately against the other goals in the performance plan.

Change management is rarely straightforward. The OCM plan may be depicted as a Gantt chart using the same tools as the IT project plan. However, in practice, OCM activities rarely have clear tasks, precedents and durations. Most OCM teams cycle through the four steps above multiple times during any OCM effort. Lessons learned at any point are incorporated into the OCM vision and communications. OCM work is not complete until the change is fully implemented and adopted by the people affected.

Who offers organizational change management certification?

A wide variety of universities and associations offer change management certificates and certifications. These include:

The University of Virginia's Darden School offers a "Managing Individual and Organizational Change" certificate designed to create resilient leaders and adaptable teams that can guide enterprise change.

Prosci's Change Management Certification Program is built around a change management methodology with supporting tools that participants apply to a current project.

Cornell University's SC Johnson College of Business offers a series of online courses leading to a certificate as well as professional development credits with the Society for Human Resources Management (SHRM)

The Society for Human Resources Management (SHRM) offers two certifications: the SHRM Certified Professional (SHRM-CP) and the SHRM Senior Certified Professional (SHRM-SCP).

MIT Sloan School offers "Leading Change in Complex Organizations" as part of a Management and Leadership certification.

Michigan State University's Eli Broad College of Business offers a professional Certificate in Change Management focusing on helping organizations alter existing processes, grow, introduce new products, reorganize or undertake other actions to be more competitive.

Stanford University's Organizational Renewal program focuses on design thinking and innovation to implement change within an enterprise.

The Association for Talent Development focuses on improving efficiency and service quality through a six step change model taught through case studies.

Northwestern University's School of Professional Studies focus on structured change approaches used to introduce new products, improved quality, IT systems, etc.

For more information on additional change management cert opportunities, see "7

change management certifications to boost your IT career."

Why do individuals resist change?

Resistance is a natural part of the change process. When expectations are disrupted, individuals often feel uncomfortable. Even positive changes such as a marriage or the birth of a child can cause discomfort. Here are some of the reasons why employees resist change and how it affects the change management process:

Inability. Individuals may lack the necessary skills or knowledge to operate in the new environment. Fear of the unknown can keep people from fully participating in training. Some worry they will not be able to understand how to operate the new system and will be overshadowed by smarter colleagues. Other groups may lack the resources to operate in the changed environment. This can become a problem during an acquisition if the acquiring company folds a department from the acquired company into its department without appropriately increasing staffing. When acquisitions are justified by claiming the merged companies will eliminate redundant jobs, management is sometimes tempted to eliminate staff before the merger is fully complete. Mergers that occur on paper but not in reality disappoint customers, fragment staff loyalty, and erode IT service levels. For more on this see "Half-baked mergers."

Unwillingness. People who don't believe in the change usually resist the change. Reasons vary but can include: They see no value to the new way of operating; they believe the change is too difficult; they perceive the change as too risky. Other people may believe the wrong option was selected. Still others worry their job will be less important and they will no longer be experts.

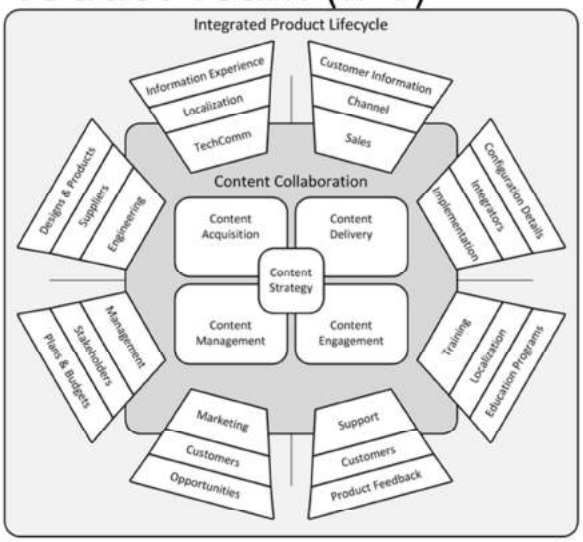
Change fatigue. Change requires a great deal of mental effort. People who switch languages as they travel from country to country find themselves drained at the end of each day even if everyone they visit attempts to speak the traveler's native language. The mental effort to understand the words spoken by individuals who do not speak a language well requires intense concentration. Too many new systems, reorganizations, mergers, or other changes can also create change fatigue. After a time, most people crave stability; at some point few people will make the extra effort required to undertake one more change.

Personal issues. Few people lead perfect lives and most worry about something. Individuals close to retirement, facing divorce, serious illness, or other personal issues frequently resist all changes in order to feel they retain some control over their life. Intellectually, these individuals may understand the reasons for the change but emotionally they often find it difficult or impossible to embrace the change. Handling each special case with compassion builds support for the change while insensitive handing can turn the rest of the enterprise against the change.

Resistance is not necessarily a sign of disloyalty or incompetence. Usually, it shows that the resisting individuals either don't agree with the vision or lack the ability to implement the change. The best change management programs encourage people to

discuss their concerns and never suppress dissent. After all, issues cannot be addressed if the OCM team does not know they exist.

Integrated Product Team (IPT)



© 2018 Al-Nafi. All Rights Reserved.

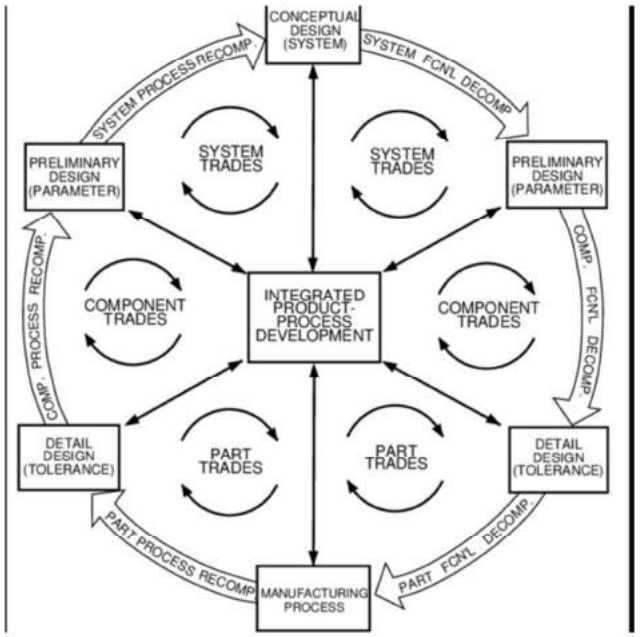
Reference https://en.wikipedia.org/wiki/Integrated_product_team

An **integrated product team** (**IPT**) is a multidisciplinary group of people who are collectively responsible for delivering a defined product or process.[1]

IPTs are used in complex development programs/projects for review and decision making. The emphasis of the IPT is on involvement of all stakeholders (users, customers, management, developers, contractors) in a collaborative forum. IPTs may be addressed at the program level, but there may also be Oversight IPTs (OIPTs), or Working-level IPTs (WIPTs).[2] IPTs are created most often as part of structured systems engineering methodologies, focusing attention on understanding the needs and desires of each stakeholder.

IPTs were introduced to the U.S. DoD in 1995 as part of the major acquisition reforms to the way goods and services were acquired.[3]

Integrated Product and Process Development (IPPD)



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.managementstudyguide.com/integrated-product-and-process-development.htm

Integrated Product and Process Development - Meaning, Advantages and Key Factors

Introduction

Objective of any organization is to provide customer satisfaction by building product and services, which not only satisfy needs and want but also create value for them. This requires product design based on the customer feedback and production process which not only minimizes cost but also provides a competitive advantage. However, most organizations tend to follow conventional production method and process.

However, in the global age of new technology and competition organization have to re-invent the way they cater to needs of customer, focus on specialization and customization is ever increasing. Given this scenario it is imperative for the organization to integrate technology and innovation within the framework of integrated product and process development.

Integrated Product and Process Development (IPPD)

Integrated product and process development combines the product design processes along with the process design process to create a new standard for producing competitive and high-quality products.

Integration of new technologies and methods provide a complete new dimension to product design process. This process starts with defining of the requirements of products based on the customer feedback while considering the design layout and other constraints. Once the finer details are finalized, they are fed into CAD models where extensive testing and modeling are done to get the best product.

With integration of production method and technology with product design, it is natural for integration of product design and process design. Therefore, integrated product and process development can be defined as a process starting from product idea to development of final product through modern technology and process management practices while minimizing cost and maximizing efficiency.

Advantages of Integrated Product and Process Development (IPPD)

Organization stands to benefit greatly from the implementation of IPPD. Some of the advantages are as follows:

Using modern technologies and implement logical steps in production design, the actual production is likely to come down, thereby reducing product delivery time.

Through optimum usage of resources and using efficient process, organizations are able to minimize cost of production thus improving profitability of the organization.

Since extensive uses of CAD model are employed chances are of product or design failure are greatly reduced thus reducing risk for organization.

As the focus is solely in delivering value to customer, quality is paramount importance and achieved through technology and methods.

Key Factors for IPPD

There are certain factors, which can vastly improve IPPD. These factors are as follows:

IPPD success is greatly dependent on agreement on the end objective which is the successful address to customer requirements. All the stakeholders and management should be aligned to the single objective.

Since this is a scientific approach, its success dependent on building up of plan, implementation of plan and constant review of the implemented plan.

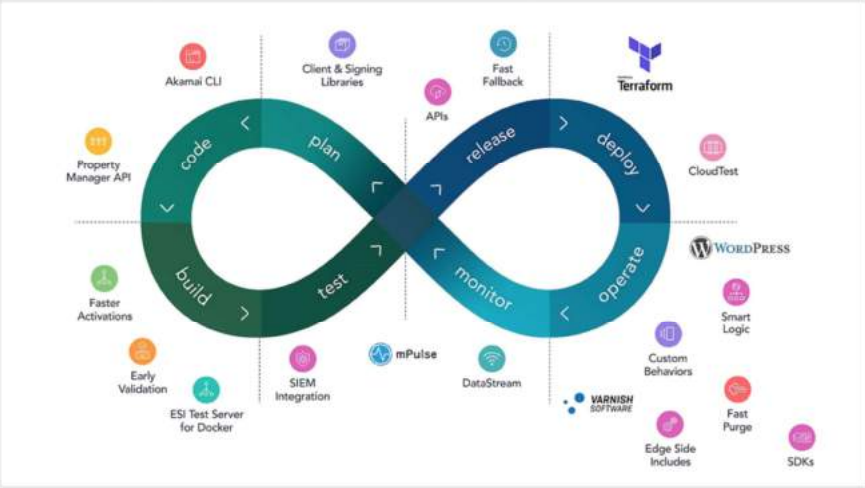
With implementation of modern methods and technology comes usage of modern tools and systems. This tools, and systems need to be integrated within the organization framework.

Skilled manpower is another essential; therefore, organization need to make investment in human capital.

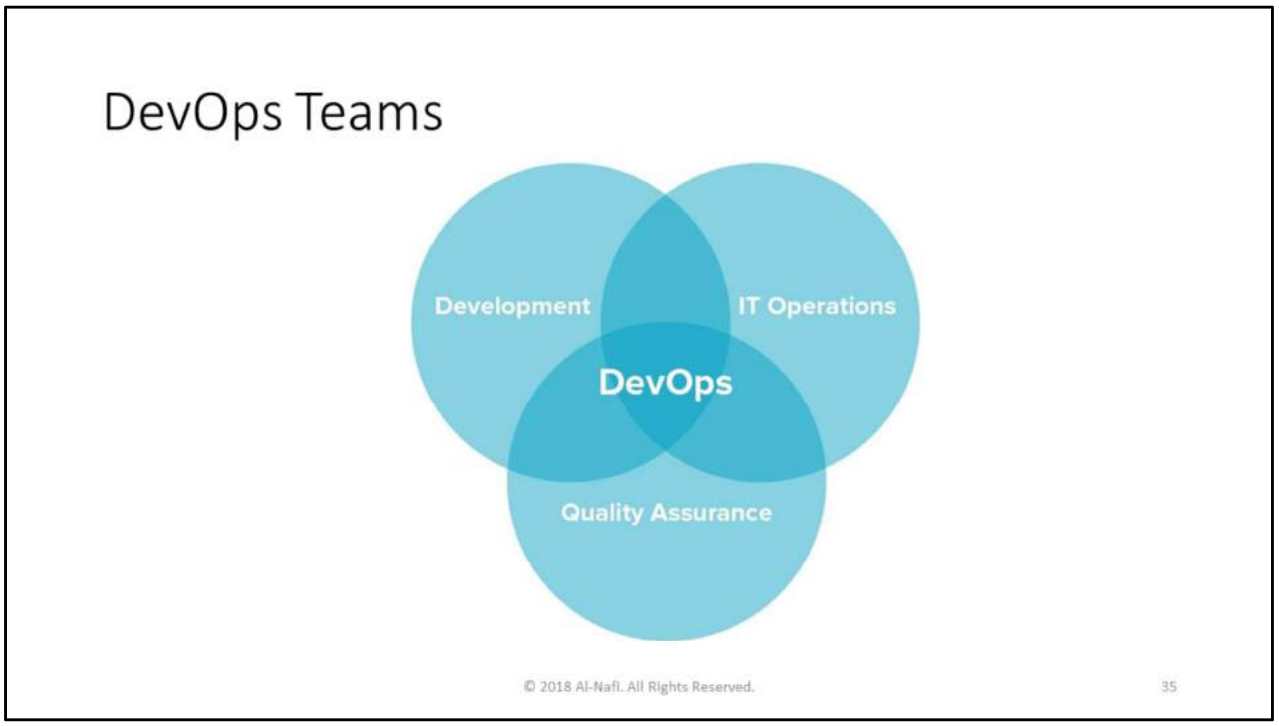
Customer is the focal point of IPPD. Therefore, constant feedback from them is essential for IPPD to be a success.

Therefore, IPPD is approach design to address all the concern of modern organization in the globalized world.

DevOps



© 2018 Al-Nafi. All Rights Reserved.



Reference https://www.atlassian.com/devops

What is DevOps?
DevOps is a set of practices that automates the processes between software development and IT teams, in order that they can build, test, and release software faster and more reliably. The concept of DevOps is founded on building a culture of collaboration between teams that historically functioned in relative siloes. The promised benefits include increased trust, faster software releases, ability to solve critical issues quickly, and better manage unplanned work.

DevOps gave us an edge
“DevOps has helped us do very frequent releases, giving us an edge on time to market. We are now able to make daily product releases as opposed to 6-month releases, and push fixes to our customers in a span of a few hours.”
— Hamesh Chawla, VP of Engineering at Zephyr

At Atlassian, DevOps is the next most famous portmanteau (combining of two

words) next to Brangelina (Brad Pitt and Angelina Jolie), bringing together the best of software development and IT operations. And like our jokes, it requires some explaining.

At its essence, DevOps is a culture, a movement, a philosophy.

It's a firm handshake between development and operations that emphasizes a shift in mindset, better collaboration, and tighter integration. It unites agile, continuous delivery, automation, and much more, to help development and operations teams be more efficient, innovate faster, and deliver higher value to businesses and customers.

Who's doing DevOps?

Chef is the company behind the Chef Automate platform for DevOps workflows. Tens of thousands of developers use Chef to test, automate, and manage infrastructure. At the forefront of the DevOps evolution, the Seattle-based company has been releasing products like Chef, InSpec, Habitat, and Chef Automate to advance new ways of developing and shipping software and applications. To experiment with and refine its own DevOps practices, Chef relies on the Atlassian platform.

Learn more

History of DevOps

The DevOps movement started to coalesce some time between 2007 and 2008, when IT operations and software development communities got vocal about what they felt was a fatal level of dysfunction in the industry.

They railed against the traditional software development model, which called for those who write the code to be organizationally and functionally apart from those who deploy and support that code.

Developers and IT/Ops professionals had separate (and often competing) objectives, separate department leadership, separate key performance indicators by which they were judged, and often worked on separate floors or even separate buildings. The result was siloed teams concerned only with their own fiefdoms, long hours, botched releases, and unhappy customers.

Surely there’s a better way, they said. So the two communities got together and started talking – with people like Patrick Dubois, Gene Kim, and John Willis driving the conversation.

What began in online forums and local meet-ups is now a major theme in the software zeitgeist, which is probably what brought you here! You and your team are feeling the pain caused by siloed teams and broken lines of communication within your company.

You’re using agile methodologies for planning and development, but still struggling to get that code out the door without a bunch of drama. You’ve heard a few things about DevOps and the seemingly magical effect it can have on teams and think “I want some of that magic.”

The bad news is that DevOps isn’t magic, and transformations don’t happen

overnight. The good news is that you don’t have to wait for upper management to roll out a large-scale initiative. By understanding the value of DevOps and making small, incremental changes, your team can embark on the DevOps journey right away. Let’s look at each of these benefits in detail.

Infrastructure as code allowed us to perform 10x more builds without adding a single person to our team.— Michael Knight, Build Engineer at Atlassian

What's in it for you?

Collaboration and trust

Culture is the #1 success factor in DevOps. Building a culture of shared responsibility, transparency and faster feedback is the foundation of every high performing DevOps team.

Teams that work in siloes often don't adhere to the 'systems thinking' of DevOps. 'Systems thinking' is being aware of how your actions not only affect your team, but all the other teams involved in the release process. Lack of visibility and shared goals means lack of dependency planning, misaligned priorities, finger pointing, and 'not our problem' mentality, resulting in slower velocity and substandard quality. DevOps is that change in mindset of looking at the development process holistically and breaking down the barrier between Dev and Ops.

Release faster and work smarter

Speed is everything. Teams that practice DevOps release more frequently, with higher quality and stability.

Lack of automated test and review cycles block the release to production and poor incident response time kills velocity and team confidence. Disparate tools and processes increase OPEX, lead to context switching, and slow down momentum. Through automation and standardized tools and processes, teams can increase productivity and release more frequently with fewer hiccups.

Accelerate time to resolution

The team with the fastest feedback loop is the team that thrives. Full transparency and seamless communication enable DevOps teams to minimize downtime and resolve issues faster than ever before.

If critical issues aren't resolved quickly, customer satisfaction tanks. Key issues slip through the cracks in the absence of open communication, resulting in increased tension and frustration among teams. Open communication helps Dev and Ops teams swarm on issues, fix incidents, and unblock the release pipeline faster.

Better manage unplanned work

Unplanned work is a reality that every team faces–a reality that most often impacts team productivity. With established processes and clear prioritization, the Dev and Ops teams can better manage unplanned work while continuing to focus on planned work.

Transitioning and prioritizing unplanned work across different teams and systems is inefficient and distracts from work at hand. However, through raised visibility and

proactive retrospection, teams can better anticipate and share unplanned work.

The CALMS Framework for DevOps

Culture

Automation

Lean

Measurement

Sharing

Culture

If we could sum up DevOps culture in one word, it'd be "collaboration" – and if we were allowed two words, they'd be "cross-functional collaboration." (Ok, that's more like three words.)

All the tooling and automation in the world are useless if they aren't accompanied by a genuine desire on the part of development and IT/Ops professionals to work together. Because DevOps doesn't solve tooling problems. It solves human problems. Therefore, it's unlikely you'll poke your head out of the cubicle one day, look around, and discover that teams at your company embody DevOps culture. But there are simple things you can do to nurture it.

Think of DevOps much like agile, but with the operations included. Forming project- or product-oriented teams to replace function-based teams is a step in the right direction. Include development, QA, product management, design, operations, project management, and any other skill set the project requires. At Atlassian, we even embed marketing with our product teams.

Few things foster collaboration like sharing a common goal and having a plan to reach it together. At some companies, switching suddenly to project-based teams is too much, too soon. So take smaller steps. Development teams can – and should – invite appropriate members of the operations team to join sprint planning sessions, daily stand-ups, and sprint demos. Operations teams can invite key developers. It's an agile and organic way to keep on the pulse of each other's projects, ideas, and struggles. The time spent listening and cross-pollinating subject-area knowledge pays for itself by making release management and emergency troubleshooting far more efficient.

And speaking of emergencies, they're an effective test of DevOps culture. Do developers, operations, and customer support swarm on the problem and resolve it as a team? Does everyone start with the assumption that their teammates made the best decisions possible with the information and resources they had at the time? Is the incident post-mortem about fixing processes instead of pointing fingers? If the answer is "yes," that's a good indication that your team functions with DevOps culture at its core.

Note that the most successful companies are on board with DevOps culture across every department, and at all levels of the org chart. They have open channels of communication, and talk regularly. They make sure everyone's goals are aligned, and adjust as needed. They assume that keeping customers happy is just as much product

management's responsibility as it is the development team's responsibility. They understand that DevOps isn't one team's job. It's everyone's job.

Teams that practice DevOps deploy 30x more frequently, have 60x fewer failures, and recover 160x faster.— Puppet Labs 2016 State of DevOps Report

Automation

Investing in automation eliminates repetitive manual work, yields repeatable processes, and creates reliable systems.

Build, test, deploy, and provisioning automation are typical starting points for teams who don't have them in place already. And hey: what better reason for developers, testers, and operators to work together than building systems to benefit everyone?

Teams new to automation usually start with continuous delivery: the practice of running each code change through a gauntlet of automated tests, often facilitated by cloud-based infrastructure, then packaging up successful builds and promoting them up toward production using automated deploys. As you might guess, continuous delivery is not a quick and easy thing to set up, but the return on investment is well worth it.

Why? Computers execute tests more rigorously and faithfully than humans. These tests catch bugs and security flaws sooner, allowing developers to fix them more easily. And the automated deploys alert IT/Ops to server "drift" between environments, which reduces or eliminates surprises when it's time to release.

Another of DevOps' major contributions is the idea of "configuration as code." Developers strive to create modular, composable applications because they are more reliable and maintainable. That same thinking can be extended to the infrastructure that hosts them, whether it lives in the cloud or on the company's own network. True, systems are always changing. But we can create a facade of immutability by using code for provisioning so that re-provisioning a compromised server becomes faster than repairing it – not to mention more reliable. It reduces risk, too. Both development and operations can incorporate new languages or technologies via the provisioning code, and share the updates with each other. Compatibility issues become immediately apparent, instead of manifesting in the middle of a release.

"Configuration as code" and "continuous delivery" aren't the only types of automation seen in the DevOps world, but they're worth special mention because they help break down the wall between development and operations. And when DevOps uses automated deploys to send thoroughly tested code to identically provisioned environments, "Works on my machine!" becomes irrelevant.

Lean

When we hear "lean" in the context of software, we usually think about eliminating low-value activities and moving quickly – being scrappy, being agile. Even more relevant for DevOps are the concepts of continuous improvement and embracing failure.

A DevOps mindset sees opportunities for continuous improvement everywhere.

Some are obvious, like holding regular retrospectives so your team's processes can improve. Others are subtle, like A/B testing different on-boarding approaches for new users of your product.

We have agile development to thank for making continuous improvement a mainstream idea. Early adopters of the agile methodology proved that a simple product in the hands of customers today is more valuable than a perfect product in the hands of customers six months from now. If the product is improved continuously, customers will stick around.

And guess what: failure is inevitable. So you might as well set up your team to absorb it, recover, and learn from it (some call this "being anti-fragile"). At Atlassian, we believe that if you're not failing once in a while, you're not trying hard enough.

We challenge our teams with big, hairy, audacious goals and make sure they have the autonomy and the resources to meet them. We hire smart, ambitious people and expect them to fail sometimes.

In the context of DevOps, failure is not a punishable offense. Teams assume that things are bound to go pear-shaped at some point, so they build for fast detection and rapid recovery. (Read up on Nexflix's Chaos Monkey for an excellent example.)

Postmortems focus on where processes fell down and how to strengthen them – not on which team member f'ed up the code. Why? Because continuous improvement and failure go hand in hand.

DevOps has evolved so that development owns more operations – and that's how Chef works. We can't just throw it over the wall anymore. Our engineers are responsible for QA, writing, and running their own tests to get the software out to customers.— Julian Dunn, Product Manager at Chef

Measurement

It's hard to prove your continuous improvement efforts are actually improving anything without data. Fortunately, there are loads of tools and technologies for measuring performance like how much time users spend in your product, whether that blog post generated any sales, or how often critical alerts pop up in your logs.

Although you can measure just about anything, that doesn't mean you have to (or should) measure everything. Take a page from agile development and start with the basics:

How long did it take to go from development to deployment?

How often do recurring bugs or failures happen?

How long does it take to recover after a system failure?

How many people are using your product right now?

How many users did you gain / lose this week?

With a solid foundation in place, it's easier to capture more sophisticated metrics around feature usage, customer journeys, and service level agreements (SLAs). The information you get comes in handy when it's time for road mapping and spec'ing out your next big move.

All this juicy data will help your team make decisions, but it’s even more powerful when shared with other teams – especially teams in other departments. For example, your marketing team wants shiny new features they can sell. But meanwhile, you’re seeing high customer churn because the product is awash in technical debt. Providing user data that supports your roadmap – even if it’s light on features and heavy on fixes - makes it easier to build consensus and get buy in from stakeholders.

Devops isn't any single person's job. It's everyone's job.— Christophe Capel, Principal Product Manager, Jira Service Desk

Sharing

The long-standing friction between development and operations teams is largely due to a lack of common ground. We believe that sharing responsibility and success will go a long way toward bridging that divide. Developers can win instant goodwill by helping to carry one of operations’ biggest burdens: the pager. DevOps is big on the idea that the same people who build an application should be involved in shipping and running it.

This doesn’t mean that you hire developers and simply expect them to be excellent operators as well. It means that developers and operators pair with each other in each phase of the application’s lifecycle.

Teams that embrace DevOps often have a rotating role whereby developers address issues caught by end users while, at the same, troubleshooting production problems. This person responds to urgent customer-reported issues, creating patches when necessary, and works through the backlog of customer-reported defects. The “developer on support” learns a lot about how the application is used in the wild. And by being highly available to the operations team, the development teams builds trust and mutual respect.

Slogging through the rough patches together makes celebrating successes all the more sweet. You’ll know DevOps culture has taken hold at your company when you see the development team bring bagels for the operations team on release day. Positive feedback from peers motivates us as much as our paychecks and career ambitions. Publicly recognizing a teammate who detected a nasty bug before it went live means a lot. If your department has discretionary budget for employee kudos, don’t let it go unused!

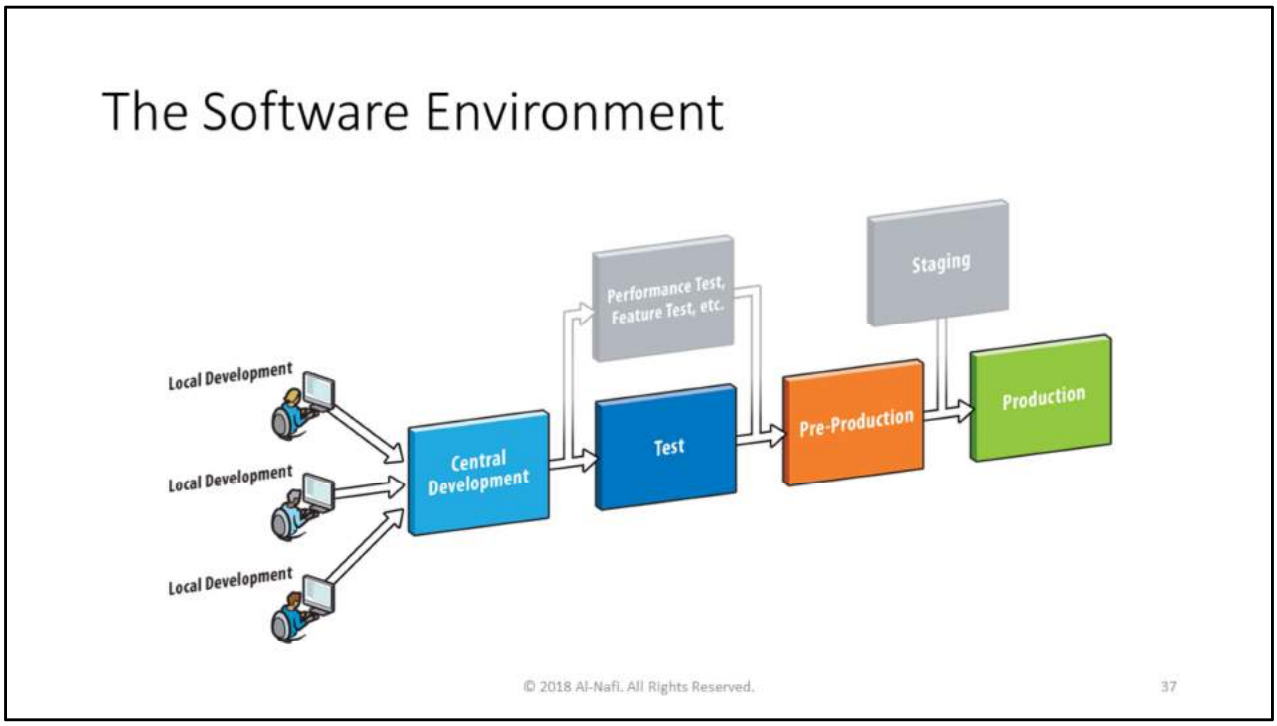
Secure Coding Guidelines and Standards



© 2018 Al-Nafi. All Rights Reserved.

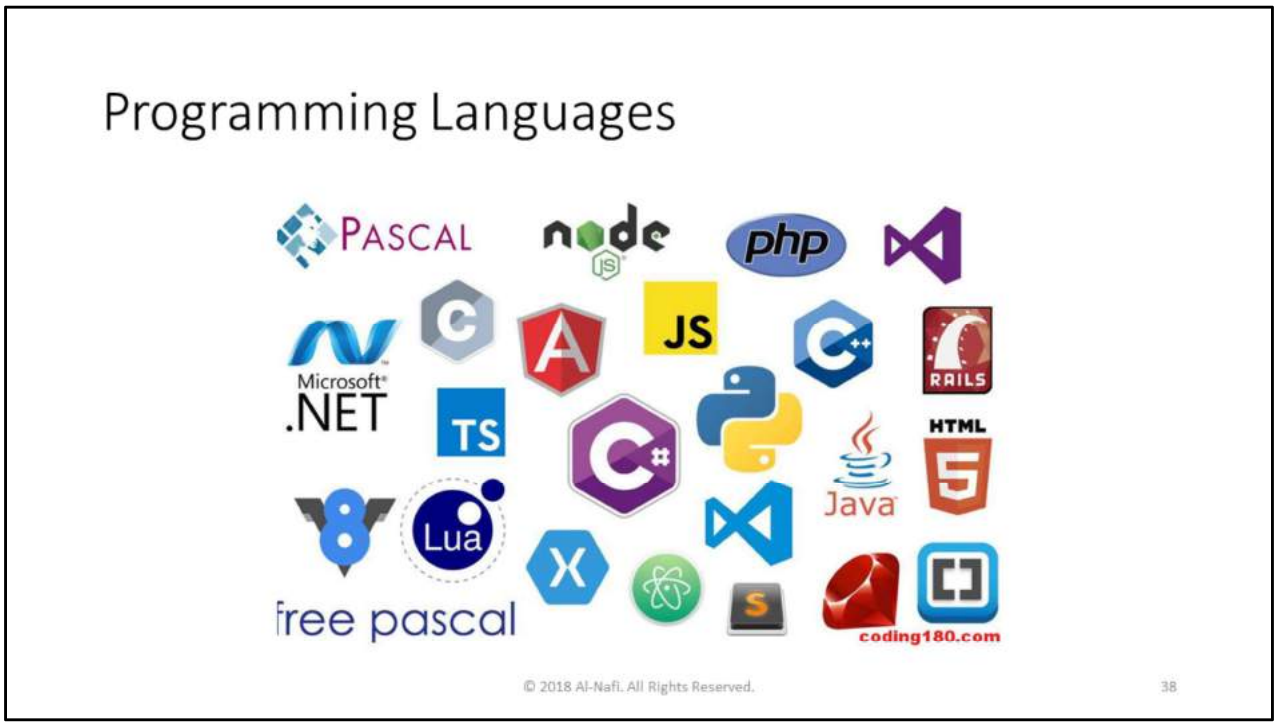
Reference https://en.wikipedia.org/wiki/Secure_coding

Secure coding is the practice of developing computer software in a way that guards against the accidental introduction of security vulnerabilities. Defects, bugs and logic flaws are consistently the primary cause of commonly exploited software vulnerabilities.[1] Through the analysis of thousands of reported vulnerabilities, security professionals have discovered that most vulnerabilities stem from a relatively small number of common software programming errors. By identifying the insecure coding practices that lead to these errors and educating developers on secure alternatives, organizations can take proactive steps to help significantly reduce or eliminate vulnerabilities in software before deployment.



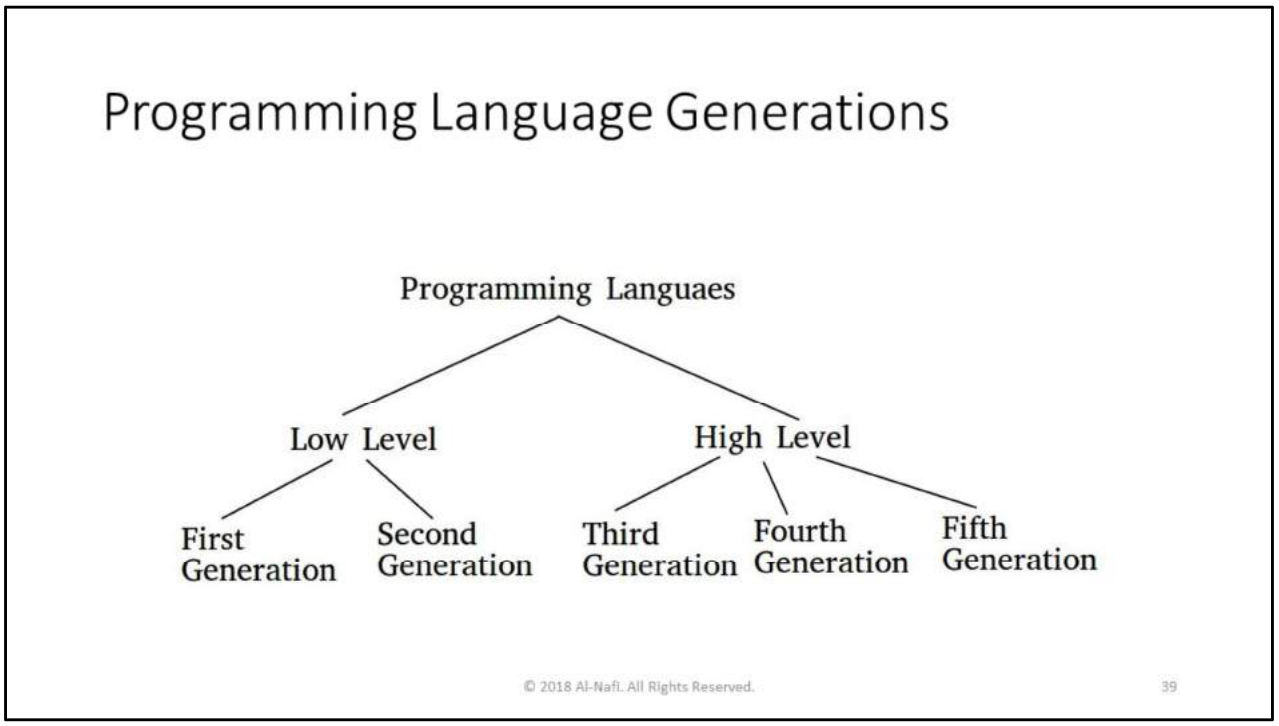
Today's architectures rely heavily on software and applications. The architecture itself includes hardware resources, including the typical components such as the central processing unit (CPU), memory, input/output processing, and storage. The operating system, which is fundamental to any technology architecture, is responsible for controlling not only the hardware resources, but also in providing security mechanisms to protect them, as well as providing resource access permissions and safeguards against misuse. Applications are used by the architecture to allow the interaction and interface to the users. Applications today provide much more functionality than ever, also making them very easy to exploit by attackers through vulnerabilities that may exist in the functionality provided. Security controls, therefore, need to be designed and built into the software to allow the users more control over the functionality, but at the same time, protect against exploits and vulnerabilities, and ultimately to protect the value of the information being processed through the application. There are many vulnerabilities and exploits that can be introduced in the application, such as when a buffer overflow attack takes advantage of improper parameter checking within the application.

Another such example might be inadequate data validation that can lead to all kinds of escalation of privileges and other exploits. Today's software environments are also distributed, meaning they are
connected to many other environments, architectures, networks, etc. Distributed applications provide a particular challenge in terms of adequate security due to the complexity of the information being
passed by components in the distributed architectures. The architectures that software is part of today are complex and ever changing. The functionality that software provides today is much more complex as well, and so protecting from a security perspective is also very challenging. Protecting the application itself and the environment that it run in begins with designing security into the functionality of the application that is written in some sort of programming language.



During development phases, developers need to write code in some sort of programming language. There are many programming languages that have been developed over the years. A programming
language is a set of instructions that tell the computer what operations to perform. Programming languages have evolved in generations, and each language is characterized into one of the typical generations characterized below. Those in the earlier classification level are closer in form to the binary language of the computer. Both machine and assembly languages are considered low-level languages. As programming languages have evolved, they have become easier and more similar to the language people use to communicate. In other words, they have become higher level languages. High-level languages are easier to use by developers than low-level languages and in some cases, can be used to produce programs more quickly and more efficiently. In addition, high-level languages are considered to be more beneficial because they enforce coding standards and development methods that can enforce a better level of more security. On the other hand, higher-level languages can also work against proper security as they can automate certain functions and provide complicated functionality for the application, implemented by the programming environment or tool, the internal details of which may be

poorly understood by the designers and developers. As a result, it may be possible that high-level languages may introduce possibilities of security vulnerabilities in ways that may not be apparent to the designers, developer, and security professionals.



Reference https://www.includehelp.com/basics/generations-of-programming-language.aspx

Generations of programming language

Programming languages have been developed over the year in a phased manner. Each phase of developed has made the programming language more user-friendly, easier to use and more powerful. Each phase of improved made in the development of the programming languages can be referred to as a generation. The programming language in terms of their performance reliability and robustness can be grouped into five **different generations**,

First generation languages (1GL)
Second generation languages (2GL)
Third generation languages (3GL)
Fourth generation languages (4GL)
Fifth generation languages (5GL)

1. First Generation Language (Machine language)

The first generation programming language is also called low-level programming language because they were used to program the computer system at a very low

level of abstraction. i.e. at the machine level. The machine language also referred to as the native language of the computer system is the first generation programming language. In the machine language, a programmer only deals with a binary number.

Advantages of first generation language

They are translation free and can be directly executed by the computers.

The programs written in these languages are executed very speedily and efficiently by the CPU of the computer system.

The programs written in these languages utilize the memory in an efficient manner because it is possible to keep track of each bit of data.

2. Second Generation language (Assembly Language)

The second generation programming language also belongs to the category of low-level- programming language. The second generation language comprises assembly languages that use the concept of mnemonics for the writing program. In the assembly language, symbolic names are used to represent the opcode and the operand part of the instruction.

Advantages of second generation language

It is easy to develop understand and modify the program developed in these languages are compared to those developed in the first generation programming language.

The programs written in these languages are less prone to errors and therefore can be maintained with a great case.

3. Third Generation languages (High-Level Languages)

The third generation programming languages were designed to overcome the various limitations of the first and second generation programming languages. The languages of the third and later generation are considered as a high-level language because they enable the programmer to concentrate only on the logic of the programs without considering the internal architecture of the computer system.

Advantages of third generation programming language

It is easy to develop, learn and understand the program.

As the program written in these languages are less prone to errors they are easy to maintain.

The program written in these languages can be developed in very less time as compared to the first and second generation language.

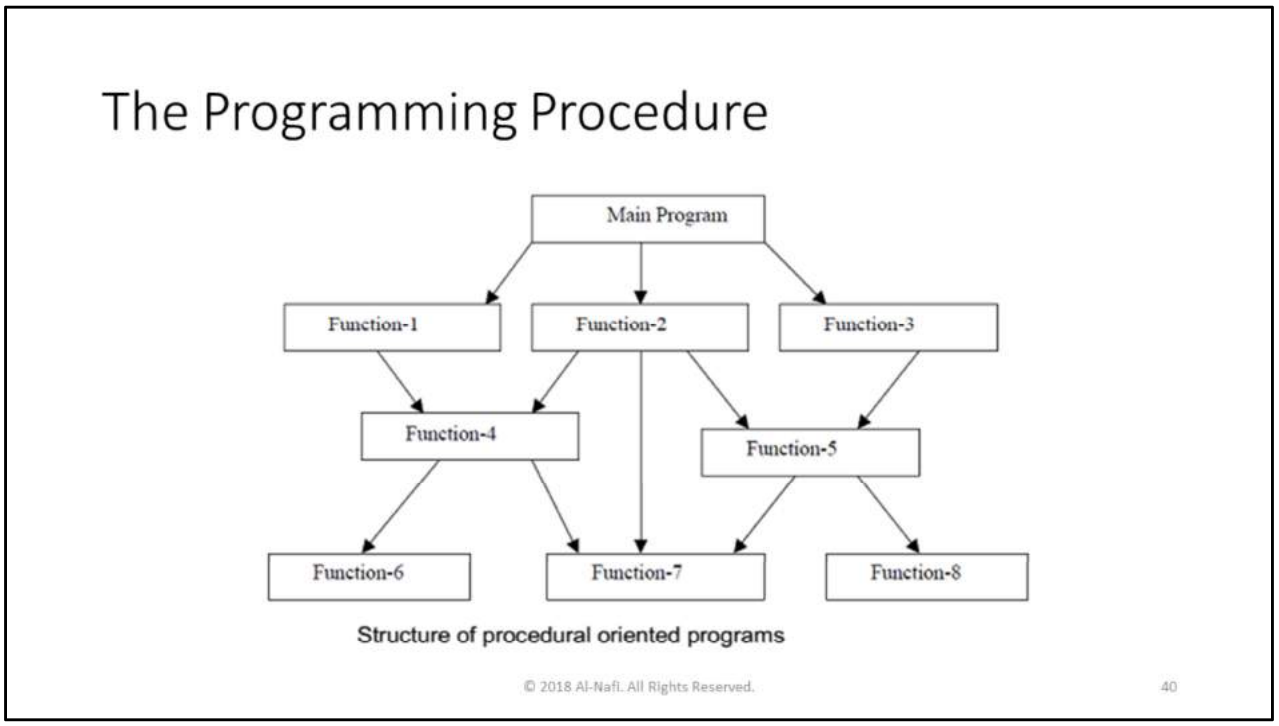
Examples: FORTRAN, ALGOL, COBOL, C++, C

4. Fourth generation language (Very High-level Languages)

The languages of this generation were considered as very high-level programming languages required a lot of time and effort that affected the productivity of a programmer. The fourth generation programming languages were designed and developed to reduce the time, cost and effort needed to develop different types of software applications.

Advantages of fourth generation languages

These programming languages allow the efficient use of data by implementing the various database.
They require less time, cost and effort to develop different types of software applications.
The program developed in these languages are highly portable as compared to the programs developed in the languages of other generation.
Examples: SOL, CSS, coldfusion
5. Fifth generation language (Artificial Intelligence Language)
The programming languages of this generation mainly focus on constraint programming. The major fields in which the fifth generation programming language are employed are Artificial Intelligence and Artificial Neural Networks
Advantages of fifth generation languages
These languages can be used to query the database in a fast and efficient manner.
In this generation of language, the user can communicate with the computer system in a simple and an easy manner.
Examples: mercury, prolog, OPS5



Reference

Procedural programming is a programming paradigm, derived from structured programming,[*citation needed*] based on the concept of the *procedure call*. Procedures, also known as routines, subroutines, or functions, simply contain a series of computational steps to be carried out. Any given procedure might be called at any point during a program's execution, including by other procedures or itself. The first major procedural programming languages appeared circa 1957–1964, including Fortran, ALGOL, COBOL, PL/I and BASIC.[1] Pascal and C were published circa 1970–1972.
Computer processors provide hardware support for procedural programming through a stack register and instructions for calling procedures and returning from them. Hardware support for other types of programming is possible, but no attempt was commercially successful (for example Lisp machines or Java processors).[*contradictory*]

Procedures and modularity[edit]
Main article: Modular programming

Modularity is generally desirable, especially in large, complicated programs. Inputs are usually specified syntactically in the form of *arguments* and the outputs delivered as *return values*.
Scoping is another technique that helps keep procedures modular. It prevents the procedure from accessing the variables of other procedures (and vice versa), including previous instances of itself, without explicit authorization.
Less modular procedures, often used in small or quickly written programs, tend to interact with a large number of variables in the execution environment, which other procedures might also modify.
Because of the ability to specify a simple interface, to be self-contained, and to be reused, procedures are a convenient vehicle for making pieces of code written by different people or different groups, including through programming libraries.
Comparison with other programming paradigms[edit]
Imperative programming[edit]
Procedural programming languages are also imperative languages, because they make explicit references to the state of the execution environment. This could be anything from *variables* (which may correspond to processor registers) to something like the position of the "turtle" in the Logo programming language.
Often, the terms "procedural programming" and "imperative programming" are used synonymously. However, procedural programming relies heavily on blocks and scope, whereas imperative programming as a whole may or may not have such features. As such, procedural languages generally use reserved words that act on blocks, such as if, while, and for, to implement control flow, whereas non-structured imperative languages use goto statements and branch tables for the same purpose.
Object-oriented programming[edit]
The focus of procedural programming is to break down a programming task into a collection of variables, data structures, and subroutines, whereas in object-oriented programming it is to break down a programming task into objects that expose behavior (methods) and data (members or attributes) using interfaces. The most important distinction is that while procedural programming uses procedures to operate on data structures, object-oriented programming bundles the two together, so an "object", which is an instance of a class, operates on its "own" data structure.[2]
Nomenclature varies between the two, although they have similar semantics:

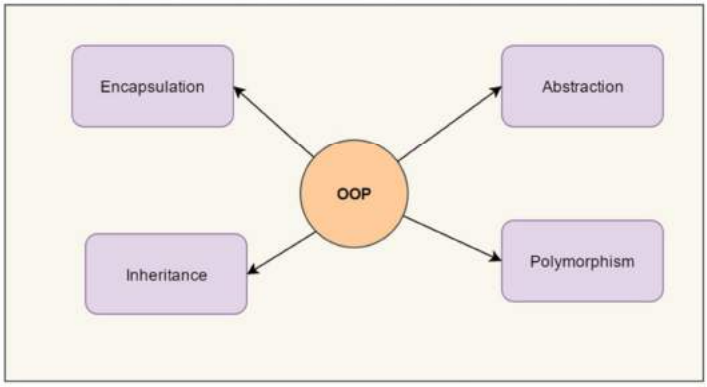
Functional programming[edit]
The principles of modularity and code reuse in practical functional languages are fundamentally the same as in procedural languages, since they both stem from structured programming. So for example:
Procedures correspond to functions. Both allow the reuse of the same code in various parts of the programs, and at various points of its execution.
By the same token, procedure calls correspond to function application.

Functions and their invocations are modularly separated from each other in the same manner, by the use of function arguments, return values and variable scopes.
The main difference between the styles is that functional programming languages remove or at least deemphasize the imperative elements of procedural programming. The feature set of functional languages is therefore designed to support writing programs as much as possible in terms of pure functions:
Whereas procedural languages model execution of the program as a sequence of imperative commands that may implicitly alter shared state, functional programming languages model execution as the evaluation of complex expressions that only depend on each other in terms of arguments and return values. For this reason, functional programs can have a free order of code execution, and the languages may offer little control over the order in which various parts of the program are executed. (For example, the arguments to a procedure invocation in Scheme are executed in an arbitrary order.)
Functional programming languages support (and heavily use) first-class functions, anonymous functions and closures, although these concepts are being included in newer procedural languages.
Functional programming languages tend to rely on tail call optimization and higher-order functions instead of imperative looping constructs.
Many functional languages, however, are in fact impurely functional and offer imperative/procedural constructs that allow the programmer to write programs in procedural style, or in a combination of both styles. It is common for input/output code in functional languages to be written in a procedural style.
There do exist a few esoteric functional languages (like Unlambda) that eschew structured programming precepts for the sake of being difficult to program in (and therefore challenging). These languages are the exception to the common ground between procedural and functional languages.

Logic programming[edit]

In logic programming, a program is a set of premises, and computation is performed by attempting to prove candidate theorems. From this point of view, logic programs are declarative, focusing on what the problem is, rather than on how to solve it.
However, the backward reasoning technique, implemented by SLD resolution, used to solve problems in logic programming languages such as Prolog, treats programs as goal-reduction procedures. Thus clauses of the form:
H :- B_1, …, B_n.have a dual interpretation, both as procedures
to show/solve H, show/solve B_1 and … and $B_n$and as logical implications:
B_1 and … and B_n implies H.Experienced logic programmers use the procedural interpretation to write programs that are effective and efficient, and they use the declarative interpretation to help ensure that programs are correct.

Object-Oriented Technology and Programming



Four Pillars of Object Oriented Programming

© 2018 Al-Nafi. All Rights Reserved.

Reference https://medium.com/@cancerian0684/what-are-four-basic-principles-of-object-oriented-programming-645af8b43727

There are 4 major principles that make an language Object Oriented. These are Encapsulation, Data Abstraction, Polymorphism and Inheritance. These are also called as four pillars of Object Oriented Programming.

Encapsulation

Encapsulation is the mechanism of hiding of data implementation by restricting access to public methods. Instance variables are kept private and accessor methods are made public to achieve this.

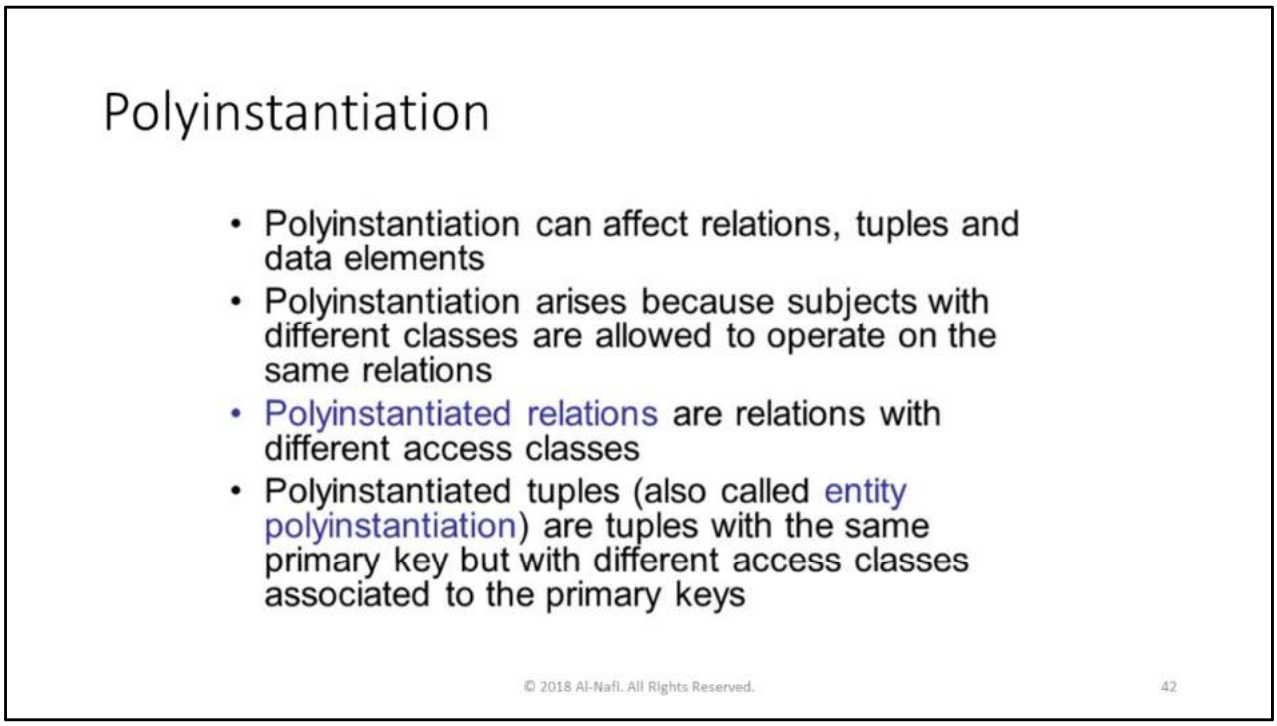
For example, we are hiding the name and dob attributes of person class in the below code snippet.

Encapsulation — private instance variable and public accessor methods.

```
public class Employee {
private String name;
private Date dob; public String getName() {
return name;
} public void setName(String name) {
```

this.name = name;
} **public Date** getDob() {
return dob;
} **public void** setDob(**Date** dob) {
this.dob = dob;
}
}**Abstraction**
Abstract means a concept or an Idea which is not associated with any particular instance. Using abstract class/Interface we express the intent of the class rather than the actual implementation. In a way, one class should not know the inner details of another in order to use it, just knowing the interfaces should be good enough.
Inheritance
Inheritances expresses “is-a” and/or “has-a” relationship between two objects. Using Inheritance, In derived classes we can reuse the code of existing super classes. In Java, concept of “is-a” is based on class inheritance (using extends) or interface implementation (using implements).
For example, FileInputStream "is-a" InputStream that reads from a file.
Polymorphism
It means one name many forms. It is further of two types — static and dynamic. Static polymorphism is achieved using method overloading and dynamic polymorphism using method overriding. It is closely related to inheritance. We can write a code that works on the superclass, and it will work with any subclass type as well.
Example
Java collections framework has an interface
called java.util.Collection, ArrayList and TreeSet are two different implementation of this interface. ArrayList maintains the insertion order of elements
while TreeSet orders its elements by their natural order or comparator(if supplied).
Now if we write a method that accepts a collection and prints its elements, the actual object (ArrayList or TreeSet) at runtime will decide the behavior of this method.
Polymorphic print method
public void print(**Collection**<**String**> collection) {
for (**String** s : collection) {
System.out.println("s = " + s);
}
}Passing an ArrayList
Collection<**String**> collection1 = **new ArrayList**<>();
collection1.add("A");
collection1.add("D");
collection1.add("B");
collection1.add("C");

print(collection1); **(1)**elements will be printed as per the insertion order of elements into arraylist
Program output
s = A
s = D
s = B
s = CPassing an TreeSet
Collection<**String**> collection2 = **new TreeSet**<>();
collection2.add("A");
collection2.add("D");
collection2.add("B");
collection2.add("C");
print(collection2); **(1)**elements will be printed as per the natural order
Program output
s = A
s = B
s = C
s = DWe just saw that print() method's behavior is determined by the actual type of object passed to it at run time. That's polymorphism!
Important Facts
Other than objects of type java.lang.Object, all java objects are polymorphic i.e. they pass the IS-A test for their own type as well as for class Object.
A reference variable's type determines the methods that can be invoked on the object that variable is referencing to. In the example above, print() method can only invoke methods that are listed on Collection interface irrespective the type of actual object passed to this method.
Polymorphic method invocation applies only to the instance methods (not to static methods, not to variables). Only overriden instance methods are dynamically invoked based on the real object's type at runtime.



Reference

Polyinstantiation in computer science is the concept of type (class, database row or otherwise) being instantiated into multiple independent instances (objects, copies). It may also indicate, such as in the case of database polyinstantiation, that two different instances have the same name (identifier, primary key).

Operating system security[edit]
In Operating system security, polyinstantiation is the concept of creating a user or process specific view of a shared resource. I.e. Process **A** cannot affect process **B** by writing malicious code to a shared resource, such as UNIX directory **/tmp**.[1][2] Polyinstantiation of shared resources have similar goals as process isolation, an application of virtual memory, where processes are assigned their own isolated virtual address space to prevent process **A** writing into the memory space of process **B**.

Database[edit]

In databases, polyinstantiation is database-related SQL (structured query language) terminology. It allows a relation to contain multiple rows with the same primary key; the multiple instances are distinguished by their security levels.[3] It occurs because of mandatory policy. Depending on the security level established, one record contains sensitive information, and the other one does not, that is, a user will see the record's information depending on his/her level of confidentiality previously dictated by the company's policy[4]

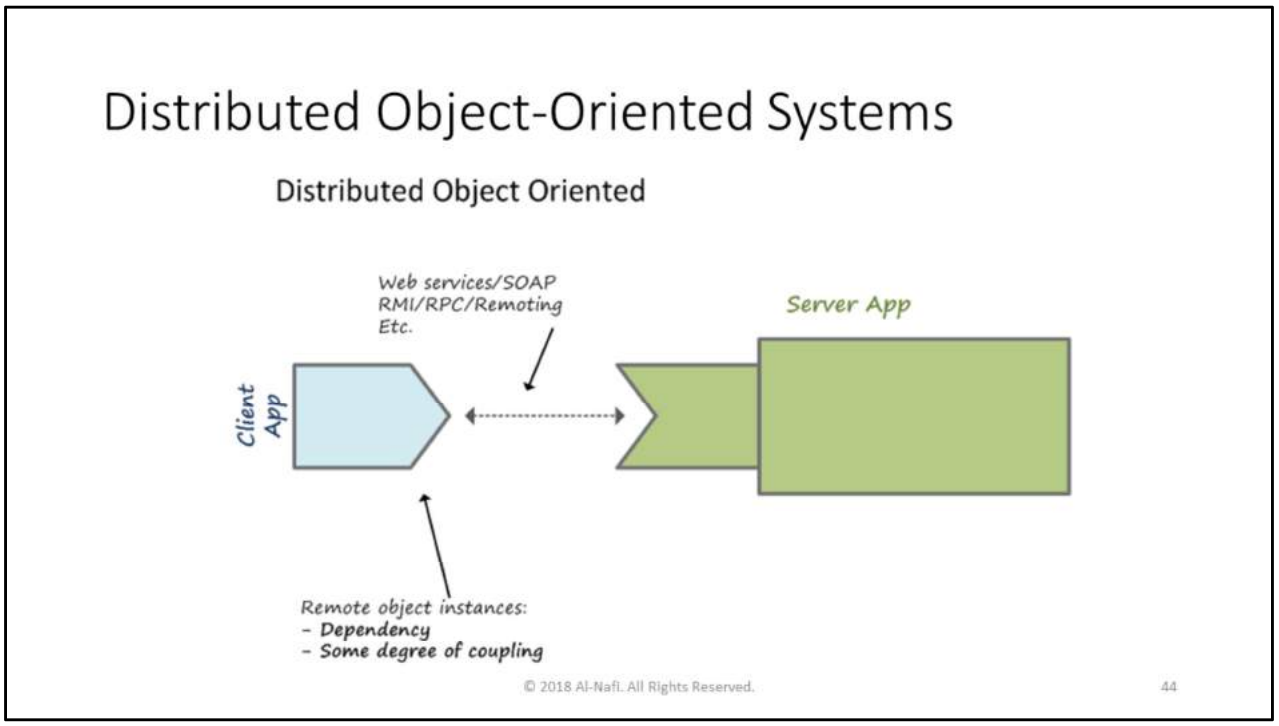
Cryptography[edit]
In cryptography, polyinstantiation is the existence of a cryptographic key in more than one secure physical location.

Object-Oriented Security



© 2018 Al-Nafi. All Rights Reserved.

in object-oriented systems, objects are encapsulated. Encapsulation protects the object by denying direct access to view or interact with what is located inside the object, this is referred to as data hiding. It is not possible to see what is contained in the object because it is encapsulated. Encapsulation can be used to protect the object, since it does not allow any other object to see data
from outside. This makes sense from a security perspective because no object should be able to access or see another object's data.

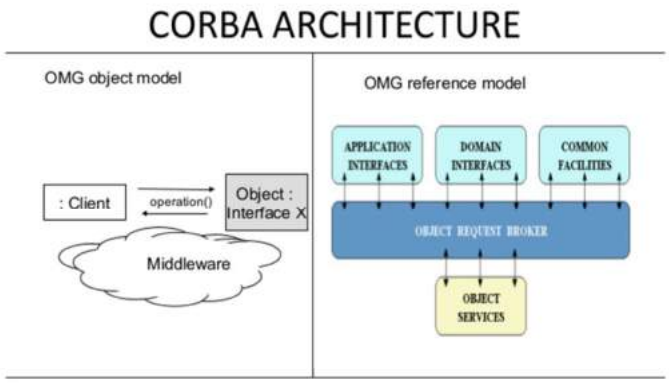


The trend in computing over the last number of decades has been to move toward a new age of distributed computing. Distributed computing allows the sharing of resources. The same concept of distributed environments can be applied in software development. Distributed development architectures allow applications to be divided into logical objects that are called components, and each component can exist in different locations. The components can then communicate with each other, and programs can call the components as required. This development architecture allows applications to download code from remote machines onto a user's local host in a manner that is seamless to the user. Applications today can be built using this distributed architecture constructed with software systems that are based on distributed objects. Examples may include Common Object
Request Broker Architecture (CORBA), Java Remote Method Invocation (JRMI), Enterprise JavaBean (EJB), and Distributed Component Object Model (DCOM).

A distributed object-oriented system allows parts of the system to be located on separate computers within a network. The object system itself is a compilation of reusable self-contained objects of code

designed to perform specific business functions. Objects can communicate with each other, even though they may reside on different machines across the network. To standardize this process, the Object Management Group (OMG) created a standard for finding objects, initiating objects, and sending requests to the objects. The standard is called the Object Request Broker (ORB), which is part of the Common Object Request Broker Architecture (CORBA) mentioned above.

Common Object Request Broker Architecture (CORBA)



Common Object Request Broker Architecture

© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Common_Object_Request_Broker_Architecture

The **Common Object Request Broker Architecture** (**CORBA**) is a standard defined by the Object Management Group (OMG) designed to facilitate the communication of systems that are deployed on diverse platforms. CORBA enables collaboration between systems on different operating systems, programming languages, and computing hardware. CORBA uses an object-oriented model although the systems that use the CORBA do not have to be object-oriented. CORBA is an example of the distributed object paradigm.

CORBA enables communication between software written in different languages and running on different computers. Implementation details from specific operating systems, programming languages, and hardware platforms are all removed from the responsibility of developers who use CORBA. CORBA normalizes the method-call semantics between application objects residing either in the same address-space (application) or in remote address-spaces (same host, or remote host on a network). Version 1.0 was released in October 1991.
CORBA uses an interface definition language (IDL) to specify the interfaces that

objects present to the outer world. CORBA then specifies a *mapping* from IDL to a specific implementation language like C++ or Java. Standard mappings exist for Ada, C, C++, C++11, COBOL, Java, Lisp, PL/I, Object Pascal, Python, Ruby and Smalltalk. Non-standard mappings exist for C#, Erlang, Perl, Tcl and Visual Basic implemented by object request brokers (ORBs) written for those languages.

The CORBA specification dictates there shall be an ORB through which an application would interact with other objects. This is how it is implemented in practice:

The application simply initializes the ORB, and accesses an internal *Object Adapter*, which maintains things like reference counting, object (and reference) instantiation policies, and object lifetime policies.

The Object Adapter is used to register instances of the *generated code classes*. Generated code classes are the result of compiling the user IDL code, which translates the high-level interface definition into an OS- and language-specific class base for use by the user application. This step is necessary in order to enforce CORBA semantics and provide a clean user process for interfacing with the CORBA infrastructure.

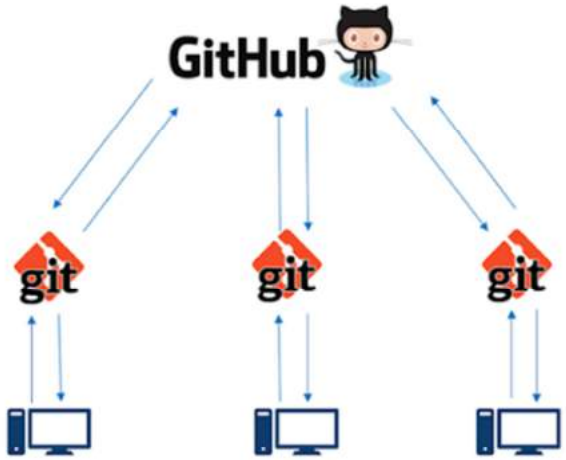
Some IDL mappings are more difficult to use than others. For example, due to the nature of Java, the IDL-Java mapping is rather straightforward and makes usage of CORBA very simple in a Java application. This is also true of the IDL to Python mapping. The C++ mapping requires the programmer to learn datatypes that predate the C++ Standard Template Library (STL). By contrast, the C++11 mapping is easier to use, but requires heavy use of the STL. Since the C language is not object-oriented, the IDL to C mapping requires a C programmer to manually emulate object-oriented features.

In order to build a system that uses or implements a CORBA-based distributed object interface, a developer must either obtain or write the IDL code that defines the object-oriented interface to the logic the system will use or implement. Typically, an ORB implementation includes a tool called an IDL compiler that translates the IDL interface into the target language for use in that part of the system. A traditional compiler then compiles the generated code to create the linkable-object files for use in the application. This diagram illustrates how the generated code is used within the CORBA infrastructure:

This figure illustrates the high-level paradigm for remote interprocess communications using CORBA. The CORBA specification further addresses data typing, exceptions, network protocols, communication timeouts, etc. For example: Normally the server side has the Portable Object Adapter (POA) that redirects calls either to the local servants or (to balance the load) to the other servers. The CORBA specification (and thus this figure) leaves various aspects of distributed system to the application to define including object lifetimes (although reference counting semantics are available to applications), redundancy/fail-over, memory management,

dynamic load balancing, and application-oriented models such as the separation between display/data/control semantics (e.g. see Model–view–controller), etc.
In addition to providing users with a language and a platform-neutral remote procedure call (RPC) specification, CORBA defines commonly needed services such as transactions and security, events, time, and other domain-specific interface models.

Libraries and Toolsets



© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Library_(computing)

Read this article further to clear your concepts. Though the definition below is more than enough.

"Software library" redirects here. It is not to be confused with library software.
This article is about a software development concept. For a repository of digital assets, see Digital library.
Illustration of an application which uses libvorbisfile to play an Ogg Vorbis file
In computer science, a **library** is a collection of non-volatile resources used by computer programs, often for software development. These may include configuration data, documentation, help data, message templates, pre-written code and subroutines, classes, values or type specifications. In IBM's OS/360 and its successors they are referred to as partitioned data sets.
A **library** is also a collection of implementations of behavior, written in terms of a language, that has a well-defined interface by which the behavior is invoked. For instance, people who want to write a higher level program can use a library to make system calls instead of implementing those system calls over and over again.

In addition, the behavior is provided for reuse by multiple independent programs. A program invokes the library-provided behavior via a mechanism of the language. For example, in a simple imperative language such as C, the behavior in a library is invoked by using C's normal function-call. What distinguishes the call as being to a library function, versus being to another function in the same program, is the way that the code is organized in the system.

Library code is organized in such a way that it can be used by multiple programs that have no connection to each other, while code that is part of a program is organized to be used only within that one program. This distinction can gain a hierarchical notion when a program grows large, such as a multi-million-line program. In that case, there may be internal libraries that are reused by independent sub-portions of the large program. The distinguishing feature is that a library is organized for the purposes of being reused by independent programs or sub-programs, and the user only needs to know the interface and not the internal details of the library.

The value of a library lies in the reuse of the behavior. When a program invokes a library, it gains the behavior implemented inside that library without having to implement that behavior itself. Libraries encourage the sharing of code in a modular fashion, and ease the distribution of the code.

The behavior implemented by a library can be connected to the invoking program at different program lifecycle phases. If the code of the library is accessed during the build of the invoking program, then the library is called a static library.[1] An alternative is to build the executable of the invoking program and distribute that, independently of the library implementation. The library behavior is connected after the executable has been invoked to be executed, either as part of the process of starting the execution, or in the middle of execution. In this case the library is called a dynamic library (loaded at runtime). A dynamic library can be loaded and linked when preparing a program for execution, by the linker. Alternatively, in the middle of execution, an application may explicitly request that a module be loaded.

Most compiled languages have a standard library although programmers can also create their own custom libraries. Most modern software systems provide libraries that implement the majority of the system services. Such libraries have commoditized the services which a modern application requires. As such, most code used by modern applications is provided in these system libraries.



Library Benefits

The benefits of libraries are many. Software libraries can contain well coded objects that are implemented properly, well-secured, and kept up to date with security patches and an iterative feedback mechanism to address bugs and faults as they are identified. Software libraries can also have the following advantages:

Increased dependability: Reused software that has been developed and tested as such can be more dependable than new software. This is because the software can be tested to reveal any design and implementation faults and therefore, these can be fixed and then reused over and over again.

Reduced process risk: If software exists, organizations know exactly the cost of creating that software. This is an important factor for project management as it reduces the margin of error in estimating project costs. This is particularly true in large-scale development projects.

Effective use of specialists: Instead of developers doing the same work on different projects, specialists can develop reusable software on different projects. These

specialists can develop reusable software that encapsulates their knowledge. This can include security specialists.

Standards compliance: Some standards, such as user interface standards, can be implemented as a set of standard reusable components. For example, if menus in a user interface are implemented using reusable components, all applications present the same menu formats to users. The use of standard user interfaces improves dependability as users are less likely to make mistakes when presented with a familiar interface.

Accelerated development: In many cases, bringing a system to market as early as possible is often more important than overall development costs. Reusing well developed software can speed up system production because both development and validation time should be reduced.

Programming Tools



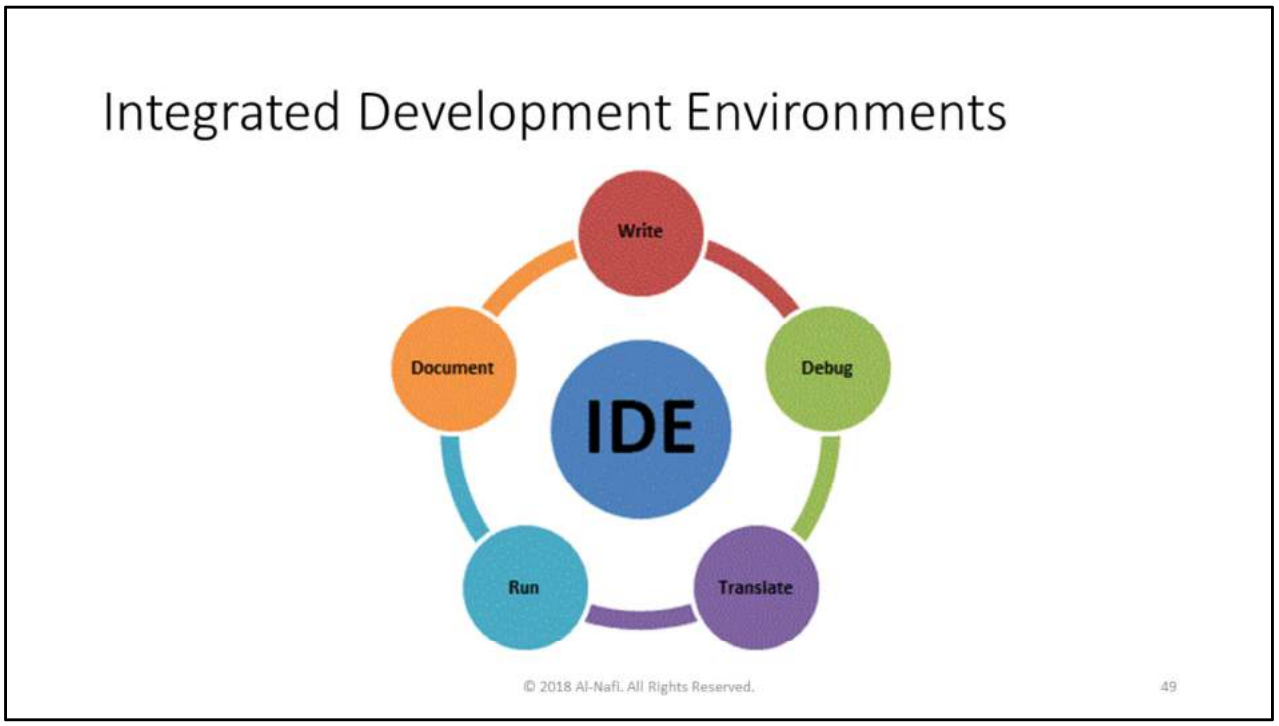
© 2018 Al-Nafi. All Rights Reserved.

A programming tool or software development tool is a program or application that software developers use to create, debug, maintain, and also, support development efforts and applications. Typically, programming tools exist such as the following:

- Binary compatibility analysis tools
- Bug databases
- Build tools
- Code coverage
- Compilation and linking tools
- Debuggers
- Documentation generators
- Library interface generators
- Integration tools
- Memory debuggers
- Revision control tools
- Scripting languages
- Search tools
- Source code editors
- Source code generation tools

- Static code analysis tools
- Unit testing tools

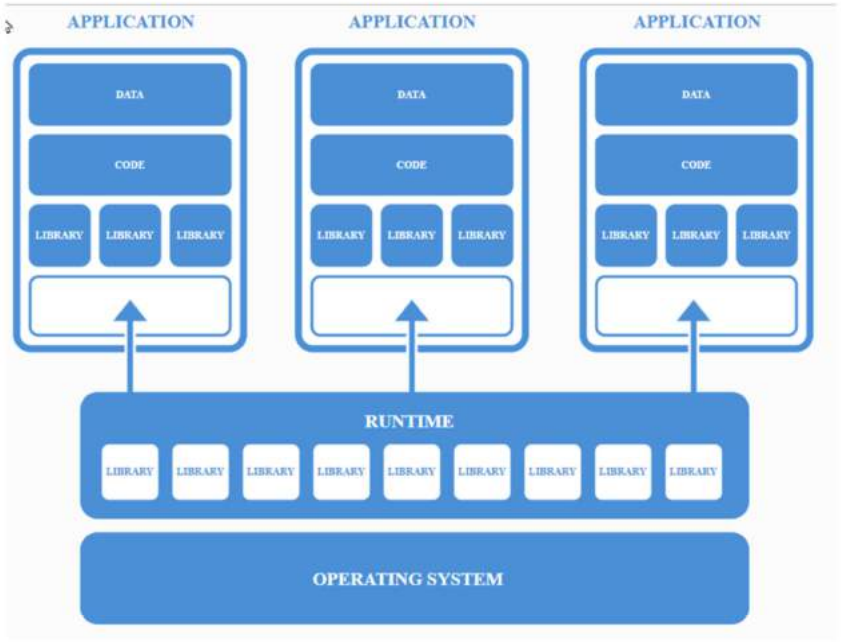
The focus of the security professional needs to be on awareness of the existence and availability of these toolsets and how they may pertain to the security of the systems that the security professional is being asked to manage and maintain. Reliance on experts in this area as needed to help better understand the impact of the use of one or more of these items in a production system is very important for the overall security to be addressed properly in the system.



Integrated development environments (IDEs) combine the features of many tools and capabilities into one environment for use by the developer and other stakeholders. Integrated development environments are designed to maximize developer productivity by providing re-usable components with similar user interfaces.

Integrated development environments also present a single architecture in which all development may be done. The environment typically consists of a source code editor, build automation tools, and debuggers. They may also have a class browser, an object browser, and a class hierarchy diagram for use in object-oriented software development. Sometimes, version control is also included as part of the environment to help organizations manage the development of a graphical user interface (GUI). An IDE for OOP usually features a class browser, tools to produce class hierarchy diagrams, and an object inspector. By using such a comprehensive toolset, developers can realize many benefits, including more efficient access and use of system resources. From a security perspective, more efficient use of security controls can also be a benefit.

Runtime



© 2018 Al-Nafi. All Rights Reserved.

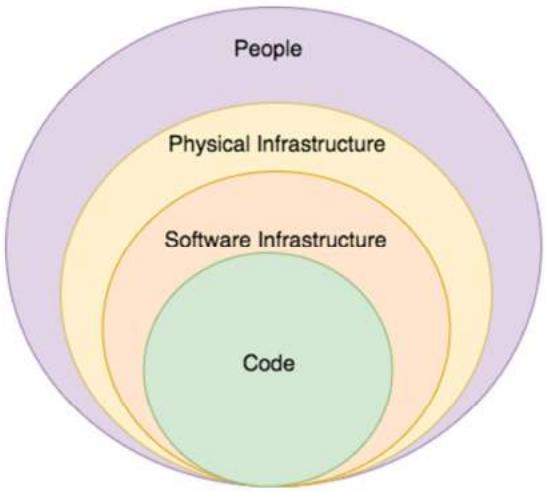
Reference http://docs.flatpak.org/en/latest/basic-concepts.html

Runtimes

Runtimes provide the basic dependencies that are used by applications. Each application must be built against a runtime, and this runtime must be installed on a host system in order for the application to run (Flatpak can automatically install the runtime required by an application). Multiple different runtimes can be installed at the same time, including different versions of the same runtime.

Runtimes are distribution agnostic and do not depend on particular distribution versions. This means that they provide a stable, cross-distribution base for applications, and allow applications to continue to work irrespective of operating system updates.

Security Weaknesses at Source Code Level and Secure Coding Practices



© 2018 Al-Nafi. All Rights Reserved.

Since applications represent the largest attack vector, there are a number of weaknesses and threats that are important to be aware of and address. These include but are not limited to the following list described below. Secure coding practices in development environments need to be addressed to limit the exposure to the same list. Proper awareness, education, training and security skills need to become part of the culture of the development environment to address security properly. The security professional needs to be heavily involved in addressing and minimizing the risks associated with the following topics.

Social Engineering

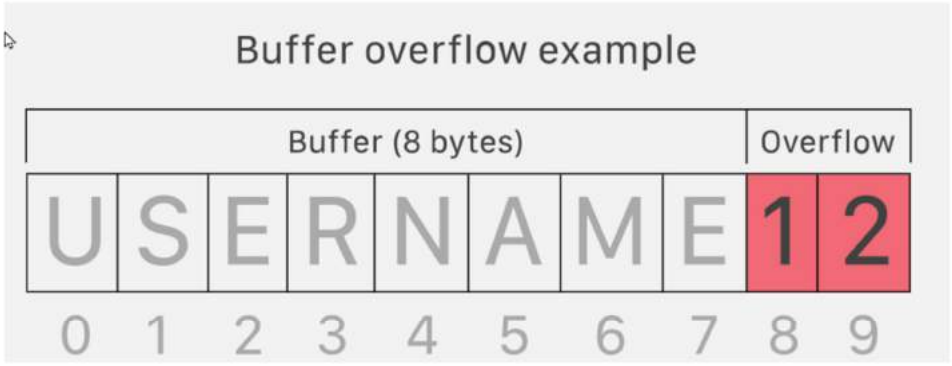


© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.onespan.com/blog/social-engineering-win-battle-trust-infographic

Please review this info graphics as it explains all the key concepts.

Buffer Overflow



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.cloudflare.com/learning/security/threats/buffer-overflow/

What is buffer overflow?
Buffer overflow is an anomaly that occurs when software writing data to a buffer overflows the buffer's capacity, resulting in adjacent memory locations being overwritten. In other words, too much information is being passed into a container that does not have enough space, and that information ends up replacing data in adjacent containers. Buffer overflows can be exploited by attackers with a goal of modifying a computer's memory in order to undermine or take control of program execution.

What's a buffer?
A buffer, or data buffer, is an area of physical memory storage used to temporarily store data while it is being moved from one place to another. These buffers typically live in RAM memory. Computers frequently use buffers to help improve performance; most modern hard drives take advantage of buffering to efficiently access data, and many online services also use buffers. For example, buffers are frequently used in online video streaming to prevent interruption. When a video is

streamed, the video player downloads and stores perhaps 20% of the video at a time in a buffer and then streams from that buffer. This way, minor drops in connection speed or quick service disruptions won't affect the video stream performance.

Buffers are designed to contain specific amounts of data. Unless the program utilizing the buffer has built-in instructions to discard data when too much is sent to the buffer, the program will overwrite data in memory adjacent to the buffer.

Buffer overflows can be exploited by attackers to corrupt software. Despite being well-understood, buffer overflow attacks are still a major security problem that torment cyber-security teams. In 2014 a threat known as 'heartbleed' exposed hundreds of millions of users to attack because of a buffer overflow vulnerability in SSL software.

How do attackers exploit buffer overflows?
An attacker can deliberately feed a carefully crafted input into a program that will cause the program to try and store that input in a buffer that isn't large enough, overwriting portions of memory connected to the buffer space. If the memory layout of the program is well-defined, the attacker can deliberately overwrite areas known to contain executable code. The attacker can then replace this code with his own executable code, which can drastically change how the program is intended to work. For example if the overwritten part in memory contains a pointer (an object that points to another place in memory) the attacker's code could replace that code with another pointer that points to an exploit payload. This can transfer control of the whole program over to the attacker's code.

Who is vulnerable to buffer overflow attacks?
Certain coding languages are more susceptible to buffer overflow than others. C and C++ are two popular languages with high vulnerability, since they contain no built-in protections against accessing or overwriting data in their memory. Windows, Mac OSX, and Linux all contain code written in one or both of these languages.
More modern languages like Java, PERL, and C# have built-in features that help reduce the chances of buffer overflow, but cannot prevent it altogether.

How to protect against buffer overflow attacks
Luckily, modern operating systems have runtime protections which help mitigate buffer overflow attacks. Let's explore 2 common protections that help mitigate the risk of exploitation:
Address space randomization - Randomly rearranges the address space locations of key data areas of a process. Buffer overflow attacks generally rely on knowing the exact location of important executable code, randomization of address spaces makes

that nearly impossible.

Data execution prevention - Marks certain areas of memory either executable or non-executable, preventing an exploit from running code found in a non-executable area.

Software developers can also take precautions against buffer overflow vulnerabilities by writing in languages that have built-in protections or using special security procedures in their code.

Despite precautions, new buffer overflow vulnerabilities continue to be discovered by developers, sometimes in the wake of a successful exploitation. When new vulnerabilities are discovered, engineers need to patch the affected software and ensure that users of the software get access to the patch.

What are the different types of buffer overflow attacks?

There are a number of different buffer overflow attacks which employ different strategies and target different pieces of code. Below are a few of the most well-known.

Stack overflow attack - This is the most common type of buffer overflow attack and involves overflowing a buffer on the call stack*.

Heap overflow attack - This type of attack targets data in the open memory pool known as the heap*.

Integer overflow attack - In an integer overflow, an arithmetic operation results in an integer (whole number) that is too large for the integer type meant to store it; this can result in a buffer overflow.

Unicode overflow - A unicode overflow creates a buffer overflow by inserting unicode characters into an input that expect ASCII characters. (ASCII and unicode are encoding standards that let computers represent text. For example the letter ‘a’ is represented by the number 97 in ASCII. While ASCII codes only cover characters from Western languages, unicode can create characters for almost every written language on earth. Because there are so many more characters available in unicode, many unicode characters are larger than the largest ASCII character.)

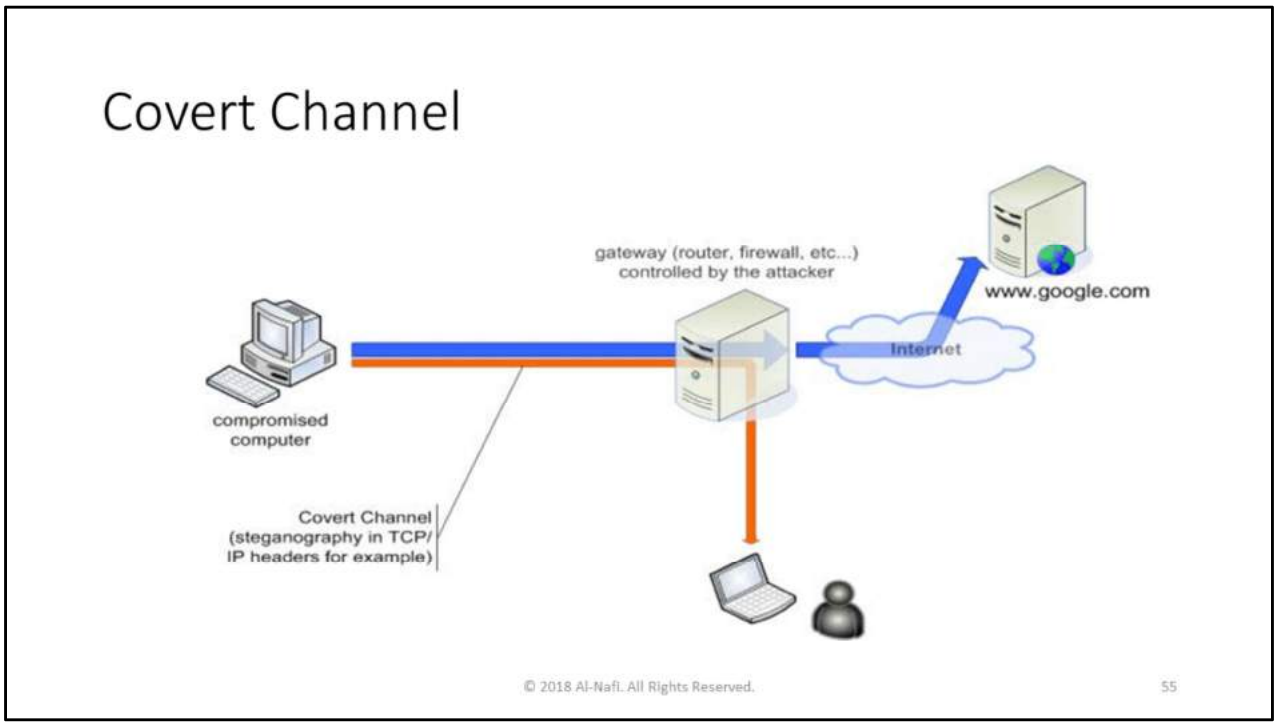
*Computers rely on two different memory allocation models, known as the stack and the heap; both live in the computer’s RAM. The stack is neatly organized and holds data in a Last-In, First-Out model. Whatever piece of data was most recently placed in the stack will be the first to come out, kind of like how the last bullet inserted into an ammunition magazine will be the first to be fired. The heap is a disorganized pool of extra memory, data does not enter or leave the heap in any particular order. Since accessing memory from the stack is much faster than accessing from the heap, the heap is generally reserved for larger pieces of data or data that a programmer wants to manage explicitly.

Citizen Programmers or Developers



© 2018 Al-Nafi. All Rights Reserved.

As we have explained above, today, technology environments are equipped with scripting and programming tools as part of their functional environments. The ability to provide more functionality
in application environments is so that these functions can be performed by the users themselves, instead of having them be programmed into the application by developers. These tools may allow all computer users to create their own utilities and reusable elements. This can be negative from a security perspective as users now have access to very powerful capabilities that may be misused
by the users as they are not focused on security or have security training. They may not be aware of the increased risk as a result of their increased functionality. If this type of unsupervised functionality is allowed, then a single user may have complete control over an application or process. This may violate separation of duties requirements. Putting powerful tool and capabilities at the user level requires mitigation of the increased risks that this may pose.



Reference https://blog.insiderattack.net/covert-channels-and-data-exfiltration-a7c73f01dc8c

The above is a very detailed article which is recommended to be read slowly at least 2-3 times. For the sake of CISSP exam you only need to understand the definition and how the attack vector works. But we will cover all of these attacks during network and web application hacking inshAllah.

A covert channel may be defined as a communication channel that allows processes to transfer information in such a way to violate some security policy or requirement. This is an information flow issue. Even though there are protection mechanisms in place, if unauthorized information can be transferred using a signaling mechanism or a storage mechanism, using some way that is not normally considered to be able to communicate, then a covert channel may exist. In simplified terms, it is any flow of information, unintentional or inadvertent, that enables an unauthorized observer to have access to the sensitive information. This may allow the observer to infer more sensitive information than is allowed.

There are two defined types of covert channels, storage and timing.

A storage covert channel involves the direct or indirect reading of storage locations by one process and a direct or indirect reading of the same storage location by another process. Typically, a covert storage channel involves memory locations or sectors on a disk that may be shared by two subjects at different security levels. This could include hard drive space, cache, or other typically used memory types in computer architectures.

A timing covert channel depends upon being able to influence the rate or timing issue that some other process is able to acquire resource. Examples of this may be the CPU, memory, or I/O devices. The variation in rate may be used to pass signals that may be used to infer more sensitive information. Essentially, the process signals information to another process by modulating its own use of system resources in such a way that this manipulation affects the real response time observed by the second process and therefore, may signal sensitive information.

Timing channels may be very difficult to detect as a result.

Malformed Input Attacks



Attacks input validation can help prevent

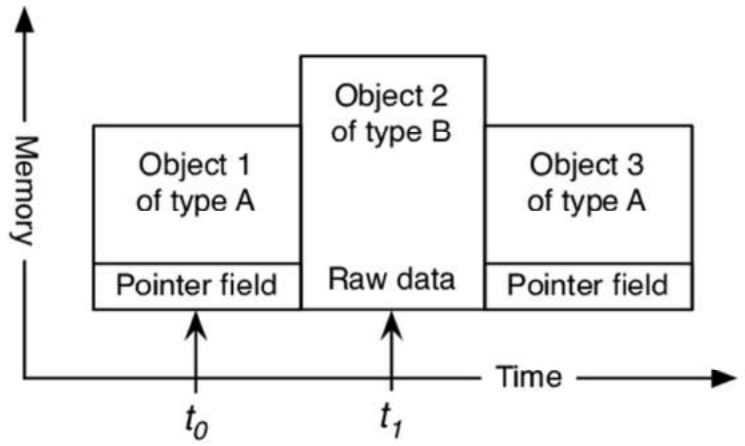
© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.owasp.org/index.php/Testing_for_Input_Validation
Please read the above article as it will provide a good knowledge. We will cover this attack in a practical manner during our web application hacking inshAllah.

A number of attacks that use input from the user and somehow inject or modify such input currently exist and are known. The known ones may be able to be detected by various detection systems and can be possibly protected against. However, new attacks relying on configuring user input in unusual ways may not be detected. For example, an attack that redirected a web browser to an alternate site might be caught by a firewall through the detection of the uniform resource locator (URL) of an inappropriate website. If, however, the URL was expressed in a Unicode format rather than ASCII, the firewall would likely fail to recognize the content, whereas the web browser would convert the information without difficulty. Here is another example. Many websites allow query access to databases but place filters on the requests to control access as part of access control. When requests using the Structured Query Language (SQL) are allowed, the use of certain syntactical structures in the query can fool the filters into seeing the query as a comment instead of an instruction, and as a result, the resulting query may be submitted to

the database engine and retrieve more information than was intended. In another instance, a site that allows users to input information for later retrieval by other users, such as a blog, may fail to detect when such input comes in the form of active scripting. This is the basis of a very well-known type of attack known as cross-site scripting. As we've seen above, technically, buffer overflows are also a form of malformed input.

Memory Reuse (Object Reuse)



© 2018 Al-Nafi. All Rights Reserved.

All architectures use memory to process information and data. Memory management involves sections of memory allocated to one process for a while, then de-allocated, then reallocated to other processes. This can include random access memory (RAM), cache, or simply hard drive space.

The problem from a security perspective is that because residual information may remain when a section of memory is reassigned to a new process after a previous process is finished with it, that information remaining on that object may be very sensitive. The architecture should ensure that memory is zeroed out completely or overwritten completely before it should be allocated to a new process. As a result, there should be no sensitive information that remains residually in memory carrying over from one process to another. While memory locations are of primary concern in this regard, developers should also be careful with the reuse of other resources that can contain sensitive information such as buffers, disk space, and other shared resources. Other examples of storage that may be very vulnerable to this type of problem is the paging or swap file on the disk. It is frequently left unprotected and may contain an enormous amount of sensitive information. Note that this is a perfect example of storage covert channel, as discussed earlier.

Executable Content/Mobile Code



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.owasp.org/index.php/Unsafe_Mobile_Code

Unsafe Mobile Code

*This is a **Vulnerability**. To view all vulnerabilities, please see the Vulnerability Category page.*

Mobile code, such as a Java Applet, is code that is transmitted across a network and executed on a remote machine. Because mobile code developers have little if any control of the environment in which their code will execute, special security concerns become relevant. One of the biggest environmental threats results from the risk that the mobile code will run side-by-side with other, potentially malicious, mobile code. Because all of the popular web browsers execute code from multiple sources together in the same JVM, many of the security guidelines for mobile code are focused on preventing manipulation of your objects' state and behavior by adversaries who have access to the same virtual machine where your program is running.

Access Violation

The program violates secure coding principles for mobile code by returning a private array variable from a public access method.

Returning a private array variable from a public access method allows the calling code to modify the contents of the array, effectively giving the array public access and contradicting the intentions of the programmer who made it private.

Example

The following Java Applet code mistakenly returns a private array variable from a public access method.

public final class urlTool extends Applet { private URL[] urls; public URL[] getURLs() { return urls; } ... } .

Dangerous Array Declaration

The program violates secure coding principles for mobile code by declaring an array public, final, and static.

In most cases an array declared public, final, and static is a bug. Because arrays are mutable objects, the final constraint requires that the array object itself be assigned only once, but makes no guarantees about the values of the array elements. Since the array is public, a malicious program can change the values stored in the array. In most situations the array should be made private.

Example

The following Java Applet code mistakenly declares an array public, final, and static.

public final class urlTool extends Applet { public final static URL[] urls; ... } **Dangerous Public Field**

The program violates secure coding principles for mobile code by declaring a member variable public but not final.

All public member variables in an Applet and in classes used by an Applet should be declared final to prevent an attacker from manipulating or gaining unauthorized access to the internal state of the Applet.

Example

The following Java Applet code mistakenly declares a member variable public but not final.

public final class urlTool extends Applet { public URL url; ... } **Inner Class**

The program violates secure coding principles for mobile code by making use of an inner class.

Inner classes quietly introduce several security concerns because of the way they are translated into Java bytecode. In Java source code, it appears that an inner class can be declared to be accessible only by the enclosing class, but Java bytecode has no concept of an inner class, so the compiler must transform an inner class declaration into a peer class with package level access to the original outer class. More insidiously, since an inner class can access private fields in their enclosing class, once an inner class becomes a peer class in bytecode, the compiler converts private fields

accessed by the inner class into protected fields.

Example

The following Java Applet code mistakenly makes use of an inner class.

public final class urlTool extends Applet { private final class urlHelper { ... } ... } **Public finalize() Method**

The program violates secure coding principles for mobile code by declaring a finalize()method public.

A program should never call finalize explicitly, except to call super.finalize() inside an implementation of finialize(). In mobile code situations, the otherwise error prone practice of manual garbage collection can become a security threat if an attacker can maliciously invoke one of your finalize() methods because it is declared with public access. If you are using finalize() as it was designed, there is no reason to declare finalize() with anything other than protected access.

Example

The following Java Applet code mistakenly declares a public finalize() method.

public final class urlTool extends Applet { public void finalize() { ... } ... } Risk Factors

Talk about the factors that make this vulnerability likely or unlikely to actually happen

Discuss the technical impact of a successful exploit of this vulnerability

Consider the likely [business impacts] of a successful attack

Time of Check vs. Time of Use (TOCTOU)

It's a race condition



© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Time-of-check_to_time-of-use

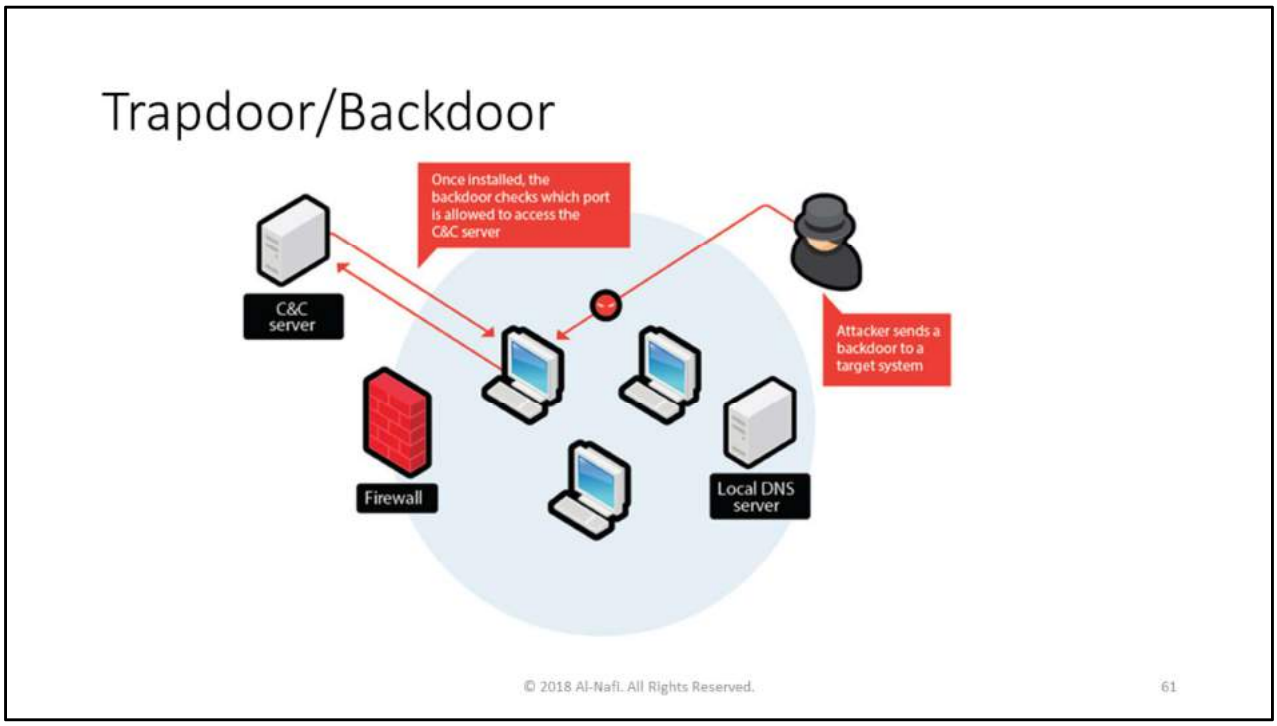
Time of Check vs. Time of Use (TOCTOU) is seemingly a very common type of attack that occurs when control information changes between the time the system security functions check the contents of variables and the time the variables actually are used during operations. Control information is information that is used to make security decisions. This might be a very good example. A user logs onto a system in the morning and later is dismissed. As a result of the termination, the security administrator removes the user from the user database and disables the account. However, because the user did not log off, the account still has access to the system and, as far as the system is concerned, still has privileges. Here is another example. A connection between two machines may drop. If an attacker manages to attach to one of the ports used for this link before the failure is detected, the invader can hijack the session by pretending to be one of the trusted hosts. A good way to deal with this would be to force periodic re-authentication on a regular basis.

Between-the-Lines Attack



© 2018 Al-Nafi. All Rights Reserved.

A similar attack to the above is called a between-the-lines entry. This occurs when the telecommunication lines used by an authorized user are tapped into and data falsely inserted or injected. To avoid this, the telecommunication lines should be physically secured so that they cannot be accessed by unauthorized individuals, and users should not leave telecommunication lines open
when they finished with them and those lines are not being used anymore.



Reference http://www.4threatsremoval.com/what-is-backdoors-top-10-backdoors/

A backdoor, also known as a trapdoor, is a way to access a computer software without getting detected or stopped by security programs installed on the PC. This term can refer to both the legitimate means of access as well as the applications that are used for remote attacks by hackers. Legitimate undocumented backdoors are used in order to allow the administrator to enter a computer system for troubleshooting, upkeep, or other reasons. Secret portals that are used by cyber criminals, on the other hand, can allow them to enter computers for malicious purposes.

Built-in legitimate backdoors are generally used for maintenance. Some of them are protected by a username and a password. In most cases, users have no idea that these backdoors exist, and only the makers of the programs are aware of them. Backdoors were a hot topic back in 2013, when Edward Snowden released the information about the NSA's effort to make companies install backdoors into their software. These backdoors would allow the intelligence agencies to enter systems unnoticed and access the data necessary to them.

The 10 biggest software backdoors of all time

Back Orifice

The DSL backdoor that wouldn't die

The PGP full-disk encryption backdoor

Backdoors in pirated copies of commercial WordPress plug-ins

The Joomla plug-in backdoor

The ProFTPD backdoor

The Borland Interbase backdoor

The Linux backdoor (insert a subtle backdoor)

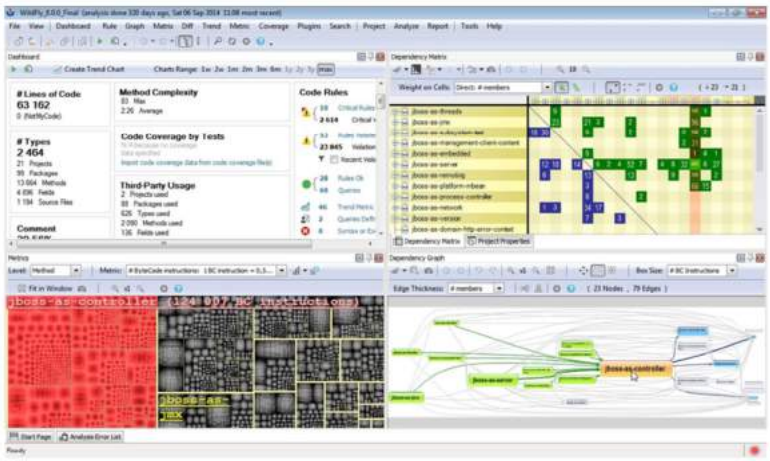
The tcpdump backdoor

The Flawed hardening backdoor

Although administrative backdoors also create vulnerabilities in software and systems, the real trouble comes when hackers install their own backdoors onto the computer. Cyber criminals can use backdoors in order to access the system without the user's notice and steal their personal data as well as drop other malicious applications onto the PC. Both of these actions can lead to serious consequences. If cyber crooks gain access to the user's personal information, this can result in identity theft or financial losses, while the addition of other malware onto the computer could result in system damage, data corruption, browsing issues, and more.

What should be a priority for computer users is that they install reliable anti-malware tools capable of detecting backdoors and eliminating them from the system and keep these tools updated. This would ensure that they do not have to deal with the above-mentioned problems. If you think that there may be malicious programs on your PC already, you should download and implement the malware prevention and removal tool as soon as you can. It will help you detect backdoors and other threats, clean your computer, and keep it safeguarded from future attacks.

Source Code Analysis Tools



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.owasp.org/index.php/Source_Code_Analysis_Tools

Source Code Analysis Tools

Source code analysis tools, also referred to as Static Application Security Testing (SAST) Tools, are designed to analyze source code and/or compiled versions of code to help find security flaws.

Some tools are starting to move into the IDE. For the types of problems that can be detected during the software development phase itself, this is a powerful phase within the development life cycle to employ such tools, as it provides immediate feedback to the developer on issues they might be introducing into the code during code development itself. This immediate feedback is very useful, especially when compared to finding vulnerabilities much later in the development cycle.

Strengths and Weaknesses

Strengths

Scales well -- can be run on lots of software, and can be run repeatedly (as with nightly builds or continuous integration)

Useful for things that such tools can automatically find with high confidence, such as buffer overflows, SQL Injection Flaws, and so forth

Output is good for developers -- highlights the precise source files, line numbers, and even subsections of lines that are affected

Weaknesses

Many types of security vulnerabilities are difficult to find automatically, such as authentication problems, access control issues, insecure use of cryptography, etc. The current state of the art only allows such tools to automatically find a relatively small percentage of application security flaws. However, tools of this type are getting better.

High numbers of false positives.

Frequently can't find configuration issues, since they are not represented in the code.

Difficult to 'prove' that an identified security issue is an actual vulnerability.

Many of these tools have difficulty analyzing code that can't be compiled. Analysts frequently can't compile code because they don't have the right libraries, all the compilation instructions, all the code, etc.

Important Selection Criteria

Requirement: Must support your programming language, but not usually a key factor once it does.

Types of vulnerabilities it can detect (out of the OWASP Top Ten?) (plus more?)

How accurate is it? False Positive/False Negative rates?

Does the tool have an OWASP Benchmark score?

Does it understand the libraries/frameworks you use?

Does it require a fully buildable set of source?

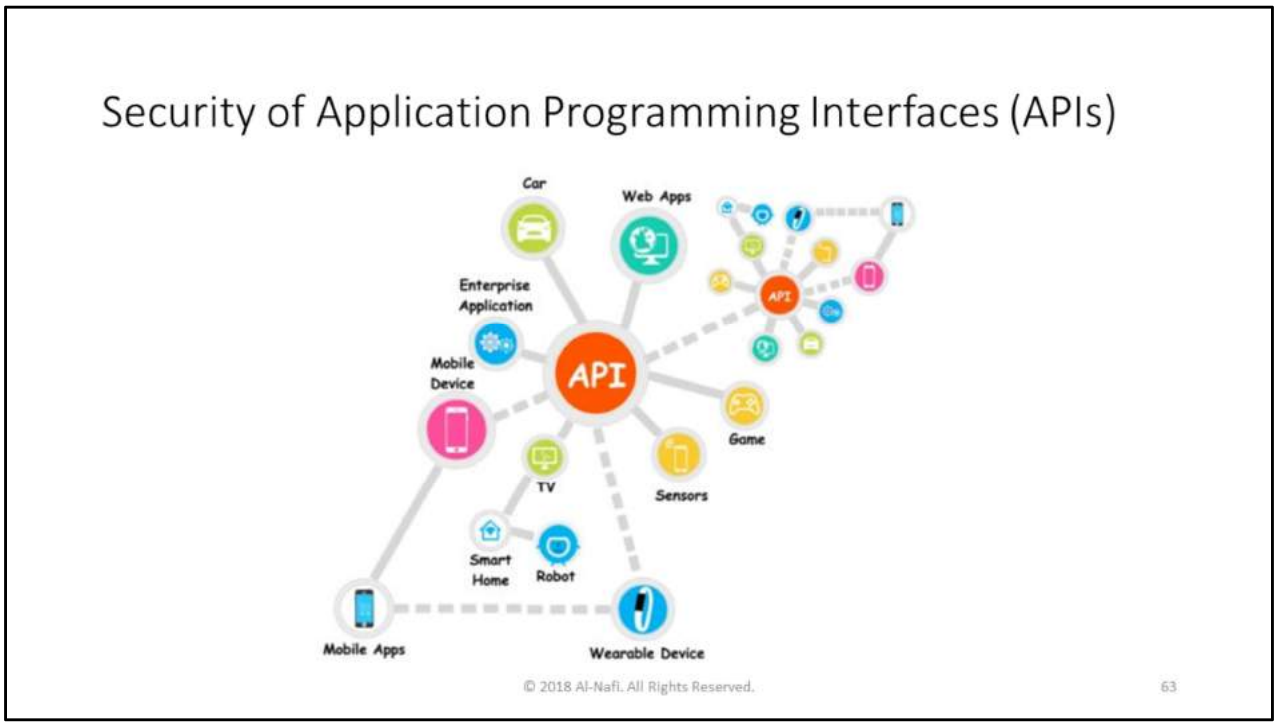
Can it run against binaries instead of source?

Can it be integrated into the developer's IDE?

How hard is it to setup/use?

Can it be run continuously and automatically?

License cost for the tool. (Some are sold per user, per org, per app, per line of code analyzed. Consulting licenses are frequently different than end user licenses.)



Reference https://www.redhat.com/en/topics/security/api-security

What is API security?

API security is the protection of the integrity of APIs—both the ones you own and the ones you use. But what does that mean?

Well, you've probably heard of the Internet of Things (IoT), where computing power is embedded in everyday objects. The IoT makes it possible to connect your phone to your fridge, so that when you stop at the grocery store on the way home you know exactly what you need for that impromptu dinner party in an hour. Or maybe you're part of a DevOps team, using microservices and containers to build and deploy legacy and cloud-native apps in a fast-paced, iterative way. APIs are one of the most common ways that microservices and containers communicate, just like systems and apps. As integration and interconnectivity become more important, so do APIs.

Why is API security important?

Businesses use APIs to connect services and to transfer data. Broken, exposed, or

hacked APIs are behind major data breaches. They expose sensitive medical, financial, and personal data for public consumption. That said, not all data is the same nor should be protected in the same way. How you approach API security will depend on what kind of data is being transferred.
If your API connects to a third party application, understand how that app is funneling information back to the internet. To use the example above, maybe you don't care if someone finds out what's in your fridge, but if they use that same API to track your location you might be more concerned.

What is web API security? REST API security vs. SOAP API security.

Web API security is concerned with the transfer of data through APIs that are connected to the internet. OAuth (Open Authorization) is the open standard for access delegation. It enables users to give third-party access to web resources without having to share passwords. OAuth is the technology standard that lets you share that Corgi belly flop compilation video onto your social networks with a single "share" button.

Most API implementations are either REST (Representational State Transfer) or SOAP (Simple Object Access Protocol).
REST APIs use HTTP and support Transport Layer Security (TLS) encryption. TLS is a standard that keeps an internet connection private and checks that the data sent between two systems (a server and a server, or a server and a client) is encrypted and unmodified. This means that a hacker trying to expose your credit card information from a shopping website can neither read your data nor modify it. You know if a website is protected with TLS if the URL begins with "HTTPS" (Hyper Text Transfer Protocol Secure).
REST APIs also use JavaScript Object Notation (JSON), which is a file format that makes it easier to transfer data over web browsers. By using HTTP and JSON, REST APIs don't need to store or repackage data, making them much faster than SOAP APIs.

SOAP APIs use built-in protocols known as Web Services Security (WS Security). These protocols define a rules set that is guided by confidentiality and authentication. SOAP APIs support standards set by the two major international standards bodies, the Organization for the Advancement of Structured Information Standards (OASIS) and the World Wide Web Consortium (W3C). They use a combination of XML encryption, XML signatures, and SAML tokens to verify authentication and authorization. In general, SOAP APIs are praised for having more comprehensive security measures, but they also need more management. For these reasons, SOAP APIs are recommended for organizations handling sensitive data.

What are some of the most common API security best practices?

You probably don't keep your savings under your mattress. Most people their money

in a trusted environment (the bank) and use separate methods to authorize and authenticate payments. API security is similar. You need a trusted environment with policies for authentication and authorization.

Here are some of the most common ways you can strengthen your API security:

Use tokens. Establish trusted identities and then control access to services and resources by using tokens assigned to those identities.

Use encryption and signatures. Encrypt your data using a method like TLS(see above). Require signatures to ensure that the right users are decrypting and modifying your data, and no one else.

Identify vulnerabilities. Keep up with your operating system, network, drivers, and API components. Know how everything works together and identify weak spots that could be used to break into your APIs. Use sniffers to detect security issues and track data leaks.

Use quotas and throttling. Place quotas on how often your API can be called and track its use over history. More calls on an API may indicate that it is being abused. It could also be a programming mistake such as calling the API in an endless loop. Make rules for throttling to protect your APIs from spikes and Denial-of-Service attacks.

Use an API gateway. API gateways act as the major point of enforcement for API traffic. A good gateway will allow you to authenticate traffic as well as control and analyze how your APIs are used.

API management and security

Finally, API security often comes down to good API management. Many API management platforms support three types of security schemes. These are:

An API key that is a single token string (i.e. a small hardware device that provides unique authentication information).

Basic Authentication (APP ID / APP Key) that is a two token string solution (i.e. username and password).

OpenID Connect (OIDC) that is a simple identity layer on top of the popular OAuth framework (i.e. it verifies the user by obtaining basic profile information and using an authentication server).

When you select an API manager know which and how many of these security schemes it can handle, and have a plan for how you can incorporate the API security practices outlined above.

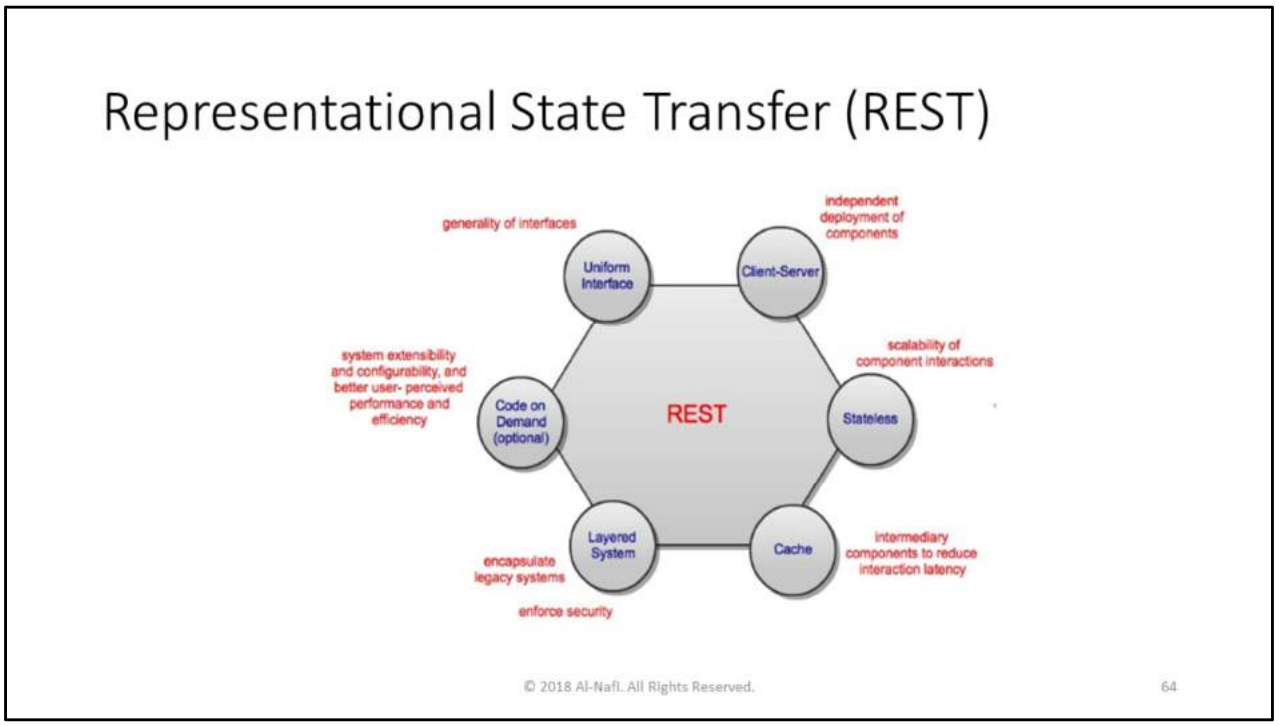
Why choose Red Hat for API management and security

Data breaches are scary, but you can take steps toward better security. APIs are worth the effort, you just need to know what to look for. A lot of it comes down to continuous security measures, asking the right questions, knowing which areas need attention, and using an API manager that you can trust. We are here to help.

At Red Hat, we recommend our award-winning Red Hat 3scale API Management. It includes:

An API manager which manages the API, applications, and developer roles

A traffic manager (an API gateway) that enforces the policies from the API manager
An identity provider (IDP) hub that supports a wide range of authentication protocols
At the API gateway, Red Hat 3scale API Management decodes timestamped tokens that expire; checks that the client identification is valid; and confirms the signature using a public key.



Reference https://searchapparchitecture.techtarget.com/definition/REST-REpresentational-State-Transfer

REST (REpresentational State Transfer)

REST (REpresentational State Transfer) is an architectural style for developing web services. REST is popular due to its simplicity and the fact that it builds upon existing systems and features of the internet's Hypertext Transfer Protocol (HTTP) in order to achieve its objectives, as opposed to creating new standards, frameworks and technologies.

Advantages of REST

A primary benefit of using REST, both from a client and server's perspective, is REST-based interactions happen using constructs that are familiar to anyone who is accustomed to using the internet's HTTP.

An example of this arrangement is REST-based interactions all communicate their status using standard HTTP status codes. So, a 404 means a requested resource wasn't found; a 401 code means the request wasn't authorized; a 200 code means everything is OK; and a 500 means there was an unrecoverable application error on

the server.

Similarly, details such as encryption and data transport integrity are solved not by adding new frameworks or technologies, but instead by relying on well-known Secure Sockets Layer (SSL) encryption and Transport Layer Security (TLS). So, the entire REST architecture is built upon concepts with which most developers are already familiar.

REST is also a language-independent architectural style. REST-based applications can be written using any language, be it Java, Kotlin, .NET, AngularJS or JavaScript. As long as a programming language can make web-based requests using HTTP, it is possible for that language to be used to invoke a RESTful API or web service. Similarly, RESTful web services can be written using any language, so developers tasked with implementing such services can choose technologies that work best for their situation.

The other benefit of using REST is its pervasiveness. On the server side, there are a variety of REST-based frameworksfor helping developers create RESTful web services, including RESTlet and Apache CXF. From the client side, all of the new JavaScript frameworks, such as JQuery, Node.js, Angular and EmberJS, all have standard libraries built into their APIs that make invoking RESTful web services and consuming the XML- or JSON-based data they return a relatively straightforward endeavor.

Disadvantages of REST

The benefit of REST using HTTP constructs also creates restrictions, however. Many of the limitations of HTTP likewise turn into shortcomings of the REST architectural style. For example, HTTP does not store state-based information between request-response cycles, which means REST-based applications must be stateless and any state management tasks must be performed by the client.

Similarly, since HTTP doesn't have any mechanism to send push notifications from the server to the client, it is difficult to implement any type of services where the server updates the client without the use of client-side polling of the server or some other type of web hook.

From an implementation standpoint, a common problem with REST is the fact that developers disagree with exactly what it means to be REST-based. Some software developers incorrectly consider anything that isn't SOAP-based to be RESTful. Driving this common misconception about REST is the fact that it is an architectural style, so there is no reference implementation or definitive standard that will confirm whether a given design is RESTful. As a result, there is discourse as to whether a given API conforms to REST-based principles.

Alternatives to REST

Alternate technologies for creating SOA-based systems or creating APIs for invoking

remote microservices include XML over HTTP (XML-RPC), CORBA, RMI over IIOP and the Simple Object Access Protocol (SOAP).
Each technology has its own set of benefits and drawbacks, but the compelling feature of REST that sets it apart is the fact that, rather than asking a developer to work with a set of custom protocols or to create a special data format for exchanging messages between a client and a server, REST insists the best way to implement a network-based web service is to simply use the basic construct of the network protocol itself, which in the case of the internet is HTTP.
This is an important point, as REST is not intended to apply just to the internet; rather, its principles are intended to apply to all protocols,
including WEBDAV and FTP.

REST vs. SOAP

The two competing styles for implementing web services are REST and SOAP. The fundamental difference between the two is the philosophical approach the two have to remotely invocations.
REST takes a resource-based approach to web-based interactions. With REST, you locate a resource on the server, and you choose to either update that resource, delete it or get some information about it.
With SOAP, the client doesn't choose to interact directly with a resource, but instead calls a service, and that service mitigates access to the various objects and resources behind the scenes.
SOAP has also built a large number of frameworks and APIs on top of HTTP, including the Web Services Description Language (WSDL), which defines the structure of data that gets passed back and forth between the client and the server.
Some problem domains are served well by the ability to stringently define the message format or can benefit from using various SOAP-related APIs, such as WS-Eventing, WS-Notification and WS-Security. There are times when HTTP cannot provide the level of functionality an application might require, and in these cases, using SOAP is preferable.

REST URIs and URLs

Most people are familiar with the way URLs and URIs work on the web. A RESTful approach to developing applications asserts that requesting information about a resource should be as simple as invoking its URL.
For example, if a client wanted to invoke a web service that listed all of the quizzes available here at TechTarget, the URL to the web service would look something like this:
www.techtarget.com/restfulapi/**quizzes**
When invoked, the web service might respond with the following JSON string listing all of the available quizzes, one of which is about DevOps:
{ "quizzes" : ["Java", "DevOps", "IoT"] }
To get the DevOps quiz, the web service might be called using the following URL:

www.techtarget.com/restfulapi/**quizzes/DevOps**
Invoking this URL would return a JSON string listing all of the questions in the DevOps quiz. To get an individual question from the quiz, the number of the question would be added to the URL. So, to get the third question in the DevOps quiz, the following RESTful URL would be used:
www.techtarget.com/restfulapi/**quizzes/DevOps/3**
Invoking that URL might return a JSON string such as the following:
{ "Question" : {"query":"What is your DevOps role?", "optionA":"Dev", "optionB":"Ops"} }
As you can see, the REST URLs in this example are structured in a logical and meaningful way that identifies the exact resource being requested.

JSON and XML REST data formats

The example above shows JSON used as the data exchange format for the RESTful interaction. The two most common data exchange formats are JSON and XML, and many RESTful web services can use both formats interchangeably, as long as the client can request the interaction to happen in either XML or JSON.
Note that while JSON and XML are popular data exchange formats, REST itself does not put any restrictions on what the format should be. In fact, some RESTful web services exchange binary data for the sake of efficiency. This is another benefit to working with REST-based web services, as the software architect is given a great deal of freedom in terms of how best to implement a service.

REST and the HTTP methods

The example above only dealt with accessing data.
The default operation of HTTP is GET, which is intended to be used when getting data from the server. However, HTTP defines a number of other methods, including PUT, POST and DELETE.
The REST philosophy asserts that to delete something on the server, you would simply use the URL for the resource and specify the DELETE method of HTTP. For saving data to the server, a URL and the PUT method would be used. For operations that are more involved than simply saving, reading or deleting information, the POST method of HTTP can be used.

History of REST

REST was first coined by computer scientist Roy Fielding in his year-2000 Ph.D. dissertation at the University of California, titled *Architectural Styles and the Design of Network-based Software Architectures*.
Chapter 5 of the dissertation, "Representational State Transfer (REST)," described Fielding's beliefs about how best to architect distributed hypermedia systems. Fielding noted a number of boundary conditions that describe how REST-based systems should behave. These conditions are referred to as REST constraints, with four of the key constraints described below:
Use of a uniform interface (UI). As stated earlier, resources in REST-based systems

should be uniquely identifiable through a single URL, and only by using the underlying methods of the network protocol, such as DELETE, PUT and GET with HTTP, should it be possible to manipulate a resource.

Client-server-based. In a REST-based system, there should be a clear delineation between the client and the server. UI and request-generating concerns are the domain of the client. Meanwhile, data access, workload management and security are the domain of the server. This separation allows loose coupling between the client and the server, and each can be developed and enhanced independent of the other.

Stateless operations. All client-server operations should be stateless, and any state management that is required should happen on the client, not the server.

RESTful resource caching. The ability to cache resources between client invocations is a priority in order to reduce latency and improve performance. As a result, all resources should allow caching unless an explicit indication is made that it is not possible.

Developing REST APIs in Java

To accommodate the growing popularity of REST-based systems, a number of frameworks have arisen to assist developers in the creation of RESTful web services. Some of the more popular open source frameworks for creating Java-based, RESTful web services include Apache CXF, Jersey, Restlet, Apache Wink, Spring Data and JBoss' RESTeasy.

The general approach of each of these frameworks is to help developers build RESTful web services using semantics with which Java developers are familiar, including Java Platform (Enterprise Edition), the Servlet API and annotations, while at the same time, offering built-in classes and methods that make it easier to keep in line with the basic tenets of REST.

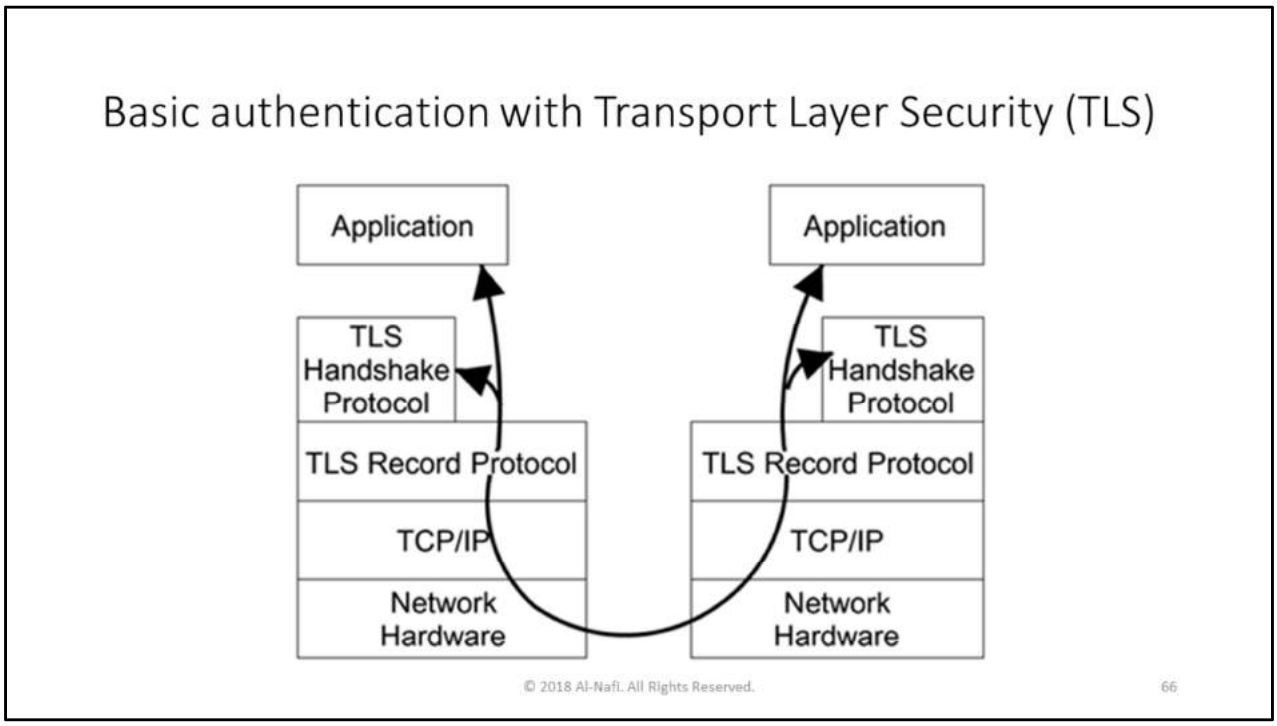
REST and the IoT

Given the near ubiquity of REST APIs and the explosive number of devices on the Internet of Things (IoT), it seems to be a perfect pairing. Compact formats based on JSON, EXI and CBOR (Concise Binary Object Representation), a JSON offshoot, are used and RESTful APIs are likewise compact.

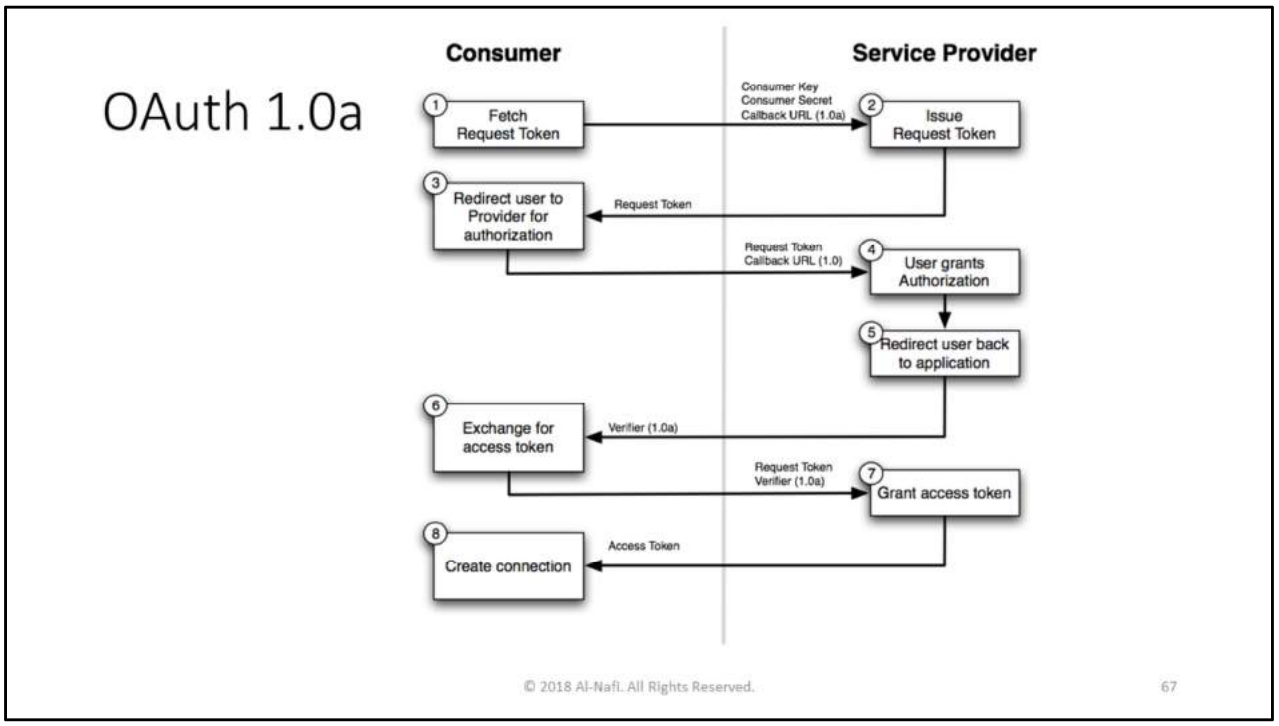
Authentication Options

- Basic authentication with Transport Layer Security (TLS)
- OAuth 1.0a
- OAuth 2

© 2018 Al-Nafi. All Rights Reserved.



Basic authentication is the easiest of the three to implement because the majority of the time, it can be implemented without additional libraries. Everything that is needed to implement basic authentication is usually included in standard framework or language library. The problem with basic authentication is that it is basic, and it offers only the absolute lowest security options of the available common protocols, so depending on requirement, it may not be enough as there are no advanced options for using this protocol. Recommendations are that basic authentication should never be used without Transport Layer Security (TLS) (formerly known as SSL) encryption because the username and password combination can easily be deduced otherwise.



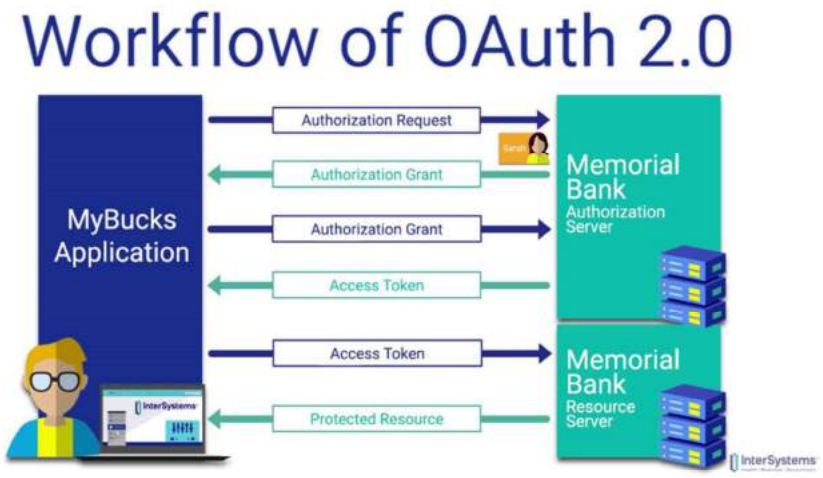
Reference https://oauth.net/core/1.0a/

You can read the above link to get a clear understanding on how Oauth works. For CISSP exam sake you need to understand how OAuth 1.0 really works. But if you want to become a network and web application pentester, then you should read the above link. Remember every protocol which is secure now can always be compromised or is waiting to be compromised. And that's what we want you to think and strive for.

OAuth 1.0a is the most secure of the three common protocols. The protocol uses a cryptographic signature that is usually HMAC-SHA1 value that combines the token secret, nonce, and other request-based security information. The great advantage of OAuth 1 is that the token secret is never sent across the wire, which completely eliminates the possibility of anyone seeing the password while in transit. This is the only one of the three protocols that can be safely used without TLS, although recommendations are always that TLS should be used based on the sensitivity of the information being transferred. However, as with any level of increased security, it usually demands a price. The price is that generating and validating signatures can become a complex process. What needs to be used are specific algorithms and a considerable set of procedures that need to be followed. However, as levels of

security have become much more needed, this issue has really disappeared as every major programming language now has a library to handle this type of activity.

OAuth 2



© 2018 Al-Nafi. All Rights Reserved. 68

This is the next evolution of what was discussed above, and OAuth 2's current specification removes signatures so there is no requirement to use cryptographic algorithms to create, generate, and validate signatures. All the encryption is now handled by TLS, which is a requirement. As a drawback, however, there may not be as many OAuth 2 libraries as there are OAuth 1a libraries, so integrating this protocol into your API may be more challenging. However, this is changing rapidly. Also useful, could be the use of a solution such as Key Management Interoperability Protocol (KMIP) V1.1. Client certificates.

Open Web Application Security Project (OWASP) REST Security Cheat Sheet



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.owasp.org/index.php/REST_Security_Cheat_Sheet

You can read the above link as it is recommended. We will cover all the key details during our network and web application hacking classes inshAllah.

Secure Coding Practices - Reference Monitors

Computer System Security and Management
Reference Monitors

Hardware security features
Hardware is the lowest layer in an IT architecture:



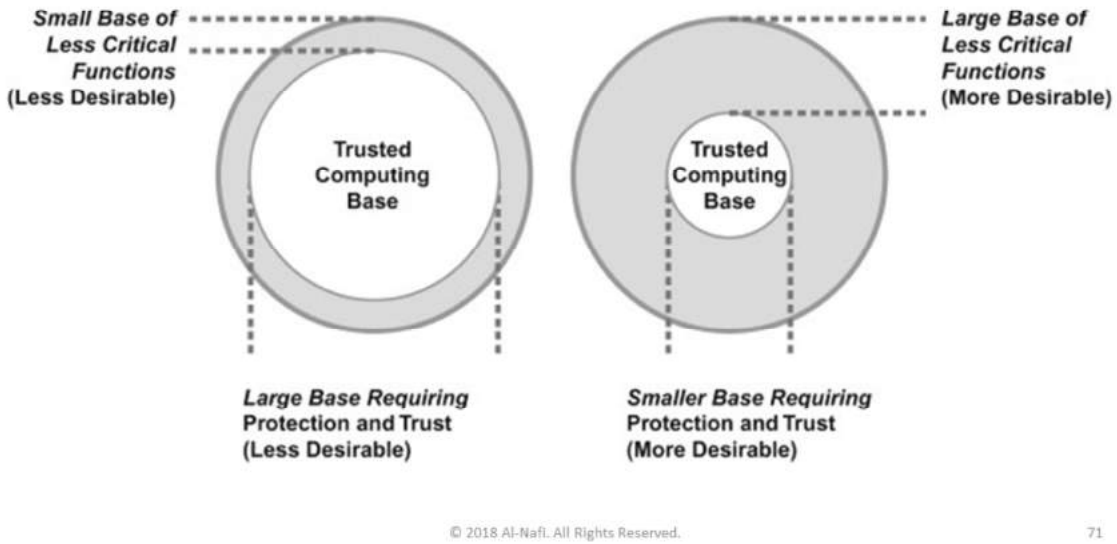
Protection in the Security Kernel

- It may be possible to evaluate security to a higher level of assurance
- Putting security mechanisms into the core of the system increase the performance. No overheads caused by security checks.
- Access control decisions are removed from applications.

© 2018 Al-Nafi. All Rights Reserved.

Secure coding practices can be referred to as developing software with the focus on securing against known and possibly vulnerabilities that may exist in the environment the applications will be running in. There are many vulnerabilities that exist and addressing them all would be impossible. However, if development of code follows a culture of focusing on security, many of the vulnerabilities can be effectively mitigated. The requirement is for these secure coding practices to be integrated into the SDLC, and coding mitigating controls as the applications are being written is an effective way of dealing with many vulnerabilities.

Secure Coding Practices - Trusted Computing Bases (TCBs)



© 2018 Al-Nafi. All Rights Reserved.

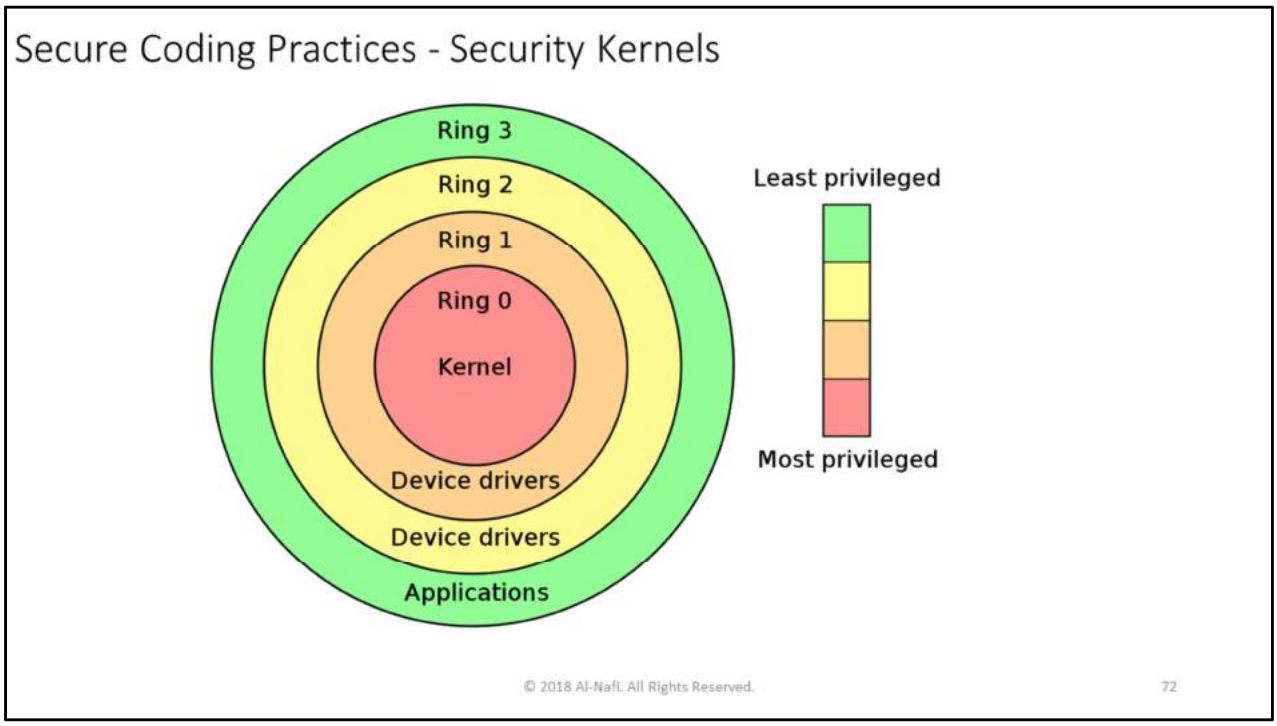
Reference https://www.sciencedirect.com/topics/computer-science/trusted-computing-base

Read the above research paper for added knowledge. Otherwise below para is good enough.

But we recommend that you read the research paper so that you have the mind set to think like an expert inshAllah.

The trusted computing base (TCB) is the collection of all the hardware, software, and firmware components within an architecture that are specifically responsible for security. The TCB is a term that is usually associated with security kernels and the reference monitor. The TCB is the collection of all of the hardware, software, and firmware within a computer system that contains all elements of the system responsible for supporting the security policy and the isolation of objects. When designed and coded properly, all of the security features within a system becomes the TCB, and therefore, can support adequate security requirements. Again, if designed and developed properly, the TCB can contain a very good trusted path

(secure methods to gain access) and a trusted shell (the environment supporting the security is secure). The trusted path is a communication channel between the user or program and the TCB. The TCB is responsible for providing the protection mechanisms necessary to ensure that the trusted path cannot be compromised in any way. The trusted shell implies that any activity taking place within the shell, or communication channel, is isolated to that channel and cannot be interacted with either from inside or outside by an untrusted party or entity.

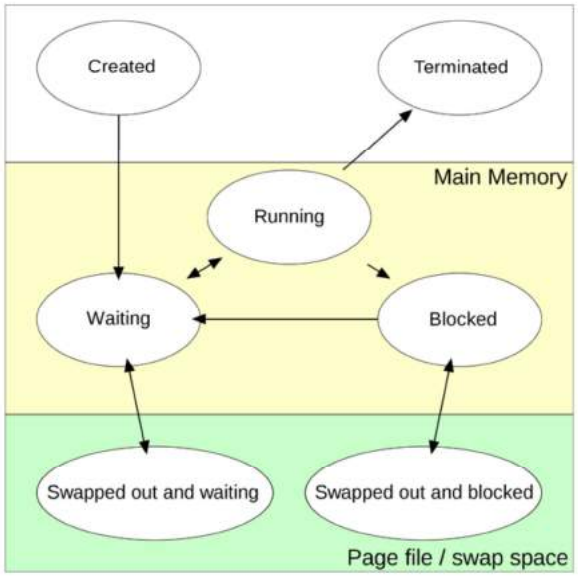


The security kernel, as mentioned above, is the implementation of the reference monitor concept. It is made up of all of the components of the TCB (the software, hardware, and firmware), and it is responsible for implementing and enforcing the reference monitor idea. A security kernel is responsible for enforcing the security policy. It must be a strict implementation of a reference monitor mechanism. The architecture of a kernel operating system is typically layered, and the kernel should be at the lowest and most primitive level. It is a small portion of the operating system through which all references to information and all changes to authorizations must pass. The kernel implements access control and information flow control between implemented objects according to the security policy. To be implemented properly and securely, the security kernel must meet three basic fundamental requirements:

- Completeness: All accesses to information must go through the kernel.
- Isolation: The kernel itself must be protected from any type of unauthorized access.
- Verifiability: The kernel must be proven to meet design specifications.

To address confidence and assurance of security capabilities of the components that make up the TCB, there are various measurement systems that can be used to verify the level of security capabilities. These measurement systems are called evaluation criteria. A number of them exist such as the Trusted Computer System Evaluation Criteria (TCSEC) and the current Common Criteria standards.

Secure Coding Practices - Processor Privilege States



© 2018 Al-Nafi. All Rights Reserved.

Read the article for added knowledge https://en.wikipedia.org/wiki/Process_state

It is highly recommended as we will covering this topic in greater detail during our stage 2 and stage 3 of the CODS. During pentesting, forensics and threat hunting inshAllah. Especially after the compromises and hacks of intel CPU recently, we will be working extensively in various projects to exploit the Processors privilege states inshAllah in CODS advance track.

Limiting processors so they can only do certain activities and capabilities can be a security control. This can be referred to as privilege states. The processor privilege states protect the processor and the activities it can perform. The earliest method of doing this was to record the processor state in a
register that could only be altered when the processor was operating in a privileged state. Instructions such as I/O requests were designed to include a reference to this register. If the register was not in a privileged state, the instructions were aborted and not performed.

The hardware itself typically controls entry into the privilege mode. For example, there are certain newer processors that prevent system code and data from being

overwritten. The idea is to have the privilege level mechanism prevent memory access by programs or data from less privileged to more privileged levels, but only if the controls are invoked and properly managed in software. In other words, hardware and software can work together to allow privileged access through processor states. The privileged levels are typically referenced in a ring architecture.

As an example, many operating systems use two processor access modes:

- User state, or sometimes referred to as problem state
- Supervisor state, or sometimes referred to as kernel state

Normal user applications should run in user mode, and operating system functions will run in supervisor mode. The privileged processor mode is called kernel mode. The kernel mode allows the
processor access to all system memory, resources, and CPU instructions appropriately. Applications should run in a non-privileged mode or what is referred to as user, or problem, state and have a limited set of capabilities, limited access to system data, and denied direct access to hardware resources. Advantages of this architecture is that problematic application software cannot disrupt the system ability to function properly. One of the major challenges of modern processing is that operating systems and applications may be most effective if run in supervisor or kernel mode at all times. Here's an example, when a user mode program calls a system service, such as reading a document from storage, the processor intercepts the call and switches the calling request to supervisor mode. When the operation is complete, the operating system switches the mode
back to user mode and allows the user mode program to continue. Earlier, we mentioned that many of these architectures are set up in ring architecture. Under the most secure operating policy, the
operating system and device drivers operate at ring level 0, also known as kernel-level or system-level privilege. At this privilege level, there are no restrictions on what a program can do. Because
programs at this level have unlimited access, security professionals should be concerned about the source of device drivers for machines that contain sensitive information. Applications and services should operate at ring level 3, also known as user-level or application-level privilege. Operating system code runs in kernel mode, it is critical that kernel mode components be carefully designed to ensure they do not violate security features. For example, if a system administrator installs a third-party device driver, it operates in kernel mode and then has access to all operating system data.
Here's the importance of understanding the security ramifications of this type of architecture. If the device driver installation software also contains malicious code, that code will also be installed and could open the system to unauthorized accesses

as a result.

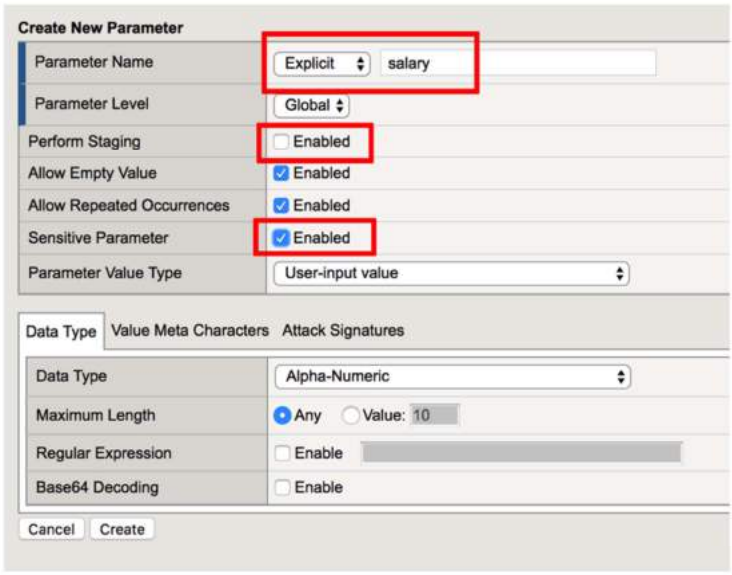
Secure Coding Practices - Security Controls for Buffer Overflows

Insecure Function	Safe Alternative
strcpy	strlcpy*, strcpy_s*
strcat	strlcat*, strcat_s*
printf/sprintf	snprintf*, sprintf_s*
gets	fgets

© 2018 Al-Nafi. All Rights Reserved.

A common problem with technology architectures is referred to as buffer overflows. This is where an application has been subjected to much more information than its buffer can handle. The problem is inadequate bounds checking, or ineffective parameter checking, which may lead to buffer overflows. As we've said, a buffer overflow is caused by improper bounds checking on input to an application. Essentially, the program fails to see if too much data is provided for an allocated space of memory, referred to as a buffer. In order to run, programs need to be loaded into memory, but if there is an overflow the data has to go somewhere. If the attack has been done creatively, that data could be malicious code that is loaded, and as a result, it may run as if it were the program itself, allowing exploits by an attacker. Buffer overflows must be corrected by developers or by directly patching fixes. Sometimes, they may be detected by reverse engineering the application's code, also referred to as disassembling programs, and looking at the actual operations of the application itself. The fix to buffer overflows is to patch for known buffer overflow conditions and also to enforce proper bounds checking and enforcement, and in some cases, proper error checking.

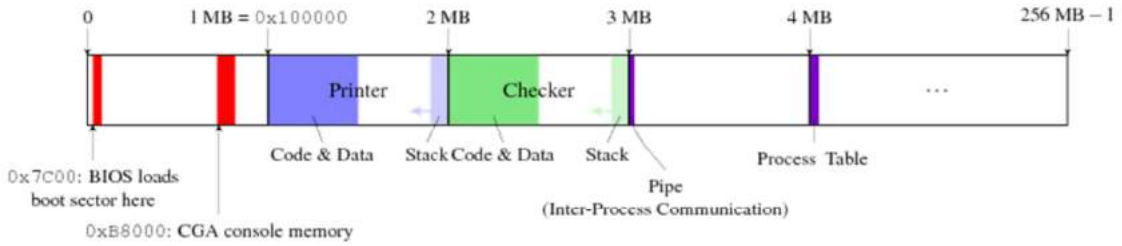
Secure Coding Practices - Controls for Incomplete Parameter Check and Enforcement



© 2018 Al-Nafi. All Rights Reserved.

Another security risk exists when all parameters, such as input, have not been fully checked for accuracy and consistency by the systems. This lack of parameter checking can lead to many attacks including buffer overflow attacks. To counter this vulnerability, systems can include some type of buffer bounds controls. Complete and effective parameter checking is something that needs to be designed, coded, and implemented by the developers and involves checking the input data to make sure the program does not allow unwanted characters, length, data types, and formats. This may be referred to as proper input data validation.

Secure Coding Practices - Process Isolation and Memory Protection

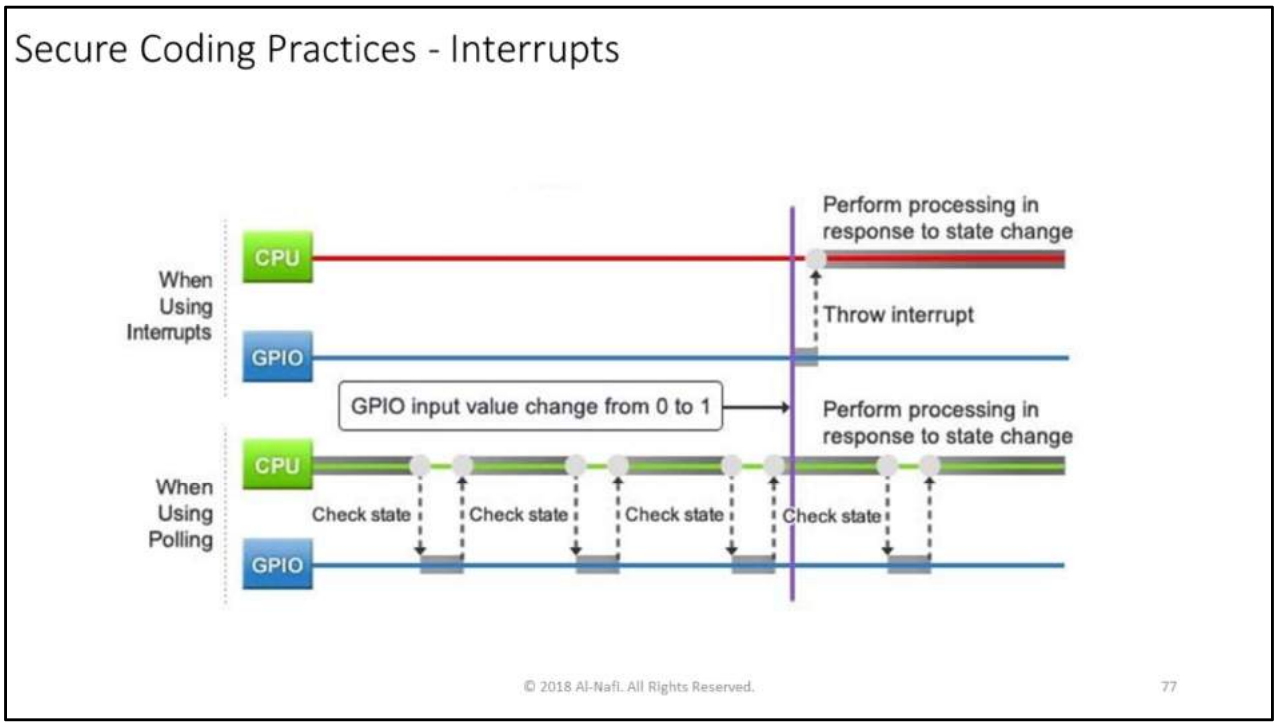


© 2018 Al-Nafi. All Rights Reserved.

Computer architectures today are multitasking. This means they can host multiple processes that are running at the same time. A process is defined as part of a computer program that is being executed in memory. Multitasking architectures today are capable of running multiple processes at the same time. For these different processes to run at the same time, they must be managed in such a way that they are able to access resources as needed, but at the same time do so without impacting any of the other processes that are running at the same time. This can become very complicated at times as processes can share memory, data, and system resources all at the same time. In other words, they may be contending for system resources all at the same time, while trying to complete their tasks.

To maintain the integrity of the operating system and of each of the processes and the data that is being accessed, it is important that accesses to resources is managed properly at all times. This requires the processes to be isolated from each other. This, very appropriately, is called process isolation. This need to isolate processes from one another within the computer architecture has to be managed to ensure that it is happening effectively and thoroughly, without exceptions and problems. This is handled by the operating system. It is the

operating system that takes care of process isolation, but it needs to partner with the CPU to enforce the process isolation through the use of interrupts and time slicing.



Reference https://www.renesas.com/us/en/support/technical-resources/engineer-school/mcu-programming-peripherals-04-interrupts.html

What Is an Interrupt?

We introduced the concept of interrupts in the second session of this series, in our discussion of timers. Consider now a similar analogy to illustrate how things might work without an interrupt: if you are boiling eggs, and you want to take them off the stove in 10 minutes, then one way to do it is to check the clock every now and then to see if the time is up. The same is true in embedded systems: if you want to wait for a specific state change to occur before doing some operation, then one way to do this is to periodically check the state. Or again, if your program is waiting for a GPIO input level; to change from 0 to 1 before executing some step, then one way to proceed is to periodically check the GPIO value. This approach—periodic checking—is referred to as polling.

While polling is a simple way to check for state changes, there's a cost. If the checking interval is too long, there can be a long lag between occurrence and detection—and you may miss the change completely, if the state changes back before you check. A shorter interval will get faster and more reliable detection, but

also consumes much more processing time and power, since many more checks will come back negative.

An alternative approach is to utilize interrupts. With this method, the state change generates an interrupt signal that causes the CPU to suspend its current operation (and save its current state), then execute the processing associated with the interrupt, and then restore its previous state and resume where it left off.

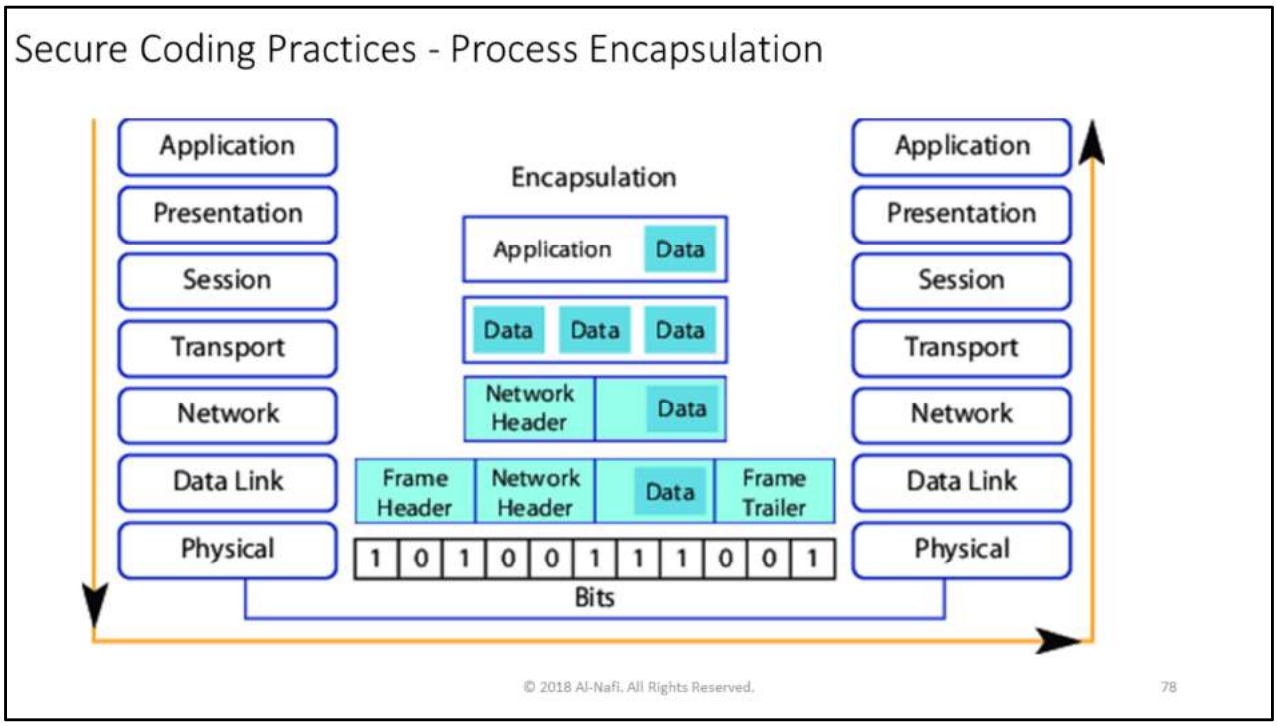
How an MCU Processes Interrupts

Interrupts can originate from both MCU-internal and MCU-external devices. An interrupt from an external switch or sensor, for example, is sometimes called "attached interrupt", as it is generated by an external device that is attached to an IRQ (interrupt request) pin on the MCU. When the relevant state change occurs, the external device sends an interrupt request signal to this pin, and this in turn generates a notification to the MCU's interrupt controller (on the RX63N, this controller is called the "ICUb").

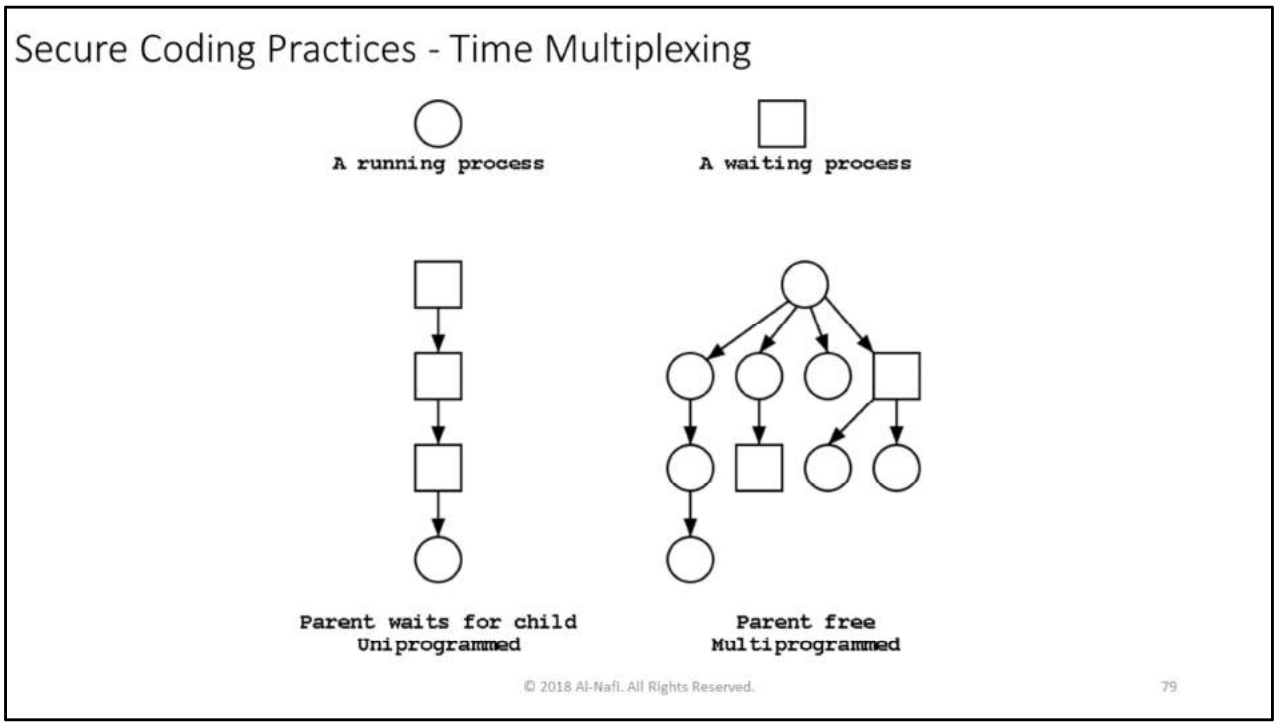
In contrast, interrupts from on-chip peripherals—internal timers, GPIO lines, UARTs, etc.—are referred to as "peripheral interrupts." These interrupt signals generate direct notification to the interrupt controller, with no need for pin attachments.

The interrupt controller's job is to pass these interrupt requests to the CPU in a coordinated way. When multiple interrupts occur, the controller must send these to the CPU in the appropriate order, based on their relative priorities. And the controller must also be aware of which interrupts are currently masked (disabled), so that it can ignore these interruptions completely.

When the CPU receives an interrupt request from the controller, it stops executing the program it is working on, and automatically saves all of the relevant working information so that it can later resume from where it left off. It then loads and executes the interrupt processing program that corresponds with the interrupt request that it received. After completing this processing, the CPU restores the saved information and resumes from where it stopped. (See Figure 2) Note that saving and resuming are handled automatically by the CPU; programmers need not concern themselves with these details.



Encapsulating a process means that you isolate that process so that no other process is able to see, understand, or interact with the internal functions of the process itself. This act of encapsulating forces processes to interact with each other through well-defined interfaces that can be overseen and managed by the operating system properly. Encapsulation effectively hides the process and its functions from other processes, thereby allowing it to engage in data hiding. Data hiding is what it sounds like, hiding data from other processes so that each of the processes running at the same time do not interfere with each other.



Reference https://cs.nyu.edu/courses/spring09/V22.0202-002/lectures/lecture-02.html

Please read the research paper via the above url to have a good understanding.

Time multiplexing allows the operating system to provide structured access by processes to resources according to a controlled and tightly managed time schedule. This schedule is defined as a short period of time, or a time slice, which will grant access to the system resources required by the process and then terminate that access once the time period has expired. That resource then becomes available to another process, again based on a time slice. Multitasking and multi-processor architectures that are common today create an additional layer of performance but also complexity with regard to time slicing or multiplexing. Due to the fact that each CPU in a computer can have more than one core, or more than one processor, the ability for the computer to process multiple requests for access to resources from processes simultaneously continues to increase, and therefore, needs to be managed properly. This is referred to as multitasking.

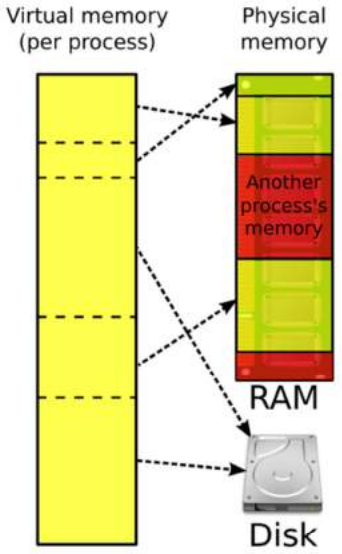
Secure Coding Practices - Naming Distinctions



© 2018 Al-Nafi. All Rights Reserved.

Naming distinctions are used to ensure that each process is assigned a unique identity within the context of the operating system and its architecture. This means that each process will be given a unique name and Process ID, or PID, ensuring that when it is referenced by the operating system, there is no confusion as to which process is being accessed by which resources. This allows all processes to be referenced properly as they execute their tasks.

Secure Coding Practices - Virtual Address Memory Mapping



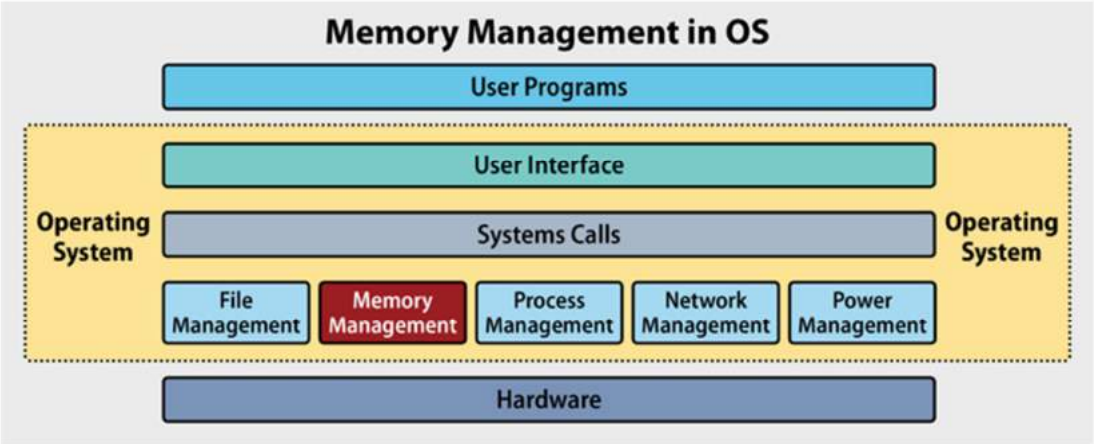
© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Virtual_memory

In computing, **virtual memory** (also **virtual storage**) is a memory management technique that provides an "idealized abstraction of the storage resources that are actually available on a given machine"[1] which "creates the illusion to users of a very large (main) memory."[2]
The computer's operating system, using a combination of hardware and software, maps memory addresses used by a program, called *virtual addresses*, into *physical addresses* in computer memory. Main storage, as seen by a process or task, appears as a contiguous address space or collection of contiguous segments. The operating system manages virtual address spaces and the assignment of real memory to virtual memory. Address translation hardware in the CPU, often referred to as a memory management unit or *MMU*, automatically translates virtual addresses to physical addresses. Software within the operating system may extend these capabilities to provide a virtual address space that can exceed the capacity of real memory and thus reference more memory than is physically present in the computer.

The primary benefits of virtual memory include freeing applications from having to manage a shared memory space, increased security due to memory isolation, and being able to conceptually use more memory than might be physically available, using the technique of paging.

Secure Coding Practices - Memory Management



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.enterprisestorageforum.com/storage-hardware/memory-management.html

Since the beginning of modern IT, the challenge of memory exhaustion has been handled by a diverse set of capabilities, usually grouped under the heading of memory management.

The reality of most compute and storage deployments is that all types of computer memory are constrained by an upper limit.

No resource on a modern system is perhaps as constrained as memory, which is always needed by operating systems, applications and storage. Without unlimited memory, at some point memory is fully consumed, which leads to system instability or data loss.

Since the beginning of modern IT, the challenge of memory exhaustion has been handled by a diverse set of capabilities, usually grouped under the heading of *memory management*.

What Is Memory Management?
Memory management is all about making sure there is as much available memory space as possible for new programs, data and processes to execute. As memory is used by multiple parts of a modern system, memory allocation and memory management can take on different forms.

Operating System - Operating systems like Microsoft Windows and Linux, can make use of physical RAM as well as hard drive swap space to manage a total pool of available memory.
Programming Languages - The C programming language requires developers to directly manage memory utilization, while other languages like Java and C#, for example, provide automatic memory management.
Applications - Applications consume and manage memory, but are often limited in memory management capabilities as defined by the underlying language and operating system.
Storage memory management - With new NVME storage drives, operating systems can benefit from faster storage drives to help expand and enable more persistent forms of memory management.
To be effective, a computer's memory management function must sit between the hardware and the operating system.

How Memory Management Works
Memory management is all about allocation and optimization of finite physical resources. Memory is not uniform - for example a 2GB RAM DIMM is not used as one large chunk of space. Rather memory allocation techniques are used for segmentation of RAM into usable blocks of memory cache.

Memory management strategies within an operating system or application typically involve an understanding of what physical address space is available in RAM and performing memory allocation to properly place, move and remove processes from memory address space.

Types of Memory Addresses
Static and dynamic memory allocation in an operating system is linked to different types of memory addresses. Fundamentally, there are two core types of memory addresses:
Physical addresses - The physical address is the memory location within system RAM and is identified as a set of digits.
Logical addresses - Also sometimes referred to as virtual memory, a logical address is

what operating systems and applications access to execute code, as an abstraction of physical address space.

How does MMU convert virtual address to physical address?
The Memory Management Unit (MMU) within a computing system is the core hardware component that translates virtual logical address space to physical addresses. The MMU is typically a physical piece of hardware and is sometimes referred to as a Paged Memory Management Unit (PMMU).
The process by which the MMU converts a virtual address to a physical address is referred to as virtual address translation and makes use of a Page Directory Pointer Table (PDPT) to convert one address type to another.

The process is directly tied to page table allocation, matching and managing one address type to another. To help accelerate virtual address translation there is a caching mechanism known as the Translation Lookaside Buffer (TLB) which is also part of the virtual address to physical address translation process.

Memory Allocation: Static vs. Dynamic Loading
Applications and data can be loaded into memory in a number of different ways, with the two core approaches being static and dynamic loading.
Static Loading - Code is loaded into memory, before it is executed. Used in structured programming languages including C.
Dynamic Loading - Code is loaded into memory as needed. Used in object oriented programming languages, such as Java.
Memory Fragmentation
When memory is allocated in a system, not all of the available is always consumed in a linear manner, which can lead to fragmentation. There are two core types of memory fragmentation, internal and external
Internal Fragmentation - Memory is allocated to a process or application and isn't used, leaving un-allocated or fragmented memory.
External Fragmentation - As memory is allocated and then deallocated, there can be small spaces of memory leftover, leaving memory holes or "fragments" that aren't suitable for other processes.
Paging
Within logical address space, virtual memory is divided up using *paging*, meaning it's divided into fixed units of memory, referred to as *pages*. Pages can have different sizes, depending on the underlying system architecture and operating system The process of page table management can be intricate and complex.
For more information on how paging is handled in Linux, check out the full kernel.org documentation.
On Windows, Microsoft provides details on its paging process here.

Segmentation

Memory segmentation within the primary memory of a system is a complicated process that references specific bits within a memory unit.

Each segment within system memory gets its own address, in an effort to improve optimization and memory allocation. Segment registers are the primary mechanism by which modern systems handle memory segmentation.

Swapping

Swapping is the process by which addition memory is claimed by an operating system from a storage device.

How swapping works is an operating system defines an area of storage that is used as "swap space," that is storage space where memory process will be stored and run as physical and virtual memory space is exhausted, released and reclaimed. The usage of swap space with traditional storage is a sub-optimal way of expanding available memory as it incurs the overhead of transferring to and from physical RAM. Additionally, traditional storage devices run with slower interface speeds that RAM. Swapping however is now being revisited as a way to expand memory with the emergence of faster, PCIe SSDs which offer an interface connect speed of up to 16 Gb/s. In contrast a SATA connected SSD has a maximum connection speed of 6.0 Gb/s.

Why do We Need Memory Management?

Memory management is an essential element of all modern computing systems. With the continued used of virtualization and the need to optimize resource utilization, memory is constantly being allocated, removed, segmented, used and re-used. With memory management techniques, memory management errors, that can lead to system and application instability and failures can be mitigated

Advantages

Maximizes the availability of memory to programs

Enables re-use and reclamation of memory that is not actively in use

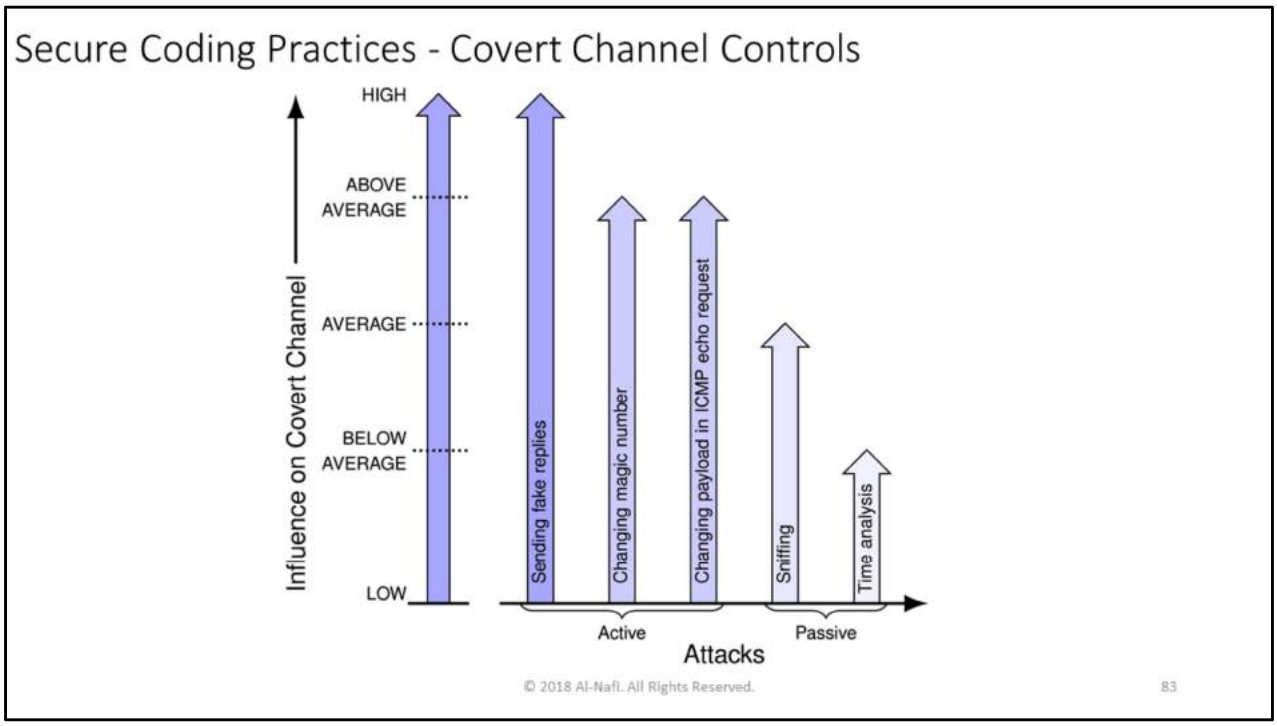
Can help to extend available physical memory with swapping

Disadvantages

Can lead to fragmentation of memory resources

Adds complexity to system operations

Introduces potential performance latency



Reference https://onlinelibrary.wiley.com/doi/full/10.1002/sec.1471

Read this as we will be working on covert channel controls and attacks during stage 2 and 3 inshAllah within CODS.

Covert channel-internal control protocols: attacks and defense

Abstract

Network covert channels have become a sophisticated means for transferring hidden information over the network. Covert channel-internal control protocols, also called *micro protocols*, have been introduced in the recent years to enhance capabilities of the network covert channels. Micro protocols are usually placed within the hidden bits of a covert channel's payload and enable features such as reliable data transfer, session management, and dynamic routing for network covert channels. These features provide adaptive and stealthy covert communication channels. Some of the micro protocol based tools exhibit vulnerabilities and are susceptible to attacks. In this paper, we demonstrate some possible attacks on micro protocols, which are capable of breaking the sophisticated covert channel communication or jeopardizing the identity of peers in such a network. These

attacks are based on the attacker's interaction with the micro protocol. We also present the defense techniques to safeguard micro protocols against such attacks. By using these techniques, micro protocol-based tools can become immune to certain attacks and lead to robust covert communication. We present our results for two micro protocol-based tools: *Ping Tunnel* and *smart covert channel tool*. Copyright © 2016 John Wiley & Sons, Ltd.

1 Introduction

Security is an important concern in the network communication. There are several vulnerabilities present in the design of networks and network protocols. Illegitimate users take advantage of such vulnerabilities and sometimes use the network for unlawful purposes. Network covert channels are hidden and potentially policy-breaking communication channels originally introduced as unforeseen communication channels by Lampson [1]. These channels are created by methods of information hiding – a discipline known to be used for censorship circumvention, malware communication, and exchange of secret information by terrorists and spies, just to mention a few scenarios [2-4]. The purpose of using network covert channels is to transfer information over the network while ensuring that the transfer is not recognized by a third party. The detection of a network covert channel is generally made difficult by using unobtrusive characteristics of the communication medium to signal hidden information.

These channels can take advantage of various environments for hiding the data, for example, hiding it in the Open Systems Interconnection (OSI) network model, Transmission Control Protocol/Internet Protocol (TCP/IP) protocol suite, and LAN environment. As described in [5], network covert channels can be classified into two different types:

Network covert storage channels embed hidden data into bits of the transferred network packets. Reserved bits or unused bits of the protocol are mainly used to transmit information. For example, DSCP bits in IP protocol.

Network covert timing channels signal hidden information through the timing information of network packets. Normally, the timing difference between network packets is used to deliver a covert message.

Covert channel-internal control protocols, also called *micro protocols*, were introduced to integrate various capabilities into covert channels such as reliable data transfer and dynamic overlay routing. The fundamental designing process for micro protocols is performed in the same way as for other communication protocols. However, a micro protocol header is placed within the hidden data transferred by a covert channel and thus size-constrained [6]. In other words, a micro protocol header contains only a few bits, and these bits are embedded in a stealthy manner into the underlying network protocol. Therefore, micro protocol-specific protocol engineering methods enhance the existing protocol engineering techniques in the design process. Micro protocols have several uses, good as well as bad [7]. For instance, the micro

protocols could help journalists to transfer illicit information over the network, as they give them the ability of hiding transferred data in an improved manner (e.g., by splitting the payload over multiple simultaneous channels). At the same time, they also provide reliable data transfer over all the channels used in covert communication. They can additionally support dynamic overlay routing in order to allow journalists to bypass critical censorship infrastructure on a global routing path. In this way, journalists can gain the ability to express their opinion freely and do not have to undergo legal restrictions.

On the other hand, the micro protocols can be exploited by illegitimate network users who can use the network covert channels as stealthy botnet command and control channels. If a botnet developer implements a network covert channel with a micro protocol, such a channel could signal commands and configuration messages between the bots and the bot master, ensuring stealthiness of the botnet's communication.

Considering this dual nature of micro protocols, we present possible attacks and defense techniques for micro protocol-based communications. The attacks are based on the idea that an attacker is able to manipulate the micro protocol and influence its communication channel. That is, he interacts with the micro protocol to counter its communication (e.g., implementing a man-in-the-middle attack). The possibility of covert channels and micro protocols cannot be completely eliminated, although it is possible to significantly reduce their presence by careful design and analysis. Certain attacks could be performed on micro protocols by sending fake protocol complying commands or fake acknowledgments. Defense techniques proposed in this paper are a set of improvement ideas which would make the micro protocol structures more secure.

Overall, this paper presents a comprehensive study of possible attacks and defense techniques for micro protocol-based tools. Our specific contributions are:

Analysis of existing micro protocol designs.

Discussion and demonstration of possible attacks on micro protocol-based tools.

Design of defense techniques to safeguard micro protocols against such attacks.

Evaluation of attacks and defense techniques on the covert channel network.

Anticipation of further developments which could lead to better micro protocol designs.

The remainder of this paper is structured as follows. Section 2 puts our work in the context of the existing research and is followed by Section 3 which provides an insight into micro protocols and attack classification. Section 4 describes the attacks on micro protocols and demonstrates these attacks on a peer-to-peer micro protocol and a dynamic overlay routing micro protocol. Section 5 presents the improved micro protocol designs in the form of defense techniques. The impact of attacks and defense is discussed in Section 6. Anticipated improvements for micro protocol designs are presented in Section 7. The conclusions and future work are given in

Section 8.

2 Related Work

Wendzel and Keller published a survey on micro protocols in 7. The authors cover the general functionality spectrum of micro protocols, capabilities of existing micro protocols, and protocol engineering means to optimize the micro protocol design. Moreover, the survey mentions two aspects which are significant for our work, namely, the *drawbacks* in the design of existing micro protocols and the *future research* to be conducted in this area.

The important drawback mentioned in 7 is that micro protocols accept out of band data, that is, non covert input that looks like covert input and is thus interpreted. The survey also highlights a clear lack of countermeasures for micro protocols and therefore the authors suggest developing them in future work.

Few implementations of micro protocols, such as Ping Tunnel 8, uses an indicator called *magic number* to indicate the presence of covert data. The packet is only considered to be part of a covert communication if the magic number equals a given value.

Mazurczyk and Kotulski presented the only known micro protocol that comprises support to evaluate the authenticity of packets 9. All other known micro protocols do not encompass such features, and they might treat non-covert input as covert input.

Similar work can be found in the area of command and control (C&C) protocols for botnets. Malware analysts aim at inserting fake commands into the C&C traffic of botnets and reverse engineering of these protocols. We expect that some C&C-related research can be applied to the world of network covert channels and vice versa. A detailed discussion of the differences between C&C protocols and micro protocols is available in 7. While C&C protocols might transfer encrypted traffic, the challenge for developing C&C countermeasures differs from that of micro protocol countermeasures as the major goal of a C&C protocol is to keep the *content* of a transmission secret, while a micro protocol's goal is to keep the communication *itself* secret. Hence, when blindly applying a countermeasure to such a *hidden* micro protocol-based communication, it may not always be clear whether the countermeasure is even applied to a covert channel or not.

There is some preliminary research conducted in the area of improving micro protocol structures so that they become less prone to vulnerabilities leading to attacks. In 10, the authors introduced an approach for the engineering of micro protocols. They defined a terminology for designing micro protocols in which the utilized protocol is called the *underlying protocol*; the area in which the covert data are embedded within the underlying protocol is called the *cover protocol*. The micro protocol with its payload is placed in the cover protocol. Their six-step engineering approach provides standard conforming, and minimal attention raising design of micro protocols, leading to improved stealthiness.

Another concept is *status updates,* which serve to minimize the traffic generated to

transfer the micro protocols. To achieve this, the header elements of micro protocols are designed in a dynamic way so that they must only be transferred if they are updated (e.g., if a destination address field must be set to a new destination address). Status updates were introduced in a micro protocol-based tool named smart covert channel tool (SCCT) 11. It is an optimized control protocol that allows the configuration of dynamic routing in covert channel overlay networks. It is implemented to achieve high stealthiness for the propagation of routing information. Different routing messages like requesting peer tables, sending peer tables, transferring topology graphs, and performing routing updates are implemented in this tool. The current implementation of this tool is based on User Datagram Protocol, but it is designed to be integrated into several network protocols, which can be utilized by software architecture of SCCT. We design one of our defense techniques for this tool, making the tool more secure.

3 Micro protocols and attack models

A micro protocol (MP) is a communication protocol whose header is placed within the hidden data transferred by a covert channel. Because of fewer possibilities for placing hidden data in a network covert channel, very limited space is available for the micro protocols. For this reason, a micro protocol structure should contain a few bits to fit itself in the limited space provided by the hidden data.

According to 7, the existing micro protocols are not optimized for a covert channel environment as their protocol headers consume more bits than required, and their behavior is not adjusted to the cover protocol. Because of this, micro protocols sometimes fail to place themselves optimally within the covert data. More anomalies are likely to occur because of the manipulation of bits, leading to easier detection of the covert channel. Another type of anomaly could be caused if the micro protocol header is not placed within the hidden data in an ideal way, that is, functionality of the underlying protocol could fail and raise attention of the users in that case 6. One example for such a case could be uncommon bit/flag combinations in network packets.

3.1 Attack classification

The attacks on micro protocols can be classified in two ways based on (i) their characteristics and (ii) attacker's knowledge about the micro protocol, as shown in Figure 1.

Figure 1 Open in figure viewerPowerPoint

Classification of attacks.

The characteristics-based classification is analogous to the covert channel countermeasure classification by Zander *et al.* 2; it categorizes the general behavior of attacks and is listed in the succeeding discussion:

preventing the covert communication containing micro protocols;
limiting the functionality of the micro protocol; or
observing the hidden communication, including its attributes.

Based on the attacker's knowledge about the micro protocol, attacks can be classified into four types (as shown in Table 1 and Figure 1).

Table 1. Attacker's knowledge about micro protocols**Type of KnowledgeCase ICase IICase IIICase IV**Presence of MP✓✓✗✗Syntax/Semantics of MP✓✗✓✗

MP, micro protocol.

In *Case I*, the attacker is fully aware of both where the micro protocol is placed and how its header structure is designed. The header structure is available for analysis, and the attacker could propose attacks by identifying the weaknesses in its design. In *Case II*, the attacker is only aware of the presence of a micro protocol-based communication but possesses no knowledge about its header structure. In certain scenarios, because of (unlikely) presence of insider's information, it is possible to manipulate such covert communication by transferring random data as *noise* to a covert channel without knowing its effect on the communication channel. It is also possible to monitor the data thoroughly on such a covert channel and decide where exactly the noise could be inserted. In *Case III*, the attacker is not aware of the presence of micro protocols but could predict the header structure design in the event of its presence. In such a case, the attacker would be able to perform a blind attack by sending falsifying commands to a system potentially involved in the covert communication (e.g., fake commands to terminate the covert communication). In the last case (*Case IV*), the attacker is unaware about both presence and structure of the micro protocol. Techniques like traffic normalization, which applies rules such as clearing unused header bits or setting header fields to default values 12, are an efficient approach for breaking such covert channels.

3.2 Levels of attack

Depending on a given micro protocol scenario, multiple characteristics-based attacks can be applied. In this work, we consider Case I (as shown in Table 1) in which we know where a micro protocol is placed and how its header structure is designed. We define attacks by analyzing structures and functioning of two micro protocol-based tool implementations: *Ping Tunnel* 8 and SCCT 11. The analysis is performed to find vulnerabilities and weaknesses in these micro protocol-based designs. The attacks are further classified as *active* and *passive*. Active attacks influence the covert channel network or its peers in a way that affects the covert communication. These methods are responsible for breaking the channel or limiting the functionality of the channel. On the other hand, passive attacks observe the behavior of the covert channel network or its users to derive the content of the transferred covert data, the throughput, or the participants involved in the hidden communication. This is possible by sniffing the data exchanged over the covert channel and determining the information patterns from the sniffed data. An attacker can observe the hidden communication with the help of these methods and conclude the purpose of a particular communication. Therefore, based on this observation, a suitable decision could be made about the covert channel (e.g., block the covert communication or

allow it to continue). This further leads an attacker to obtain more information about the network and the kind of information being exchanged by the peers.
In a scenario when an attack is performed on the legitimate covert channel where the attacker's goal is to expose useful information, the defense techniques must be used to safeguard against these attacks. We propose some improvement methods to enhance the features and capabilities of the micro protocols. These improved designs are based on the functionalities of micro protocol-based tools and aim at increasing micro protocol's immunity to certain attacks.

3.3 Capabilities of an attacker

We consider that an attacker is capable of interacting with the micro protocol. Based on this, we consider the following capabilities that an attacker can have:
Send packets (normal, fake, or modified) to other peers in the network, possibly triggering abnormal behavior.
Gain access to the network, log traffic, or modify topology of the network.
Launch DoS attacks by disabling connections between the peers after passive analysis of the network.
However, the classification of attacks mentioned in this section is not necessarily a comprehensive enumeration of all possible scenarios. Instead, we define a general context of the possible ways in which an attacker can attack a micro protocol-based communication. We made our assumptions as general as possible and included most of the vulnerabilities present in the micro protocol structure. Based on this hypothesis, we demonstrate attacks in the next section.

4 Potential attacks on micro protocols

As mentioned earlier, we consider two micro protocol-based tool implementations to design the attacks: *Ping Tunnel* and SCCT. Ping Tunnel [8] is designed for peer-to-peer communications under TCP-restricted environments. SCCT [11] is capable of building a hidden overlay network and provides dynamic overlay routing based on optimized link-state routing (OLSR). Both micro protocols were selected because of their dissimilar functionalities. We analyze the structure and behavior of both the micro protocols and categorize attacks for them.

4.1 Peer-to-peer communications micro protocol

Ping Tunnel [8] is based on a micro protocol, which is used to tunnel TCP connections to a remote host using Internet Control Message Protocol (ICMP) echo request and reply packets in a reliable manner. The set up of Ping Tunnel is based on a *client*, a *proxy*, and a *destination* host. Ping Tunnel can be used when a client desires to send TCP packets to a destination (e.g, a browser connects to a web server) but a firewall blocks TCP connections. In this case, the client sends ICMP packets containing TCP data to the proxy, which resides outside the client's network. The proxy establishes a normal TCP connection with the specified destination and sends the response TCP payload back to the client hidden in the ICMP packet.
The client performs all its communication with the proxy using ICMP echo request

packets, whereas the proxy uses ICMP echo reply packets for the same path. A *magic number* is used to differentiate Ping Tunnel's packets from normal ping packets. The packet format used to exchange messages between a client and a proxy is shown in Figure 2.

Figure 2 Open in figure viewerPowerPoint

Ping Tunnel's micro protocol data unit as described in 8.

4.2 Dynamic overlay routing micro protocol

Smart covert channel tool is a *status update*-based micro protocol implementation that establishes multi-hop covert channel routes. A *status update* is a small chunk of data, transmitted through a covert channel specifying a change in at least one setting, for example, changing the destination address in the covert channel 11. The micro protocol's dynamic routing algorithm is based on optimized link-state routing and was extended for providing different optimization means to maximize covertness (*quality of covertness*) and connection quality of a covert channel routing path. It also facilitates routers and proxies by combining *agents* (active participants in the routing process) and *drones* (optional hops that solely forward data as a proxy) in the overlay network. SCCT uses User Datagram Protocol-based tunneling, but in general, the tool is capable of utilizing arbitrary underlying protocols and hiding techniques. The tool maintains a topology table and a peer table at every peer in the overlay network and also synchronizes them among all the peers present in the network.

4.3 Attacks

We categorize the following attacks after analyzing the micro protocol header and the functionality of Ping Tunnel and SCCT:

4.3.1 Sniffing

To sniff ICMP packets sent by **Ping Tunnel**, an attacker could filter these packets from all the received ICMP echo requests and replies using a sniffing tool, for example, **tcpdump** 13 (in our case). Ping Tunnel uses a special four byte magic number to indicate the presence of Ping Tunnel's ICMP echo requests and replies. Hence, an attacker could compare the value of the first four bytes of the received ICMP echo payload with the known value of the magic number and records corresponding packets in the log files. An attacker could perform the following activities with the sniffed Ping Tunnel data:

IP tapping: An attacker could retrieve a list of IP addresses involved in a hidden network communication using the IP field of Ping Tunnel's packet. This helps in obtaining information about the involved parties in the covert communication.

Traffic analysis: An attacker could intercept and examine the data exchanged during the hidden communication. Necessary actions for the parties involved in the covert communication (whether to block or allow the involved parties on the network) would be based on the results of the traffic analysis.

To sniff the packets sent by **SCCT** † , a passive observer acting as an agent could be introduced into the overlay network. The agent could then sniff the information of

peers (IP address, id, etc.) involved in the overlay network and could also retrieve the overlay topology by sniffing routing updates. It is also possible to maintain log files of sniffed information with the timestamps. This information is useful to predict the rate at which the number of involved participants change (increase or decrease) over time in the covert network. In our implementation, the agent is able to log all the information related to the peers and the network topology in a *.dot* file. This allows the generation of network graphs using **Graphviz** 14.

4.3.2 Man-in-the-middle attack

To perform a man-in-the-middle attack on **Ping Tunnel**, an attacker could act as a man-in-the-middle by establishing independent connections with the client and the proxy while relaying messages between them. Doing this would make both the systems believe that they are directly connected to each other. In reality, the entire conversation would be controlled by an attacker. All the messages exchanged between the client and the proxy would be intercepted, resulting in breaking the communication channel. This could be performed as follows:

Case I: Sending fake replies to the client's ICMP echo requests. This would lead to the interpretation of replies at the client side, while it is up to the attacker's decision to forward these messages to the destination, or not.

Case II: Changing the magic number by replacing it with the random data. This would prevent Ping Tunnel from recognizing the packets belonging to the covert communication channel.

Case III: Changing the payload with the random data. This would render the hidden information in the payload useless.

The communication flow during the aforementioned cases is shown in Figure 3. The mechanism is implemented in two steps: In the first step, a *packet redirector* transfers all ICMP packets from the client to the attacker.

The **IPTables** 15 tool used with the PREROUTING/POSTROUTING chain (depending on the desired action) of the mangle table could be used to perform this job. In the next step, the attacker accepts all the incoming packets forwarded by the packet redirector and acts as *content modifier* to accomplish one of the three cases. The modifications could be carried out using **NFQueue** 16 to check the presence of Ping Tunnel's magic number in the payload of received ICMP packets and performs the necessary modifications on the payload. This approach is shown in Figure 4.

Figure 3 Open in figure viewerPowerPoint

Scenarios for attacking Ping Tunnel.

Figure 4 Open in figure viewerPowerPoint

The mechanism applied by the attacker.

To perform a man-in-the-middle attack in the overlay network formed by **SCCT**, a malicious agent could join the network by announcing itself as a new peer to the network. It could therefore send a routing update message to other peers and act as a man-in-the-middle peer (cf. Figure 5). By doing so, the malicious agent is actively

able to participate in the network in order to *redirect* the covert traffic through itself.

Figure 5 Open in figure viewerPowerPoint

A malicious agent in the overlay network.

Traffic redirection is useful as it enables the malicious agent to analyze the content of the traffic and decide whether to modify, block, or forward (with or without a delay) the packet. In our implementation, the malicious agent becomes one of the peers in the overlay network and, as soon as it receives a topology status update, it joins all other peers in the network. The malicious agent establish connections with all other peers by propagating the lowest possible *delay* (metric) on all the overlay network's edges it forms. When any agent in the overlay network sends a covert message to any other agent who is at a distance of more than one hop from itself, the chosen routing path to reach the destination agent would be via the malicious agent. When the message passes through the malicious agent, it could perform the following actions on the message:

Case I: Block the messages from reaching the destination.

Case II: Change the data in the payload of the transmitted messages (replace the payload with random data) and then forward it to the desired destination.

Case III: Redirection of messages for the special analysis. The traffic could be routed over a separate analysis network to carry further analysis on the content of the hidden data.

The diagrammatic representation of the three scenarios is shown in Figure 6. Agent A wants to send a message to agent B via the malicious agent M (i.e., the next hop on the path). Thus, agent A starts the transfer by sending a status update to configure the destination address of the following payload to be "B." In the next message, agent A sends a status update message to indicate the presence of payload data (payload follows directly) to the malicious agent M, which it should forward to agent B. The malicious agent M either performs the forwarding (*sniffing* as a passive observer), actively blocks the message from being transmitted to agent B (as shown in *Case I* of Figure 6) or modifies the value of the payload data and then forwards it to agent B (as shown in *Case II* of Figure 6). If the malicious agent M recognizes something suspicious in the hidden data, it could also redirect the traffic to a separate analysis network to perform further investigation (*Case III*).

Figure 6 Open in figure viewerPowerPoint

Malicious agent M interrupting the overlay communication based on status update messages.

4.3.3 Side-channel attack by timing analysis

This attack could be performed by the an attacker on the **Ping Tunnel** network. He could analyze the timings of incoming ICMP echo request packets and calculate the inter-packet time gaps between them using the following formula:

(1)

where *T* is the incoming time of the i-th ICMP echo request packet and *G* is the

inter-packet time gap between two consecutive ICMP echo request packets. The normal ICMP echo requests generally show a constant time gap produced by the *ping* program. In this way, an attacker is able to determine the difference between normal pings and pings from Ping Tunnel by comparing the time gaps of different services like SSH and FTP sent through Ping Tunnel. Each service connected via Ping Tunnel alters the inter-arrival time of the protocol, making its behavior highly dependent on the embedded service as well as Ping Tunnel's protocol design.

5 Defense against attacks

In this section, we present improvement schemes that can be applied to the micro protocol design of Ping Tunnel and SCCT. These improvements make these tools less prone to the attacks described in Section **4**.

5.1 Ping Tunnel

The key feature of Ping Tunnel used by the attacker is the four byte magic number at the beginning of the Ping Tunnel data unit. Because the magic number is fixed, it is easy for an attacker to identify the packets that belong to Ping Tunnel. In order to hide the presence of a Ping Tunnel data unit, we propose the following scheme to randomize the magic number. Ping Tunnel supports limited authentication when setting up a new Ping Tunnel proxy, by specifying a secret passphrase. This is used when performing a challenge-response authentication when creating a new tunnel. The same passphrase can be used when generating magic numbers for the packets. The new magic number has the same length as the old one and is computed as follows:

where ‖ denotes string concatenation. We use one random byte *R*, followed by the first three bytes of the SHA-256 hash of the passphrase, concatenated with the random byte *R* and the two-byte tunnel id number (id_no). Because the tunnel id is randomly generated in Ping Tunnel, this provides a total of up to 2^{24} possible magic numbers for a given passphrase. ‡ Upon receiving such a packet, the recipient in possession of the secret passphrase has all the information needed to compute the correct magic number and can compare it with the received magic number. As all of the attacks on Ping Tunnel (from Sect. 4.3) rely on a fixed magic number, the proposed scheme could prevent all such attacks. The only action an attacker can take is to modify or block *all* ICMP echo packets.

5.2 Smart covert channel tool

As presented in Section **4.3**, SCCT is vulnerable to attacks like sniffing and man-in-the-middle. In this section, we present the improvement ideas for SCCT by introducing an enhanced version of this tool. A sniffing attack enables the attacker to monitor the network passively and gather information about the network topology as well as about the joining and the leaving agents. In case of SCCT, the topology table contains information about the whole network topology and of all the involved agents. The topology table and the peer table of SCCT's overlay network, shown in Figure **7**, are presented in Table **2** and **3**. The sniffing agent relies on these two

tables for deriving the topology of the overlay network and the identity of the peers.
Table 2. A sample topology table of the old smart covert channel toolEdges between the agentsMetricsA ↔ B10A ↔ C7B ↔ D5C ↔ D12
Table 3. A sample peer table of the old smart covert channel toolAgentIP AddressCCMaskAxx.xx.xx.AA0x406Bxx.xx.xx.BB0x406Cxx.xx.xx.CC0x406Dxx.xx.xx.DD0x406
Figure 7 Open in figure viewerPowerPoint
An example of smart covert channel tool network with four agents.
In order to prevent the sniffing agent from performing such an attack, the enhanced version of SCCT introduces the concept of *known* agents. A specific agent is said to be *known* if the source agent knows the IP address of the destination agent. The IP address is only required when connecting any agent for the first time. The topology table of enhanced SCCT remains initially empty and the peer table only gives the information about the local IP address of itself. These two tables will be updated whenever any agent is connected or disconnected. The peer table is never synchronized across the network; instead, each peer keeps its own list of known agents.
The enhanced version of SCCT keeps the topology table internally the same way as the original version; however, there is no way for an agent to reveal this information through the tool. The agent can only view the part of the topology table that involves the known agents and the directly connected agents. If the destination agent is reachable only via some known or unknown intermediate agents, the information about the intermediate agents is not available to the source agent. Agents will not have any idea about the topology of other agents in the overlay network. To reach an agent in the network, the concept of path calculation is used in the same manner as in the original version of SCCT [11]. So the path is internally calculated between the agents whenever network topology changes. This hides the network topology from the agents so that sniffing will no longer be possible.
We present the working of the enhanced SCCT with the help of an example. Suppose there is an overlay network as in Figure 7 and agent D knows agent C. If agent D wants to connect to agent A, agent A is reachable; however, there is no information about the intermediate agents or the metrics. Tables 4 and 5 show how the topology table and the peer table look like from the perspective of agent D.
Table 4. Topology table of enhanced smart covert channel tool from the perspective of agent D**Edges between the agentsMetrics**D ↔⋯↔ AConnected but no informationon intermediate agentsD ↔ C12
Table 5. Peer table of enhanced smart covert channel tool from the perspective of agent DAgentIP AddressCCMaskAxx.xx.xx.AA0x406Cxx.xx.xx.CC0x406Dxx.xx.xx.DD0x406
During a man-in-the-middle attack, the malicious agent claims to be directly connected to every agent in the overlay network and influences the topology table of

other agents by introducing the lowest metrics. Consequently, the traffic from all the neighboring agents passes through the malicious agent, giving it the capability of controlling or manipulating the network traffic. The enhanced SCCT prevents this attack in two ways: it imposes a minimum metric allowed on a link, and it only allows connections to known agents. This means the attacker can only connect to the known agents and also cannot set the connection metric below the minimum, so the messages will not necessarily be routed through the malicious agent. Moreover, any modifications by an agent to the metrics of the routes not involving that agent are not propagated or accepted by other agents. The modified version of the topology table will be local to the malicious agent, and it will only affect its own routing decisions. This prevents the attacker from forcing the routing through itself by making the other connections too expensive.

6 Results and Discussion

We discuss the impact of attacks for both micro protocols separately. Providing a general view, Figures 8 and 9 describe the influence of each attack using a value between lol and high. Such an influential evaluation is necessary because the application of attacks depends on the use case. In some cases, a covert channel overlay network can only be observed and modified while keeping a low profile, while at others, a destruction of the overlay network is desired and the recognition of an attack by the covert channel users may be acceptable. Therefore, we combine the following aspects in the influence value for an attack:

the passive influence on the covert communication, that is, determining the involvement of a host in the covert communication or observing the transferred hidden content;

the limiting influence, that is, performance decrease or functionality decrease; and the active influence on the exchanged data, that is, modifying the content of the covert communication or preventing the covert communication to occur at all.

Figure 8 Open in figure viewerPowerPoint

Impact of active and passive attacks on Ping Tunnel.

Figure 9 Open in figure viewerPowerPoint

Impact of active and passive attacks on smart covert channel tool.

We also discuss the influence of defense techniques for the two micro protocol-based tools.

6.1 Impact of attacks on Ping Tunnel

It is possible to derive information patterns from the sniffed data depending on the different types of transactions performed by Ping Tunnel users. By analyzing the log files obtained by sniffing, it is clear how the information is exchanged through the covert channel. The list of IP addresses involved in the covert communication can be filtered from the sniffed data. This information is useful as it helps obtain details about the peers involved in the covert communication.

The applied man-in-the-middle attack prevents the covert communication as long as

the packets are not forwarded, or packets are forwarded with replaced payloads or magic numbers. Moreover, by introducing delays into the covert communication, the man-in-the-middle attack could decrease the performance of the connection. In all of the mentioned cases, it is likely that covert channel users become aware of the attack.

By performing the timing analysis of Ping Tunnel's ICMP packets for SSH through Ping Tunnel and FTP through Ping Tunnel, we compared their inter-packet time gaps with normal ping packets' inter-packet time gaps. The inter-packet time gaps of ICMP echo request packets for different services are shown in Figure 10. The graph shows that the average inter-packet time gap calculated from 10 consecutive packets of normal ICMP echo requests is approximately 1 s, but in the case of SSH through Ping Tunnel, this average time is reduced to 0.35 s, and in the case of FTP through Ping Tunnel, it increases to 1.5 s.

Figure 10 Open in figure viewerPowerPoint

Incoming time of ICMP echo request packets for different services through Ping Tunnel.

Although the introduction of a micro protocol influences the behavior of a covert communication, it is likely that the detection using known approaches, such as support vector machines 17, is feasible for SSH, FTP, and additional services.

The influence of the proposed attacks on Ping Tunnel are shown in Figure 8. The active attacks are considered more capable of influencing the covert channel. The influence of active attacks on the covert channel ranges between above average and high because their objective is to break the goals of a covert channel. However, passive attacks must be considered powerful as well, because the one of the goals of applying covert channels is to keep the data transfer secret. While a timing analysis may only lead to detection of a host's involvement in the covert communication, sniffing allows determining the hidden content of messages.

Different scenarios for the application of active attacks are possible. When there is no interactive connection but, for instance, an automatic covert logging connection, a man-in-the-middle attack that sends fake replies to the original sender but does not forward packets to the desired destination, allows sniffing secret information on a constant basis. However, after a fake reply is sent, the attack may still provide an option to forward packets *a posteriori*. The latter option is not provided if the attacker simply changes the magic number or the ICMP echo payload as the whole communication with the destination will break afterwards.

6.2 Impact of attacks on smart covert channel tool

The influence of attacks on the SCCT overlay network is presented in Figure 9. By implementing the redirecting attack on SCCT, it is possible to sniff and alter the routing updates, resulting in a successful integration of a man-in-the-middle agent. The sniffed traffic generated by this agent is helpful in generating the topology graphs of the overlay network. A drawback observed in this technique is the inability to

introduce a fake route when the distance of a routing path is only one hop. Two agents can still exchange information between each other without passing through the malicious agent as long as they are direct neighbors. The influence of the routing redirect is considered average as this attack primarily lays the foundation for other active and passive attacks, that is, sniffing, preventing the communication, and replacing the payload.

If the malicious agent replaces the payload with random data, the communication between agents is still provided, but the content of their communication is rendered useless. Such an attack is likely to get detected but could – when applied in a temporary and highly targeted manner – prevent a particular communication if highly sensitive information must be protected from being transferred (e.g., exfiltrated from an organizational network). The influence of blocking all the data transfers between agents is similar to replacing the content, if applied in a temporary and highly targeted manner as well.

By passive observation, future predictions could be made about the development of the dynamic routing patterns and the topology of the overlay network (decreasing or increasing in size; involvement of new participants), and the exchanged messages could be observed, making this attack influential above average.

6.3 Impact of defense techniques

By randomizing the magic number of Ping Tunnel, the attacker has no way of distinguishing the Ping Tunnel ICMP packets from other ICMP packets using the magic number; thus, it is not possible to create simple rules to detect the presence of Ping Tunnel. This leaves modifying or blocking all the ICMP echo packets as the only blind option for the attacker. The non-blind option of detecting Ping Tunnel via its generated inter-arrival times is still feasible. However, countermeasures for this could also be implemented as discussed in Section **7**.

With an introduction of enhanced SCCT, the tool is immune to the shown overlay sniffing attacks as well as the presented man-in-the middle attack. An agent will not be able to monitor the whole overlay network, unless if it is aware of all the IP addresses of all the agents present in the overlay network. This reduces the probability of conducting a sniffing attack to a great extent. Even if a sniffing agent joins the overlay network, it will only have the information of its known agents.

In a nutshell, enhanced SCCT will be immune to the following attacks:

Sniffing network packets (promiscuous mode): SCCT sends two type of packets in the overlay network, routing update packets, and message packets. Passive agents cannot use routing update packets to get information about the network topology because routing decisions are now made locally, and these packets do not contain clear information about these decisions. By inspecting the message packets, passive agents can only retrieve the information about the source and the content of the message. The information about the destination agent will no longer be retrievable because *peer id* is used to denote the destination agent in the message packet, and

the agents are only allowed to access information about peer ids of their *known* agents.
Manipulating topology and routing update information on agents: This attack is no longer possible as this information is now inaccessible, except for the known agents. The enhanced version of the tool is also immune to any kind of propagation of modified routing table information that can be exploited by malicious agents in terms of a man-in-the-middle attack.

7 Anticipated improvements for micro protocol design

As with the development of C&C protocols for botnets, we can expect that micro protocol development will become increasingly sophisticated within the next years. Means to decrease the size of micro protocol headers to cause fewer anomalies [12] and to ensure a low-attention raising header design and embedding [11] are already published. Also, a micro protocol header could be encrypted or processed by a receiver only if authenticated. So far, authentication is only implemented by Mazurczyk and Kotulski [10], making it challenging to insert fake messages by a third party.
While [10] proposes a dynamic micro protocol header structure, their approach could be extended to scramble protocol header structures using a cryptographic key shared between peers, which would increase the difficulty for analyzing the micro protocol and for introducing fake messages.
It is known that covert channel tools can apply different hiding techniques for each utilized connection and can split their *payload* over multiple communication protocols and OSI layers at the same time [6], [18]. Covert payload can even be nested in several internal layers [18]. In other words, micro protocols embedded in covert channel payload can have their *header* bits fragmented over different areas in the network protocol headers and layers [6], and it is likely that more sophisticated micro protocol designs will take advantage of this option.
Furthermore, a higher stealthiness of a network covert channel itself increases the stealthiness of all the hidden content, including the micro protocol. Many techniques are known to improve covertness, but the most of the existing micro protocols do not take advantage of these improvement techniques. For instance, considering Ping Tunnel's detectability based on inter-packet time gaps, it is imaginable that a micro protocol introduces artificial delays to adjust its traffic to the inter-arrival times of normal ping traffic.
These examples for micro protocol improvements underpin the advantages of a dynamic header structure and sophisticated embedding and transfer techniques. For this reason, additional research is necessary to enable attacks for these anticipated features.

8 Conclusion and Future work

Micro protocols increase the stealthiness and the capabilities of network covert channels. In this paper, we proposed novel approaches for attacking micro

protocol-based tools. We presented a categorization of attacks based on the knowledge of the micro protocol's presence and structure, as well as on the type of the attack (active or passive). Additionally, we described three types of attacks based on the characteristics of micro protocols: detection, limitation, and elimination. Our implementation of attacks for two micro protocol-based tools, Ping Tunnel and SCCT, confirms the feasibility of countering a micro protocol-based communication.

The categorization of these attacks also supports decision-making when countering other tools based on micro protocols. Although the proposed attacks are applied for fully-aware case (where the presence and the structure of the micro protocol header are known), they could also be applied to the other cases (e.g., where the structure of micro protocol is predicted but the presence is not detected). This would lead to the design of additional attacks. Our results show that our techniques are able to counter micro protocol-based communication in an effective manner without requiring much effort to implement them.

In some scenarios, the attacks are implemented for offensive purposes, while micro protocols are used for good purposes. To safeguard such micro protocols against attacks, we presented some defense techniques in terms of improvement ideas so that micro protocols can have a better and more robust structure. The improved micro protocol designs of Ping Tunnel and SCCT proved that they are immune to the presented sniffing and man-in-the-middle attacks. The presented improvement methods are generic and can, for this reason, be applied to other micro protocols in the future.

On one hand, the future work leads towards designing attacks for “blind” situations, that is, the situations in which the presence or the structure of a micro protocol are unknown. Such attacks will provide us with a capability of countering micro protocols at any level, regardless of their structure. On the other hand, the analysis of additional micro protocols is important to improve their design for more adaptive and stealthier communication channels.

Secure Coding Practices - Cryptography

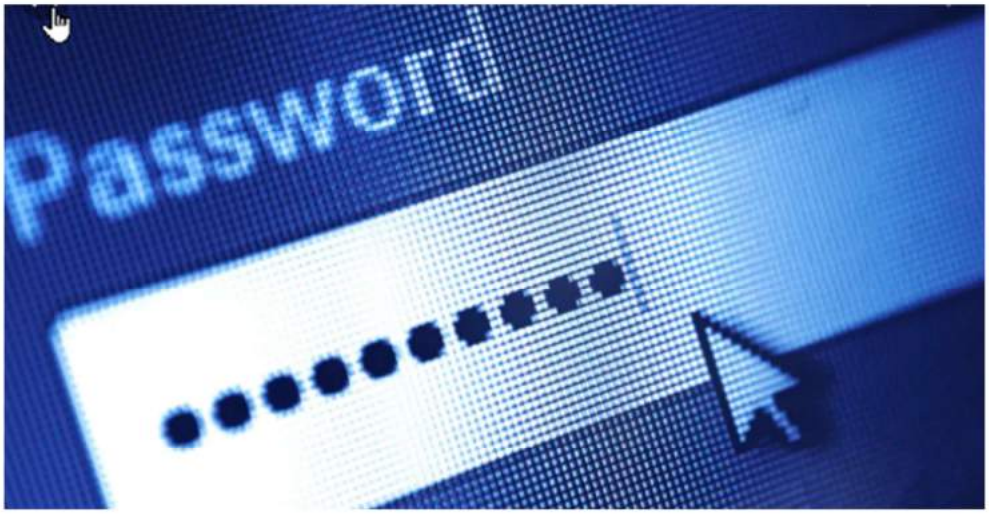


© 2018 Al-Nafi. All Rights Reserved.

We have covered all the controls in detail during our Domain 3 review.

Cryptography techniques can be implemented to protect information by transforming the data through encryption schemes and methods. Typically, they can be used to protect the confidentiality and integrity of information. Cryptography can also be used to address authenticity of communications and nonrepudiation. Cryptography today can be used in many architectures and to protect information while in motion (transit) or at rest. Encryption algorithms can be used to encrypt specific information located anywhere in the architecture.

Secure Coding Practices -Password Protection Techniques



© 2018 Al-Nafi. All Rights Reserved.

Operating system and applications can use passwords as a convenient mechanism to provide authentication services. Typically, operating systems use passwords to authenticate the user and establish access controls for resources including the system, files, or applications. Password protections offered by the operating system include controls on how the password is selected and how complex the password needs to be, password time limits, and password lengths as well. Password files stored within a computer system must be secured by the protection mechanisms of the operating system so that no one, including system administrators, will have access to passwords belonging to entities of the system. Because password files are prone to unauthorized access,
the most common solution is to encrypt password files using one-way encryption algorithms (hashing). Hashing passwords ensures that no one has access to the actual passwords.

However, there are attacks against hashed password files, such as what is referred to as a dictionary attack. There are many other types of password controls that may be offered by the architecture, such as password masking, etc. Careful implementation of these password protection measures needs to be done to

ensure protection based on the value of the architecture.

Secure Coding Practices - Inadequate Granularity of Controls



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.helpsystems.com/resources/articles/six-ws-granular-access-control

The Six Ws of Granular Access Control

Security experts are in general agreement that passwords will simply no longer suffice when it comes to system security. As the numerous breaches within the past years have shown, it is too easy to crack passwords and gain access to all the data across entire systems. So, what can an organization do to better protect its systems? This is where granular access controls, a key feature in certain identity and access management solutions, comes in.

What exactly is granular access control? With all the buzzwords floating around the cybersecurity world, it's easy to stumble across a term that could use additional explanation. This article takes a closer look at **w**hy granular access control is so effective -- by placing limitations on **w**ho can get into your organization's system, **w**here, **w**hen and ho**w** they can access it, and **w**hat they can do with it.

WHO

In its simplest definition, granular access controls define who can have access to each part of a system, as well as what they can do with that access. However, setting up permissions for each individual user is impractical and would be incredibly time consuming to track and maintain. Instead, privileges are granted based on roles defined in a corporate directory. For example, a database administrator would be granted permissions for all database servers, whereas a web administrator wouldn't need access to those particular servers and would therefore not be give permission to access it.

While these role permissions can be set up manually, this would still be immensely time consuming for an IT team, and nearly impossible to keep up with. Employees who leave an organization may still retain permissions for days, even months after their departure, leaving an organization incredibly vulnerable. Identity and access solutions turn this crucial security protocol into a doable task. Rules and permissions can be changed instantaneously, protecting users from making mistakes, and organizations from leaving doors open to private data.

HOW

Once these roles have been established and assigned, users must also authenticate their identity before logging in. This can be done in a number of ways – passwords, tokens, etc. However, as mentioned above, passwords are no longer enough when it comes to critical data. This is where identity and access management solutions come in. They can step up the level of authentication needed for roles with administrator access, adding an additional layer of protection.

How one accesses a system may seem like a simple concept, but when it comes to access, the details matter. There are different levels of connection on can make to the server. For example, an admin can securely access the server over ssh, transfer files with sftp, and escalate privileges with sudo. Some admins may only need to copy files to or from a server but won't need access to the server itself. Others may need full administrator privileges and all the commands that come with it. Granular access controls assign only the necessary connection capability to each user class.

WHAT

When using granular access controls, it is ideal to practice the principle of least privilege. That is, unless otherwise specified, a role will be assigned the least amount of access possible to a system. As a role is more defined, the necessary access becomes clearer and is assigned accordingly. For instance, a web administrator would only need access to web servers and a select number of privileged commands. However, it is not only the access to different parts of the system that are defined – the level of permissions must also be determined. As stated earlier, database and web administrators only need access to select servers and commands, while, Linux

administrators typically need access to all servers and all privileged commands. Being this explicit in access and permissions prevents accidental and intentional tampering that can result in data breaches or loss.

WHERE

While it's an advantage not only to organizations and employees that work can be accomplished from anywhere, it also requires extra vigilance. Since people can access servers everywhere, it no longer looks suspicious when an organization has numerous IP addresses logging in globally. It also isn't feasible to require IT teams to comb through these addresses to try and ensure that logins are only coming in from locations where employees are located.

Granular access controls can skip these issues altogether simply by limiting the amount of locations from where a server can be accessed. For example, if no employees are located in Canada, then no Canadian IP addresses can have access to an organization's system.

Restrictions must also be placed on the type of access users have when using VPNs (Virtual Private Networks) outside an organization's offices. Allowing for major changes from thousands of miles away leaves systems incredibly vulnerable. The highest level of access should always be reserved for those logging in directly from the physical server.

WHEN

A crucial component to configuring granular access controls for maximum security is timing. Staff in an organization rarely need access to systems or data 24 hours a day. In fact, someone signing into their account outside of normal business hours could be considered suspicious. On the other hand, someone logging in during strange hours may only indicate that they're located in an office in another country. Granular access controls are sophisticated enough to establish rules based on not only role, but on the window of time that a group can be expected to be working. Limiting access to a set timeframe can prevent an error or threat from remaining undiscovered for hours. Additionally, granular access controls can provide temporary access for a limited amount of time. For example, a contract employee could be given credentials that are set to time out at the end of their contract. Alternately, a sales person traveling abroad may be given credentials to log in from another country for the length of the trip.

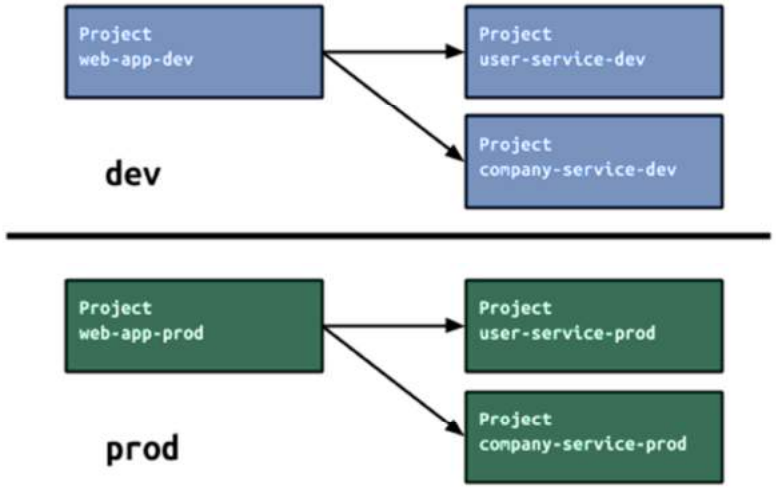
WHY

As mentioned above, passwords have become insufficient protection from internal and external breaches. The key to system infiltration comes down to credentials. Once someone has the necessary credentials to access a system and use privileged commands, the damage can be catastrophic. The more employees with these high-

level credentials, the higher the risk is that someone could get full access to the system, even with a simple phishing attack. Even when considering password vaulting technologies, a single password is still all that prevents someone from getting through the door. The flexibility of granular access control ensures there are multiple ways to prevent someone from having complete access.
Risks of malicious insider attacks are also mitigated through granular access control. Organizations that provide full credentials to every employee make it incredibly difficult to track what employees are doing, making it possible to provide an insider with an open window to private data for months without the threat of detection.

Attempting to control privileges like this manually would take numerous additional employees, who would still be unable to make updates and adjust rules in real time. Identity and access management solutions, like Powertech Identity & Access Manager (BoKS), allows IT teams to efficiently protect your organization's data. You can restrict privileges so that no one employee has full control of your system, yet still give users the credentials they need to get their work done.

Secure Coding Practices - Control and Separation of Environments



© 2018 Al-Nafi. All Rights Reserved.

In software development, there are usually various environments, these may include, for example:

- Development environment
- Quality assurance environment
- Production environment

There may be other environments, but the security issue is to control how each environment can access the application and the data that the application is processing and then provide mechanisms to keep them separate. For example, systems analysts and programmers write, compile, and perform initial testing of the application's implementation and functionality in the development environment. As the application reaches the point of being ready to be put into production, users and quality assurance people perform functional testing within the quality assurance environment. To be effective, the quality assurance configuration should simulate the production environment as closely as
possible. Once the testing has been completed, including the security testing, and stakeholders have accepted the application, it is moved into the production

environment. What is important is to keep the environments separate and isolated. Those working in any environment should be restricted to that environment only. Blended environments combine one or more of these individual environments and are generally the most difficult to control. As an example, it is generally accepted that developers
working in development environments should never have access to the production environment.
Control measures protecting the various environments are many, but they should include physical isolation of environment, physical or temporal separation of data for each environment, access control lists, content dependent access controls, role-based constraints, role definition stability, accountability, and separation of duties.

Secure Coding Practices - Software Forensics



© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Software_forensics

Software forensics is the science of analyzing software source code or binary code to determine whether intellectual property infringement or theft occurred. It is the centerpiece of lawsuits, trials, and settlements when companies are in dispute over issues involving software patents, copyrights, and trade secrets. Software forensics tools can compare code to determine correlation, a measure that can be used to guide a software forensics expert.

Past methods of software forensics[edit]
Past methods of code comparison included hashing, statistical analysis, text matching, and tokenization. These methods compared software code and produced a single measure indicating whether copying had occurred. However, these measures were not accurate enough to be admissible in court because the results were not accurate, the algorithms could be easily fooled by simple substitutions in the code, and the methods did not take into account the fact that code could be similar for reasons other than copying.

Robert Zeidman[edit]
In 2003, Robert Zeidman developed algorithms, which he incorporated in the CodeSuite tool, for multidimensional software correlation that divides software code into basic elements and determines which elements are similar, or "correlated." He also created a procedure for filtering and interpreting the correlations to eliminate reasons for correlation that are not due to copying: common algorithms, third-party code, common identifier names, common author, and automatic code generation. The combination of these algorithms and procedures resulted in a more reliable and quantitative analysis than was available previously. Zeidman's book, "The Software IP Detective's Handbook," is considered the standard textbook for software forensics[1][2] and the CodeSuite tools and methodology has been used in many software IP litigations including ConnectU v. Facebook, Symantec v. IRS Baker & McKenzie, and SCO Group, Inc. v. International Business Machines Corp.

Copyright infringement[edit]
Following the use of software tools to compare code to determine the amount of correlation, an expert can use an iterative filtering process to determine that the correlated code is due to third-party code, code generation tools, commonly used names, common algorithms, common programmers, or copying. If the correlation is due to copying, and the copier did not have the authority from the rights holder, then copyright infringement occurred.

Trade secret protection and infringement[edit]
Software can contain trade secrets, which provide a competitive advantage to a business. To determine trade secret theft, the same tools and processes can be used to detect copyright infringement. If code was copied without authority, and that code has the characteristics of a trade secret—it is not generally known, the business keeps it secret, and its secrecy maintains its value to the business—then the copied code constitutes trade secret theft.

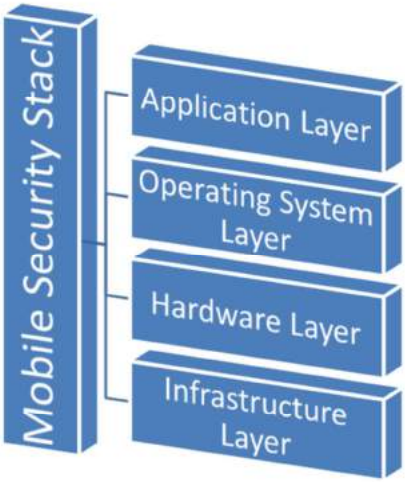
Trade secret theft can also involve the taking of code functionality without literally copying the code. Comparing code functionality is a very difficult problem that has yet to be accomplished by any algorithm in reasonable time. For this reason, finding the theft of code functionality is still mostly a manual process.

Patent infringement[edit]
As with trade secret functionality, it is not currently possible to scientifically detect software patent infringement, as software patents cover general implementation rather than specific source code. For example, a program that implements a patented invention can be written in many available programming languages, using different function names and variable names and performing operations in different

sequences. There are so many combinations of ways to implement inventions in software that even the most powerful modern computers cannot consider all combinations of code that might infringe a patent. This work is still left to human experts using their knowledge and experience, but it is a problem that many in software forensics are trying to automate by finding an algorithm or simplifying process.

Objective facts before subjective evidence[edit]
One important rule of any forensic analysis is that the objective facts must be considered first. Reviewing comments in the code or searching the Internet to find information about the companies that distribute the code and the programmers who wrote the code are useful only after the objective facts regarding correlation have been considered. Once an analysis has been performed using forensic tools and procedures, analysts can then begin looking at subjective evidence like comments in the code. If the information in that subjective evidence conflicts with the objective analysis, analysts need to doubt the subjective evidence. Fake copyright notices, open source notifications, or programmer names that were added to source code after copying took place, in order to disguise the copying, are not uncommon in real-world cases of code theft.

Secure Coding Practices - Mobile Code security



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.veracode.com/security/mobile-code-security

Mobile App and Mobile Code Security Risks
There are two main categories of mobile code security risks: (1) malicious functionality and (2) vulnerabilities.

The category of malicious functionality is a list of unwanted and dangerous mobile code behaviors that are stealthily placed in a Trojan app that the user is tricked into installing. Users think they are installing a game or utility and instead get hidden spyware, phishing UI or unauthorized premium dialing.

Malicious Functionality

- Activity monitoring and data retrieval
- Unauthorized dialing, SMS and payments
- Unauthorized network connectivity (exfiltration or command & control)
- UI impersonation

- System modification (rootkit, APN proxy config)
- Logic or time bomb

The category of mobile security vulnerabilities are errors in design or implementation that expose the mobile device data to interception and retrieval by attackers. Mobile code security vulnerabilities can also expose the mobile device or the cloud applications used from the device to unauthorized access.

Vulnerabilities

- Sensitive data leakage (inadvertent or side channel)
- Unsafe sensitive data storage
- Unsafe sensitive data transmission
- Hardcoded password/keys
- The Mobile Code Security Stack

Increasing smartphone adoption rates coupled with the rapid growth in smartphone application counts have created a scenario where private and sensitive information is being pushed to the new device perimeter at an alarming rate. The smartphone mobile device is quickly becoming ubiquitous. While there is much overlap with common operating system models, the mobile device code security model has some distinct points of differentiation.

The mobile code security stack can be broken up into four distinct layers. The lowest layer of the stack is the infrastructure layer, followed upward by the hardware, operating system and application layers. These security stack layers each define a separate section of the security model of a smartphone or mobile device.

Each layer of the mobile code security model is responsible for the security of its defined components and nothing more. The upper layers of the stack rely on all lower layers to ensure that their components are appropriately safe. This abstraction-based model allows the design of a particular mobile security mechanism to focus on a single specific area of concern without expending the resources required to analyze all layers that support its current location within the stack.

Mobile Security - Infrastructure Layer

The infrastructure layer is the lowest and thus most supportive layer of the mobile code security stack. This layer is the foundation that supports all of the other tiers of the model. The majority of the functional components at this layer are owned and

operated by a mobile carrier or infrastructure provider; however, integration into the handset occurs as data is transmitted from this tier upward.
Cellular voice and data carriers operate the infrastructure that carries all data and voice communications from end point to end point. The security of components at this level typically encompasses the protocols in use by the carriers and infrastructure providers themselves. Examples of such protocols include code division multiple access protocol (CDMA), global system for mobile communications (GSM), global positions systems (GPS), short messaging systems (SMS) and multimedia messaging systems (MMS). Due to the low foundational nature of this particular security tier, flaws or vulnerabilities discovered at this tier are generally effective across multiple platforms, multiple carriers and multiple handset providers.

Mobile Security - Hardware Layer
As we move up the stack to the second tier of the mobile code security stack, we are moving into the realm of a physical unit that is typically under the direct control of an end user. The hardware layer is identified by the individual end user premise equipment, generally in the form of a smartphone or tablet style mobile device. The hardware layer is accessible to the operating system, allowing for direct control of the physical components of the unit. This hardware is generally called the “firmware” and is upgraded by the physical manufacturer of the handset and occasionally delivered by proxy through the phone carrier. Security flaws or vulnerabilities discovered at this layer typically affect all end users who use a particular piece of hardware or individual hardware component. If a hardware flaw is discovered in a single manufacturer’s device, it is more than likely that all hardware revisions using that similar design and/or chip will be affected as well.

Mobile Security - Operating System Layer
The third tier in the mobile code security stack is the operating system layer. This layer corresponds to the software running on a device that allows communications between the hardware and the application tiers. The operating system is periodically updated with feature enhancements, patches and security fixes, which may or may not coincide with patches made to the firmware by the physical handset manufacturer. The operating system provides access to its resources via the publishing of application programming interfaces. These resources are available to be consumed by the application layer as it is the only layer higher in the stack than the operating system itself. Simultaneously, the operating system communicates with the hardware/firmware to run processes and pass data to and from the device.

Operating system flaws are a very common flaw type and currently tend to be the target of choice for attackers that wish to have a high impact. If an operating system flaw is discovered, the entire installed base of that particular operating system

revision will likely be vulnerable. It is at this layer, and above, where software is the overriding enforcement mechanism for security. Specifically due to the fact that software is relied upon, the operating system, and the application layer above, is the most common location where security flaws are discovered.

Mobile Security - Application Layer

The application tier resides at the top of the mobile security stack and is the layer that the end user directly interfaces with. The application layer is identified by running processes that utilize application programming interfaces provided by the operating system layer as an entry point into the rest of the stack.

Application layer security flaws generally result from coding flaws in applications that are either shipped with or installed onto a mobile device after deployment. These flaws come in classes that are similar to the personal computing area. Buffer overflows, insecure storage of sensitive data, improper cryptographic algorithms, hardcoded passwords and backdoored applications are only a sample set of application layer flaw classes. The result of exploitation of application layer security flaws can range from elevated operating system privilege to exfiltration of sensitive data.

How to Test for Mobile Code Security

When analyzing an individual device for security implications, take into account each of the layers of the mobile code security stack and determine the effectiveness of the security mechanisms that are in place. At each layer, determine what, if any, security mechanisms and mitigations the manufacturer has implemented and if those mechanisms are sufficient for the type of data you plan to store and access on the device.

Secure Coding Practices - Sandbox

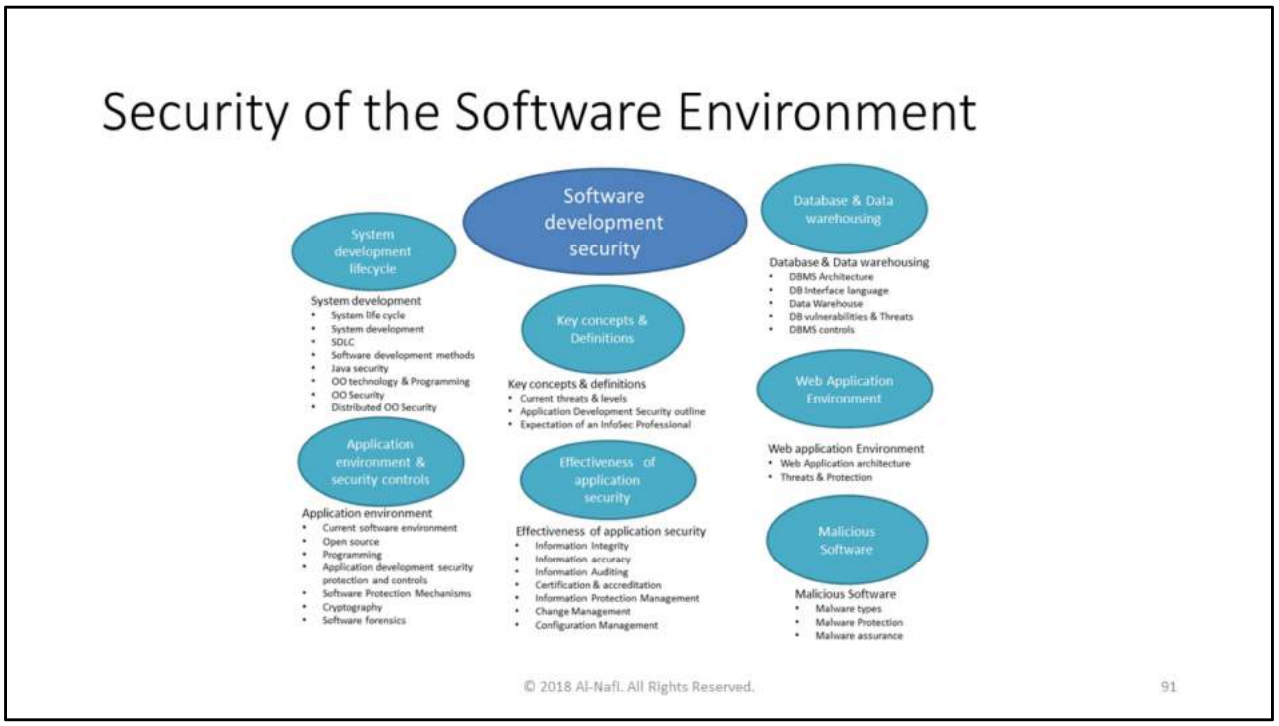


© 2018 Al-Nafi. All Rights Reserved.

One of the control mechanisms for mobile code is called a sandbox environment. As its name implies, a sandbox can be a “play” area where we can test certain pieces of code to see if they are malicious. The sandbox provides a protective area for program execution. Limits are placed on the amount of memory and processor resources the program can consume in that sandbox environment. If the program exceeds these limits, the web browser terminates the process and logs an error code and ultimately does not allow the code to run.

This can ensure the safety of the browser’s activities. As an example, in the Java sandbox security model, there is an option to provide an area for the Java code to do what it needs to do, including restricting the bounds of this area. This is exactly the idea of a sandbox. A sandbox cannot confine code and its behavior without some type of enforcement mechanism. The Java security manager makes sure all restricted code stays in the sandbox and cannot ultimately do anything outside of it. Trusted code resides outside the sandbox, and untrusted code is confined within the sandbox. By default, Java applications live outside the sandbox and Java applets are confined within the sandbox.

You can read further via https://www.windowschimp.com/best-sandbox-software/



Reference https://en.wikipedia.org/wiki/Software_development_security

Security, as part of the software development process, is an ongoing process involving people and practices, and ensures application confidentiality, integrity, and availability. Secure software is the result of security aware software development processes where security is built in and thus software is developed with security in mind.[1]

Security is most effective if planned and managed throughout every stage of software development life cycle (SDLC), especially in critical applications or those that process sensitive information.
The solution to software development security is more than just the technology.

Software development challenges[edit]
As technology advances, application environments become more complex and application development security becomes more challenging. Applications, systems, and networks are constantly under various security attacks such as malicious

code or denial of service. Some of the challenges from the application development security point of view include Viruses, Trojan horses, Logic bombs, Worms, Agents, and Applets.[2]
Applications can contain security vulnerabilities that may be introduced by software engineers either intentionally or carelessly.
Software, environmental, and hardware controls are required although they cannot prevent problems created from poor programming practice. Using limit and sequence checks to validate users' input will improve the quality of data. Even though programmers may follow best practices, an application can still fail due to unpredictable conditions and therefore should handle unexpected failures successfully by first logging all the information it can capture in preparation for auditing. As security increases, so does the relative cost and administrative overhead.
Applications are typically developed using high-level programming languages which in themselves can have security implications. The core activities essential to the software development process to produce secure applications and systems include: conceptual definition, functional requirements, control specification, design review, code review and walk-through, system test review, and maintenance and change management.
Building secure software is not only the responsibility of a software engineer but also the responsibility of the stakeholders which include: management, project managers, business analysts, quality assurance managers, technical architects, security specialists, application owners, and developers.

Basic principles[edit]
There are a number of basic guiding principles to software security. Stakeholders' knowledge of these and how they may be implemented in software is vital to software security. These include:

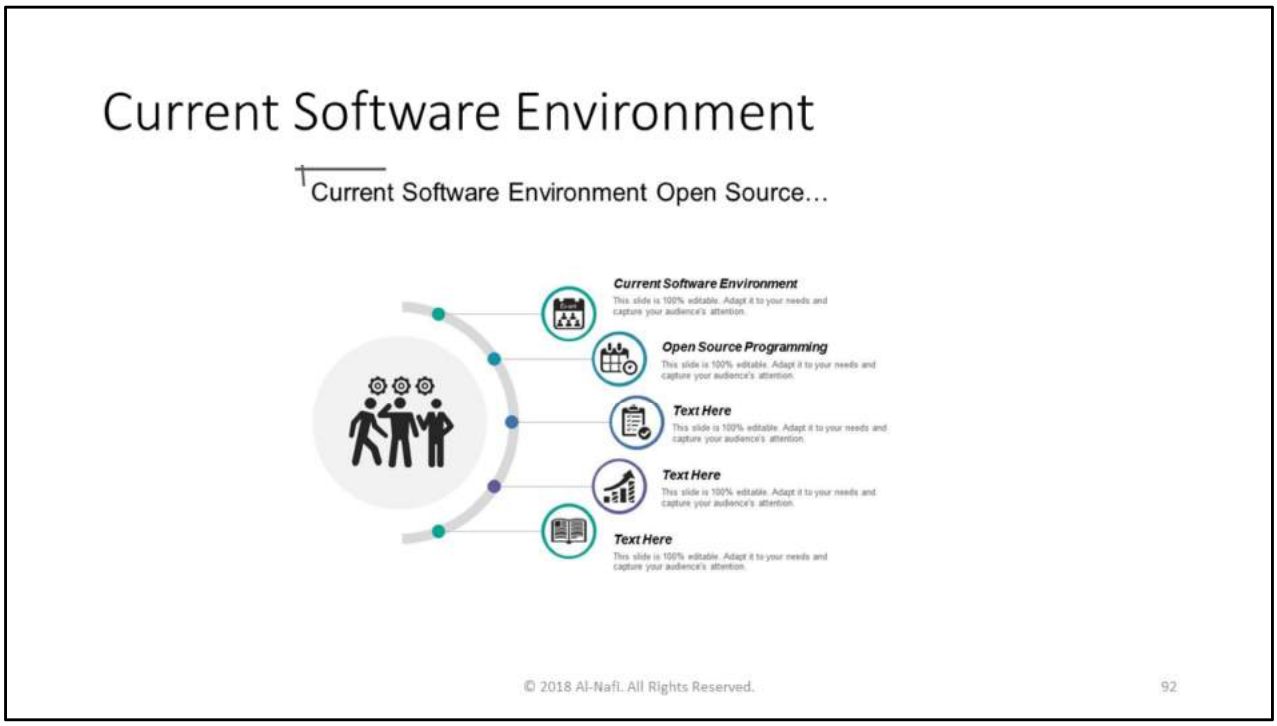
- Protection from disclosure
- Protection from alteration
- Protection from destruction
- Who is making the request
- What rights and privileges does the requester have
- Ability to build historical evidence
- Management of configuration, sessions and errors/exceptions

Basic practices[edit]
The following lists some of the recommended web security practices that are more specific for software developers.

- Sanitize inputs at the client side and server side
- Encode request/response
- Use HTTPS for domain entries

- Use only current encryption and hashing algorithms
- Do not allow for directory listing
- Do not store sensitive data inside cookies
- Check the randomness of the session
- Set secure and HttpOnly flags in cookies
- Use TLS not SSL
- Set strong password policy
- Do not store sensitive information in a form's hidden fields
- Verify file upload functionality
- Set secure response headers
- Make sure third party libraries are secured
- Hide web server information

Security testing[edit]
Common attributes of security testing include authentication, authorization, confidentiality, availability, integrity, non-repudiation, and resilience. Security testing is essential to ensure that the system prevents unauthorized users to access its resources and data. Some application data is sent over the internet which travels through a series of servers and network devices. This gives ample opportunities to unscrupulous hackers.
Summary[edit]
All secure systems implement security controls within the software, hardware, systems, and networks - each component or process has a layer of isolation to protect an organization's most valuable resource which is its data. There are various security controls that can be incorporated into an application's development process to ensure security and prevent unauthorized access.



Information systems are becoming more distributed, with a substantial increase in the use of open protocols, interfaces, and source code, as well as sharing of resources. All of these elements require that all resources be protected against unauthorized access, as well as issues related to confidentiality, integrity, and availability. Many of these safeguards are provided through software controls, especially operating system mechanisms and application software controls. The operating system must offer controls that protect the computer's resources and so must the application and system itself running on top of the operating system. In addition, the relationship between applications and the operating system, and how they communicate is also very important. Controls must be included in operating systems so that applications cannot damage or circumvent the operating system controls. And controls need to be designed and built into the application software to protect the data that ultimately it processes. A lack of adequate software protection mechanisms can leave the operating system and critical computer resources open to corruption and attack and the sensitive data open to potential disclosure, corruption, or unavailability.

The complexity of information systems today has also increased. Older computing

typically required the application running on a specific machine, aside from the hardwired functions resident in the CPU. Today, an application may be running on architectures that involve the hardware platform, CPU microcode, virtual machine server, operating system, network operating system, utilities, remote procedure calls, object request broker, database and web servers, engine application, multiple interface applications, interface utilities, API libraries, and multiple entities involved in a remote client interface. In other words, the architecture itself, and the components that make it up, has become much more complex. This ultimately requires adequate protection of all entities and components that make up the architecture.

While many of these levels have been added in the name of interoperability and standardization, the complexity introduced does make addressing the security requirements more difficult. Some of the main security requirements for applications and databases are to ensure that only valid, authorized, and authenticated users can access the sensitive information contained within the database environments and the proper enforcement of the permissions related to use of the data. It may also be required that the system or software provides some type of granularity for controlling such permissions and that possibly encryption or other appropriate logical controls are available for protecting the value of sensitive information. Other controls required may include password protection and audit mechanisms that provide assurance of the functional security controls.



Reference https://opensource.com/resources/what-open-source

The term "open source" refers to something people can modify and share because its design is publicly accessible.
The term originated in the context of software development to designate a specific approach to creating computer programs. Today, however, "open source" designates a broader set of values—what we call "the open source way." Open source projects, products, or initiatives embrace and celebrate principles of open exchange, collaborative participation, rapid prototyping, transparency, meritocracy, and community-oriented development.

What is open source software?
Open source software is software with source code that anyone can inspect, modify, and enhance.
"Source code" is the part of software that most computer users don't ever see; it's the code computer programmers can manipulate to change how a piece of software—a "program" or "application"—works. Programmers who have access to a computer program's source code can improve that program by adding features to it

or fixing parts that don't always work correctly.

What's the difference between open source software and other types of software?
Some software has source code that only the person, team, or organization who created it—and maintains exclusive control over it—can modify. People call this kind of software "proprietary" or "closed source" software.
Only the original authors of proprietary software can legally copy, inspect, and alter that software. And in order to use proprietary software, computer users must agree (usually by signing a license displayed the first time they run this software) that they will not do anything with the software that the software's authors have not expressly permitted. Microsoft Office and Adobe Photoshop are examples of proprietary software.
Open source software is different. Its authors make its source code available to others who would like to view that code, copy it, learn from it, alter it, or share it. LibreOffice and the GNU Image Manipulation Program are examples of open source software.
As they do with proprietary software, users must accept the terms of a license when they use open source software—but the legal terms of open source licenses differ dramatically from those of proprietary licenses.
Open source licenses affect the way people can use, study, modify, and distribute software. In general, open source licenses grant computer users permission to use open source software for any purpose they wish. Some open source licenses—what some people call "copyleft" licenses—stipulate that anyone who releases a modified open source program must also release the source code for that program alongside it. Moreover, some open source licenses stipulate that anyone who alters and shares a program with others must also share that program's source code without charging a licensing fee for it.
By design, open source software licenses promote collaboration and sharing because they permit other people to make modifications to source code and incorporate those changes into their own projects. They encourage computer programmers to access, view, and modify open source software whenever they like, as long as they let others do the same when they share their work.
Is open source software only important to computer programmers?
No. Open source technology and open source thinking both benefit programmers and non-programmers.
Because early inventors built much of the Internet itself on open source technologies—like the Linux operating system and the Apache Web server application—anyone using the Internet today benefits from open source software. Every time computer users view web pages, check email, chat with friends, stream music online, or play multiplayer video games, their computers, mobile phones, or gaming consoles connect to a global network of computers using open source

software to route and transmit their data to the "local" devices they have in front of them. The computers that do all this important work are typically located in faraway places that users don't actually see or can't physically access—which is why some people call these computers "remote computers."

More and more, people rely on remote computers when performing tasks they might otherwise perform on their local devices. For example, they may use online word processing, email management, and image editing software that they don't install and run on their personal computers. Instead, they simply access these programs on remote computers by using a Web browser or mobile phone application. When they do this, they're engaged in "remote computing."

Some people call remote computing "cloud computing," because it involves activities (like storing files, sharing photos, or watching videos) that incorporate not only local devices but also a global network of remote computers that form an "atmosphere" around them.

Cloud computing is an increasingly important aspect of everyday life with Internet-connected devices. Some cloud computing applications, like Google Apps, are proprietary. Others, like ownCloud and Nextcloud, are open source.

Cloud computing applications run "on top" of additional software that helps them operate smoothly and efficiently, so people will often say that software running "underneath" cloud computing applications acts as a "platform" for those applications. Cloud computing platforms can be open source or closed source. OpenStack is an example of an open source cloud computing platform.

Why do people prefer using open source software?

People prefer open source software to proprietary software for a number of reasons, including:

Control. Many people prefer open source software because they have more control over that kind of software. They can examine the code to make sure it's not doing anything they don't want it to do, and they can change parts of it they don't like. Users who aren't programmers also benefit from open source software, because they can use this software for any purpose they wish—not merely the way someone else thinks they should.

Training. Other people like open source software because it helps them become better programmers. Because open source code is publicly accessible, students can easily study it as they learn to make better software. Students can also share their work with others, inviting comment and critique, as they develop their skills. When people discover mistakes in programs' source code, they can share those mistakes with others to help them avoid making those same mistakes themselves.

Security. Some people prefer open source software because they consider it more secure and stable than proprietary software. Because anyone can view and modify open source software, someone might spot and correct errors or omissions that a program's original authors might have missed. And because so many

programmers can work on a piece of open source software without asking for permission from original authors, they can fix, update, and upgrade open source software more quickly than they can proprietary software.

Stability. Many users prefer open source software to proprietary software for important, long-term projects. Because programmers publicly distribute the source code for open source software, users relying on that software for critical tasks can be sure their tools won't disappear or fall into disrepair if their original creators stop working on them. Additionally, open source software tends to both incorporate and operate according to open standards.

Doesn't "open source" just mean something is free of charge?

No. This is a common misconception about what "open source" implies, and the concept's implications are not only economic.

Open source software programmers can charge money for the open source software they create or to which they contribute. But in some cases, because an open source license might require them to release their source code when they sell software to others, some programmers find that charging users money for *software services and support* (rather than for the software itself) is more lucrative. This way, their software remains free of charge, and they make money helping others install, use, and troubleshoot it.

While some open source software may be free of charge, skill in programming and troubleshooting open source software can be quite valuable. Many employers specifically seek to hire programmers with experience working on open source software.

What is open source "beyond software"?

At Opensource.com, we like to say that we're interested in the ways open source values and principles apply to the world *beyond software*. We like to think of open source as not only a way to develop and license computer software, but also an *attitude*.

Approaching all aspects of life "the open source way" means expressing a willingness to share, collaborating with others in ways that are transparent (so that others can watch and join too), embracing failure as a means of improving, and expecting—even encouraging—everyone else to do the same.

It also means committing to playing an active role in improving the world, which is possible only when everyone has access to the way that world is designed.

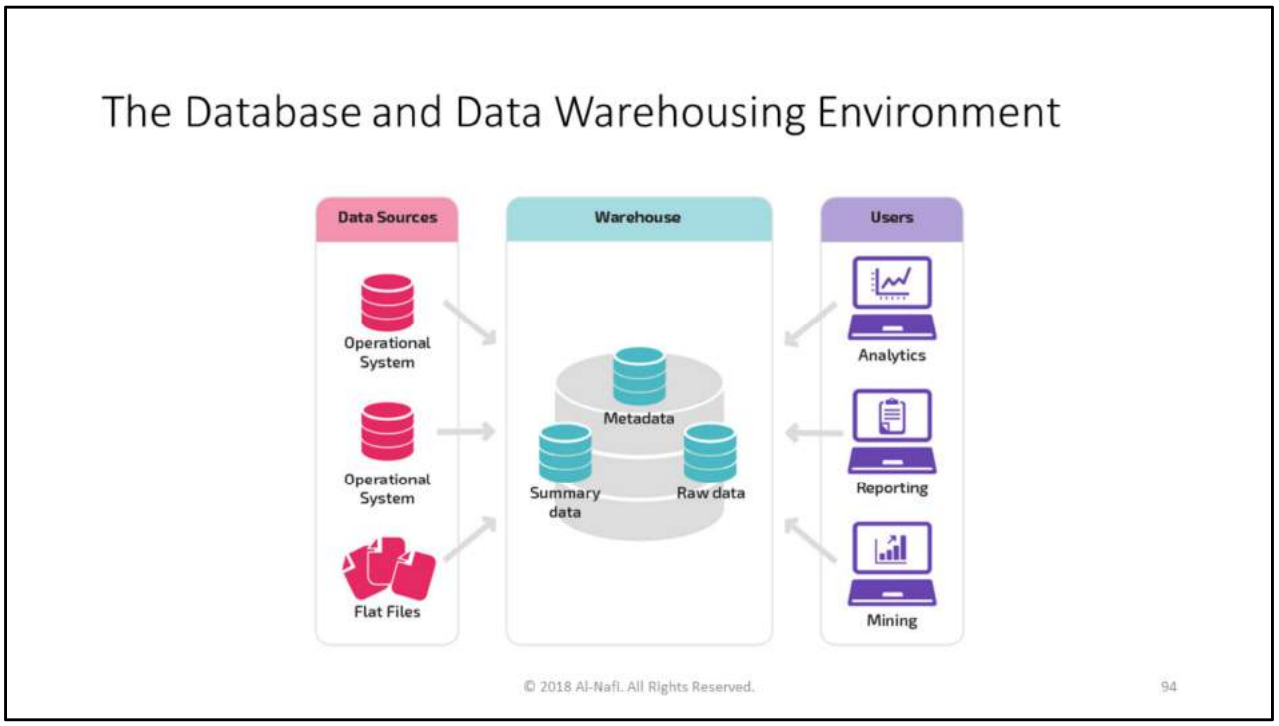
The world is full of "source code"—blueprints, recipes, rules—that guide and shape the way we think and act in it. We believe this underlying code (whatever its form) should be open, accessible, and shared—so many people can have a hand in altering it for the better.

Here, we tell stories about the impact of open source values on all areas of life—science, education, government, manufacturing, health, law, and organizational dynamics. We're a community committed to telling others how the open source way

is the *best* way, because a love of open source is just like anything else: it's better when it's shared.

Where can I learn more about open source?

We've compiled several resources designed to help you learn more about open source. We recommend you read our open source FAQs, how-to guides, and tutorials to get started.



Reference https://en.wikipedia.org/wiki/Data_warehouse

In computing, a **data warehouse** (**DW** or **DWH**), also known as an **enterprise data warehouse** (**EDW**), is a system used for reporting and data analysis, and is considered a core component of business intelligence.[1] DWs are central repositories of integrated data from one or more disparate sources. They store current and historical data in one single place[2] that are used for creating analytical reports for workers throughout the enterprise.[3]

The data stored in the warehouse is uploaded from the operational systems (such as marketing or sales). The data may pass through an operational data store and may require data cleansing[2] for additional operations to ensure data quality before it is used in the DW for reporting.

Extract, transform, load (ETL) and Extract, load, transform (E-LT) are the two main approaches used to build a data warehouse system.

ETL based Data warehousing[edit]
The typical extract, transform, load (ETL)-based data warehouse[4] uses staging, data integration, and access layers to house its key functions. The staging layer or staging database stores raw data extracted from each of the disparate source data systems. The integration layer integrates the disparate data sets by transforming the data from the staging layer often storing this transformed data in an operational data store (ODS) database. The integrated data are then moved to yet another database, often called the data warehouse database, where the data is arranged into hierarchical groups, often called dimensions, and into facts and aggregate facts. The combination of facts and dimensions is sometimes called a star schema. The access layer helps users retrieve data.[5]
The main source of the data is cleansed, transformed, catalogued, and made available for use by managers and other business professionals for data mining, online analytical processing, market research and decision support.[6] However, the means to retrieve and analyze data, to extract, transform, and load data, and to manage the data dictionary are also considered essential components of a data warehousing system. Many references to data warehousing use this broader context. Thus, an expanded definition for data warehousing includes business intelligence tools, tools to extract, transform, and load data into the repository, and tools to manage and retrieve metadata.

IBM InfoSphere DataStage, Ab Initio Software, Informatica – PowerCenter are some of the tools which are widely used to implement ETL based data warehouse.

ELT based Data warehousing[edit]
ELT based data warehousing gets rid of a separate ETL tool for data transformation. Instead, it maintains a staging area inside the data warehouse itself. In this approach, data gets extracted from heterogeneous source systems and are then directly loaded into the data warehouse, before any transformation occurs. All necessary transformations are then handled inside the data warehouse itself. Finally, the manipulated data gets loaded into target tables in the same data warehouse.

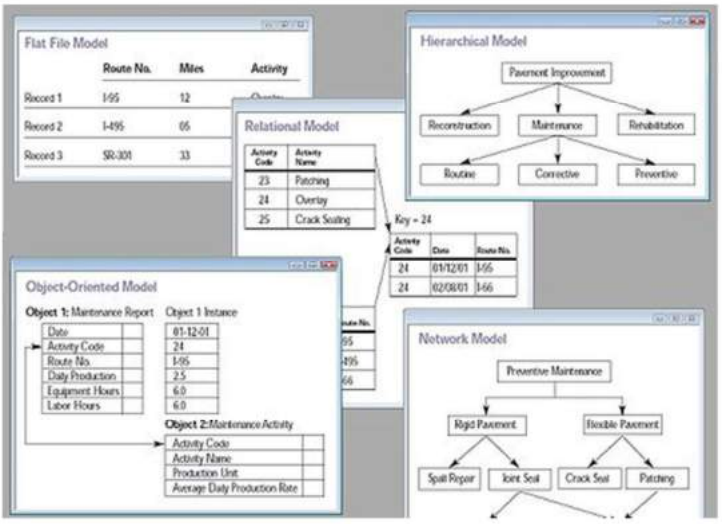
Benefits[edit]
A data warehouse maintains a copy of information from the source transaction systems. This architectural complexity provides the opportunity to:
Integrate data from multiple sources into a single database and data model. More congregation of data to single database so a single query engine can be used to present data in an ODS.
Mitigate the problem of database isolation level lock contention in transaction processing systems caused by attempts to run large, long-running, analysis queries in transaction processing databases.

Maintain data history, even if the source transaction systems do not.
Integrate data from multiple source systems, enabling a central view across the enterprise. This benefit is always valuable, but particularly so when the organization has grown by merger.
Improve data quality, by providing consistent codes and descriptions, flagging or even fixing bad data.

Present the organization's information consistently.
Provide a single common data model for all data of interest regardless of the data's source.
Restructure the data so that it makes sense to the business users.
Restructure the data so that it delivers excellent query performance, even for complex analytic queries, without impacting the operational systems.
Add value to operational business applications, notably customer relationship management (CRM) systems.
Make decision–support queries easier to write.
Organize and disambiguate repetitive data
Generic[edit]
The environment for data warehouses and marts includes the following:
Source systems that provide data to the warehouse or mart;
Data integration technology and processes that are needed to prepare the data for use;
Different architectures for storing data in an organization's data warehouse or data marts;
Different tools and applications for the variety of users;
Metadata, data quality, and governance processes must be in place to ensure that the warehouse or mart meets its purposes.
In regards to source systems listed above, R. Kelly Rainer states, "A common source for the data in data warehouses is the company's operational databases, which can be relational databases".[7]
Regarding data integration, Rainer states, "It is necessary to extract data from source systems, transform them, and load them into a data mart or warehouse".[7]
Rainer discusses storing data in an organization's data warehouse or data marts.[7]
Metadata is data about data. "IT personnel need information about data sources; database, table, and column names; refresh schedules; and data usage measures".[7]
Today, the most successful companies are those that can respond quickly and flexibly to market changes and opportunities. A key to this response is the effective and efficient use of data and information by analysts and managers.[7] A "data warehouse" is a repository of historical data that is organized by subject to support decision makers in the organization.[7] Once data is stored in a data mart or warehouse, it can be accessed.

Related systems (data mart, OLAP, OLTP, predictive analytics)[edit]
A data mart is a simple form of a data warehouse that is focused on a single subject (or functional area), hence they draw data from a limited number of sources such as sales, finance or marketing. Data marts are often built and controlled by a single department within an organization. The sources could be internal operational systems, a central data warehouse, or external data.[8] Denormalization is the norm for data modeling techniques in this system. Given that data marts generally cover only a subset of the data contained in a data warehouse, they are often easier and faster to implement.

Database Models



© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Database_model

A **database model** is a type of data model that determines the logical structure of a database and fundamentally determines in which manner data can be stored, organized and manipulated. The most popular example of a database model is the relational model, which uses a table-based format.

Examples[edit]
Common logical data models for databases include:
Hierarchical database model
It is the oldest form of data base model. It was developed by IBM for IMS (information Management System). It is a set of organized data in tree structure. DB record is a tree consisting of many groups called segments. It uses one to many relationships. The data access is also predictable

Network model
Relational model
Entity–relationship model
Enhanced entity–relationship model

Object model
Document model
Entity–attribute–value model
Star schema
An object-relational database combines the two related structures.
Physical data models include:
Inverted index
Flat file

Other models include:
Associative model
Correlational model
Multidimensional model
Multivalue model
Semantic model
XML database
Named graph
Triplestore
Relationships and functions[edit]
A given database management system may provide one or more models. The optimal structure depends on the natural organization of the application's data, and on the application's requirements, which include transaction rate (speed), reliability, maintainability, scalability, and cost. Most database management systems are built around one particular data model, although it is possible for products to offer support for more than one model.
Various physical data models can implement any given logical model. Most database software will offer the user some level of control in tuning the physical implementation, since the choices that are made have a significant effect on performance.
A model is not just a way of structuring data: it also defines a set of operations that can be performed on the data.[1] The relational model, for example, defines operations such as select (project) and join. Although these operations may not be explicit in a particular query language, they provide the foundation on which a query language is built.

Flat model[edit]
Flat File Model
Main articles: Flat file database and Spreadsheet
The flat (or table) model consists of a single, two-dimensional array of data elements, where all members of a given column are assumed to be similar values, and all

members of a row are assumed to be related to one another. For instance, columns for name and password that might be used as a part of a system security database. Each row would have the specific password associated with an individual user. Columns of the table often have a type associated with them, defining them as character data, date or time information, integers, or floating point numbers. This tabular format is a precursor to the relational model.
Early data models[edit]
These models were popular in the 1960s, 1970s, but nowadays can be found primarily in old legacy systems. They are characterized primarily by being navigational with strong connections between their logical and physical representations, and deficiencies in data independence.

Hierarchical model[edit]
Hierarchical Model
Main article: Hierarchical model
In a hierarchical model, data is organized into a tree-like structure, implying a single parent for each record. A sort field keeps sibling records in a particular order. Hierarchical structures were widely used in the early mainframe database management systems, such as the Information Management System (IMS) by IBM, and now describe the structure of XML documents. This structure allows one one-to-many relationship between two types of data. This structure is very efficient to describe many relationships in the real world; recipes, table of contents, ordering of paragraphs/verses, any nested and sorted information.

This hierarchy is used as the physical order of records in storage. Record access is done by navigating downward through the data structure using pointers combined with sequential accessing. Because of this, the hierarchical structure is inefficient for certain database operations when a full path (as opposed to upward link and sort field) is not also included for each record. Such limitations have been compensated for in later IMS versions by additional logical hierarchies imposed on the base physical hierarchy.

Network model[edit]
Network Model
Main article: Network model
The network model expands upon the hierarchical structure, allowing many-to-many relationships in a tree-like structure that allows multiple parents. It was most popular before being replaced by the relational model, and is defined by the CODASYL specification.

The network model organizes data using two fundamental concepts,

called *records* and *sets*. Records contain fields (which may be organized hierarchically, as in the programming language COBOL). Sets (not to be confused with mathematical sets) define one-to-many relationships between records: one owner, many members. A record may be an owner in any number of sets, and a member in any number of sets.
A set consists of circular linked lists where one record type, the set owner or parent, appears once in each circle, and a second record type, the subordinate or child, may appear multiple times in each circle. In this way a hierarchy may be established between any two record types, e.g., type A is the owner of B. At the same time another set may be defined where B is the owner of A. Thus all the sets comprise a general directed graph (ownership defines a direction), or *network* construct. Access to records is either sequential (usually in each record type) or by navigation in the circular linked lists.
The network model is able to represent redundancy in data more efficiently than in the hierarchical model, and there can be more than one path from an ancestor node to a descendant. The operations of the network model are navigational in style: a program maintains a current position, and navigates from one record to another by following the relationships in which the record participates. Records can also be located by supplying key values.
Although it is not an essential feature of the model, network databases generally implement the set relationships by means of pointers that directly address the location of a record on disk. This gives excellent retrieval performance, at the expense of operations such as database loading and reorganization.
Popular DBMS products that utilized it were Cincom Systems' Total
and Cullinet's IDMS. IDMS gained a considerable customer base; in the 1980s, it adopted the relational model and SQL in addition to its original tools and languages.
Most object databases (invented in the 1990s) use the navigational concept to provide fast navigation across networks of objects, generally using object identifiers as "smart" pointers to related objects. Objectivity/DB, for instance, implements named one-to-one, one-to-many, many-to-one, and many-to-many named relationships that can cross databases. Many object databases also support SQL, combining the strengths of both models.

Inverted file model[edit]
Main article: Inverted index
In an *inverted file* or *inverted index*, the contents of the data are used as keys in a lookup table, and the values in the table are pointers to the location of each instance of a given content item. This is also the logical structure of contemporary database indexes, which might only use the contents from a particular columns in the lookup table. The *inverted file data model* can put indexes in a set of files next to existing flat database files, in order to efficiently directly access needed records in these files.

Notable for using this data model is the ADABAS DBMS of Software AG, introduced in 1970. ADABAS has gained considerable customer base and exists and supported until today. In the 1980s it has adopted the relational model and SQL in addition to its original tools and languages.

Document-oriented database Clusterpoint uses inverted indexing model to provide fast full-text search for XML or JSON data objects for example.
Relational model[edit]
Two tables with a relationship
Main article: Relational model
The relational model was introduced by E.F. Codd in 1970[2] as a way to make database management systems more independent of any particular application. It is a mathematical model defined in terms of predicate logic and set theory, and implementations of it have been used by mainframe, midrange and microcomputer systems.
The products that are generally referred to as relational databases in fact implement a model that is only an approximation to the mathematical model defined by Codd. Three key terms are used extensively in relational database models: *relations*, *attributes*, and *domains*. A relation is a table with columns and rows. The named columns of the relation are called attributes, and the domain is the set of values the attributes are allowed to take.
The basic data structure of the relational model is the table, where information about a particular entity (say, an employee) is represented in rows (also called tuples) and columns. Thus, the "relation" in "relational database" refers to the various tables in the database; a relation is a set of tuples. The columns enumerate the various attributes of the entity (the employee's name, address or phone number, for example), and a row is an actual instance of the entity (a specific employee) that is represented by the relation. As a result, each tuple of the employee table represents various attributes of a single employee.
All relations (and, thus, tables) in a relational database have to adhere to some basic rules to qualify as relations. First, the ordering of columns is immaterial in a table. Second, there can't be identical tuples or rows in a table. And third, each tuple will contain a single value for each of its attributes.

A relational database contains multiple tables, each similar to the one in the "flat" database model. One of the strengths of the relational model is that, in principle, any value occurring in two different records (belonging to the same table or to different tables), implies a relationship among those two records. Yet, in order to enforce explicit integrity constraints, relationships between records in tables can also be defined explicitly, by identifying or non-identifying parent-child relationships

characterized by assigning cardinality (1:1, (0)1:M, M:M). Tables can also have a designated single attribute or a set of attributes that can act as a "key", which can be used to uniquely identify each tuple in the table.
A key that can be used to uniquely identify a row in a table is called a primary key. Keys are commonly used to join or combine data from two or more tables. For example, an *Employee* table may contain a column named *Location* which contains a value that matches the key of a *Location* table. Keys are also critical in the creation of indexes, which facilitate fast retrieval of data from large tables. Any column can be a key, or multiple columns can be grouped together into a compound key. It is not necessary to define all the keys in advance; a column can be used as a key even if it was not originally intended to be one.
A key that has an external, real-world meaning (such as a person's name, a book's ISBN, or a car's serial number) is sometimes called a "natural" key. If no natural key is suitable (think of the many people named *Brown*), an arbitrary or surrogate key can be assigned (such as by giving employees ID numbers). In practice, most databases have both generated and natural keys, because generated keys can be used internally to create links between rows that cannot break, while natural keys can be used, less reliably, for searches and for integration with other databases. (For example, records in two independently developed databases could be matched up by social security number, except when the social security numbers are incorrect, missing, or have changed.)
The most common query language used with the relational model is the Structured Query Language (SQL).

Dimensional model[edit]
The dimensional model is a specialized adaptation of the relational model used to represent data in data warehouses in a way that data can be easily summarized using online analytical processing, or OLAP queries. In the dimensional model, a database schema consists of a single large table of facts that are described using dimensions and measures. A dimension provides the context of a fact (such as who participated, when and where it happened, and its type) and is used in queries to group related facts together. Dimensions tend to be discrete and are often hierarchical; for example, the location might include the building, state, and country. A measure is a quantity describing the fact, such as revenue. It is important that measures can be meaningfully aggregated—for example, the revenue from different locations can be added together.
In an OLAP query, dimensions are chosen and the facts are grouped and aggregated together to create a summary.
The dimensional model is often implemented on top of the relational model using a star schema, consisting of one highly normalized table containing the facts, and surrounding denormalized tables containing each dimension. An alternative physical

implementation, called a snowflake schema, normalizes multi-level hierarchies within a dimension into multiple tables.
A data warehouse can contain multiple dimensional schemas that share dimension tables, allowing them to be used together. Coming up with a standard set of dimensions is an important part of dimensional modeling.

Its high performance has made the dimensional model the most popular database structure for OLAP.
Post-relational database models[edit]
Products offering a more general data model than the relational model are sometimes classified as *post-relational*.[3] Alternate terms include "hybrid database", "Object-enhanced RDBMS" and others. The data model in such products incorporates relations but is not constrained by E.F. Codd's Information Principle, which requires that

all information in the database must be cast explicitly in terms of values in relations and in no other way

— [4]

Some of these extensions to the relational model integrate concepts from technologies that pre-date the relational model. For example, they allow representation of a directed graph with trees on the nodes. The German company *sones* implements this concept in its GraphDB.
Some post-relational products extend relational systems with non-relational features. Others arrived in much the same place by adding relational features to pre-relational systems. Paradoxically, this allows products that are historically pre-relational, such as PICK and MUMPS, to make a plausible claim to be post-relational.
The resource space model (RSM) is a non-relational data model based on multi-dimensional classification.[5]

Graph model[edit]
Main article: Graph database
Graph databases allow even more general structure than a network database; any node may be connected to any other node.

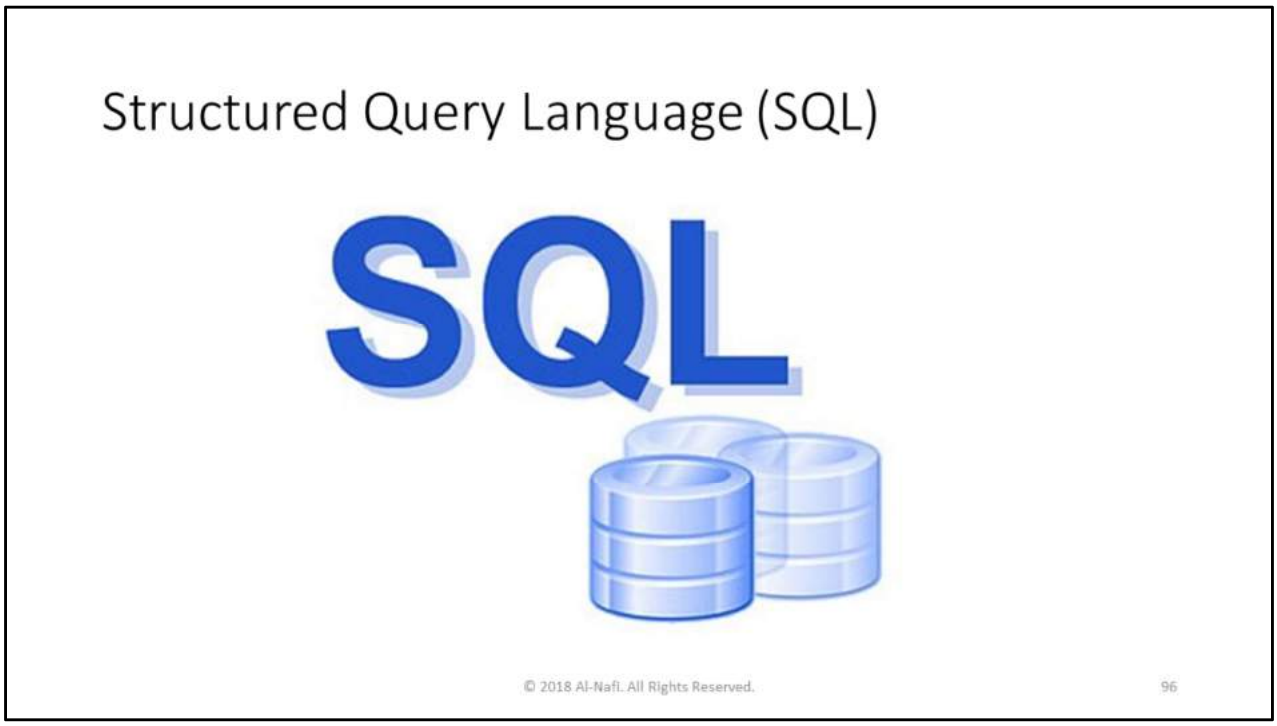
Multivalue model[edit]
Main article: MultiValue
Multivalue databases are "lumpy" data, in that they can store exactly the same way as relational databases, but they also permit a level of depth which the relational model can only approximate using sub-tables. This is nearly identical to the way XML expresses data, where a given field/attribute can have multiple right answers at the same time. Multivalue can be thought of as a compressed form of XML.
An example is an invoice, which in either multivalue or relational data could be seen

as (A) Invoice Header Table - one entry per invoice, and (B) Invoice Detail Table - one entry per line item. In the multivalue model, we have the option of storing the data as on table, with an embedded table to represent the detail: (A) Invoice Table - one entry per invoice, no other tables needed.
The advantage is that the atomicity of the Invoice (conceptual) and the Invoice (data representation) are one-to-one. This also results in fewer reads, less referential integrity issues, and a dramatic decrease in the hardware needed to support a given transaction volume.

Object-oriented database models[edit]
Object-Oriented Model
Main articles: Object-relational model and Object model
In the 1990s, the object-oriented programming paradigm was applied to database technology, creating a new database model known as object databases. This aims to avoid the object-relational impedance mismatch - the overhead of converting information between its representation in the database (for example as rows in tables) and its representation in the application program (typically as objects). Even further, the type system used in a particular application can be defined directly in the database, allowing the database to enforce the same data integrity invariants. Object databases also introduce the key ideas of object programming, such as encapsulation and polymorphism, into the world of databases.

A variety of these ways have been tried[*by whom?*]for storing objects in a database. Some[*which?*] products have approached the problem from the application programming end, by making the objects manipulated by the program persistent. This typically requires the addition of some kind of query language, since conventional programming languages do not have the ability to find objects based on their information content. Others[*which?*] have attacked the problem from the database end, by defining an object-oriented data model for the database, and defining a database programming language that allows full programming capabilities as well as traditional query facilities.
Object databases suffered because of a lack of standardization: although standards were defined by ODMG, they were never implemented well enough to ensure interoperability between products. Nevertheless, object databases have been used successfully in many applications: usually specialized applications such as engineering databases or molecular biology databases rather than mainstream commercial data processing. However, object database ideas were picked up by the relational vendors and influenced extensions made to these products and indeed to the SQL language.

An alternative to translating between objects and relational databases is to use an object-relational mapping (ORM) library.



Reference https://en.wikipedia.org/wiki/SQL

SQL (/ˌɛsˌkjuːˈɛl/ (listen) *S-Q-L*,[4] /ˈsiːkwəl/ "sequel"; **Structured Query Language**)[5][6][7] is a domain-specific language used in programming and designed for managing data held in a relational database management system (RDBMS), or for stream processing in a relational data stream management system (RDSMS). It is particularly useful in handling structured data, i.e. data incorporating relations among entities and variables.
SQL offers two main advantages over older read–write APIs such as ISAM or VSAM. Firstly, it introduced the concept of accessing many records with one single command. Secondly, it eliminates the need to specify *how* to reach a record, e.g. with or without an index.

Originally based upon relational algebra and tuple relational calculus, SQL consists of many types of statements,[8] which may be informally classed as sublanguages, commonly: a data query language (DQL),[a] a data definition language (DDL),[b] a data control language (DCL), and a data manipulation language (DML).[c][9] The scope of SQL includes data query, data manipulation

(insert, update and delete), data definition (schema creation and modification), and data access control. Although SQL is essentially a declarative language (4GL), it includes also procedural elements.
SQL was one of the first commercial languages to utilize Edgar F. Codd's relational model. The model was described in his influential 1970 paper, "A Relational Model of Data for Large Shared Data Banks".[10] Despite not entirely adhering to the relational model as described by Codd, it became the most widely used database language.[11][12]
SQL became a standard of the American National Standards Institute (ANSI) in 1986, and of the International Organization for Standardization (ISO) in 1987.[13] Since then, the standard has been revised to include a larger set of features. Despite the existence of such standards, most SQL code is not completely portable among different database systems without adjustments.

History[edit]
SQL was initially developed at IBM by Donald D. Chamberlin and Raymond F. Boyce after learning about the relational model from Ted Codd[14] in the early 1970s.[15] This version, initially called *SEQUEL* (*Structured English Query Language*), was designed to manipulate and retrieve data stored in IBM's original quasi-relational database management system, System R, which a group at IBM San Jose Research Laboratory had developed during the 1970s.[15]

Chamberlin and Boyce's first attempt of a relational database language was Square, but it was difficult to use due to subscript notation. After moving to the San Jose Research Laboratory in 1973, they began work on SEQUEL.[14] The acronym SEQUEL was later changed to SQL because "SEQUEL" was a trademark of the UK-based Hawker Siddeley Dynamics Engineering Limited company.[16]
After testing SQL at customer test sites to determine the usefulness and practicality of the system, IBM began developing commercial products based on their System R prototype including System/38, SQL/DS, and DB2, which were commercially available in 1979, 1981, and 1983, respectively.[17]

In the late 1970s, Relational Software, Inc. (now Oracle Corporation) saw the potential of the concepts described by Codd, Chamberlin, and Boyce, and developed their own SQL-based RDMS with aspirations of selling it to the U.S. Navy, Central Intelligence Agency, and other U.S. government agencies. In June 1979, Relational Software, Inc. introduced the first commercially available implementation of SQL, Oracle V2 (Version2) for VAX computers.

By 1986, ANSI and ISO standard groups officially adopted the standard "Database

Language SQL" language definition. New versions of the standard were published in 1989, 1992, 1996, 1999, 2003, 2006, 2008, 2011[14] and, most recently, 2016.

Design[edit]

SQL deviates in several ways from its theoretical foundation, the relational model and its tuple calculus. In that model, a table is a set of tuples, while in SQL, tables and query results are lists of rows: the same row may occur multiple times, and the order of rows can be employed in queries (e.g. in the LIMIT clause).

Critics argue that SQL should be replaced with a language that returns strictly to the original foundation: for example, see *The Third Manifesto*. However, no known proof exists that such uniqueness cannot be added to SQL itself[*citation needed*], or at least a variation of SQL. In other words, it's quite possible that SQL can be "fixed" or at least improved in this regard such that the industry may not have to switch to a completely different query language to obtain uniqueness. Debate on this remains open.

Syntax[edit]

Main article: SQL syntax

{\displaystyle \left.{\begin{array}{rl}\textstyle {\mathtt {UPDATE~clause}}&\{{\mathtt {UPDATE\ country}}\\\textstyle {\mathtt {SET~clause}}&\{{\mathtt {SET\ population=~}}\overbrace {\mathtt {population+1}} ^{\mathtt {expression}}\\\textstyle {\mathtt {WHERE~clause}}&\{{\mathtt {WHERE\ \underbrace {{name=}\overbrace {'USA'} ^{expression}} _{predicate};}}\end{array}}\right\}{\textstyle {\texttt {statement}}}}

A chart showing several of the SQL language elements that compose a single statement

The SQL language is subdivided into several language elements, including:

Clauses, which are constituent components of statements and queries. (In some cases, these are optional.)[18]

Expressions, which can produce either scalar values, or tables consisting of columns and rows of data

Predicates, which specify conditions that can be evaluated to SQL three-valued logic (3VL) (true/false/unknown) or Boolean truth values and are used to limit the effects of statements and queries, or to change program flow.

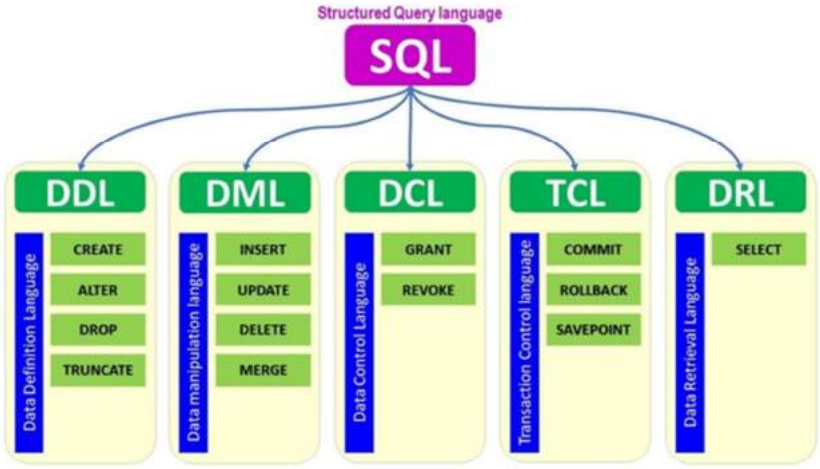
Queries, which retrieve the data based on specific criteria. This is an important element of *SQL*.

Statements, which may have a persistent effect on schemata and data, or may control transactions, program flow, connections, sessions, or diagnostics.

SQL statements also include the semicolon (";") statement terminator. Though not required on every platform, it is defined as a standard part of the SQL grammar.

Insignificant whitespace is generally ignored in SQL statements and queries, making it easier to format SQL code for readability.

SQL Sublanguages



© 2018 Al-Nafi. All Rights Reserved.

SQL actually consists of these three sublanguages:

The Data Definition Language (DDL) is used to create databases, tables, views, and keys specifying the links between tables. Because it is administrative in nature, users of SQL rarely use DDL commands as they should be restricted to database administrators.

DDL also has nothing to do with the population of the database, which is accomplished by Data Manipulation Language (DML),
used to query and extract data, insert new records, delete old records, and update existing records.

System and database administrators utilize Data Control Language (DCL) to control access to data. It provides the security
control aspects of SQL and should be the security professional's area of concern.

DDL-Data Definition Language
– Used to define Database objects like TABLE, VIEW,SEQUENCE,INDEX,SYNONYM

creation or modification or removing.
–CREATE,ALTER,DROP,TRUNCATE,RENAME are the DDL commands

DML – Data Manipulation Language
-Used to manipulate the data in Database objects like table, view, index ..etc,.
-INSERT, UPDATE, DELETE are the DML commands.

DRL/DQL-Data Retrieval Language/Data Query Language
-used to retrieve information from the database objects. it is for read only purpose.
– SELECT is the DQL or DRL command.

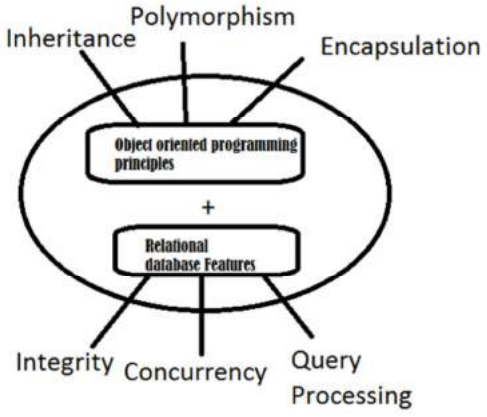
TCL – Transaction Control Language
-Transaction control statement are use to apply the changes permanently save into database.
-COMMIT, ROLLBACK, SAVEPOINT, ROLLBACK TO are the TCL commands.

DCL – Data Control Language
Data control statements are use to give privileges to access limited data or share the information between users.
-GRANT,REVOKE ,AUDIT,COMMENT, ANALYZE are the DCL commands.

SCL- Session Control Language
Session control statement are manage properties dynamically of a user session.
-ALTER SESSION,SET ROLL are the SCL commands.

Partial reference https://www.interviewsansar.com/2018/11/16/sql-sub-languages/

Object-Oriented (OO) Database Model

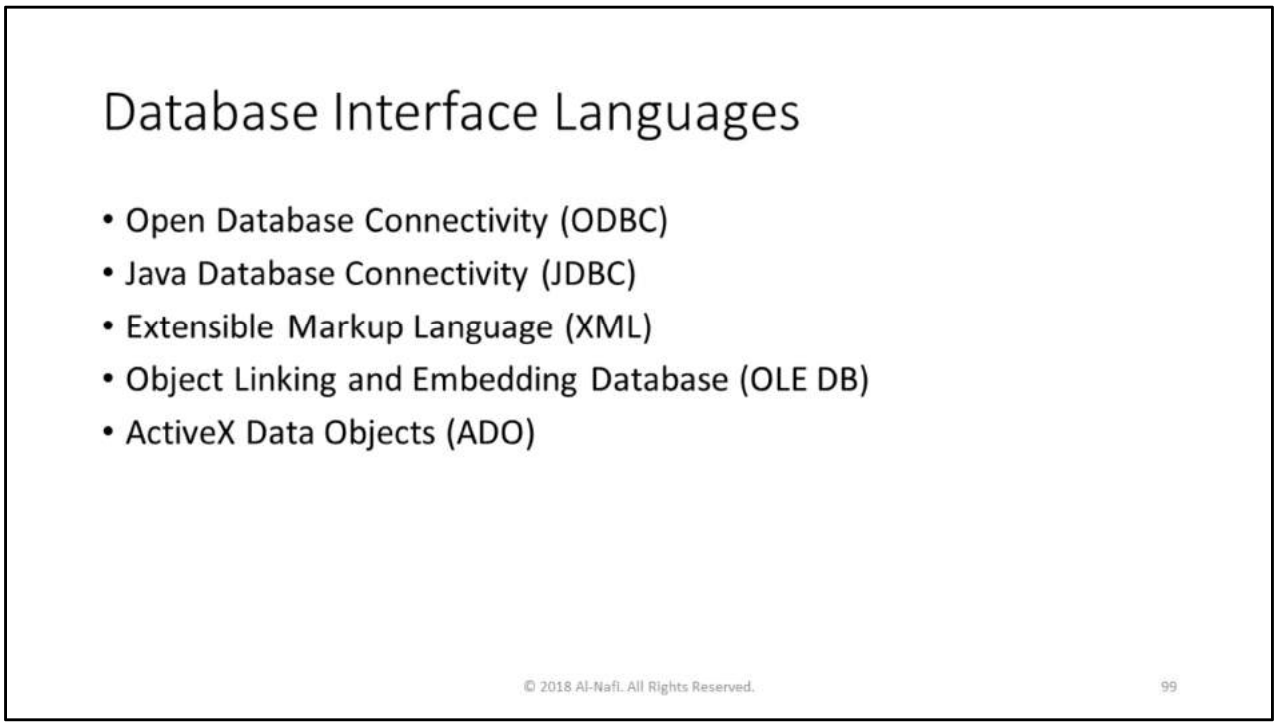


© 2018 Al-Nafi. All Rights Reserved.

The object-oriented (OO) database model is one of the newest database models. It is very similar to OOP languages, and as such, the OO database model stores data as objects. The objects are a collection of public and private data elements and the set of operations that can be executed on those data elements. Because the data objects contain their own operations, any call to data potentially has the full range of database functions available, and therefore, must be secured properly. Because of the nature of objects being the driver in this model, the OO model does not necessarily require a high-level language, such as SQL, because the functions are contained within the objects themselves. An advantage of not having a query language allows the OO DBMS to interact with applications without the language overhead. There is no need for a language in between.

A natural evolution of the above DBMS models has seen relational models being used together with OO functions and interfaces to
create what is called an object-relational model. This is basically a hybrid model, taking the advantages of each, relational and OO.
The hybrid model allows organizations to maintain their current relational database software and, at the same time, provide an upgrade path for future technologies by

supporting the OO capabilities.



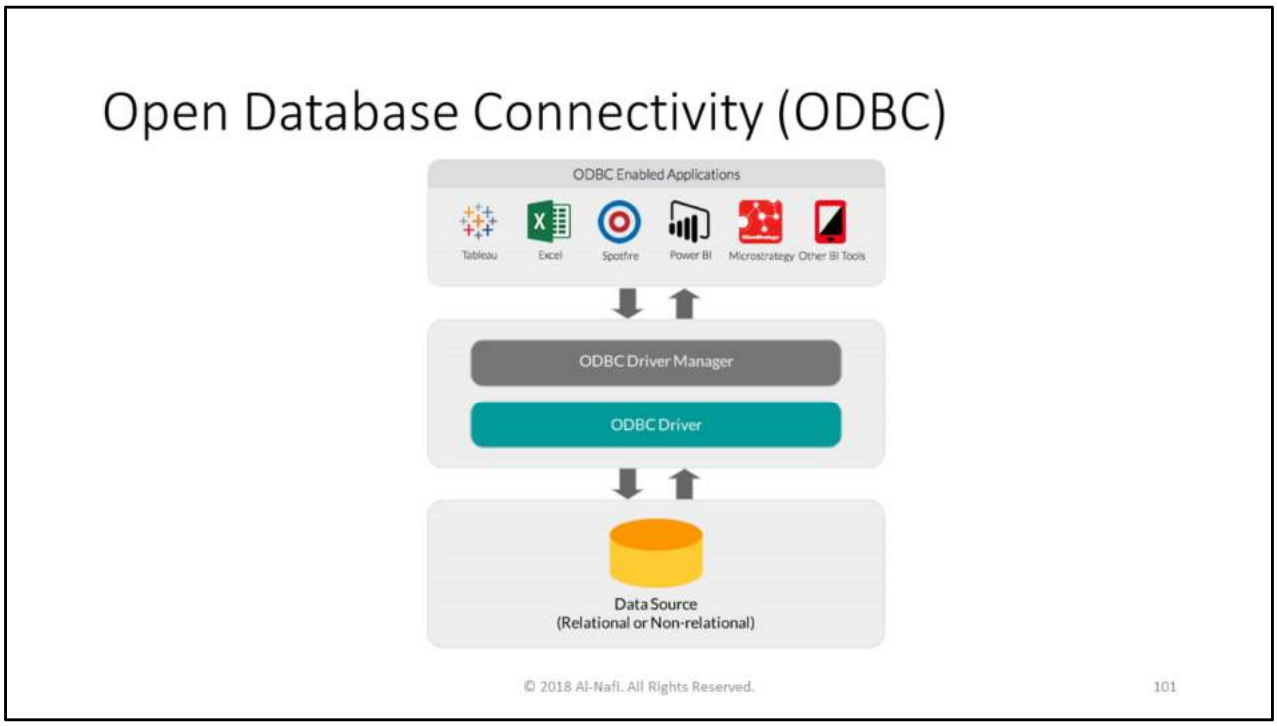
The existence of legacy databases has proven a difficult challenge for managing new database access requirements. To provide an interface that combines newer systems and legacy systems that are still being used by many organizations, several standardized access methods have evolved. These are referred to as Database Interface Languages, and some of them include the following:

- Open Database Connectivity (ODBC)
- Java Database Connectivity (JDBC)
- Extensible Markup Language (XML)
- Object Linking and Embedding Database (OLE DB)
- ActiveX Data Objects (ADO)

The purpose of all of these languages is to provide a gateway to the data contained in the legacy systems as well as the newer database systems.

- Start from here

© 2018 Al-Nafi. All Rights Reserved.



ODBC is considered to be the dominant means of standardized data access. It was developed and maintained by Microsoft, most database vendors use it as an interface method to allow an application to communicate with a database either locally or remotely over a network. It is really considered to be an API that is used to provide a connection between applications and databases. It was designed so that databases could connect without having to use specific database commands and features. It acts as the middle component that facilitates access between applications and databases.

ODBC commands are used in application programs that then translates them into the commands required by the specific database system. This allows programs to be linked between any DBMS with a minimum of code changes. It allows users to specify which database is being used and can be easily updated as new database technologies enter the market. ODBC is considered to be a very powerful tool. However, because it needs to operate as a system entity, it has vulnerabilities that can be exploited. The following is a discussion of some of the ODBC security issues.

ODBC Security Issues

- The username and password for the database are stored in plaintext. To prevent disclosure of this information, the files need to be protected. For example, if an HTML document was calling an ODBC data source, the HTML source must be protected to ensure that the username and password in plaintext cannot be read.

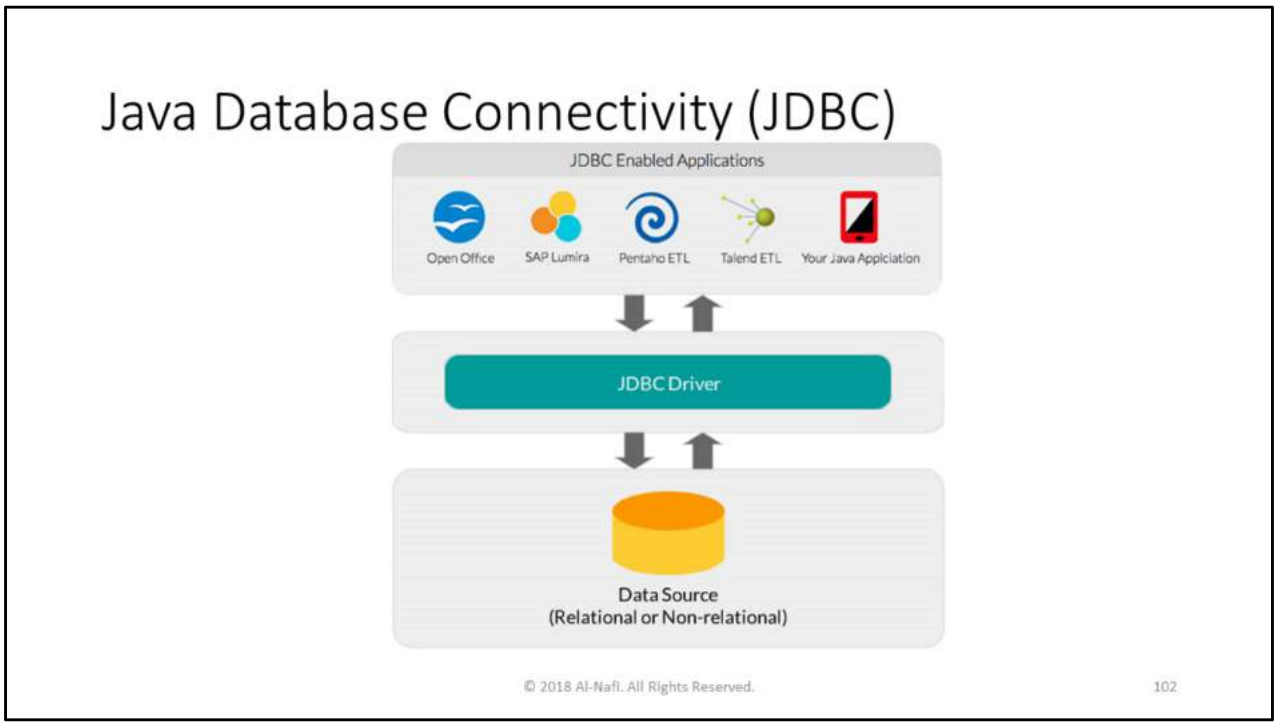
- The HTML should call a common gateway interface (CGI) that has the authentication details because HTML can be viewed in a browser.

- The returned data is sent as clear text over the network.

- Verification of the access level of the user using the ODBC application may be inadequate in some cases.

- Calling applications must be checked to ensure they do not attempt to combine data from multiple data sources, thus allowing data aggregation that may lead to unauthorized inference.

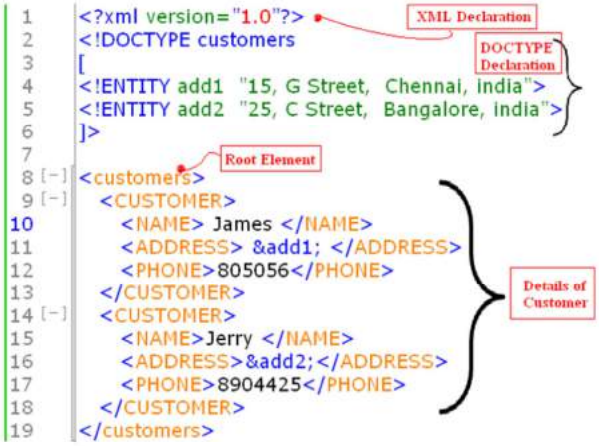
- Every calling applications or API must be checked properly to ensure they do not attempt to exploit the ODBC drivers and somehow gain elevated system access.



As we have seen above, ODBC is Microsoft's answer to providing an interface between applications and the database environment.
JDBC is Sun Microsystems' technology. It is an API used to connect Java programs to database environments. It is used to connect a
Java program to a database either directly or also by connecting through ODBC, depending on whether the database vendor has
created the necessary drivers for Java.

Regardless of the interface used to connect the user to the database, there are some very important security controls to consider in this environment. These include how and where the user will be authenticated, controlling user access properly, and auditing user actions to provide accountability. As security is very important in these environments, Java has a number of capabilities driven toward security, but these must be deliberately and properly implemented to secure the database calls and applications.

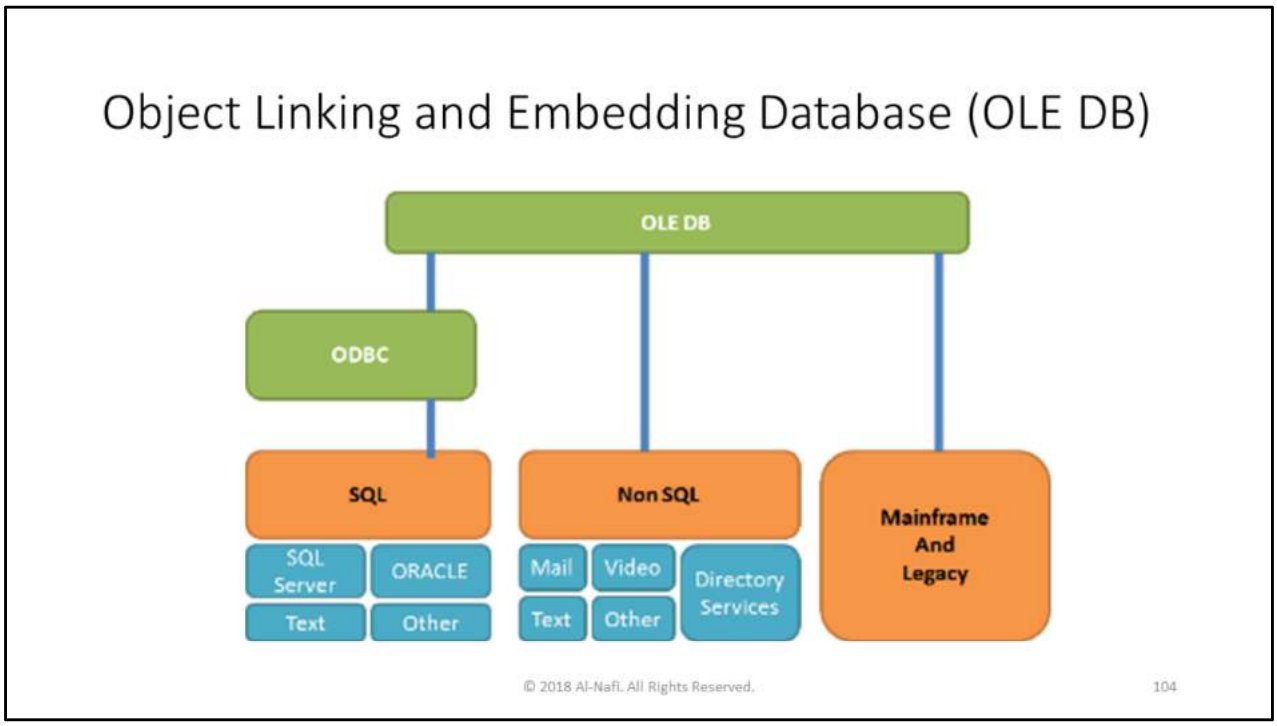
Extensible Markup Language (XML)




```
1  <?xml version="1.0"?>
2  <!DOCTYPE customers
3  [
4  <!ENTITY add1  "15, G Street,  Chennai, india">
5  <!ENTITY add2  "25, C Street,  Bangalore, india">
6  ]>
7
8  <customers>
9    <CUSTOMER>
10     <NAME> James </NAME>
11     <ADDRESS> &add1; </ADDRESS>
12     <PHONE>805056</PHONE>
13   </CUSTOMER>
14   <CUSTOMER>
15     <NAME>Jerry </NAME>
16     <ADDRESS>&add2;</ADDRESS>
17     <PHONE>8904425</PHONE>
18   </CUSTOMER>
19 </customers>
```

© 2018 Al-Nafi. All Rights Reserved.

XML is referred to as a markup language that is used to store and transport data across networks. Much like HTML, it is widely used
across the internet to represent data structures used in web services. XML can also be used to make database calls as it is used
to store and transport data, and as such, XML applications must be reviewed for how authentication of users is established, access
controls are implemented, auditing of user actions is implemented and stored, and confidentiality of sensitive data is maintained.



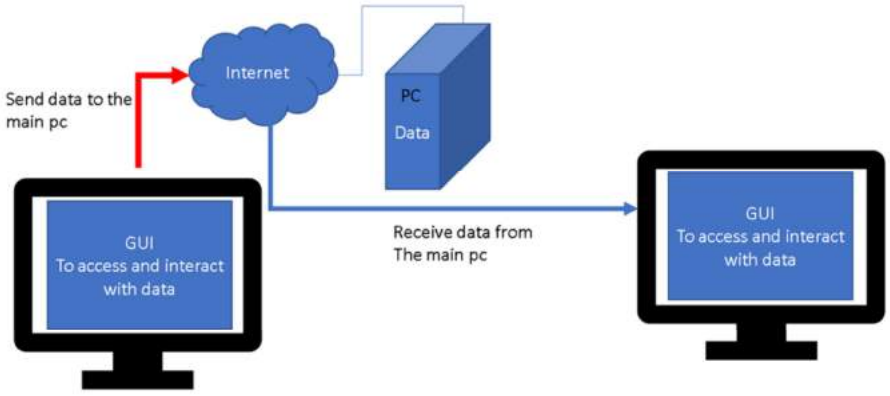
Object Linking and Embedding (OLE) is a Microsoft technology that allows an object, such as an Excel spreadsheet, to be embedded or linked to the inside of another object, such as a Word document. This capability makes OLE very flexible in

making data calls. The Component Object Model (COM) is the protocol that allows OLE to work properly. OLE allows users to
share a single source of data for a particular object. The document contains the name of the file containing the data, along with a
picture of the data. The way OLE works is that when the source is updated, all the documents using the data are also updated.
As part of the OLE technology, there is something called OLE DB, which is an interface language designed by Microsoft to link data
across various DBMSs. It is an open specification that is designed to build on the success of ODBC by providing an open standard for accessing all kinds of data across different environments. It enables organizations to easily take advantage of information contained not only in data within a database environment, but also when accessing data from other types of data sources.
The OLE DB interfaces are based on the COM, and as such, they provide

applications with uniform access to data regardless of the information source. The OLE DB separates the data into components that can run as middleware on a client or server across a wide variety of applications. The OLE DB architecture provides for components such as direct data access interfaces, query engines, cursor engines, optimizers, business rules, and transaction managers.

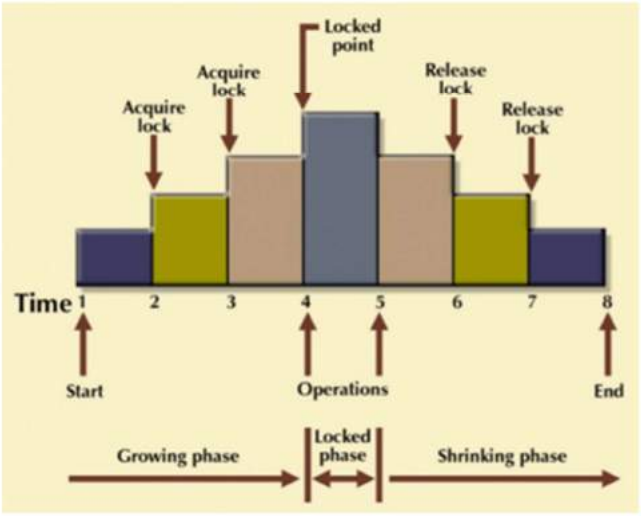
As with any powerful interface language, when organizations are developing databases and determining how data may be linked through the applications accessing those databases, security must be addressed during the development stage. If OLE DB is considered, there are optional OLE DB interfaces that can be implemented to support the administration of security information. OLE DB interfaces allow for authentication and authorization for access to data among components and applications. The OLE DB can also provide a clear view of the security mechanisms that are supported by the operating system and the database components.

Accessing Databases through the Internet



© 2018 Al-Nafi. All Rights Reserved.

DBMS Controls



© 2018 Al-Nafi. All Rights Reserved.

Reference https://www.guru99.com/dbms-concurrency-control.html

DBMS Concurrency Control: Two Phase, Timestamp, Lock-Based Protocol
What is Concurrency Control?
Concurrency control is the procedure in DBMS for managing simultaneous operations without conflicting with each another. Concurrent access is quite easy if all users are just reading data. There is no way they can interfere with one another. Though for any practical database, would have a mix of reading and WRITE operations and hence the concurrency is a challenge.
Concurrency control is used to address such conflicts which mostly occur with a multi-user system. It helps you to make sure that database transactions are performed concurrently without violating the data integrity of respective databases. Therefore, concurrency control is a most important element for the proper functioning of a system where two or multiple database transactions that require access to the same data, are executed simultaneously.

Potential problems of Concurrency

Here, are some issues which you will likely to face while using the Concurrency Control method:

Lost Updates occur when multiple transactions select the same row and update the row based on the value selected

Uncommitted dependency issues occur when the second transaction selects a row which is updated by another transaction (**dirty read**)

Non-Repeatable Read occurs when a second transaction is trying to access the same row several times and reads different data each time.

Incorrect Summary issue occurs when one transaction takes summary over the value of all the instances of a repeated data-item, and second transaction update few instances of that specific data-item. In that situation, the resulting summary does not reflect a correct result.

Why use Concurrency method?

Reasons for using Concurrency control method is DBMS:

To apply Isolation through mutual exclusion between conflicting transactions

To resolve read-write and write-write conflict issues

To preserve database consistency through constantly preserving execution obstructions

The system needs to control the interaction among the concurrent transactions. This control is achieved using concurrent-control schemes.

Concurrency control helps to ensure serializability

Example

Assume that two people who go to electronic kiosks at the same time to buy a movie ticket for the same movie and the same show time.

However, there is only one seat left in for the movie show in that particular theatre. Without concurrency control, it is possible that both moviegoers will end up purchasing a ticket. However, concurrency control method does not allow this to happen. Both moviegoers can still access information written in the movie seating database. But concurrency control only provides a ticket to the buyer who has completed the transaction process first.

Concurrency Control Protocols

Different concurrency control protocols offer different benefits between the amount of concurrency they allow and the amount of overhead that they impose.

Lock-Based Protocols

Two Phase

Timestamp-Based Protocols

Validation-Based Protocols

Lock-based Protocols

A lock is a data variable which is associated with a data item. This lock signifies that operations that can be performed on the data item. Locks help synchronize access to the database items by concurrent transactions.

All lock requests are made to the concurrency-control manager. Transactions proceed only once the lock request is granted.

Binary Locks: A Binary lock on a data item can either locked or unlocked states.

Shared/exclusive: This type of locking mechanism separates the locks based on their uses. If a lock is acquired on a data item to perform a write operation, it is called an exclusive lock.

1. Shared Lock (S):

A shared lock is also called a Read-only lock. With the shared lock, the data item can be shared between transactions. This is because you will never have permission to update data on the data item.

For example, consider a case where two transactions are reading the account balance of a person. The database will let them read by placing a shared lock. However, if another transaction wants to update that account's balance, shared lock prevent it until the reading process is over.

2. Exclusive Lock (X):

With the Exclusive Lock, a data item can be read as well as written. This is exclusive and can't be held concurrently on the same data item. X-lock is requested using lock-x instruction. Transactions may unlock the data item after finishing the 'write' operation.

For example, when a transaction needs to update the account balance of a person. You can allows this transaction by placing X lock on it. Therefore, when the second transaction wants to read or write, exclusive lock prevent this operation.

3. Simplistic Lock Protocol

This type of lock-based protocols allows transactions to obtain a lock on every object before beginning operation. Transactions may unlock the data item after finishing the 'write' operation.

4. Pre-claiming Locking

Pre-claiming lock protocol helps to evaluate operations and create a list of required data items which are needed to initiate an execution process. In the situation when all locks are granted, the transaction executes. After that, all locks release when all of its operations are over.

Starvation

Starvation is the situation when a transaction needs to wait for an indefinite period to acquire a lock.

Following are the reasons for Starvation:

When waiting scheme for locked items is not properly managed

In the case of resource leak

The same transaction is selected as a victim repeatedly

Deadlock

Deadlock refers to a specific situation where two or more processes are waiting for each other to release a resource or more than two processes are waiting for the

resource in a circular chain.

Two Phase Locking (2PL) Protocol

Two-Phase locking protocol which is also known as a 2PL protocol. It is also called P2L. In this type of locking protocol, the transaction should acquire a lock after it releases one of its locks.

This locking protocol divides the execution phase of a transaction into three different parts.

In the first phase, when the transaction begins to execute, it requires permission for the locks it needs.

The second part is where the transaction obtains all the locks. When a transaction releases its first lock, the third phase starts.

In this third phase, the transaction cannot demand any new locks. Instead, it only releases the acquired locks.

The Two-Phase Locking protocol allows each transaction to make a lock or unlock request in two steps:

Growing Phase: In this phase transaction may obtain locks but may not release any locks.

Shrinking Phase: In this phase, a transaction may release locks but not obtain any new lock

It is true that the 2PL protocol offers serializability. However, it does not ensure that deadlocks do not happen.

In the above-given diagram, you can see that local and global deadlock detectors are searching for deadlocks and solve them with resuming transactions to their initial states.

Strict Two-Phase Locking Method

Strict-Two phase locking system is almost similar to 2PL. The only difference is that Strict-2PL never releases a lock after using it. It holds all the locks until the commit point and releases all the locks at one go when the process is over.

Centralized 2PL

In Centralized 2 PL, a single site is responsible for lock management process. It has only one lock manager for the entire DBMS.

Primary copy 2PL

Primary copy 2PL mechanism, many lock managers are distributed to different sites. After that, a particular lock manager is responsible for managing the lock for a set of data items. When the primary copy has been updated, the change is propagated to the slaves.

Distributed 2PL

In this kind of two-phase locking mechanism, Lock managers are distributed to all sites. They are responsible for managing locks for data at that site. If no data is replicated, it is equivalent to primary copy 2PL. Communication costs of Distributed 2PL are quite higher than primary copy 2PL

Timestamp-based Protocols

The timestamp-based algorithm uses a timestamp to serialize the execution of concurrent transactions. This protocol ensures that every conflicting read and write operations are executed in timestamp order. The protocol uses the **System Time or Logical Count as** a Timestamp.

The older transaction is always given priority in this method. It uses system time to determine the time stamp of the transaction. This is the most commonly used concurrency protocol.

Lock-based protocols help you to manage the order between the conflicting transactions when they will execute. Timestamp-based protocols manage conflicts as soon as an operation is created.

Example:

Suppose there are there transactions T1, T2, and T3. T1 has entered the system at time 0010 T2 has entered the system at 0020 T3 has entered the system at 0030 Priority will be given to transaction T1, then transaction T2 and lastly Transaction T3.

Advantages:

Schedules are serializable just like 2PL protocols

No waiting for the transaction, which eliminates the possibility of deadlocks!

Disadvantages:

Starvation is possible if the same transaction is restarted and continually aborted

Characteristics of Good Concurrency Protocol

An ideal concurrency control DBMS mechanism has the following objectives:

Must be resilient to site and communication failures.

It allows the parallel execution of transactions to achieve maximum concurrency.

Its storage mechanisms and computational methods should be modest to minimize overhead.

It must enforce some constraints on the structure of atomic actions of transactions.

Summary

Concurrency control is the procedure in DBMS for managing simultaneous operations without conflicting with each another.

Lost Updates, dirty read, Non-Repeatable Read, and Incorrect Summary Issue are problems faced due to lack of concurrency control.

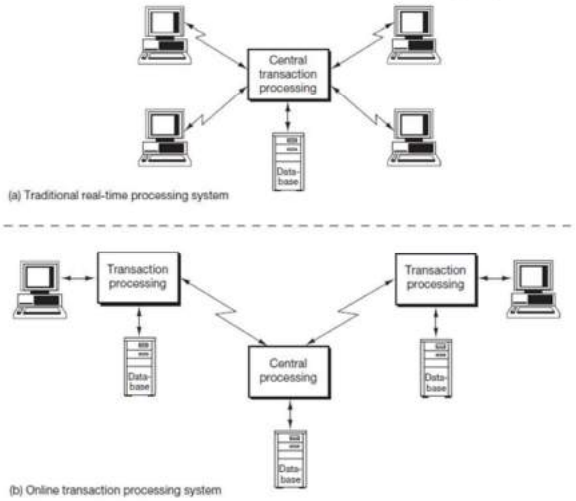
Lock-Based, Two-Phase, Timestamp-Based, Validation-Based are types of Concurrency handling protocols

The lock could be Shared (S) or Exclusive (X)

Two-Phase locking protocol which is also known as a 2PL protocol needs transaction should acquire a lock after it releases one of its locks. It has 2 phases growing and shrinking.

The timestamp-based algorithm uses a timestamp to serialize the execution of concurrent transactions. The protocol uses the **System Time or Logical Count as** a Timestamp.

Online Transaction Processing (OLTP)



© 2018 Al-Nafi. All Rights Reserved.

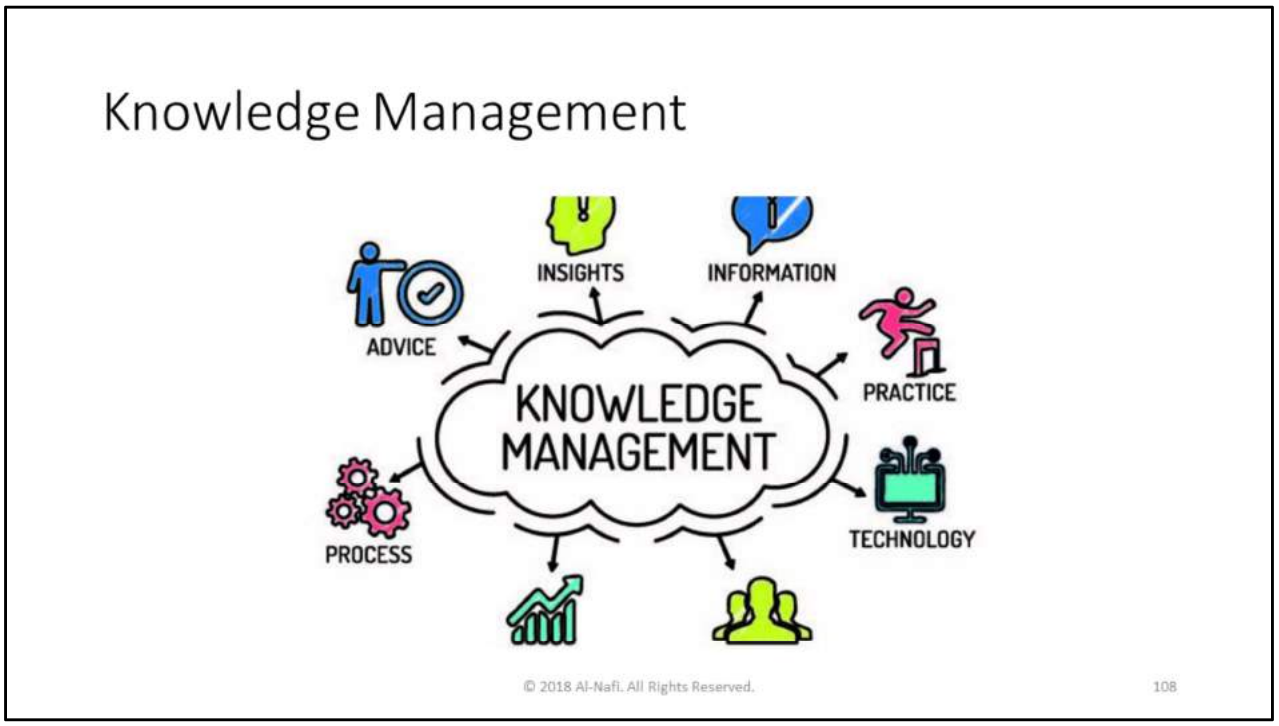
Reference https://en.wikipedia.org/wiki/Online_transaction_processing

In **Online transaction processing** (**OLTP**), information systems typically facilitate and manage transaction-oriented applications.
The term "transaction" can have two different meanings, both of which might apply: in the realm of computers or database transactions it denotes an atomic change of state, whereas in the realm of business or finance, the term typically denotes an exchange of economic entities (as used by, e.g., Transaction Processing Performance Council or commercial transactions.[1]):50 OLTP may use transactions of the first type to record transactions of the second.
OLTP has also been used to refer to processing in which the system responds immediately to user requests. An automated teller machine (ATM) for a bank is an example of a commercial transaction processing application. Online transaction processing applications have high throughput and are insert- or update-intensive in database management. These applications are used concurrently by hundreds of users. The key goals of OLTP applications are availability, speed, concurrency and recoverability.[2] Reduced paper trails and the faster, more accurate forecast for revenues and expenses are both examples of how OLTP makes things simpler for

businesses. However, like many modern online information technology solutions, some systems require offline maintenance, which further affects the cost-benefit analysis of an online transaction processing system.
OLTP is typically contrasted to OLAP (online analytical processing), which is generally characterized by much more complex queries, in a smaller volume, for the purpose of business intelligence or reporting rather than to process transactions. Whereas OLTP systems process all kinds of queries (read, insert, update and delete), OLAP is generally optimized for read only and might not even support other kinds of queries. OLTP also operates differently from batch processing and grid computing.[1]:15
OLTP is contrasted to OLEP (online event processing), which is based on distributed event logs to offer strong consistency in large-scale heterogeneous systems.[3] Whereas OLTP is associated with short atomic transactions, OLEP allows for more flexible distribution patterns and higher scalability, but with increased latency and without guaranteed upper bound to the processing time.

Overview[edit]
An OLTP system is an accessible data processing system in today's enterprises. Some examples of OLTP systems include order entry, retail sales, and financial transaction systems.[4] Online transaction processing systems increasingly require support for transactions that span a network and may include more than one company. For this reason, modern online transaction processing software uses client or server processing and brokering software that allows transactions to run on different computer platforms in a network.
In large applications, efficient OLTP may depend on sophisticated transaction management software (such as CICS) and/or database optimization tactics to facilitate the processing of large numbers of concurrent updates to an OLTP-oriented database.
For even more demanding decentralized database systems, OLTP brokering programs can distribute transaction processing among multiple computers on a network. OLTP is often integrated into service-oriented architecture (SOA) and Web services.
Online transaction processing (OLTP) involves gathering input information, processing the data and updating existing data to reflect the collected and processed information. As of today, most organizations use a database management system to support OLTP. OLTP is carried in a client-server system.
Online transaction process concerns about concurrency and atomicity. Concurrency controls guarantee that two users accessing the same data in the database system will not be able to change that data or the user has to wait until the other user has finished processing, before changing that piece of data. Atomicity controls guarantee that all the steps in a transaction are completed successfully as a group. That is, if any steps between the transaction fail, all other steps must fail also.[5]
Systems design[edit]

To build an OLTP system, a designer must know that the large number of concurrent users does not interfere with the system's performance. To increase the performance of an OLTP system, a designer must avoid excessive use of indexes and clusters.
The following elements are crucial for the performance of OLTP systems:[2]
Rollback segments
Rollback segments are the portions of database that record the actions of transactions in the event that a transaction is rolled back. Rollback segments provide read consistency, rollback transactions, and recovery of the database.[6]Clusters
A cluster is a schema that contains one or more tables that have one or more columns in common. Clustering tables in a database improves the performance of join operations.[7]Discrete transactions
A discrete transaction defers all change to the data until the transaction is committed. It can improve the performance of short, non-distributed transactions.[8]Block size
The data block size should be a multiple of the operating system's block size within the maximum limit to avoid unnecessary I/O.[9]Buffer cache size
SQL statements should be tuned to use the database buffer cache to avoid unnecessary resource consumption.[10]Dynamic allocation of space to tables and rollback segments
Transaction processing monitors and the multi-threaded server
A transaction processing monitor is used for coordination of services. It is like an operating system and does the coordination at a high level of granularity and can span multiple computing devices.[11]Partition (database)
Partition use increases performance for sites that have regular transactions while still maintaining availability and security.Database tuning
With database tuning, an OLTP system can maximize its performance as efficiently and rapidly as possible.



Reference https://en.wikipedia.org/wiki/Knowledge_management

Knowledge management (**KM**) is the process of creating, sharing, using and managing the knowledge and information of an organisation.[1] It refers to a multidisciplinary approach to achieving organisational objectives by making the best use of knowledge.[2]

An established discipline since 1991[*citation needed*], KM includes courses taught in the fields of business administration, information systems, management, library, and information sciences.[3][4] Other fields may contribute to KM research, including information and media, computer science, public health and public policy.[5] Several universities offer dedicated master's degrees in knowledge management.

Many large companies, public institutions and non-profit organisations have resources dedicated to internal KM efforts, often as a part of their business strategy, IT, or human resource management departments.[6] Several consulting companies provide advice regarding KM to these organisations.[6]

Knowledge management efforts typically focus on organisational objectives such as improved performance, competitive advantage, innovation, the sharing of lessons learned, integration and continuous improvement of the organisation.[7] These

efforts overlap with organisational learning and may be distinguished from that by a greater focus on the management of knowledge as a strategic asset and on encouraging the sharing of knowledge.[2][8] KM is an enabler of organisational learning.[9][10]

History[edit]

Knowledge management efforts have a long history, including on-the-job discussions, formal apprenticeship, discussion forums, corporate libraries, professional training, and mentoring programs.[2][10] With increased use of computers in the second half of the 20th century, specific adaptations of technologies such as knowledge bases, expert systems, information repositories, group decision support systems, intranets, and computer-supported cooperative work have been introduced to further enhance such efforts.[2]

In 1999, the term personal knowledge management was introduced; it refers to the management of knowledge at the individual level.[11]

In the enterprise, early collections of case studies recognised the importance of knowledge management dimensions of strategy, process and measurement.[12][13] Key lessons learned include people and the cultural norms which influence their behaviors are the most critical resources for successful knowledge creation, dissemination and application; cognitive, social and organisational learning processes are essential to the success of a knowledge management strategy; and measurement, benchmarking and incentives are essential to accelerate the learning process and to drive cultural change.[13] In short, knowledge management programs can yield impressive benefits to individuals and organisations if they are purposeful, concrete and action-orientated.

Research[edit]

KM emerged as a scientific discipline in the early 1990s.[14] It was initially supported by individual practitioners, when Skandia hired Leif Edvinsson of Sweden as the world's first Chief Knowledge Officer (CKO).[15] Hubert Saint-Onge (formerly of CIBC, Canada), started investigating KM long before that.[2] The objective of CKOs is to manage and maximise the intangible assets of their organisations.[2] Gradually, CKOs became interested in practical and theoretical aspects of KM, and the new research field was formed.[16] The KM idea has been taken up by academics, such as Ikujiro Nonaka (Hitotsubashi University), Hirotaka Takeuchi (Hitotsubashi University), Thomas H. Davenport (Babson College) and Baruch Lev (New York University).[3][17]

In 2001, Thomas A. Stewart, former editor at *Fortune* magazine and subsequently the editor of *Harvard Business Review*, published a cover story highlighting the importance of intellectual capital in organisations.[18] The KM discipline has been gradually moving towards academic maturity.[2] First, is a trend toward higher

cooperation among academics; single-author publications are less common. Second, the role of practitioners has changed.[16] Their contribution to academic research declined from 30% of overall contributions up to 2002, to only 10% by 2009.[19] Third, the number of academic knowledge management journals has been steadily growing, currently reaching 27 outlets.[20]

Multiple KM disciplines exist; approaches vary by author and school.[16][21] As the discipline matured, academic debates increased regarding theory and practice, including:

Techno-centric with a focus on technology, ideally those that enhance knowledge sharing and creation.[22][23]

Organisational with a focus on how an organisation can be designed to facilitate knowledge processes best.[6]

Ecological with a focus on the interaction of people, identity, knowledge, and environmental factors as a complex adaptive system akin to a natural ecosystem.[24][25]

Regardless of the school of thought, core components of KM roughly include people/culture, processes/structure and technology. The details depend on the perspective.[26] KM perspectives include:

community of practice[27]

social network analysis[28]

intellectual capital[29]

information theory[14][15]

complexity science[30]

constructivism[31][32]

The practical relevance of academic research in KM has been questioned[33] with action research suggested as having more relevance[34] and the need to translate the findings presented in academic journals to a practice.[12]

Dimensions[edit]

Different frameworks for distinguishing between different 'types of' knowledge exist.[10] One proposed framework for categorising the dimensions of knowledge distinguishes tacit knowledge and explicit knowledge.[30] Tacit knowledge represents internalised knowledge that an individual may not be consciously aware of, such as to accomplish particular tasks. At the opposite end of the spectrum, explicit knowledge represents knowledge that the individual holds consciously in mental focus, in a form that can easily be communicated to others.[16][35]

The Knowledge Spiral as described by Nonaka & Takeuchi.

Ikujiro Nonaka proposed a model (SECI, for Socialisation, Externalisation, Combination, Internalisation) which considers a spiraling interaction between explicit knowledge and tacit knowledge.[36] In this model, knowledge follows a cycle in which implicit knowledge is 'extracted' to become explicit knowledge, and explicit knowledge is 're-internalised' into implicit knowledge.[36]

Hayes and Walsham (2003) describe knowledge and knowledge management as two different perspectives.[37] The content perspective suggests that knowledge is easily stored; because it may be codified, while the relational perspective recognises the contextual and relational aspects of knowledge which can make knowledge difficult to share outside the specific context in which it is developed.[37]
Early research suggested that KM needs to convert internalised tacit knowledge into explicit knowledge to share it, and the same effort must permit individuals to internalise and make personally meaningful any codified knowledge retrieved from the KM effort.[6][38]
Subsequent research suggested that a distinction between tacit knowledge and explicit knowledge represented an oversimplification and that the notion of explicit knowledge is self-contradictory.[11] Specifically, for knowledge to be made explicit, it must be translated into information (i.e., symbols outside our heads).[11][39] More recently, together with Georg von Krogh and Sven Voelpel, Nonaka returned to his earlier work in an attempt to move the debate about knowledge conversion forward.[4][40]
A second proposed framework for categorising knowledge dimensions distinguishes embedded knowledge of a system outside a human individual (e.g., an information system may have knowledge embedded into its design) from embodied knowledge representing a learned capability of a human body's nervous and endocrine systems.[41]
A third proposed framework distinguishes between the exploratory creation of "new knowledge" (i.e., innovation) vs. the transfer or exploitation of "established knowledge" within a group, organisation, or community.[37][42] Collaborative environments such as communities of practice or the use of social computing tools can be used for both knowledge creation and transfer.[42]

Strategies[edit]

Knowledge may be accessed at three stages: before, during, or after KM-related activities.[29] Organisations have tried knowledge capture incentives, including making content submission mandatory and incorporating rewards into performance measurement plans.[43] Considerable controversy exists over whether such incentives work and no consensus has emerged.[7]
One strategy to KM involves actively managing knowledge (push strategy).[7][44] In such an instance, individuals strive to explicitly encode their knowledge into a shared knowledge repository, such as a database, as well as retrieving knowledge they need that other individuals have provided (codification).[44]
Another strategy involves individuals making knowledge requests of experts associated with a particular subject on an ad hoc basis (pull strategy).[7][44] In such an instance, expert individual(s) provide insights to requestor (personalisation).[30]
Hansen et al. defined the two strategies.[45] Codification focuses on collecting and storing codified knowledge in electronic databases to make it

accessible.[46] Codification can therefore refer to both tacit and explicit knowledge.[47] In contrast, personalisation encourages individuals to share their knowledge directly.[46] Information technology plays a less important role, as it is only facilitates communication and knowledge sharing.

Other knowledge management strategies and instruments for companies include:[7][24][30]

Knowledge sharing (fostering a culture that encourages the sharing of information, based on the concept that knowledge is not irrevocable and should be shared and updated to remain relevant)

- Make knowledge-sharing a key role in employees' job description
- Inter-project knowledge transfer
- Intra-organisational knowledge sharing
- Inter-organisational knowledge sharing

Proximity & architecture (the physical situation of employees can be either conducive or obstructive to knowledge sharing)

Storytelling (as a means of transferring tacit knowledge)

Cross-project learning

After-action reviews

Knowledge mapping (a map of knowledge repositories within a company accessible by all)

Communities of practice

Expert directories (to enable knowledge seeker to reach to the experts)

Expert systems (knowledge seeker responds to one or more specific questions to reach knowledge in a repository)

Best practice transfer

Knowledge fairs

Competency-based management (systematic evaluation and planning of knowledge related competences of individual organisation members)

Master–apprentice relationship, Mentor-mentee relationship, job shadowing

Collaborative software technologies (wikis, shared bookmarking, blogs, social software, etc.)

Knowledge repositories (databases, bookmarking engines, etc.)

Measuring and reporting intellectual capital (a way of making explicit knowledge for companies)

Knowledge brokers (some organisational members take on responsibility for a specific "field" and act as first reference on a specific subject)

Motivations[edit]

Multiple motivations lead organisations to undertake KM.[35] Typical considerations include:[30]

Making available increased knowledge content in the development and provision of products and services

Achieving shorter development cycles
Facilitating and managing innovation and organisational learning
Leveraging expertises across the organisation
Increasing network connectivity between internal and external individuals
Managing business environments and allowing employees to obtain relevant insights and ideas appropriate to their work
Solving intractable or wicked problems
Managing intellectual capital and assets in the workforce (such as the expertise and know-how possessed by key individuals or stored in repositories)

KM technologies[edit]

Knowledge management (KM) technology can be categorised:

Groupware—Software that facilitates collaboration and sharing of organisational information. Such applications provide tools for threaded discussions, document sharing, organisation-wide uniform email, and other collaboration-related features.
Workflow systems—Systems that allow the representation of processes associated with the creation, use and maintenance of organisational knowledge, such as the process to create and utilise forms and documents.
Content management and document management systems—Software systems that automate the process of creating web content and/or documents. Roles such as editors, graphic designers, writers and producers can be explicitly modeled along with the tasks in the process and validation criteria. Commercial vendors started either to support documents or to support web content but as the Internet grew these functions merged and vendors now perform both functions.
Enterprise portals—Software that aggregates information across the entire organisation or for groups such as project teams.
eLearning—Software that enables organisations to create customised training and education. This can include lesson plans, monitoring progress and online classes.
Planning and scheduling software—Software that automates schedule creation and maintenance. The planning aspect can integrate with project management software.[22]
Telepresence—Software that enables individuals to have virtual "face-to-face" meetings without assembling at one location. Videoconferencing is the most obvious example.
Ontological Approach—An ontology-based knowledge model for knowledge management. This model can facilitate knowledge discovery that provides users with insight for decision making.[48]

These categories overlap. Workflow, for example, is a significant aspect of a content or document management systems, most of which have tools for developing enterprise portals.[7][49]

Proprietary KM technology products such as Lotus Notes defined proprietary formats for email, documents, forms, etc. The Internet drove most vendors to adopt Internet

formats. Open-source and freeware tools for the creation of blogs and wikis now enable capabilities that used to require expensive commercial tools.[34][50]
KM is driving the adoption of tools that enable organisations to work at the semantic level,[51] as part of the Semantic Web.[52] Some commentators have argued that after many years the Semantic Web has failed to see widespread adoption,[53][54][55] while other commentators have argued that it has been a success.[56]
Knowledge Barriers[edit]
Just like knowledge transfer and knowledge sharing, the term “knowledge barriers” is not a uniformly defined term and differs in its meaning depending on the author. [57] Knowledge barriers can be associated with high costs for both companies and individuals[58] [59] [60].

Web Application Environment



© 2018 Al-Nafi. All Rights Reserved.

The web application environment is where web applications run on a server and hosts the interface that web users use to interact with organizations. As the web application environment is accessible to everyone out on the web, it becomes really important to protect theentire web application architecture and its components. If the web server can be compromised in some way, it may offer the attacker a platform from which to mount probes or other nefarious activities. Also, such unauthorized access may provide the attacker with intelligence about the organization such as corporate sales and projects and can also provide a way by which the attacker may be able to gain access to the enterprise's proprietary and sensitive intellectual property. Current statistics indicate that most attacks are conducted at the application level, either against the web server application itself, in-house scripts, or the common front-end applications used for e-commerce activities. There are many vulnerabilities and exploits that exist in the application layer, especially the web application environment. Therefore, attacks on the application software are much more likely to succeed than attacks on the underlying platforms. Once the application has been breached, an attack on the operating system, and other components of the architecture becomes generally possible.

Factors that Make Websites Vulnerable

- Websites are designed to be widely accessible and are usually heavily advertised as well, therefore, a very large number of people will have information about the web site and its architecture.
- Web server software does make provisions for logging of traffic, but many administrators either turn off logging altogether or reduce the logging to minimal levels.
- The standard security tools of firewalls and intrusion detection systems can be applied but are not particularly well suited to protecting such public websites:
    - In the case of firewalls, a website must have standard ports open for specific traffic.
    - Intrusion detection systems (IDSs) must be tuned properly and maintained adequately to provide any useful information from the flood of data. Websites will see all kinds of traffic, from different locations, requesting connections, web pages, submitting form information, or even updating search engine facts.

Web Application Threats and Protection

Specific protections that may be helpful include the following:

- Having a particular assurance sign-off process for web servers
- Hardening the operating system used on such servers, which would include at the very least removing default configurations and accounts, configuring permissions and privileges correctly, and keeping up to date with vendor patches
- Extending web and network vulnerability scans prior to deployment
- Deploying IDS and advanced intrusion prevention system (IPS) technology
- Using application proxy firewalls

Disabling any unnecessary documentation and libraries
Ensure administrative interfaces are removed or secured appropriately
Only allow access from authorized hosts or networks, and then use strong (multi-factor) user authentication
Do not hard code the authentication credentials into the application itself, and ensure the security of the credentials using certificates or similar high-trust authentication mechanisms
Use account lockout and extended logging and audit, and protect all authentication traffic with encryption
Ensure the interface is at least as secure as the rest of the application and most often secure it at a higher level

Because of the accessibility of web systems and applications, and the vulnerabilities

and exploits available, input validation becomes essential to address as part of securing this environment. Application proxy firewalls are very effective, but they need to make
sure the proxies are able to deal with problems of known exploits such as buffer overflows, authentication issues, scripting, the passing of commands to the underlying platform (that includes issues related to database engines, such as SQL commands), encoding issues (such as Unicode), and URL encoding and translation. In particular, the application proxy firewalls may need to address issues of the passing of input data to in-house and custom-developed software, ensuring validation of input to those systems. In other words, the biggest challenge when data is being passed from anything to anything else becomes adequate data validation.

In regard to session management, we need to remember that Hypertext Transfer Protocol (HTTP) is a stateless technology, and therefore, periods of apparent attachment to the server are controlled by other technologies, such as cookies or URL data, that must be both protected and validated. If cookies are needed, or allowed, they should always be encrypted. Also, time validation needs to be included as part of session management, which typically means to disallow sequential, calculable, or predictable cookies, session numbers, or URL data. Instead, always use random and unique indicators.

As usual, with any application related environment, web application environments should always validate all input and output, fail secure (closed), and make your application or system as simple as possible. Use secure network design and penetration testing to validate secure designs and to identify potential vulnerabilities and threats to be mitigated and use defense in depth. Some other specific security controls to consider in a web system are not to cache secure pages, confirm that all encryption used meets industry standards, monitor your code vendors for security patches and alerts, log any and all critical transactions and milestones, handle exceptions properly, do not trust any data from the client, and do not automatically trust data from other servers, partners, or any other part of the application itself.



Reference https://www.owasp.org/index.php/Main_Page

Please review the above link properly.

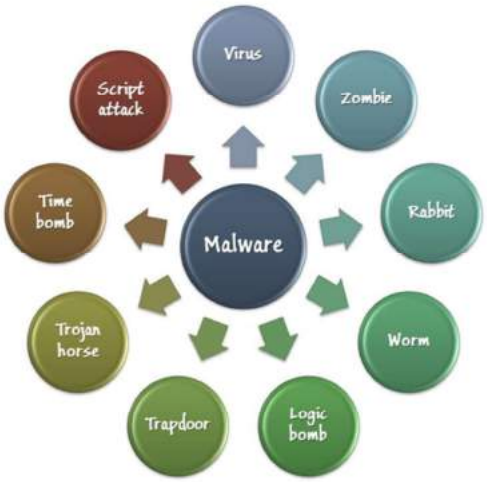
Open Web Application Security Project (OWASP) Framework

One very helpful resource for the secure development of web environments, including web applications, is the Open Web Application Security Web Project (OWASP). OWASP provides a number of helpful frameworks focused on the secure deployment
of web applications. OWASP has several guides and resources available for secure web application development:

- Development Guide
- Code Review Guide
- Testing Guide
- Top Ten Web Application Security Vulnerabilities
- OWASP Mobile

Given the prevalence of web-based and cloud-based solutions that organizations can standardize on, OWASP provides an easily accessible and complete framework with processes for web application security that has become very valuable in current web application environments. The security professional should be familiar with the “top ten” web application vulnerabilities and also how to mitigate them. This knowledge needs to be enforced in web development and deployment areas of the organization, together with other valuable resources from OWASP and possibly other frameworks used by professionals and stakeholders involved in web solution deployment.

Malicious Software (Malware)



© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Malware

Malware (a portmanteau for **malicious software**) is any software intentionally designed to cause damage to a computer, server, client, or computer network[1][2] (by contrast, software that causes *unintentional* harm due to some deficiency is typically described as a software bug). A wide variety of types of malware exist, including computer viruses, worms, Trojan horses, ransomware, spyware, adware, and scareware.
Programs are also considered malware if they secretly act against the interests of the computer user. For example, at one point Sony music Compact discs silently installed a rootkit on purchasers' computers with the intention of preventing illicit copying, but which also reported on users' listening habits, and unintentionally created extra security vulnerabilities.[3]
A range of antivirus software, firewalls and other strategies are used to help protect against the introduction of malware, to help detect it if it is already present, and to recover from malware-associated malicious activity and attacks.[4]

Purposes[edit]
Many early infectious programs, including the first Internet Worm, were written as experiments or pranks. Today, malware is used by both black hat hackers and governments, to steal personal, financial, or business information.[5][6]
Malware is sometimes used broadly against government or corporate websites to gather guarded information,[7] or to disrupt their operation in general. However, malware can be used against individuals to gain information such as personal identification numbers or details, bank or credit card numbers, and passwords.
Since the rise of widespread broadband Internet access, malicious software has more frequently been designed for profit. Since 2003, the majority of widespread viruses and worms have been designed to take control of users' computers for illicit purposes.[8] Infected "zombie computers" can be used to send email spam, to host contraband data such as child pornography,[9] or to engage in distributed denial-of-service attacks as a form of extortion.[10]
Programs designed to monitor users' web browsing, display unsolicited advertisements, or redirect affiliate marketing revenues are called spyware. Spyware programs do not spread like viruses; instead they are generally installed by exploiting security holes. They can also be hidden and packaged together with unrelated user-installed software.[11] The Sony BMG rootkit was intended to preventing illicit copying; but also reported on users' listening habits, and unintentionally created extra security vulnerabilities.[3]
Ransomware affects an infected computer system in some way, and demands payment to bring it back to its normal state. There are two variations of ransomware, being crypto ransomware and locker ransomware.[12] With the locker ransomware just locking down a computer system without encrypting its contents. Whereas the traditional ransomware is one that locks down your system and encrypts the contents of the system. For example, programs such as CryptoLocker encrypt files securely, and only decrypt them on payment of a substantial sum of money.[13]
Some malware is used to generate money by click fraud, making it appear that the computer user has clicked an advertising link on a site, generating a payment from the advertiser. It was estimated in 2012 that about 60 to 70% of all active malware used some kind of click fraud, and 22% of all ad-clicks were fraudulent.[14]
In addition to criminal money-making, malware can be used for sabotage, often for political motives. Stuxnet, for example, was designed to disrupt very specific industrial equipment. There have been politically motivated attacks that have spread over and shut down large computer networks, including massive deletion of files and corruption of master boot records, described as "computer killing." Such attacks were made on Sony Pictures Entertainment (25 November 2014, using malware known as Shamoon or W32.Disttrack) and Saudi Aramco (August 2012).[15][16]
Infectious malware[edit]
Main articles: Computer virus and Computer worm

The best-known types of malware, viruses and worms, are known for the manner in which they spread, rather than any specific types of behavior. A computer virus is software that embeds itself in some other executable software (including the operating system itself) on the target system without the user's knowledge and consent and when it is run, the virus is spread to other executables. On the other hand, a *worm* is a stand-alone malware software that *actively* transmits itself over a network to infect other computers. These definitions lead to the observation that a virus requires the user to run an infected software or operating system for the virus to spread, whereas a worm spreads itself.[17]

Concealment[edit]

These categories are not mutually exclusive, so malware may use multiple techniques.[18] This section only applies to malware designed to operate undetected, not sabotage and ransomware.

See also: Polymorphic packer

Viruses[edit]

Main article: Computer virus

A computer virus is software usually hidden within another seemingly innocuous program that can produce copies of itself and insert them into other programs or files, and that usually performs a harmful action (such as destroying data).[19] An example of this is a PE infection, a technique, usually used to spread malware, that inserts extra data or executable code into PE files.[20]

Screen-locking ransomware[edit]

Main article: Ransomware

'Lock-screens', or screen lockers is a type of “cyber police” ransomware that blocks screens on Windows or Android devices with a false accusation in harvesting illegal content, trying to scare the victims into paying up a fee.[21] Jisut and SLocker impact Android devices more than other lock-screens, with Jisut making up nearly 60 percent of all Android ransomware detections.[22]

Trojan horses[edit]

Main article: Trojan horse (computing)

A Trojan horse is a harmful program that misrepresents itself to masquerade as a regular, benign program or utility in order to persuade a victim to install it. A Trojan horse usually carries a hidden destructive function that is activated when the application is started. The term is derived from the Ancient Greek story of the Trojan horse used to invade the city of Troy by stealth.[23][24][25][26][27]

Trojan horses are generally spread by some form of social engineering, for example, where a user is duped into executing an e-mail attachment disguised to be unsuspicious, (e.g., a routine form to be filled in), or by drive-by download. Although their payload can be anything, many modern forms act as a backdoor, contacting a controller which can then have unauthorized access to the affected computer.[28] While Trojan horses and backdoors are not easily detectable by

themselves, computers may appear to run slower due to heavy processor or network usage.
Unlike computer viruses and worms, Trojan horses generally do not attempt to inject themselves into other files or otherwise propagate themselves.[29]
In spring 2017 Mac users were hit by the new version of Proton Remote Access Trojan (RAT)[30] trained to extract password data from various sources, such as browser auto-fill data, the Mac-OS keychain, and password vaults.[31]

Rootkits[edit]

Main article: Rootkit

Once malicious software is installed on a system, it is essential that it stays concealed, to avoid detection. Software packages known as *rootkits* allow this concealment, by modifying the host's operating system so that the malware is hidden from the user. Rootkits can prevent a harmful process from being visible in the system's list of processes, or keep its files from being read.[32]
Some types of harmful software contain routines to evade identification and/or removal attempts, not merely to hide themselves. An early example of this behavior is recorded in the Jargon File tale of a pair of programs infesting a Xerox CP-V time sharing system:

> Each ghost-job would detect the fact that the other had been killed, and would start a new copy of the recently stopped program within a few milliseconds. The only way to kill both ghosts was to kill them simultaneously (very difficult) or to deliberately crash the system.[33]

Backdoors[edit]

Main article: Backdoor (computing)

A backdoor is a method of bypassing normal authentication procedures, usually over a connection to a network such as the Internet. Once a system has been compromised, one or more backdoors may be installed in order to allow access in the future,[34] invisibly to the user.
The idea has often been suggested that computer manufacturers preinstall backdoors on their systems to provide technical support for customers, but this has never been reliably verified. It was reported in 2014 that US government agencies had been diverting computers purchased by those considered "targets" to secret workshops where software or hardware permitting remote access by the agency was installed, considered to be among the most productive operations to obtain access to networks around the world.[35] Backdoors may be installed by Trojan horses, worms, implants, or other methods.[36][37]

Evasion[edit]

Since the beginning of 2015, a sizable portion of malware utilizes a combination of many techniques designed to avoid detection and analysis.[38] From the more common, to the least common:
evasion of analysis and detection by fingerprinting the environment when executed.[39]

confusing automated tools' detection methods. This allows malware to avoid detection by technologies such as signature-based antivirus software by changing the server used by the malware.[40]
timing-based evasion. This is when malware runs at certain times or following certain actions taken by the user, so it executes during certain vulnerable periods, such as during the boot process, while remaining dormant the rest of the time.
obfuscating internal data so that automated tools do not detect the malware.[41]
An increasingly common technique (2015) is adware that uses stolen certificates to disable anti-malware and virus protection; technical remedies are available to deal with the adware.[42]
Nowadays, one of the most sophisticated and stealthy ways of evasion is to use information hiding techniques, namely stegomalware. A survey on stegomalware was published by Cabaj et al. in 2018.[43]

Vulnerability[edit]

Main article: Vulnerability (computing)

In this context, and throughout, what is called the "system" under attack may be anything from a single application, through a complete computer and operating system, to a large network.
Various factors make a system more vulnerable to malware:

Security defects in software[edit]

Malware exploits security defects (security bugs or vulnerabilities) in the design of the operating system, in applications (such as browsers, e.g. older versions of Microsoft Internet Explorer supported by Windows XP[44]), or in vulnerable versions of browser plugins such as Adobe Flash Player, Adobe Acrobat or Reader, or Java SE.[45][46] Sometimes even installing new versions of such plugins does not automatically uninstall old versions. Security advisories from plug-in providers announce security-related updates.[47] Common vulnerabilities are assigned CVE IDs and listed in the US National Vulnerability Database. Secunia PSI[48] is an example of software, free for personal use, that will check a PC for vulnerable out-of-date software, and attempt to update it.
Malware authors target bugs, or loopholes, to exploit. A common method is exploitation of a buffer overrun vulnerability, where software designed to store data in a specified region of memory does not prevent more data than the buffer can accommodate being supplied. Malware may provide data that overflows the buffer, with malicious executable code or data after the end; when this payload is accessed it does what the attacker, not the legitimate software, determines.

Insecure design or user error[edit]

Early PCs had to be booted from floppy disks. When built-in hard drives became common, the operating system was normally started from them, but it was possible to boot from another boot device if available, such as a floppy disk, CD-ROM, DVD-ROM, USB flash drive or network. It was common to configure the computer to boot

from one of these devices when available. Normally none would be available; the user would intentionally insert, say, a CD into the optical drive to boot the computer in some special way, for example, to install an operating system. Even without booting, computers can be configured to execute software on some media as soon as they become available, e.g. to autorun a CD or USB device when inserted.
Malware distributors would trick the user into booting or running from an infected device or medium. For example, a virus could make an infected computer add autorunnable code to any USB stick plugged into it. Anyone who then attached the stick to another computer set to autorun from USB would in turn become infected, and also pass on the infection in the same way.[49] More generally, any device that plugs into a USB port - even lights, fans, speakers, toys, or peripherals such as a digital microscope - can be used to spread malware. Devices can be infected during manufacturing or supply if quality control is inadequate.[49]
This form of infection can largely be avoided by setting up computers by default to boot from the internal hard drive, if available, and not to autorun from devices.[49] Intentional booting from another device is always possible by pressing certain keys during boot.
Older email software would automatically open HTML email containing potentially malicious JavaScript code. Users may also execute disguised malicious email attachments. The *2018 Data Breach Investigations Report* by Verizon, cited by CSO Online, states that emails are the primary method of malware delivery, accounting for 92% of malware delivery around the world.[50][51]

Over-privileged users and over-privileged code[edit]

Main article: principle of least privilege

In computing, privilege refers to how much a user or program is allowed to modify a system. In poorly designed computer systems, both users and programs can be assigned more privileges than they should have, and malware can take advantage of this. The two ways that malware does this is through overprivileged users and overprivileged code.
Some systems allow all users to modify their internal structures, and such users today would be considered over-privileged users. This was the standard operating procedure for early microcomputer and home computer systems, where there was no distinction between an *administrator* or *root*, and a regular user of the system. In some systems, non-administrator users are over-privileged by design, in the sense that they are allowed to modify internal structures of the system. In some environments, users are over-privileged because they have been inappropriately granted administrator or equivalent status.
Some systems allow code executed by a user to access all rights of that user, which is known as over-privileged code. This was also standard operating procedure for early microcomputer and home computer systems. Malware, running as over-privileged code, can use this privilege to subvert the system. Almost all currently popular

operating systems, and also many scripting applications allow code too many privileges, usually in the sense that when a user executes code, the system allows that code all rights of that user. This makes users vulnerable to malware in the form of e-mail attachments, which may or may not be disguised.

Use of the same operating system[edit]

Homogeneity can be a vulnerability. For example, when all computers in a network run the same operating system, upon exploiting one, one worm can exploit them all:[52] In particular, Microsoft Windows or Mac OS X have such a large share of the market that an exploited vulnerability concentrating on either operating system could subvert a large number of systems. Introducing diversity purely for the sake of robustness, such as adding Linux computers, could increase short-term costs for training and maintenance. However, as long as all the nodes are not part of the same directory service for authentication, having a few diverse nodes could deter total shutdown of the network and allow those nodes to help with recovery of the infected nodes. Such separate, functional redundancy could avoid the cost of a total shutdown, at the cost of increased complexity and reduced usability in terms of single sign-on authentication.

Anti-malware strategies[edit]

Main article: Antivirus software

As malware attacks become more frequent, attention has begun to shift from viruses and spyware protection, to malware protection, and programs that have been specifically developed to combat malware. (Other preventive and recovery measures, such as backup and recovery methods, are mentioned in the computer virus article).

Anti-virus and anti-malware software[edit]

A specific component of anti-virus and anti-malware software, commonly referred to as an on-access or real-time scanner, hooks deep into the operating system's core or kernel and functions in a manner similar to how certain malware itself would attempt to operate, though with the user's informed permission for protecting the system. Any time the operating system accesses a file, the on-access scanner checks if the file is a 'legitimate' file or not. If the file is identified as malware by the scanner, the access operation will be stopped, the file will be dealt with by the scanner in a pre-defined way (how the anti-virus program was configured during/post installation), and the user will be notified.[*citation needed*] This may have a considerable performance impact on the operating system, though the degree of impact is dependent on how well the scanner was programmed. The goal is to stop any operations the malware may attempt on the system before they occur, including activities which might exploit bugs or trigger unexpected operating system behavior.

Anti-malware programs can combat malware in two ways:

They can provide real time protection against the installation of malware software on a computer. This type of malware protection works the same way as that of antivirus

protection in that the anti-malware software scans all incoming network data for malware and blocks any threats it comes across.
Anti-malware software programs can be used solely for detection and removal of malware software that has already been installed onto a computer. This type of anti-malware software scans the contents of the Windows registry, operating system files, and installed programs on a computer and will provide a list of any threats found, allowing the user to choose which files to delete or keep, or to compare this list to a list of known malware components, removing files that match.[53]
Real-time protection from malware works identically to real-time antivirus protection: the software scans disk files at download time, and blocks the activity of components known to represent malware. In some cases, it may also intercept attempts to install start-up items or to modify browser settings. Because many malware components are installed as a result of browser exploits or user error, using security software (some of which are anti-malware, though many are not) to "sandbox" browsers (essentially isolate the browser from the computer and hence any malware induced change) can also be effective in helping to restrict any damage done.[*citation needed*]
Examples of Microsoft Windows antivirus and anti-malware software include the optional Microsoft Security Essentials[54] (for Windows XP, Vista, and Windows 7) for real-time protection, the Windows Malicious Software Removal Tool[55] (now included with Windows (Security) Updates on "Patch Tuesday", the second Tuesday of each month), and Windows Defender (an optional download in the case of Windows XP, incorporating MSE functionality in the case of Windows 8 and later).[56] Additionally, several capable antivirus software programs are available for free download from the Internet (usually restricted to non-commercial use).[57] Tests found some free programs to be competitive with commercial ones.[57][58][59] Microsoft's System File Checker can be used to check for and repair corrupted system files.
Some viruses disable System Restore and other important Windows tools such as Task Manager and Command Prompt. Many such viruses can be removed by rebooting the computer, entering Windows safe mode with networking,[60] and then using system tools or Microsoft Safety Scanner.[61]
Hardware implants can be of any type, so there can be no general way to detect them.

Website security scans[edit]

As malware also harms the compromised websites (by breaking reputation, blacklisting in search engines, etc.), some websites offer vulnerability scanning.[62][63][64][65] Such scans check the website, detect malware, may note outdated software, and may report known security issues.

"Air gap" isolation or "parallel network"[edit]

As a last resort, computers can be protected from malware, and infected computers can be prevented from disseminating trusted information, by imposing an "air

gap" (i.e. completely disconnecting them from all other networks). However, malware can still cross the air gap in some situations. For example, removable media can carry malware across the gap.[citation needed]
"AirHopper",[66] "BitWhisper",[67] "GSMem" [68] and "Fansmitter" [69] are four techniques introduced by researchers that can leak data from air-gapped computers using electromagnetic, thermal and acoustic emissions.

Grayware[edit]

See also: Privacy-invasive software and Potentially unwanted program

Grayware is a term applied to unwanted applications or files that are not classified as malware, but can worsen the performance of computers and may cause security risks.[70]
It describes applications that behave in an annoying or undesirable manner, and yet are less serious or troublesome than malware. Grayware encompasses spyware, adware, fraudulent dialers, joke programs, remote access tools and other unwanted programs that may harm the performance of computers or cause inconvenience. The term came into use around 2004.[71]
Another term, potentially unwanted program (PUP) or potentially unwanted application (PUA),[72] refers to applications that would be considered unwanted despite often having been downloaded by the user, possibly after failing to read a download agreement. PUPs include spyware, adware, and fraudulent dialers. Many security products classify unauthorised key generators as grayware, although they frequently carry true malware in addition to their ostensible purpose.
Software maker Malwarebytes lists several criteria for classifying a program as a PUP.[73] Some types of adware (using stolen certificates) turn off anti-malware and virus protection; technical remedies are available.[42]

History of viruses and worms[edit]

Before Internet access became widespread, viruses spread on personal computers by infecting executable programs or boot sectors of floppy disks. By inserting a copy of itself into the machine code instructions in these programs or boot sectors, a virus causes itself to be run whenever the program is run or the disk is booted. Early computer viruses were written for the Apple II and Macintosh, but they became more widespread with the dominance of the IBM PC and MS-DOS system. The first IBM PC virus in the "wild" was a boot sector virus dubbed (c)Brain,[74] created in 1986 by the Farooq Alvi brothers in Pakistan.[75] Executable-infecting viruses are dependent on users exchanging software or boot-able floppies and thumb drives so they spread rapidly in computer hobbyist circles.[citation needed]
The first worms, network-borne infectious programs, originated not on personal computers, but on multitasking Unix systems. The first well-known worm was the Internet Worm of 1988, which infected SunOS and VAX BSD systems. Unlike a virus, this worm did not insert itself into other programs. Instead, it exploited security holes (vulnerabilities) in network server programs and started itself running as a

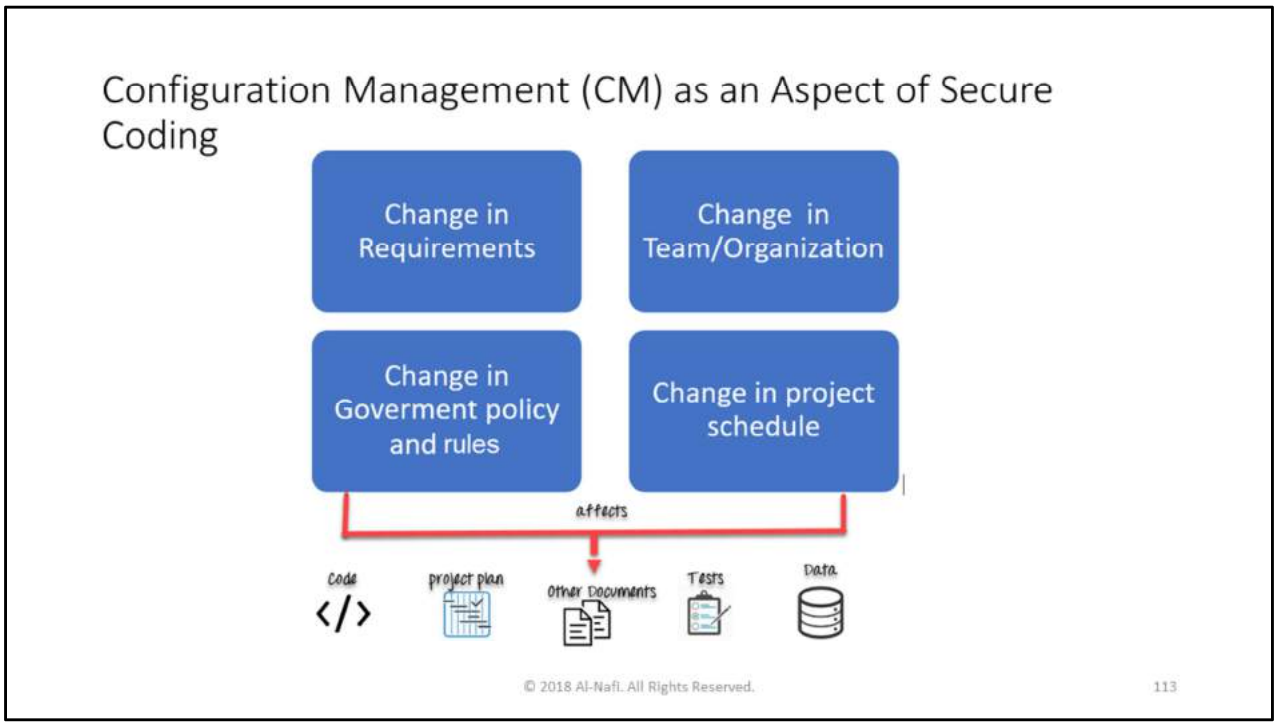
separate process.[76] This same behavior is used by today's worms as well.[77][78]
With the rise of the Microsoft Windows platform in the 1990s, and the flexible macros of its applications, it became possible to write infectious code in the macro language of Microsoft Word and similar programs. These *macro viruses* infect documents and templates rather than applications (executables), but rely on the fact that macros in a Word document are a form of executable code.[79]

Security of Code Repositories



© 2018 Al-Nafi. All Rights Reserved.

As part of good security management, it's very important to ensure the safety of application code while it is being developed, as well as during usage and while at rest in the enterprise. Code is typically stored in what are called code repositories. In today's
environments and trends, the security of code repositories can pose a challenge for several reasons. With the move to offshoring application development, the code being developed may not be available to the enterprise directly, and likewise, the development
environment may be unavailable for management and inspection. The protection of code repositories needs to be handled just like any other valuable asset through a combination of logical and physical access controls and mechanisms, as well as protecting the
integrity and availability of the content of code repositories.

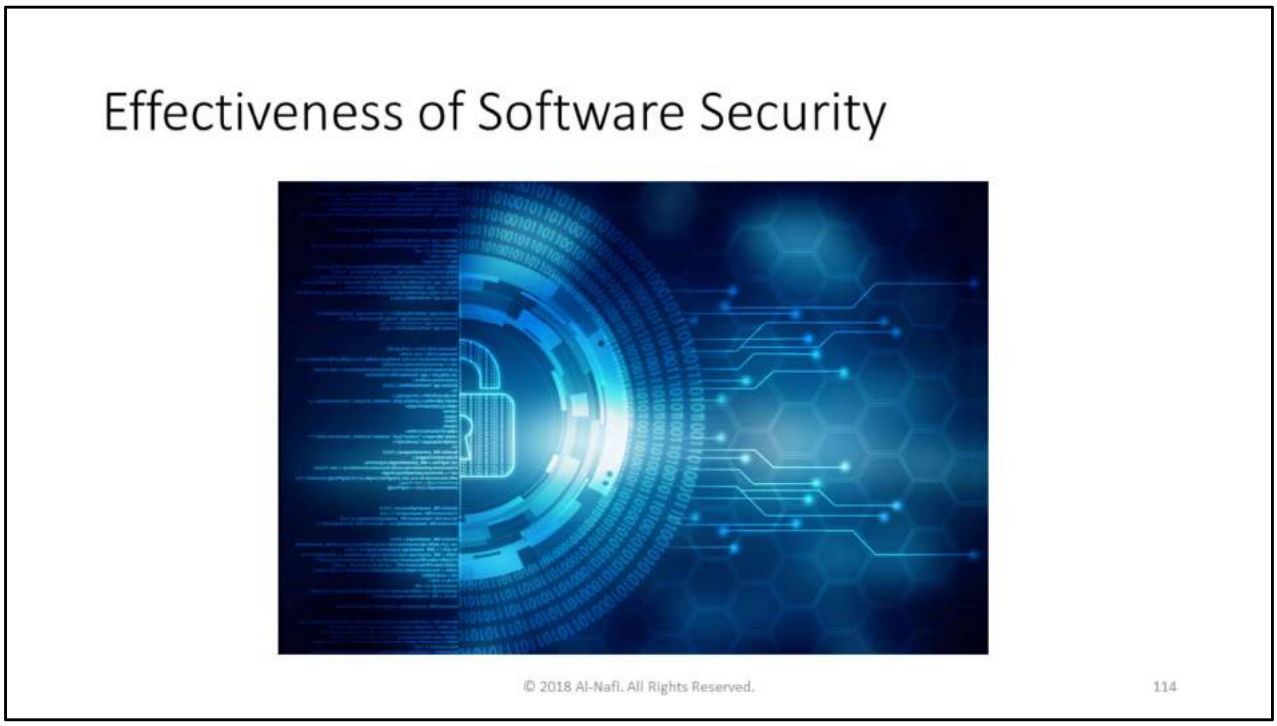


Configuration Management (CM)
For software and applications, configuration management (CM) refers to monitoring and managing changes to a program or documentation. The goal is to guarantee the integrity of the code, availability, and usage of the correct version of all system components such as the software code, design documents, documentation, and control files.

CM, therefore, involves reviewing every change made to a system.
This includes identifying, controlling, accounting for, and auditing all changes. The process would include the following:
l The first step is to identify any changes that are made.
l Controlling occurs when every change is subject to some type of documentation that must be reviewed and approved by an authorized individual.
l Accounting is recording and reporting on the configuration of the software or hardware throughout any change procedures.
l Auditing allows the completed change to be verified, especially ensuring that any changes did not affect the security policy or protection mechanisms that are implemented.

Successful CM requires a well-defined and understood
set of policies and standards that clearly define the following:
l The set of artifacts (configuration items) under the jurisdiction
of CM
l How artifacts are named

How artifacts enter and leave the controlled set
l How an artifact under CM is allowed to change
l How different versions of an artifact under CM are made
available and under what conditions each one can be used
l How CM tools are used to enable and enforce CM

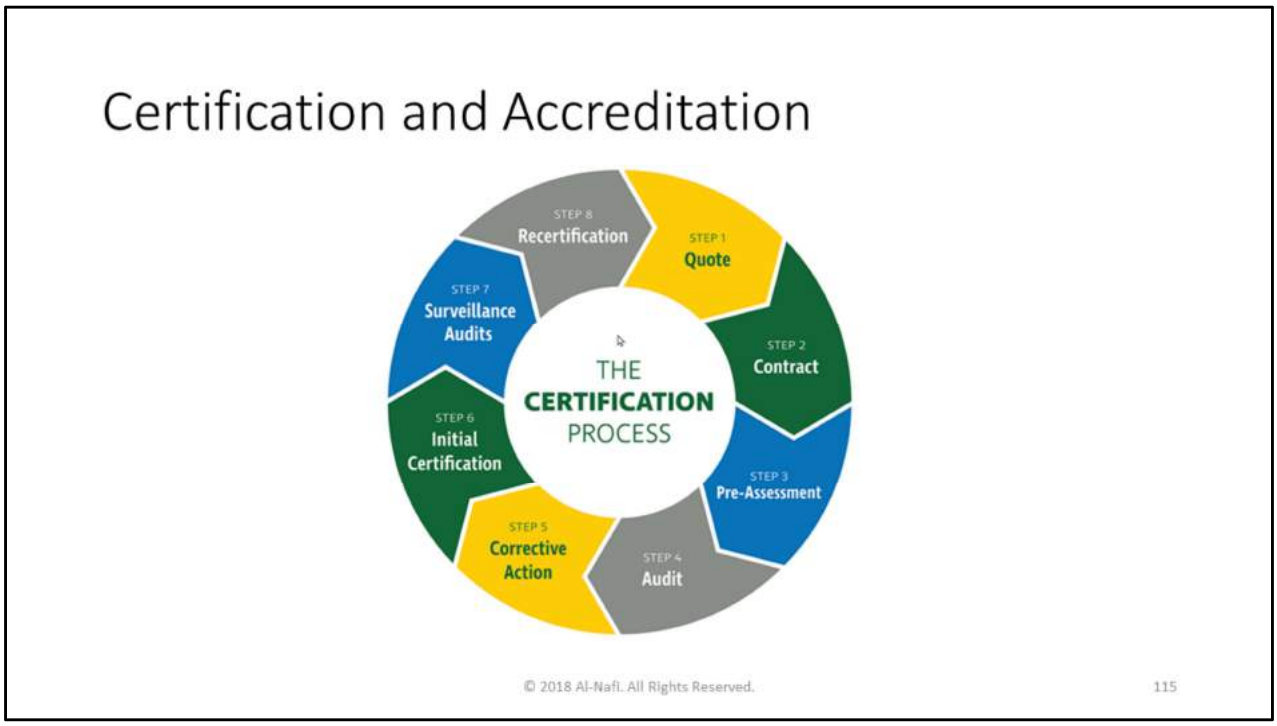


Effectiveness of Software Security

As we have seen, application software has become an integral component in every organization over the last number of decades, and building better applications, that have the proper security controls built-in based on requirements becomes very important. As part of this importance, organizations need to evaluate the effectiveness of the applications development process, including how security is involved and ultimately that the security designed into the application is indeed effective based on the organization's requirements.

The best way to evaluate the effectiveness of application development and software security is through having an efficient and secure process itself and through testing and assurance mechanisms. Providing meaningful metrics that are evaluated, meaningful, and provided to stakeholders allows organizations to have assurance that the effectiveness of software security is indeed at the levels required based on goals and objectives. Providing meaningful metrics that reflect on use cases can give
organizations a more comprehensive view of how secure applications actually are.

Use cases are tangible outcomes of a program and can definitely be useful in applications security testing. They are essentially scores for how well the security functions in certain test situations. By measuring the quality of each use case, organizations can have a clear understanding of how well the applications provide security.



https://en.wikipedia.org/wiki/Certification_and_Accreditation

Certification and accreditation (C&A or CnA) is a process for implementing any formal process. It is a systematic procedure for evaluating, describing, testing, and authorizing systems or activities prior to or after a system is in operation. The process is used extensively across the world.

Certification is a comprehensive evaluation of a process, system, product, event, or skill, typically measured against some existing norm or standard. Industry and/or trade associations will often create certification programs to test and evaluate the skills of those performing services within the interest area of that association. Testing laboratories may also certify that certain products meet pre-established standards, or governmental agencies may certify that a company is meeting existing regulations (e.g., emission limits).

Accreditation is the formal declaration by a neutral third party that the certification program is administered in a way that meets the relevant norms or standards of certification program (e.g., ISO/IEC 17024).

National bodies[edit]

Many nations have established specific bodies.

United Kingdom[edit]

In the United Kingdom, for example, an organization known as United Kingdom Accreditation Service (UKAS) has been established as the nation's official accreditation body. Most European nations have similar organizations established to provide accreditation services within their borders.

United States[edit]

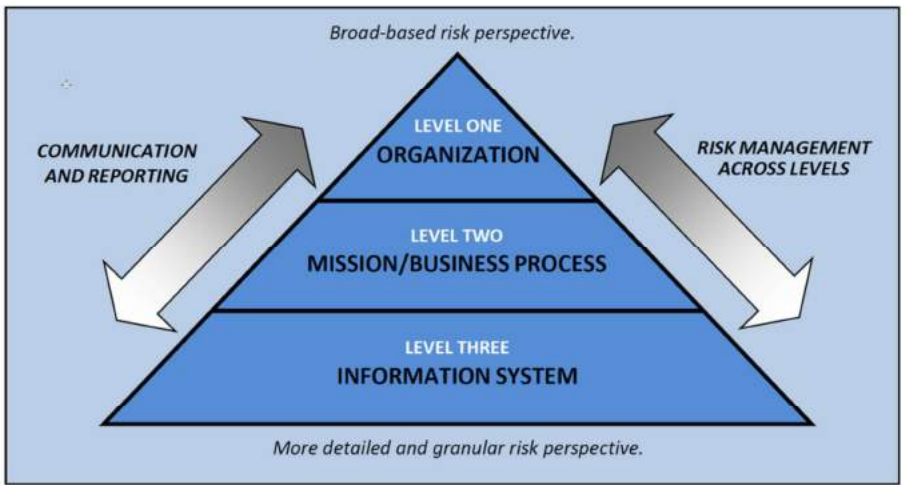
There is no such "approved" accreditation body within the United States, however. As a result, over the years multiple accreditation bodies have become established to address the accreditation needs of specific industries or market segments. Some of these accreditation services are for profit entities, however the majority are not-for-profit bodies that provide accreditation services as part of their mission.

Information security[edit]

Certification and accreditation is a two-step process that ensures security of information systems.[1] Certification is the process of evaluating, testing, and examining security controls that have been pre-determined based on the data type in an information system. The evaluation compares the current systems' security posture with specific standards. The certification process ensures that security weaknesses are identified and plans for mitigation strategies are in place. On the other hand, accreditation is the process of accepting the residual risks associated with the continued operation of a system and granting approval to operate for a specified period of time.

In IT governance, the primary reason why certification and accreditation (C&A) process is being performed on critical systems is to ensure that the security compliance has been technically evaluated. Certified and accredited systems are systems that have had their security compliance technically evaluated for optimal performance in a specific environment and configuration. These certified systems are hereby evaluated to run in a specific working environment.

NIST SP 800-37 Revision 2



© 2018 Al-Nafi. All Rights Reserved.

Please download and read this document as it is a key document.

https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-37r2.pdf

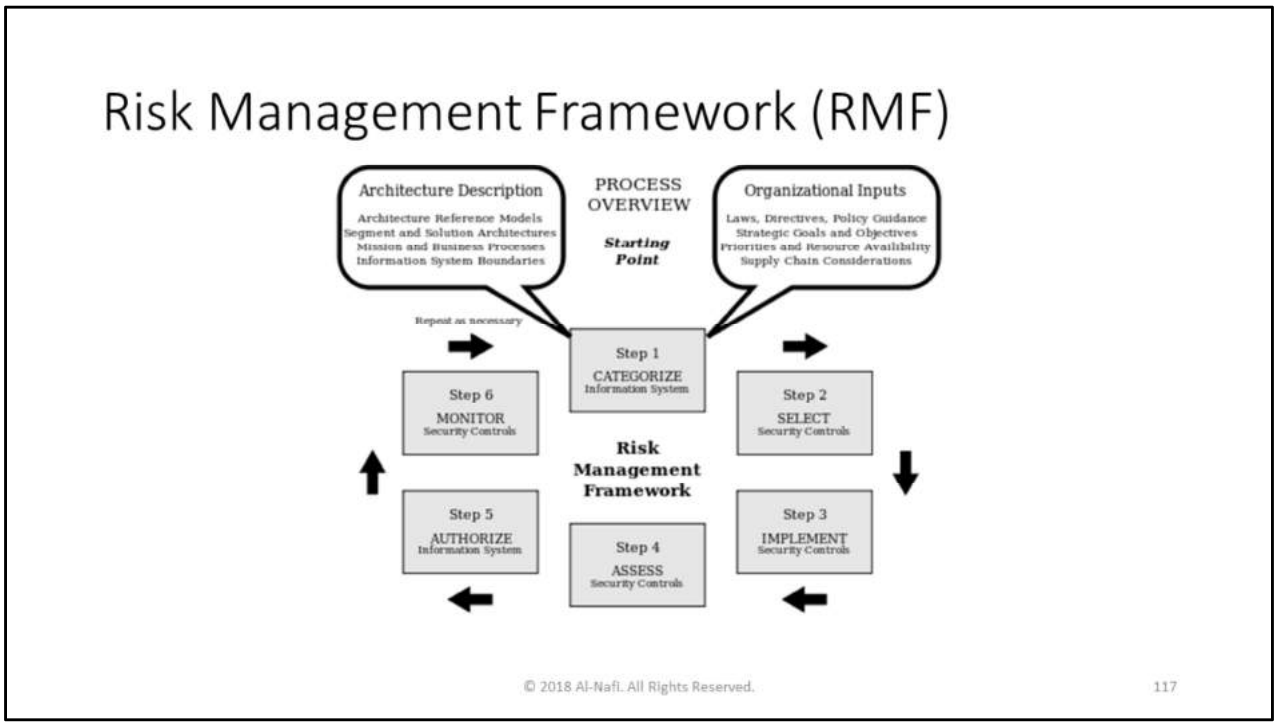
Simply read it atleast once.

The U.S. National Institute of Standards and Technology (NIST) has developed and published a document, SP 800-37 Revision 1: Guide for Applying the Risk Management Framework to Information Systems that recommends a security authorization process and procedures to ensure the risk management process is applied into application development and how security is involved to ensure the effectiveness of software and its security capabilities. As we've seen above, the process of certification and accreditation can be very useful, but the NIST SP 800-37 Revision 1 guidance has provided a way to create a change in the traditional thought process surrounding certification and accreditation and extends it. The revised process emphasizes the following:

- Building information security capabilities into information systems through the application of state-of-the-practice management, operational, and technical

security controls

- Maintaining awareness of the security state of information systems on an ongoing basis though enhanced monitoring processes
- Providing essential information to senior leaders to facilitate decisions regarding the acceptance of risk to organizational operations and assets, individuals, and other organizations, arising from the operation and use of information systems



Reference https://csrc.nist.gov/projects/risk-management/risk-management-framework-(RMF)-Overview

Reference https://en.wikipedia.org/wiki/Risk_management_framework

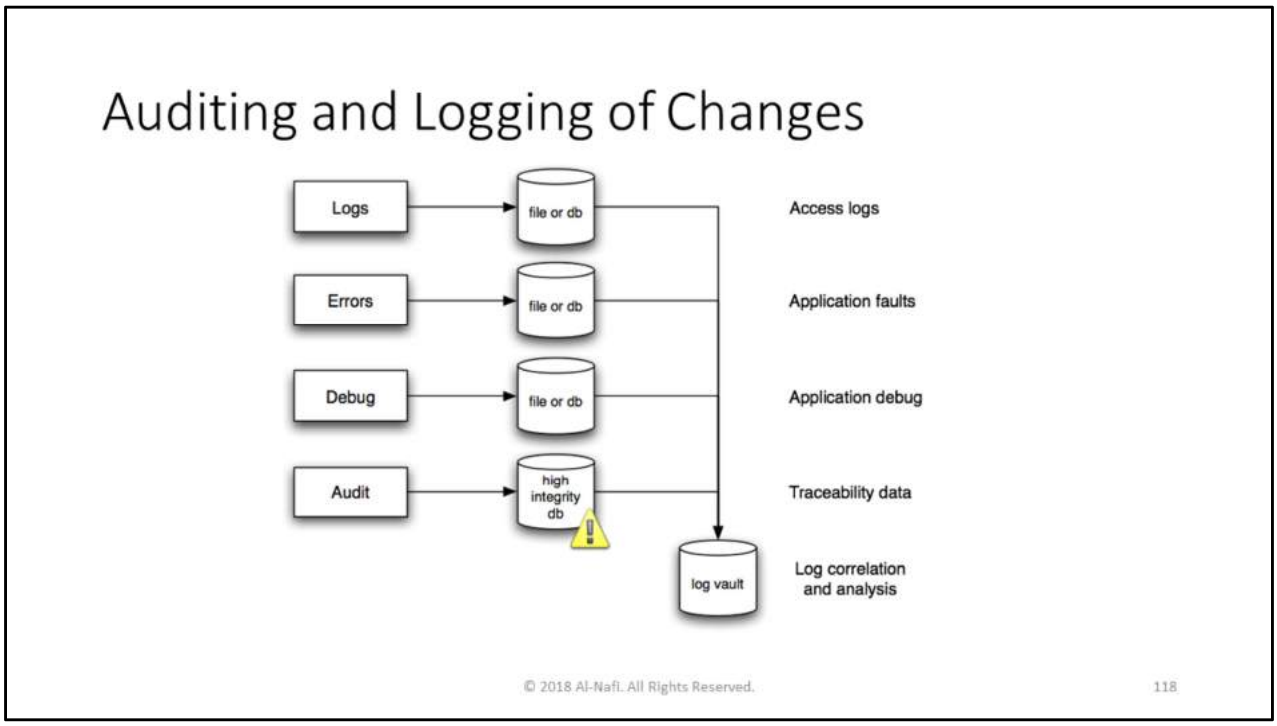
Please read this document once so that you understand as it will help you tremendously during your cyber security career.

The **Risk Management Framework** is a United States federal government policy and standards to help secure information systems (computers and networks) developed by National Institute of Standards and Technology.
The two main publications that cover the details of RMF are NIST Special Publication 800-37, "Guide for Applying the Risk Management Framework to Federal Information Systems", and NIST Special Publication 800-53, "Security and Privacy Controls for Federal Information Systems and Organizations".
NIST Special Publication 800-37, "Guide for Applying the Risk Management Framework to Federal Information Systems", developed by the Joint Task Force Transformation Initiative Working Group, transforms the traditional Certification

and Accreditation (C&A) process into the six-step Risk Management Framework (RMF).
The Risk Management Framework (RMF), illustrated at right, provides a disciplined and structured process that integrates information security and risk management activities into the system development life cycle.[1]

The RMF steps include:
Categorize the information system and the information processed, stored, and transmitted by that system based on an impact analysis. Vested party is identified.
Select an initial set of baseline security controls for the information system based on the security categorization; tailoring and supplementing the security control baseline as needed based on an organizational assessment of risk and local conditions. If any overlays apply to the system it will be added in this step
Implement the security controls identified in the **Step 2 SELECTION** are applied in this step.
Assess third party entity assess the controls and verifies that the controls are properly applied to the system.
Authorize the information system is granted or denied an Authority to Operate (ATO), in some cases it may be postponed while certain items are fixed. The ATO is based off the report from the Assessment phase.
Monitor the security controls in the information system are monitored in a pre-planned fashion documented earlier in the process. ATO is good for 3 years, every 3 years the process needs to be repeated.

Risks[edit]
During its lifecycle, an information system will encounter many types of risk that affect the overall security posture of the system and the security controls that must be implemented. The RMF process supports early detection and resolution of risks. Risk can be categorized at high level as infrastructure risks, project risks, application risks, information asset risks, business continuity risks, outsourcing risks, external risks and strategic risks. Infrastructure risks focus on the reliability of computers and networking equipment. Project risks focus on budget, timeline and system quality. Application risks focus on performance and overall system capacity. Information asset risks focus on the damage, loss or disclosure to an unauthorized part of information assets. Business continuity risks focus on maintaining a reliable system with maximum up-time. Outsourcing risks focus on the impact of 3rd party supplier meeting their requirements. [2] External risks are items outside the information system control that impact the security of the system. Strategic risks focuses on the need of information system functions to align with the business strategy that the system supports. [3]



Systems, applications, architecture, and network device reporting is important to the overall health and security of systems. Every network device, operating system, or application, and indeed, component of architectures should provide some form of logging capabilities.

Reference https://logz.io/blog/monitoring-logging-compliance/

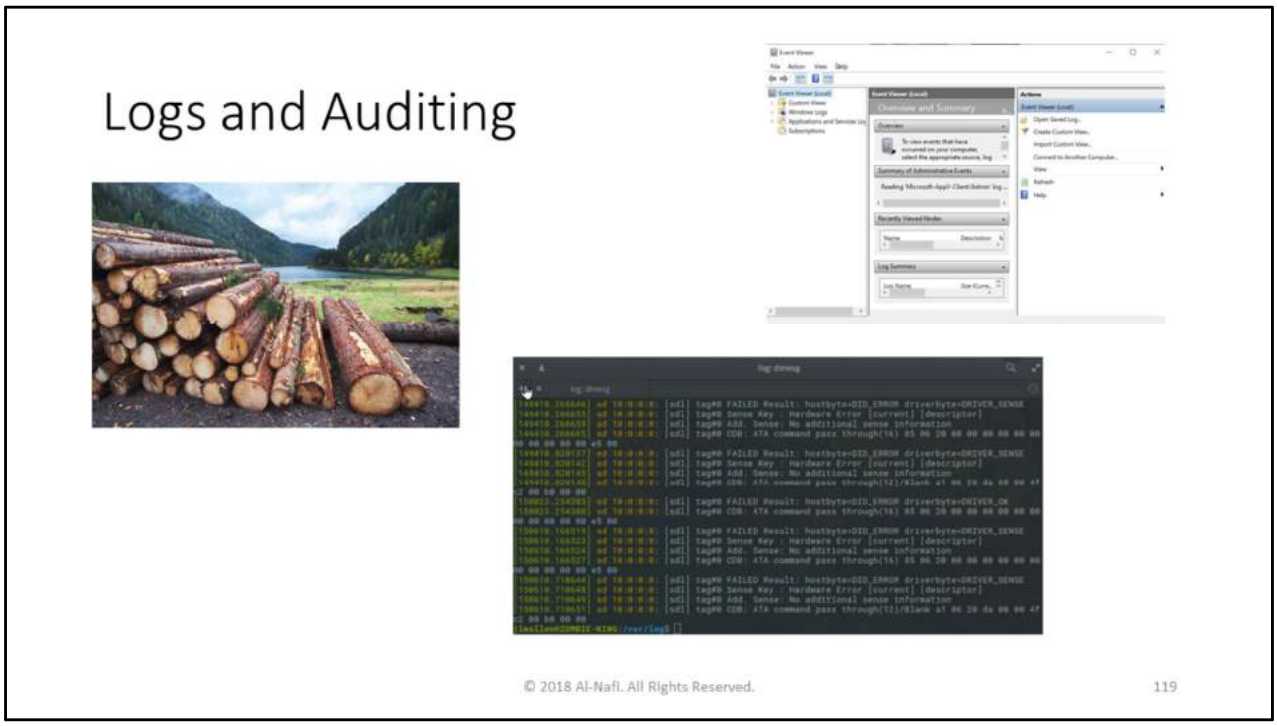
Addressing compliance requirements for monitoring and logging can be a challenge for any organization no matter how experienced or skilled the people responsible are. Compliance requirements are often not well understood by technical teams and there is not much instruction on how to comply with a compliance program. In this article, we'll discuss what some of these new compliance programs mean, why they are important, and how you can comply with your logging and monitoring system.

What is a compliance program?
The goal of compliance is to provide stronger security in a verifiable manner. The

way they do this is to create standards or regulations that stipulate where an organization must meet a minimum level of strong security practices. To meet these, it may include technical controls; such as logging/ monitoring software, strong configuration control, or administrative controls; such as policy, procedure, and training.
While compliance sets a minimum level of requirements, it is up to the organization to determine the level of control needed. Most commonly, this is done via an assessment of risk or threat in context with the requirement. If it can be shown that a compliance control requirement is sufficient, then the control should be at least what is required but can vary based on the assessment.

Who is affected by these regulations?
This next section includes a table of some common compliance programs – often referred to by the regulation or security framework associated with it. A regulation may be very broad and impact any business when it focuses on specific types of data (just as a state privacy law protects any citizen's data regardless of location) or it may focus on a specific sector of the economy (like healthcare or energy utilities).
Many businesses may find they are within the scope of a regulation to a limited degree, such as when a business self-insures its employees, it falls into some HIPAA regulation requirements even though the business has no health-industry orientation. If you're wondering if you might be impacted by a regulation, the question is typically determined by your legal or security department.

What do the compliance programs say?
The regulation may be how a program is identified, but often it is also a framework or associated document that provides instruction on how a compliance effort is to be implemented and assessed. It is wise to distribute these framework requirements to staff (sometimes in addition to the regulation) in order to explain what is required.



A log is a record of security relevant actions and events that have taken place on a computer architecture. Logs:

- Provide a clear view of who owns a process, what action was initiated, when it was initiated, where the action occurred, and why the process ran
- Are the primary record keepers of system and network activity l Are particularly helpful in capturing the pertinent information to explain what happened and why in the event that security controls experience failures

Reference https://en.wikipedia.org/wiki/Log_file

In computing, a **log file** is a file that records either events that occur in an operating system or other software runs,[1] or messages between different users of a communication software. **Logging** is the act of keeping a log. In the simplest case, messages are written to a single log file.

A transaction log is a file (i.e., log) of the communications between a system and the users of that system,[2] or a data collection method that automatically captures the type, content, or time of transactions made by a person from a terminal with

that system.[3] For Web searching, a transaction log is an electronic record of interactions that have occurred during a searching episode between a Web search engine and users searching for information on that Web search engine.

Many operating systems, software frameworks and programs include a logging system. A widely used logging standard is syslog, defined in Internet Engineering Task Force (IETF) RFC 5424). The syslog standard enables a dedicated, standardized subsystem to generate, filter, record, and analyze log messages. This relieves software developers of having to design and code their own ad hoc logging systems.[4][5][6]

Event logs[edit]

Event logs record events taking place in the execution of a system in order to provide an audit trail that can be used to understand the activity of the system and to diagnose problems. They are essential to understand the activities of complex systems, particularly in the case of applications with little user interaction (such as server applications).

It can also be useful to combine log file entries from multiple sources. This approach, in combination with statistical analysis, may yield correlations between seemingly unrelated events on different servers. Other solutions employ network-wide querying and reporting.[7][8]

Transaction logs[edit]

Main article: Transaction log

Most database systems maintain some kind of **transaction log**, which are not mainly intended as an audit trail for later analysis, and are not intended to be human-readable. These logs record changes to the stored data to allow the database to recover from crashes or other data errors and maintain the stored data in a consistent state. Thus, database systems usually have both general event logs and transaction logs.

Message logs[edit]

Internet Relay Chat (IRC), instant messaging (IM) programs, peer-to-peer file sharing clients with chat functions, and multiplayer games (especially MMORPGs) commonly have the ability to automatically log or save textual communication, both public (IRC channel/IM conference/MMO public/party chat messages) and private chat messages between users. [13] Message logs are almost universally plain text files, but IM and VoIP clients (which supports textual chat, e.g. Skype) might save them in HTML files or in a custom format to ease reading and encryption.

Internet Relay Chat (IRC)[edit]

In the case of IRC software, message logs often include system/server messages and entries related to channel and user changes (e.g. topic change, user

joins/exits/kicks/bans, nickname changes, user status changes), making them more like a combined message/event log of the channel in question, but such a log isn't comparable to a true IRC server event log, because it only records user-visible events for the time frame the user spent being connected to a certain channel.

Instant messaging[edit]
Instant messaging and VoIP clients often offer the chance to store encrypted logs to enhance the user's privacy. These logs require a password to be decrypted and viewed, and they are often handled by their respective writing application..

Transaction log analysis[edit]
The use of data stored in transaction logs of Web search engines, Intranets, and Web sites can provide valuable insight into understanding the information-searching process of online searchers.[14] This understanding can enlighten information system design, interface development, and devising the information architecture for content collections.

Please read this article and we will cover more during our PCI DSS and ISO 27001 courses. Reference https://logz.io/blog/monitoring-logging-compliance/

Addressing compliance requirements for monitoring and logging can be a challenge for any organization no matter how experienced or skilled the people responsible are. Compliance requirements are often not well understood by technical teams and there is not much instruction on how to comply with a compliance program. In this article, we'll discuss what some of these new compliance programs mean, why they are important, and how you can comply with your logging and monitoring system.

What is a compliance program?
The goal of compliance is to provide stronger security in a verifiable manner. The way they do this is to create standards or regulations that stipulate where an organization must meet a minimum level of strong security practices. To meet these, it may include technical controls; such as logging/ monitoring software, strong configuration control, or administrative controls; such as policy, procedure, and training.
While compliance sets a minimum level of requirements, it is up to the organization to determine the level of control needed. Most commonly, this is done via an assessment of risk or threat in context with the requirement. If it can be shown that a compliance control requirement is sufficient, then the control should be at least what is required but can vary based on the assessment.

Who is affected by these regulations?
This next section includes a table of some common compliance programs – often

referred to by the regulation or security framework associated with it. A regulation may be very broad and impact any business when it focuses on specific types of data (just as a state privacy law protects any citizen's data regardless of location) or it may focus on a specific sector of the economy (like healthcare or energy utilities).
Many businesses may find they are within the scope of a regulation to a limited degree, such as when a business self-insures its employees, it falls into some HIPAA regulation requirements even though the business has no health-industry orientation. If you're wondering if you might be impacted by a regulation, the question is typically determined by your legal or security department.

What do the compliance programs say?
The regulation may be how a program is identified, but often it is also a framework or associated document that provides instruction on how a compliance effort is to be implemented and assessed. It is wise to distribute these framework requirements to staff (sometimes in addition to the regulation) in order to explain what is required.

Why and when do you have to comply?
The penalty mechanism helps to provide the answers to the "**why** and **when** do we have to comply" question. In many instances, a compliance program costs more in fines and fees than if the company were to just comply.
For instance, PCI fines range between $5000–$100,000/month[XVIII].. For US State Privacy breach laws (in 48 states) the average fine is $50–$90 per person included in the breach. This does not exclude any other civil litigation that often follows a breach event. HIPAA fines reach up to $1.5M per incident[XIX], and GDPR fines[XX] are up to 20 million Euros or 4 percent of the annual global business, whichever is highest.
These high costs make companies pay attention to key aspects of the compliance program – such as adhering to a compliance schedule. The compliance program, once engaged, typically starts a strict timing for audits, and in many cases, the timeframe for fixing or rectifying non-compliance findings.
More on the subject:
Announcing Logz.io Security Analytics
How to Stay Ahead of Data Retention Requirements – Part 1
How Log Analytics Improves Your Zero Trust Security Model
The compliance framework is where compliance programs get challenging, and where technical staff may get involved. You'll want to start by reading the actual text of the framework. Most compliance frameworks are typically publicly available[XXI] so you can read about the requirements for the organization to follow.
Where is the compliance program applied?
The challenge with a framework is that it is usually somewhat "high level" for most technical staff. This can be a point of frustration – most frameworks are not very prescriptive.

There are a few aspects to keep in mind: first, there is often a question of scope, defining **where** the compliance program is to be applied. While this may be obvious, it can also be strategic when a company can segregate a high-risk function to limit the costs of security.

Second, it is most common to find that "objectives" or security control requirements are 1) categorized into common business operations and key functions (such as Human Resources, Access Control, Physical Security, Computer Operations, Encryption). Then 2) language is used to describe the outcome or components of a "compliant" environment, but without defining the specific control.

Finally, it is becoming more common for objectives to be couched in language that assumes a critical risk determination to be part of the solution to affect the features of the control. Therefore, these are all areas that can be part of a company compliance approach. Some examples of compliance language may help provide some context for the topic of logging or monitoring.

Examples of compliance language for logging/event monitoring

NERC CIP-007-5 Table R4 – Security Event Monitoring

R4. Each Responsible Entity shall implement, in a manner that identifies, assesses, and corrects deficiencies, one or more documented processes that collectively include each of the applicable requirement parts in CIP-007-5 Table R4 – Security Event Monitoring. [Violation Risk Factor: Medium] [Time Horizon: Same Day Operations and Operations Assessment].

M4. Evidence must include each of the documented processes that collectively include each of the applicable requirement parts in CIP-007-5 Table R4 – Security Event Monitoring and additional evidence to demonstrate implementation as described in the Measures column of the table.

ISO 27001 – A.12.4 – Logging and Monitoring

Objective: To record events and generate evidence.

Control 12.4.1 A.12.4.1 Event logging – Event logs recording user activities, exceptions, faults, and information security events shall be produced, kept and regularly reviewed.

Control A.12.4.2 Protection of log information – Logging facilities and log information shall be protected against tampering and unauthorized access.

Control A.12.4.3 Administrator and operator logs – System administrator and system operator activities shall be logged, and the logs protected and regularly reviewed.

Control A.12.4.4 Clock synchronization –The clocks of all relevant information processing systems within an organization or security domain shall be synchronized to a single reference time source.

PCI DSS (Requirement 10): Track and monitor all access to network resources and cardholder data logging mechanisms and the ability to track user activities are critical for effective forensics and vulnerability management. The presence of logs in all environments allows for thorough tracking and analysis if something goes wrong.

Determining the cause of a compromise is very difficult without system activity logs.
10.1 Establish a process for linking all access to system components to each individual user – especially access done with administrative privileges.
10.2 Implement automated audit trails for all system components for reconstructing these events: all individual user accesses to cardholder data; all actions taken by any individual with root or administrative privileges; access to all audit trails; invalid logical access attempts; use of identification and authentication mechanisms; initialization of the audit logs; creation and deletion of system-level objects.
10.3 Record audit trail entries for all system components for each event, including at a minimum: user identification, type of event, date and time, success or failure indication, the origin of the event, and identity or name of affected data, system component or resource.
10.4 Using time synchronization technology, synchronize all critical system clocks and times and implement controls for acquiring, distributing, and storing time.
10.5 Secure audit trails so they cannot be altered.
10.6 Review logs for all system components related to security functions at least daily.
10.7 Retain audit trail history for at least one year; at least three months of history must be immediately available for analysis.

Takeaways

As you can see from the examples above, the compliance requirements do not specify a specific product or solution – it is up to the organization to choose how to address the requirements to achieve compliance. Such solutions should ideally bring auditable outcomes so that an assessor or auditor could verify that the control in place meets the expectations of the compliance program.

A second takeaway is that the topic – logging and monitoring – is fairly similar across compliance programs. Many controls would be the same between compliance programs because the underlying systems and technology risks are so similar.

Finally, there are typically assessment guides provided with many compliance frameworks. With a little research, you will find multiple guidelines and checklists for deploying or assessing compliance with your program. Many are likely to be instructions to auditors and can give you “insight” as to how you might be assessed and what sort of evidence is expected. Advance planning can save you time and effort while illustrating to compliance auditors that your organization is staying ahead of the goals for the program.

Final Recommendations

We have discussed the who, what, why, when, and where of compliance and want to leave you with a couple final recommendations:

1) Consider the scope of the compliance program to ensure that your controls include the system components, facilities, products, and business processes that are included in the compliance program. Avoid focusing too narrowly on the scope of compliance.

2) Interpretation of control requirements can be challenging. One way to address complex technical control configuration is to work towards standard security best practices with all products, tools, and utilities at your enterprise, and consider performing a risk assessment to help deploy the controls most applicable.

Change Management

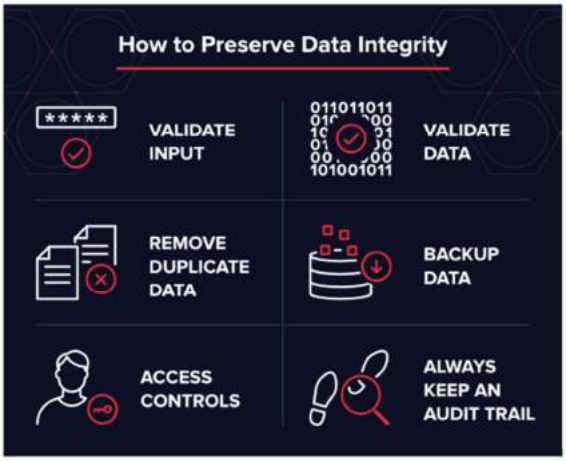


© 2018 Al-Nafi. All Rights Reserved.

Read this article https://www.prosci.com/resources/articles/what-is-change-management

Organizations need to understand change and change management as integral elements in any successful enterprise security architecture. They need to make sure that changes to applications and other systems already in production are made in a rigorous and controlled way to ensure quality assurance of the change. As part of this, organizations need to be able to plan for change, manage it through a well-defined
lifecycle, approve changes, document it, and roll it back if required. There are many practices and guides available that organizations can use as frameworks to guide change management and change control.

Information Integrity, Accuracy, and Auditing



© 2018 Al-Nafi. All Rights Reserved.

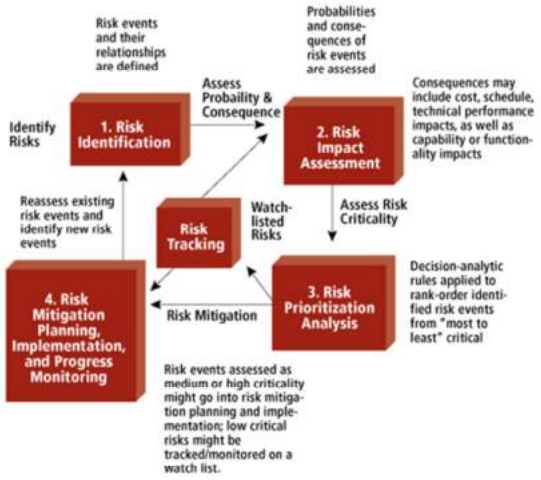
Information integrity means that organizations need to have procedures in place that should be applied to compare or reconcile what was processed against what was supposed to be processed. For example, controls can compare totals or check sequence numbers to make sure the right operations were performed on the correct data elements.

Another element of integrity is information accuracy. Because decisions are made based on information, the accuracy of information becomes very important to ensure as information is processed by applications. To check input accuracy, data validation and verification checks should be incorporated into the appropriate applications. Other controls that may be required are character checks to compare input characters against the expected type of actual characters, such as numbers or letters. This is sometimes known as sanity checking by developers and others involved in applications. Range checks verify input data against predetermined upper and lower limits to make sure they fit within those ranges. Relationship checks compare input data with data on a master record file somewhere else to ensure the correct relationships. Reasonableness checks will compare input data with an expected standard that is also considered to be another form of sanity

checking. Transaction limits check input data against set ceilings on specified transactions to make sure
they don't exceed the limits set as being the specified ceiling or upper limit.

Information auditing is important because vulnerabilities may exist in the development and software lifecycles and therefore, as a result, there is a likelihood that attacks and vulnerabilities may be exploited. Auditing procedures can assist in detecting any abnormal activities that may indicate vulnerabilities are being exploited. A secure information system must provide authorized personnel with the ability to audit any action that can potentially cause unauthorized access to, damage to, or in some
way affect the release of sensitive and valuable information. The level and type of auditing depends on the auditing requirements of the installed software and the sensitivity of data that is processed or stored on the system. The key point is that the audit results provide information on what types of unauthorized activities have taken place and who or what processes took the action to be able to drive the corrective actions necessary at that point.

Risk Analysis and Mitigation



© 2018 Al-Nafi. All Rights Reserved.

Risk is defined as an event or occurrence that has a probability of having an impact to an application project should that risk occur. Being able to identify the risks and mitigate them as part of application security effectiveness is also very important.

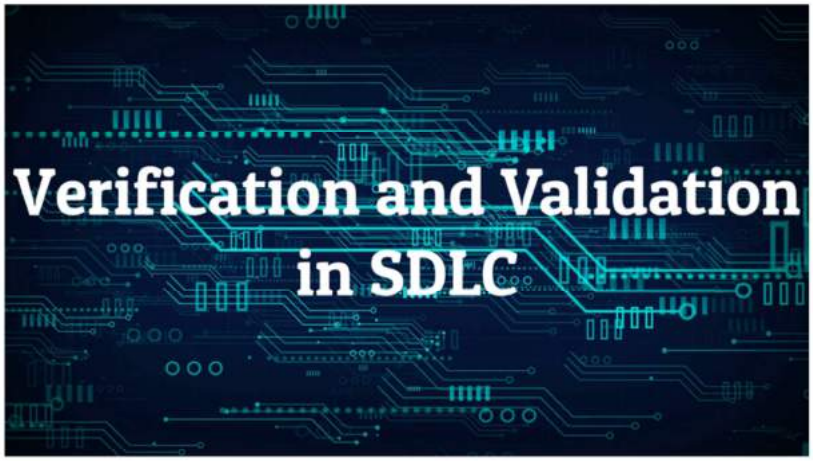
Risk Management



© 2018 Al-Nafi. All Rights Reserved.

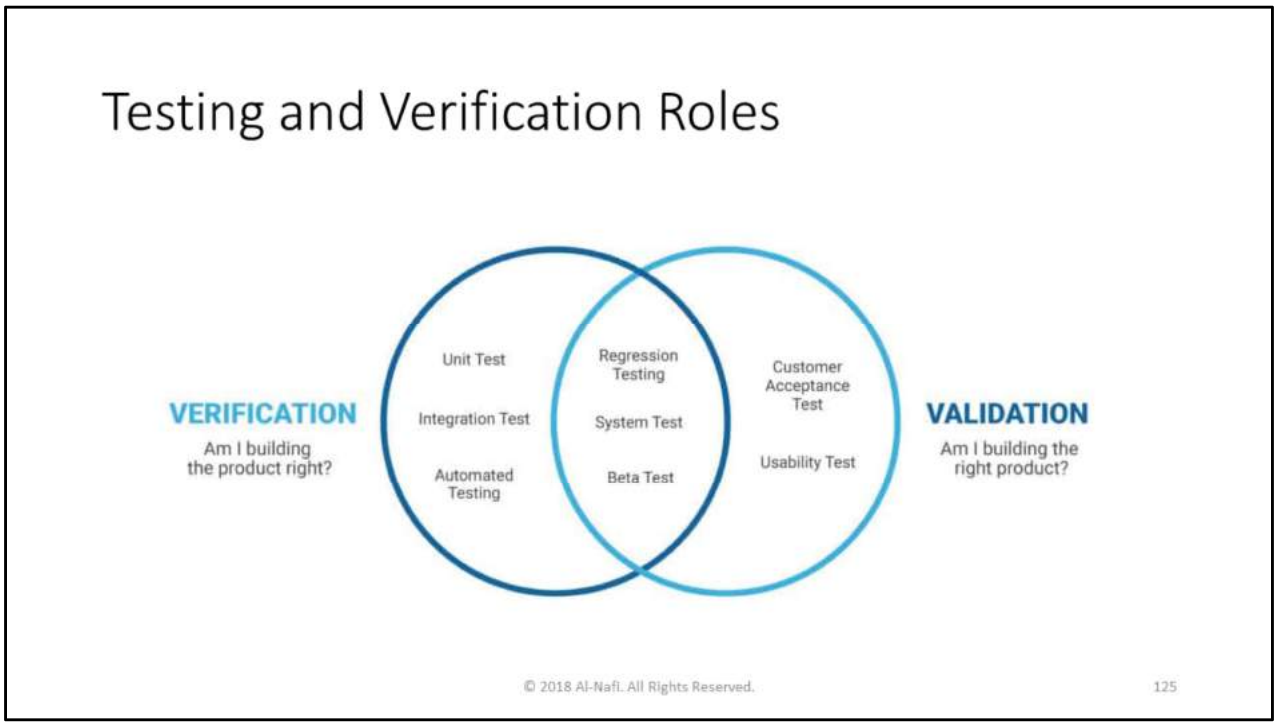
Risk management is an ongoing process that continues through the life of a project. It includes processes for risk management planning, identification, analysis, monitoring, and control. Many of these processes are updated throughout the project lifecycle as new risks can be identified at any time and need to be mitigated as they are identified
and analyzed. It is the objective of risk management to mitigate or treat the risk, and therefore, the probability and impact of events adverse to the project.

Testing and Verification SDLC



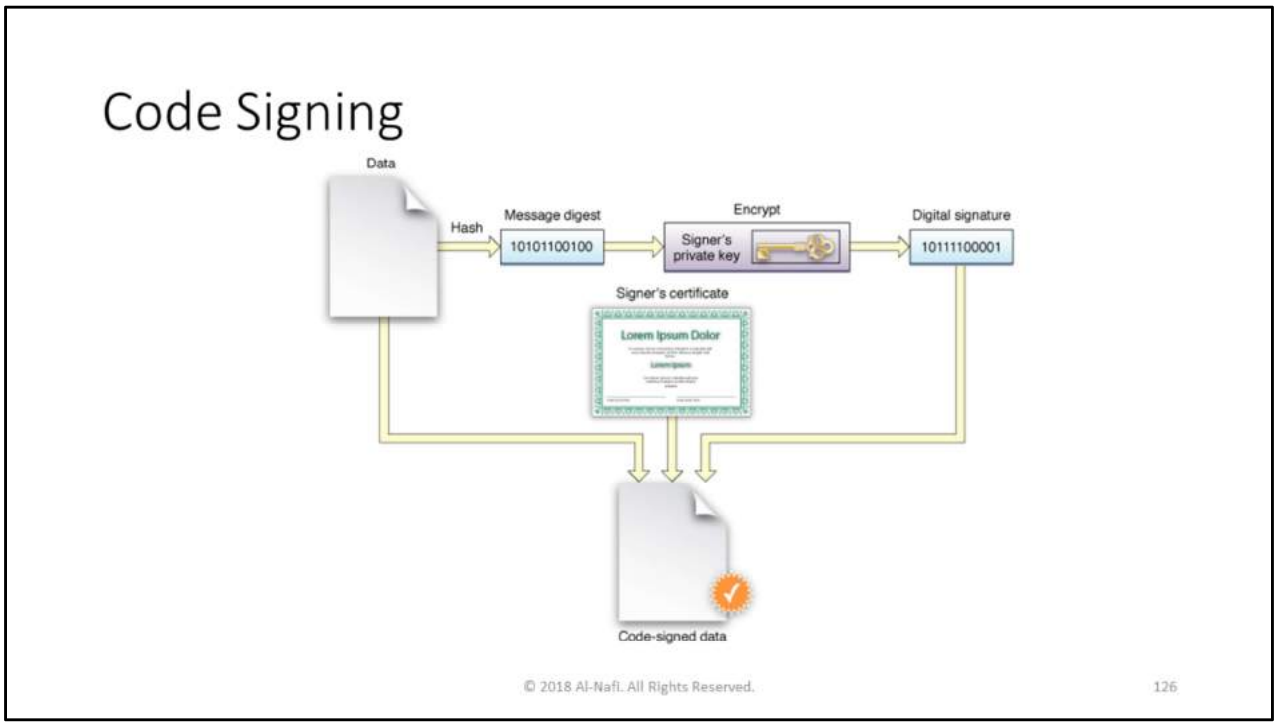
© 2018 Al-Nafi. All Rights Reserved.

When mitigations are implemented, they must be tested. In mature and efficient SDLC environments, this is often done as part of the promotion between development environments by the quality assurance and testing teams.



Reference https://www.plutora.com/blog/verification-vs-validation

Security findings should be addressed by the development team the same as any other change request with the condition that the security assessor or another independent entity verifies and validates the flaw has indeed been remediated. These roles need to be distinct and separate. In large organizations, independent verification and validation teams work to determine if security findings and flaws are truly resolved. They do this by testing and using other assurance methods. This process should also involve the audit group to independently verify that the findings have been addressed. In other words, the developer or system owner does not
authoritatively declare the risk mitigated without the concurrence of an independent party that includes security and audit and possibly other stakeholders. In addition to testing of mitigations, the developer should be encouraged to use code signing as another means of integrity checking for the code they are producing.



Reference https://en.wikipedia.org/wiki/Code_signing

Code signing is the process of digitally signing executables and scripts to confirm the software author and guarantee that the code has not been altered or corrupted since it was signed. The process employs the use of a cryptographic hash to validate authenticity and integrity.[1]
Code signing can provide several valuable features. The most common use of code signing is to provide security when deploying; in some programming languages, it can also be used to help prevent namespace conflicts. Almost every code signing implementation will provide some sort of digital signature mechanism to verify the identity of the author or build system, and a checksum to verify that the object has not been modified. It can also be used to provide versioning information about an object or to store other meta data about an object.[2]

The efficacy of code signing as an authentication mechanism for software depends on the security of underpinning signing keys. As with other public key infrastructure (PKI) technologies, the integrity of the system relies on publishers securing their private keys against unauthorized access. Keys stored in software on general-

purpose computers are susceptible to compromise. Therefore, it is more secure, and best practice, to store keys in secure, tamper-proof, cryptographic hardware devices known as hardware security modules or HSMs.[3]

Providing security[edit]

Many code signing implementations will provide a way to sign the code using a system involving a pair of keys, one public and one private, similar to the process employed by TLS or SSH. For example, in the case of .NET, the developer uses a private key to sign their libraries or executables each time they build. This key will be unique to a developer or group or sometimes per application or object. The developer can either generate this key on their own or obtain one from a trusted certificate authority (CA).[4]

Code signing is particularly valuable in distributed environments, where the source of a given piece of code may not be immediately evident - for example Java applets, ActiveX controls and other active web and browser scripting code. Another important usage is to safely provide updates and patches to existing software.[5] Windows, Mac OS X, and most Linux distributions provide updates using code signing to ensure that it is not possible for others to maliciously distribute code via the patch system. It allows the receiving operating system to verify that the update is legitimate, even if the update was delivered by third parties or physical media (disks).

Code signing is used on Windows and Mac OS X to authenticate software on first run, ensuring that the software has not been maliciously tampered with by a third-party distributor or download site. This form of code signing is not used on Linux because of that platform's decentralized nature, the package manager being the predominant mode of distribution for all forms of software (not just updates and patches), as well as the open-source model allowing direct inspection of the source code if desired. Debian-based Linux distributions (among others) validate downloaded packages using public key cryptography.[6]

Trusted identification using a certificate authority (CA)[edit]

The public key used to authenticate the code signature should be traceable back to a trusted root authority CA, preferably using a secure public key infrastructure (PKI). This does not ensure that the code itself can be trusted, only that it comes from the stated source (or more explicitly, from a particular private key).[7] A CA provides a root trust level and is able to assign trust to others by proxy. If a user trusts a CA, then the user can presumably trust the legitimacy of code that is signed with a key generated by that CA or one of its proxies. Many operating systems and frameworks contain built-in trust for one or more existing CAs (such as Entrust Datacard, VeriSign/Symantec, DigiCert, Comodo, GoDaddy and GlobalSign). It is also commonplace for large organizations to implement a private CA, internal to the organization, which provides the same features as public CAs, but it is only trusted

within the organization.

Extended Validation (EV) Code Signing[edit]
Extended validation (EV) code signing certificates are subject to additional validation and technical requirements, which are summarized in the CA/Browser Forum's EV Code Signing Certificate Guidelines. These guidelines are based on the CA/B Forum's Baseline Requirements and Extended Validation Guidelines. In addition to validation requirements specific to EV, the EV code signing guidelines stipulate that the "the Subscriber's private key is generated, stored and used in a crypto module that meets or exceeds the requirements of FIPS 140-2 level 2."[8]
Certain applications, such as signing Windows 10 kernel-mode drivers, require an EV code signing certificate.[9] Additionally, Microsoft's IEBlog states that Windows programs "signed by an EV code signing certificate can immediately establish reputation with SmartScreen reputation services even if no prior reputation exists for that file or publisher." [10]
As of December 4, 2019, Microsoft listed the following providers of EV code signing certificates:[11]
SSL.com
Symantec
Certum
Entrust
GlobalSign
Sectigo (formerly Comodo)
DigiCert
Alternative to CAs[edit]
The other model is where developers can choose to provide their own self-generated key. In this scenario, the user would normally have to obtain the public key in some fashion directly from the developer to verify the object is from them for the first time. Many code signing systems will store the public key inside the signature. Some software frameworks and OSs that check the code's signature before executing will allow you to choose to trust that developer from that point on after the first run. An application developer can provide a similar system by including the public keys with the installer. The key can then be used to ensure that any subsequent objects that need to run, such as upgrades, plugins, or another application, are all verified as coming from that same developer.

Time-stamping[edit]
Time-stamping was designed to circumvent the trust warning that will appear in the case of an expired certificate. In effect, time-stamping extends the code trust beyond the validity period of a certificate.[12]
In the event that a certificate has to be revoked due to a compromise, a specific date

and time of the compromising event will become part of the revocation record. In this case, time-stamping helps establish whether the code was signed before or after the certificate was compromised.[12]

Code-Signing in Xcode[edit]
Developers need to sign their iOS and tvOS apps before running them on any real device and before uploading them to the App Store. This is needed to prove that the developer owns a valid Apple Developer ID. An application needs a valid profile or certificate so that it can run on the devices.

Problems[edit]
Like any security measure, code signing can be defeated. Users can be tricked into running unsigned code, or even into running code that refuses to validate, and the system only remains secure as long as the private key remains private.[13][14]
It is also important to note that code signing does not protect the end user from any malicious activity or unintentional software bugs by the software author — it merely ensures that the software has not been modified by anyone other than the author. Sometimes, sandbox systems do not accept certificates, because of a false time-stamp or because of an excess usage of RAM.

Implementations[edit]
IBM's Lotus Notes has had PKI signing of code from Release 1, and both client and server software have execution control lists to control what levels of access to data, environment and file system are permitted for given users. Individual design elements, including active items such as scripts, actions and agents, are always signed using the editor's ID file, which includes both the editor's and the domain's public keys. Core templates such as the mail template are signed with a dedicated ID held by the Lotus template development team.

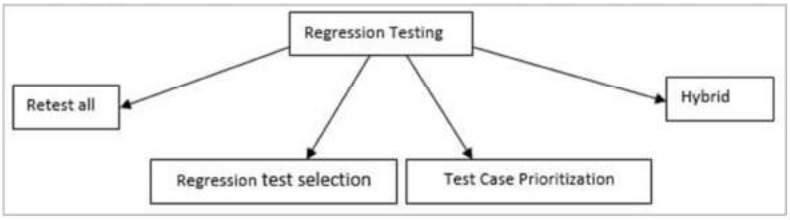
Microsoft implements a form of code signing (based on Authenticode) provided for Microsoft tested drivers. Since drivers run in the kernel, they can destabilize the system or open the system to security holes. For this reason, Microsoft tests drivers submitted to its WHQL program. After the driver has passed, Microsoft signs that version of the driver as being safe. On 32-bit systems only, installing drivers that are not validated with Microsoft is possible after accepting to allow the installation in a prompt warning the user that the code is unsigned. For .NET (managed) code, there is an additional mechanism called Strong Name Signing that uses Public/Private keys and SHA-1 hash as opposed to certificates. However, Microsoft discourages reliance on Strong Name
Signing as a replacement for Authenticode.[15]

Unsigned code in gaming and consumer devices[edit]
In the context of consumer devices such as games consoles, unsigned code is often used to refer to an application which has not been signed with the cryptographic key normally required for software to be accepted and executed. Most console games have to be signed with a secret key designed by the console maker or the game will not load on the console. There are several methods to get unsigned code to execute which include software exploits, the use of a modchip, a technique known as the swap trick or running a softmod.

It may not initially seem obvious why simply copying a signed application onto another DVD does not allow it to boot. On the Xbox, the reason for this is that the Xbox executable file (XBE) contains a media-type flag, which specifies the type of media that the XBE is bootable from. On nearly all Xbox software, this is set such that the executable will only boot from factory produced discs so simply copying the executable to burnable media is enough to stop the execution of the software. However, since the executable is signed, simply changing the value of the flag is not possible as this alters the signature of the executable causing it to fail validation when checked.

Proliferation of connected devices[edit]
Proliferation of connected devices is required for having the ability to identify and authenticate devices. This ability applies to the area of Internet of Things (IoT). IoT requires devices to be trusted. In the IoT, code signing in the software release process ensures the integrity of IoT device software and firmware updates, and defends against the risks associated with tampering the device or the code embedded in it. Device credentialing enables control over the manufacturing process of high technology products and protects against unauthorized production of counterfeits. Together with code signing, the technology ensures physical authenticity and the authenticity and integrity of the code they possess at the time of manufacture through the use of a digital birth certificate, or during subsequent upgrades through code validation any time during the product lifecycle. This is creating a new dimension for code signing, and elevating the security awareness and need to maintain private signing keys secured within a dedicated protected environment to establish a root of trust for the entire system. Given the prevalence of malware and Advanced Persistent Threats (APTs), many software vendors, providers of online services, enterprise IT organizations and manufacturers of high-technology IoT devices are under pressure to increase the security of their high technology manufacturing and code signing process.[16]

Regression Testing



© 2018 Al-Nafi. All Rights Reserved.

Reference https://en.wikipedia.org/wiki/Regression_testing

Regression testing (rarely *non-regression testing*[1]) is re-running functional and non-functional tests to ensure that previously developed and tested software still performs after a change.[2] If not, that would be called a *regression*. Changes that may require regression testing include bug fixes, software enhancements, configuration changes, and even substitution of electronic components.[3] As regression test suites tend to grow with each found defect, test automation is frequently involved. Sometimes a change impact analysis is performed to determine an appropriate subset of tests (*non-regression analysis*[4]).

Background[edit]
As software is updated or changed, or reused on a modified target, emergence of new faults and/or re-emergence of old faults is quite common. Sometimes re-emergence occurs because a fix gets lost through poor revision control practices (or simple human error in revision control). Often, a fix for a problem will be "fragile" in that it fixes the problem in the narrow case where it was first observed but not in more general cases which may arise over the lifetime of the software. Frequently, a

fix for a problem in one area inadvertently causes a software bug in another area. Finally, it may happen that, when some feature is redesigned, some of the same mistakes that were made in the original implementation of the feature are made in the redesign.

Therefore, in most software development situations, it is considered good coding practice, when a bug is located and fixed, to record a test that exposes the bug and re-run that test regularly after subsequent changes to the program.[5] Although this may be done through manual testing procedures using programming techniques, it is often done using automated testing tools.[6] Such a test suite contains software tools that allow the testing environment to execute all the regression test cases automatically; some projects even set up automated systems to re-run all regression tests at specified intervals and report any failures (which could imply a regression or an out-of-date test).[7] Common strategies are to run such a system after every successful compile (for small projects), every night, or once a week. Those strategies can be automated by an external tool.

Regression testing is an integral part of the extreme programming software development method. In this method, design documents are replaced by extensive, repeatable, and automated testing of the entire software package throughout each stage of the software development process. Regression testing is done after functional testing has concluded, to verify that the other functionalities are working. In the corporate world, regression testing has traditionally been performed by a software quality assurance team after the development team has completed work. However, defects found at this stage are the most costly to fix. This problem is being addressed by the rise of unit testing. Although developers have always written test cases as part of the development cycle, these test cases have generally been either functional tests or unit tests that verify only intended outcomes. Developer testing compels a developer to focus on unit testing and to include both positive and negative test cases.[8]

Techniques[edit]

The various regression testing techniques are:

Retest all[edit]

This technique checks all the test cases on the current program to check its integrity. Though it is expensive as it needs to re-run all the cases, it ensures that there are no errors because of the modified code.[9]

Regression test selection[edit]

Unlike Retest all, this technique runs a part of the test suite (owing to the cost of retest all) if the cost of selecting the part of the test suite is less than the Retest all technique.[9]

Test case prioritization[edit]

Prioritize the test cases so as to increase a test suite's rate of fault detection. Test case prioritization techniques schedule test cases so that the test cases that are

higher in priority are executed before the test cases that have a lower priority.[9]

Types of test case prioritization[edit]

General prioritization – Prioritize test cases that will be beneficial on subsequent versions

Version-specific prioritization – Prioritize test cases with respect to a particular version of the software.

Hybrid[edit]

This technique is a hybrid of regression test selection and test case prioritization.[9]

Benefits and drawbacks[edit]

Regression testing is performed when changes are made to the existing functionality of the software or if there is a bug fix in the software. Regression testing can be achieved through multiple approaches, if a *test all* approach is followed, it provides certainty that the changes made to the software have not affected the existing functionalities, which are unaltered.[10]

In agile software development—where the software development life cycles are very short, resources are scarce, and changes to the software are very frequent—regression testing might introduce a lot of unnecessary overhead.[10]

In a software development environment which tends to use black box components from a third party, performing regression testing can be tricky, as any change in the third-party component may interfere with the rest of the system (and performing regression testing on a third-party component is difficult, because it is an unknown entity).[10]

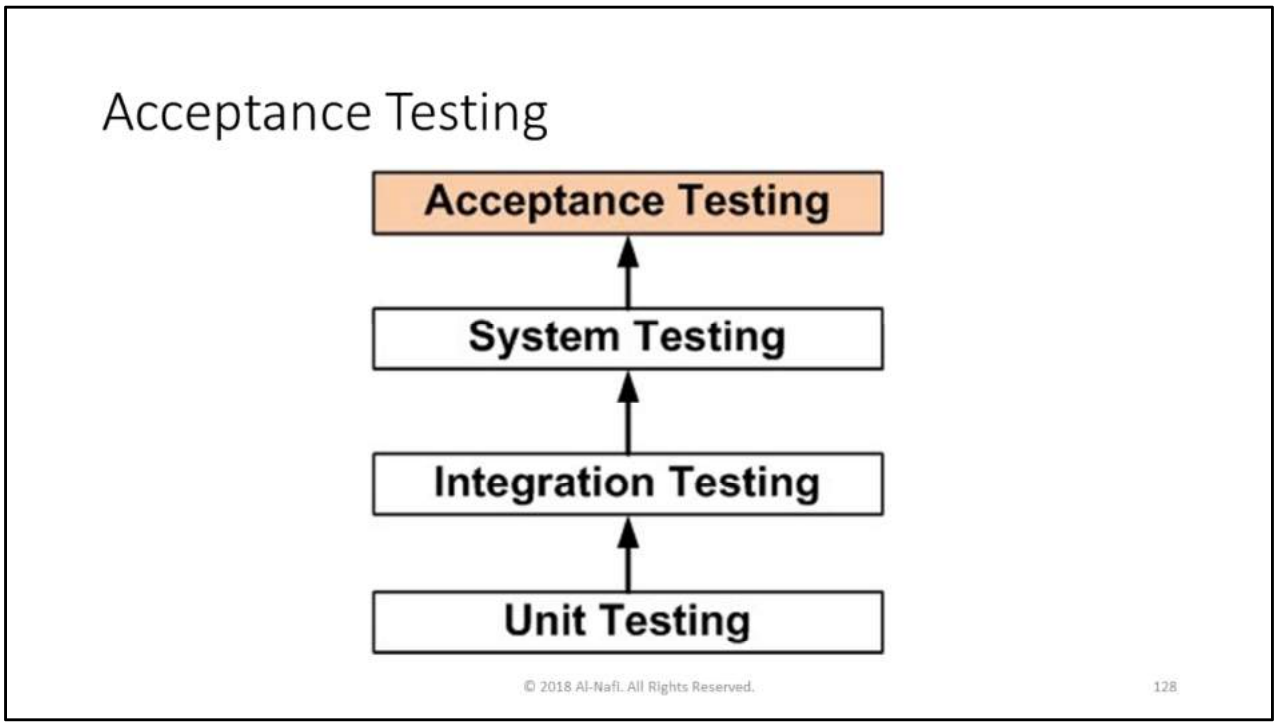
Uses[edit]

Regression testing can be used not only for testing the *correctness* of a program but often also for tracking the quality of its output.[11] For instance, in the design of a compiler, regression testing could track the code size and the time it takes to compile and execute the test suite cases.

> Also as a consequence of the introduction of new bugs, program maintenance requires far more system testing per statement written than any other programming. Theoretically, after each fix, one must run the entire batch of test cases previously run against the system to ensure that it has not been damaged in an obscure way. In practice, such *regression testing* must indeed approximate this theoretical idea, and it is very costly.
>
> — *Fred Brooks, The Mythical Man Month, p. 122*

Regression tests can be broadly categorized as functional tests or unit tests. Functional tests exercise the complete program with various inputs. Unit tests exercise individual functions, subroutines, or object methods. Both functional testing tools and unit-testing tools tend to be automated and are often third-party products that are not part of the compiler suite. A functional test may be a scripted series of program inputs, possibly even involving an automated mechanism for controlling mouse movements and clicks. A unit test may be a set of separate functions within the code itself or a driver layer that links to the code without altering the code being

tested.



Reference http://softwaretestingfundamentals.com/acceptance-testing/

ACCEPTANCE TESTING is a level of software testing where a system is tested for acceptability. The purpose of this test is to evaluate the system's compliance with the business requirements and assess whether it is acceptable for delivery.

Definition by ISTQB

acceptance testing: Formal testing with respect to user needs, requirements, and business processes conducted to determine whether or not a system satisfies the acceptance criteria and to enable the user, customers or other authorized entity to determine whether or not to accept the system.

Analogy

During the process of manufacturing a ballpoint pen, the cap, the body, the tail and clip, the ink cartridge and the ballpoint are produced separately and unit tested separately. When two or more units are ready, they are assembled and Integration Testing is performed. When the complete pen is integrated, System Testing is performed. Once System Testing is complete, Acceptance Testing is performed so as to confirm that the ballpoint pen is ready to be made available to the end-users.

Method

Usually, Black Box Testing method is used in Acceptance Testing. Testing does not normally follow a strict procedure and is not scripted but is rather ad-hoc.

Tasks

Acceptance Test Plan

- Prepare
- Review
- Rework
- Baseline

Acceptance Test Cases/Checklist

- Prepare
- Review
- Rework
- Baseline

Acceptance Test

- Perform

When is it performed?

Acceptance Testing is the fourth and last level of software testing performed after System Testing and before making the system available for actual use.

Who performs it?

Internal Acceptance Testing (Also known as Alpha Testing) is performed by members of the organization that developed the software but who are not directly involved in the project (Development or Testing). Usually, it is the members of Product Management, Sales and/or Customer Support.

External Acceptance Testing is performed by people who are not employees of the organization that developed the software.

- *Customer Acceptance Testing* is performed by the customers of the organization that developed the software. They are the ones who asked the organization to develop the software. [This is in the case of the software not being owned by the organization that developed it.]
- *User Acceptance Testing* (Also known as Beta Testing) is performed by the end users of the software. They can be the customers themselves or the customers' customers.

جزاك اللهُ

To ask questions, Join the Al Nafi Official Group

https://www.facebook.com/groups/alnafi/

(This group is only for members to ask questions)

© 2018 Al-Nafi. All Rights Reserved.